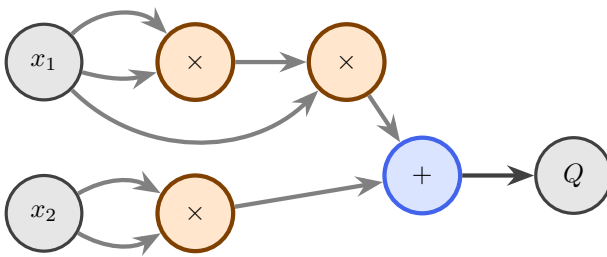


ZKDL Lecture Notes

Version 0.4



Abstract

Due to the rise of zero-knowledge technologies and their applications in various fields such as Blockchain or anonymous identity management, it is essential to develop a comprehensive understanding of the underlying mechanisms. However, the existing resources on the topic are either too high-level or too low-level, making it hard for regular practicing engineers to understand the practical implications of zero-knowledge protocols.

This book aims to bridge this gap by providing a complete, practical guide to the state-of-the-art techniques in zero-knowledge cryptography, such as Σ -protocols, zk-SNARKs (Groth16 in particular), PlonK and more. We gathered all the necessary information in one place, and tried to make it easy to follow, with numerous examples and code snippets. We attach exercises to each chapter to help you understand the material better. Despite the book's practical focus, we preserve the mathematical rigor where suitable and necessary.

Contents

1	Preface	9
2	Notation	11
I	Mathematics Crash Course	14
3	Introduction to Number Theory	15
3.1	Division, GCD, LCM	15
3.2	Extended Euclidean algorithm	20
3.3	Prime numbers	21
3.4	Fundamental Theorem of Arithmetic	23
3.5	Modular arithmetic	25
3.6	Chinese Remainder Theorem	30
3.7	Euler's Theorem	31
3.8	Exercises	33
4	Introduction to Abstract Algebra	35
4.1	Groups	35
4.2	Subgroups	38
4.3	Cyclic Groups	38
4.4	Cosets and Lagrange's Theorem*	41
4.5	Isomorphisms and Endomorphisms	42
4.6	Fields	44
4.7	Finite Fields	45
4.8	Some Fun: Group Implementation in Rust	45
4.9	Exercises	50
5	Polynomials	53
5.1	Basic Definition	53
5.2	Roots and divisibility	54
5.3	Interpolation	55
5.4	Schwartz-Zippel Lemma	57
5.5	Some Fun: Shamir's Secret Sharing	57
5.6	Exercises	59
6	Linear Algebra Basics	60
6.1	Vector Space	60
6.2	Matrix	61
6.3	Inner Product	64
6.4	Hadamard Product	64
6.5	Outer Product	65

7	Fields Extension	66
7.1	General Definition	66
7.2	Polynomial Quotient Ring	67
7.3	Multiplicative Group of a Finite Field	70
7.4	Algebraic Closure	71
7.5	Exercises	72
II	Cryptography Tools	74
8	Basics of Security Analysis	75
8.1	Cipher Semantic Security	75
8.2	Message recovery attacks	78
8.3	Discrete Logarithm Assumption	79
8.4	Computational Diffie-Hellman	80
8.5	Decisional Diffie-Hellman	80
8.6	Exercises	82
9	Elliptic Curves and Pairing	85
9.1	Classical Definition	85
9.2	Discrete Logarithm Problem on Elliptic Curves	89
9.3	Relations	89
9.4	Elliptic Curve in Projective Coordinates	92
9.4.1	Projective Space	92
9.4.2	Elliptic Curve Equation in Projective Form	95
9.4.3	General Projective Coordinates	96
9.4.4	Fast Addition	97
9.4.5	Scalar Multiplication Basic Implementation	99
9.5	Elliptic Curve Pairing	100
9.5.1	Case Study: BLS Signature	103
9.5.2	Case Study: Verifying Quadratic Equations	103
9.6	Exercises	104
10	Commitment Schemes	107
10.1	Hash-based commitments	108
10.2	Pedersen commitments	109
10.3	Vector commitments	110
10.4	Polynomial commitment	112
10.5	Exercises	113
III	Foundational Zero-Knowledge Protocols	114
11	Introduction to Zero-Knowledge Proofs	115
11.1	Motivation	115

11.2	Relations and Languages	116
11.3	Interactive Probabilistic Proofs	117
11.3.1	Example: Quadratic Residue Test	118
11.4	Zero-Knowledge	120
11.4.1	The Verifier's View	121
11.4.2	The Simulation Paradigm	122
11.5	Proof of Knowledge	123
11.6	Fiat-Shamir Heuristic	124
11.6.1	Random Oracle	124
11.6.2	Fiat-Shamir Transformation	125
11.7	Exercises	128
12	Sigma Protocols	131
12.1	Schnorr's Identification Protocol	131
12.2	Schnorr's Signature Scheme	133
12.3	Sigma Protocols	134
12.4	More Sigma Protocol Examples	136
12.4.1	Okamoto's Protocol for Representations	136
12.4.2	Chaum-Pedersen protocol for DH-triplets	137
12.5	Generalizing Sigma Protocols	138
12.6	Combining Sigma Protocols	139
12.6.1	The AND Sigma Protocol	139
12.6.2	The OR Sigma Protocol	140
12.7	Okamoto Protocol Sage Implementation	141
12.7.1	Prover Implementation	141
12.7.2	Verifier Implementation	143
12.8	Exercises	144
13	Arithmetic Circuits. R1CS	146
13.1	What is zk-SNARK?	146
13.1.1	Informal Overview	146
13.1.2	Formal Definition	147
13.2	Arithmetic Circuits	150
13.2.1	What is Arithmetic Circuit?	150
13.2.2	More advanced examples	151
13.2.3	Circuit Satisfiability Problem	153
13.3	Rank-1 Constraint System	154
13.3.1	Constraint Definition	155
13.3.2	Why Rank-1?	157
14	Quadratic Arithmetic Program	158
14.1	R1CS in Matrix Form	158
14.2	Polynomial Interpolation	160
14.3	Putting All Together!	162
14.4	Probabilistically Checkable Proofs	164

14.5	QAP as a Linear PCP	166
15	Pairing-based SNARKs	168
15.1	Building Pairing-based SNARK	168
15.1.1	Attempt #1: Encrypted Verification	168
15.1.2	Attempt #2: Including Proof of Exponent	170
15.1.3	Attempt #3: Making SNARK Sound	171
15.1.4	Attempt #4: Splitting the Extended Witness	174
15.1.5	Attempt #5: Making SNARK Zero-Knowledge	176
15.2	Real Protocols	180
15.3	Pinocchio Protocol	180
15.4	Groth16 Protocol	183
16	Circom	185
16.1	Journey Begins	185
16.2	From Theory to Practice: Circom Basics	185
16.3	Arguments, Functions, and Vars	186
16.4	Theoretical Recap: Using the Learned Concepts	187
16.5	From R1CS to Proof Generation	189
16.6	Parallel and Custom Keywords	193
16.7	Generating and Verifying Proofs	194
17	PlonK	197
17.1	Number Theoretical Transform	197
17.1.1	Barycentric Interpolation	197
17.1.2	Multiplicative Cyclic Subgroup	198
17.1.3	Fast Polynomial Multiplication	200
17.2	Plonk Arithmetization	203
17.2.1	Execution Trace	203
17.2.2	Encode the program	204
17.2.3	Custom Gates	206
17.2.4	Public Inputs	206
17.2.5	Matrices to Polynomials	208
17.2.6	Summary	212
17.3	Plonk Prover and Verifier	212
17.3.1	Gadgets	213
17.3.2	Proving	214
17.3.3	Verification	215
18	Bulletproofs	218
18.1	Introduction	218
18.2	Notation	218
18.3	Zero-knowledge multiplication	218
18.3.1	Naïve polynomial multiplication protocol	219
18.3.2	Optimized polynomial multiplication protocol	221

18.3.3	Zero-knowledge multiplication protocol	222
18.3.4	Zero-knowledge inner-product protocol	223
18.4	Inner-product argument	224
18.4.1	Inner-product compression	225
18.4.2	Proving $\mathcal{R}'_{ip}$	227
18.4.3	Zero-knowledge extension of inner-product argument	231
18.5	Inner-product based polynomial commitment scheme	232
18.6	Range proofs	233
18.7	Arithmetic circuits proofs	238
18.7.1	Arithmetization	238
18.7.2	Proving a circuit satisfiability	241

IV Sum-Check-based Protocols and Lookup Checks 248

19	Sum-Check Protocol	249
19.1	Multivariate Polynomials	249
19.2	Multilinear Extension	250
19.3	The Sum-Check Protocol	251
19.3.1	Protocol Description	251
19.3.2	Sum-Check Protocol Implementation	253
19.4	Sumcheck Applications	258
19.4.1	The #SAT Problem	258
19.4.2	Matrix Multiplication Verification (MatMul Check-Sum Protocol)	259
19.4.3	MatMul Sum-Check Protocol Implementation	260
20	GKR Protocol	264
20.1	Motivation	264
20.2	Protocol Description	264
20.2.1	Circuit Representation	264
20.2.2	Protocol Specification	266
21	Offline Memory Checking	269
21.1	Sum-Check-based Grand Product Protocol	269
21.2	Randomized Permutation Check	271
21.3	Offline Memory Checking	273
22	Lookup Checks	277
22.1	Motivation	277
22.2	Plookup Protocol	278
22.2.1	plookup Precise Scheme	280
22.3	Logup	282
22.3.1	Preliminaries	282
22.3.2	Logarithmic Derivative	283

22.3.3	Applying SumCheck	285
22.3.4	Precise Scheme	286
23	UltraGroth	288
23.1	Protocol Motivation	288
23.1.1	Historical Notes	288
23.2	Lookup Checks	289
23.2.1	Some other implications	289
23.3	UltraGroth	290
23.4	Two-round Interactive R1CS	290
23.5	Generalizing to multiple rounds	291
23.6	Efficiency Analysis	293
23.7	Practical Implementation and Further Steps	293
23.8	Acknowledgements	293
23.9	Summary	293
V	Concluding Remarks	294
24	Acknowledgements	301

1 Preface

Motivation. Cryptography is wonderful. If you are holding this book, you are probably already know the significance of cryptography in the modern informational infrastructure: from classical standard protocols such as Transport Layer Security (TLS) to the more recent cryptographic advancements used in Blockchain technologies. This book is exactly about the latter.

In particular, as the book's name suggests, we consider the *zero-knowledge cryptography*, which is a cornerstone of numerous privacy-preserving protocols. However, in its current state, the zero-knowledge technology is *tremendously* hard to work with:

- The technology is relatively new, so there are almost no high-level abstractions that can be used to simplify the design of zero-knowledge protocols for regular developers.
- The technology is based on advanced mathematics, which is hard to understand without a proper background.
- The amount of available resources on the topic is surprisingly limited. What's more, the resources are very diverse: they are either (a) too high-level, which makes you understand what the protocol *should* do, but you still have no idea how to implement it, (b) or too low-level, consisting of numerous security proofs and obscure theorems with little practical implications. Note that in the latter case you typically still have no clue what was going on, unless you spend a significant amount of time studying the topic.

Of course, this book cannot solve all the aforementioned problems at once. However, we do hope that this book will serve as a complete, practical guide to most state-of-the-art techniques in zero-knowledge cryptography for practicing engineers. We gathered all the necessary information in one place, and tried to make it easy-to-follow, with a lot of examples and code snippets. Yet, we still try to preserve the mathematical rigor where suitable and necessary.

Who are we? We are *Distributed Lab* — a Ukrainian cryptography and engineering team focused on cryptography related projects (primarily in Bitcoin and EVM ecosystems). We have been working on numerous zero-knowledge solutions involving optimizing advanced algebraic system components, ranging from fast Groth16 RSA or 384-bit Brainpool ECDSA verification to PlonK circuits acceleration of BN254 elliptic curve operations and pairing. We are significantly contributing to the Bitcoin ecosystem and developing payment and social applications with the focus on privacy (e.g. *Rarimo*).

Who is this book for? This book does require basic background in Mathematics. Of course, we will try to explain all the necessary concepts from scratch (such as basic number and group theory, security analysis), but this material is rather supplementary and is not the main focus of the book. That being said, you do not need Math PhD, but be prepared: the book is challenging.

How to use this book? The book is structured in the following way:

1. The first part of the book, consisting of sections from [Section 3](#) to [Section 8](#), is dedicated to the theoretical background on Mathematics and Cryptography needed for understanding the zero-knowledge protocols. If you feel comfortable with polynomials, groups, elliptic curves and commitment schemes, you can skip this part.
2. The main part of the book starts from [Section 11](#), where we first define the zero-knowledge proofs and explain the basic concepts behind them. We then proceed to the more advanced topics, such as Sigma protocols ([Section 12](#)), SNARKs (from [Section 13](#) to [Section 16](#)), and PlonK ([Section 17](#)).

Note on References. This book currently contains only a limited number of references explicitly cited in the text. However, before publication, we intend to incorporate a comprehensive list of sources that have contributed to its content. This will ensure proper acknowledgment of all relevant works and contributors.

2 Notation

Before delving into the world of zero-knowledge, we need to settle the notation which we are going to use throughout the book.

Set Theory

A **set** is a well-defined collection of objects. The objects that belong to a set are called its **elements**. We will denote sets by capital letters, if a is an element of the set A , we write $a \in A$.

Let us enumerate some fundamental sets:

- $\mathbb{N} = \{1, 2, 3, \dots\}$ – a set of natural numbers.
- $\mathbb{Z} = \{0, \pm 1, \pm 2, \pm 3, \dots\}$ – a set of integers.
- $\mathbb{Q} = \{\frac{n}{m} : n \in \mathbb{Z}, m \in \mathbb{N}\}$ – a set of rational numbers.
- $\mathbb{R}$ – a set of real numbers. Examples: 2.2, $-3.90.12$, 1.4, $-6.7, \dots$
- $\mathbb{R}_{>0}$ – a set of positive real numbers. Examples: 2.6, 10.4, 100.2.
- $\mathbb{C}$ – a set of complex numbers¹. Examples: $1 + 2i$, $5i$, $-7 - 5.7i, \dots$

To represent the number of elements in a set A , we write $|A|$. If the set is finite, $|A| \in \mathbb{N}$, otherwise $|A| = \infty$. $A \subset B$ denotes “ A is a subset of B ” (meaning that all elements of A are also in B , e.g., $\mathbb{Q} \subset \mathbb{R}$).

$A \cap B$ means the intersection of A and B (a set of elements belonging to both A and B), while $A \cup B$ – the union of A and B (the set of elements belonging to either A or B). $A \setminus B$ denotes the set difference (the set of elements belonging to A , but not B). $\overline{A}$ denotes the complement of A (the set of elements not belonging to A). All operations are illustrated in **Figure 2.1** (this picture is typically called the *Venn Diagram*).

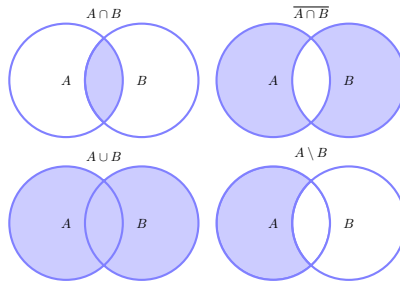


Figure 2.1: Set operations illustrated with Venn diagrams.

As you may have noticed, to define the set we typically write $\{f(a) : \phi(a)\}$, where $f(a)$ is some function and $\phi(a)$ is a predicate (function, inputting a and

¹Complex number is an expression in a form $x + iy$ for $i^2 = -1$

returning true/false if a certain condition on a is met). For example, $\{x^3 : x \in \mathbb{R}, x^2 = 4\}$ is “a set of values x^3 which are the real solutions to equation $x^2 = 4$ ”. It is quite easy to see that this set is simply $\{2^3, (-2)^3\} = \{8, -8\}$.

Cartesian Products and Functions

The notation $A \times B$ means a set of pairs (a, b) where $a \in A$ and $b \in B$ (or, written shortly, $A \times B = \{(a, b) : a \in A, b \in B\}$), called a Cartesian product. We additionally introduce notation $A^n := A \times A \times \cdots \times A$ – Cartesian product n times. For example, $\mathbb{Q}^3$ is a set of triplets (a, b, c) where $a, b, c \in \mathbb{Q}$, while $\mathbb{Q}^2 \times \mathbb{R}$ is a set of triplets (a, b, c) where $a, b \in \mathbb{Q}$ and $c \in \mathbb{R}$.

Subsets of $A \times B$ are called relations. We define a function $f \subset A \times B$ from a set A to a set B , as the special type of relation in which for each element $a \in A$ there is a *unique* element $b \in B$ such that $(a, b) \in f$. In other words, for every element in A we assigns a unique element in B . We usually write $f : A \rightarrow B$ to denote such a relation and write $f(a) = b$ instead of writing down $(a, b) \in f$.

We say the function $f : A \rightarrow B$ is **injective** if condition $a_1 = a_2$ implies $f(a_1) = f(a_2)$. Function f is said to be **surjective** if for each $b \in B$ there exists $a \in A$ such that $f(a) = b$. A function that is both injective and surjective is called a **bijection**.

Logic

Statement beginning with $\forall$ means “for all...”. For instance, $(\forall a \in A \subset \mathbb{R}) : \{a < 1\}$ is read as: “For any a in set A (which is a subset of real numbers), it is true that $a < 1$ ”. Or, more shortly, “Any (real) a from A is less than 1”.

Statement beginning from $\exists$ means “there exists such...”. Let us consider the following example: $(\exists \varepsilon > 0)(\forall a \in A) : \{a > \varepsilon\}$ is read as “there exists such a positive ε such that for any element a from A , a is greater than ε ”, or, more concisely, “there exists a positive constant ε such that any element from A is greater than ε ”.

Statement beginning from $\exists!$ means “there exists a unique...”. For example, $(\exists! x \in \mathbb{R}_{>0}) : \{x^2 = 4\}$ is read as “there exists a unique positive real x such that $x^2 = 4$ ”.

Symbol $\wedge$ means “and”. For example, $\{x \in \mathbb{R} : x^2 = 4 \wedge x > 0\}$ is read as “a set of real x such that $x^2 = 4$ and x is positive”. Of course, $\{x \in \mathbb{R} : x^2 = 4 \wedge x > 0\} = \{2\}$.

Symbol $\vee$ means “or”. For example, $\{x \in \mathbb{R} : x^2 = 4 \vee x^2 = 9\}$ is read as “a set of real x such that either $x^2 = 4$ or $x^2 = 9$ ”, this set is equal to $\{\pm 2, \pm 3\}$.

Sequences, Matrices, and Vectors

A **sequence** is a mapping from natural numbers $\mathbb{N}$ to some specific set X . To denote the sequence, we write $\{x_n\}_{n \in \mathbb{N}} \subset X$. If the sequence iterates through some specific set $\mathcal{I} \subseteq \mathbb{N}$, we write $\{x_n\}_{n \in \mathcal{I}}$. For example, $\{x_n\}_{n \in \{1,2,5\}} =$

(x_1, x_2, x_5) . We write vectors in bold font, e.g., $\mathbf{x} = (x_1, x_2, x_3) \in \mathbb{F}^3$, while matrices are denoted by capital letters in regular font. Here is an example matrix:

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \in \mathbb{F}^{2 \times 2}$. For additional linear algebra notation, refer to [Section 6](#), where we introduce the very basics of linear algebra.

Finally, we denote the set $\{0, \dots, n-1\}$ for $n \in \mathbb{N}$ by $[n]$. This notation is used very extensively in the book, so make sure to remember it. For example, instead of writing $\sum_{k=0}^{n-1} k$ we sometimes write $\sum_{k \in [n]} k = \frac{n(n-1)}{2}$. Or, instead of specifying n conditions $z_1 = f(x_0), z_2 = f(x_1), \dots, z_n = f(x_{n-1})$ explicitly, we simply say $z_{i+1} = f(x_i)$ for all $i \in [n]$.

Algorithms and Probabilities

Suppose we specified the algorithm A that takes some input x and returns some output y . We write $y \leftarrow A(x)$ to denote that the algorithm A is executed on input x and returns output y . To denote some party in the protocol we use calligraphic font, e.g., $\mathcal{P}$ typically denotes the prover while $\mathcal{V}$ denotes the verifier. Such parties are algorithms themselves, so we can easily write $y \leftarrow \mathcal{P}(x)$ to denote that the prover, based on x , has outputted y .

To denote the probability of an event E happening, we write $\Pr[E]$. For example, if E is an event of rolling a dice and getting a number 4, then $\Pr[E] = \frac{1}{6}$. To denote the random sampling from the finite set X , we write $x \leftarrow \$ X$. For example, $x \leftarrow \$ \{1, 2, 3, 4, 5, 6\}$ denotes the random sampling of x from the set $\{1, 2, 3, 4, 5, 6\}$.

To denote the probability of both E and F happening, we write $\Pr[E, F]$ (as opposed to $E \cap F$). To denote the conditional probability, we write $\Pr[E|F]$, which means the probability of E happening given that F has already happened. In practice, we place the protocol execution in F , while E is the event of the protocol outputting a certain result. For example,

$$\Pr[x \equiv 1 \pmod{3} \mid x \leftarrow \text{RollDice}(\cdot)] = \frac{1}{3}.$$

When event F can be written concisely, we write it in the subindex of $\Pr$. For instance, $\Pr_{x \leftarrow \$ [6]}[x \equiv 0 \pmod{2}] = \frac{1}{2}$.

Part I

Mathematics Crash Course

As any cryptography subfield, Cryptography requires a solid knowledge of Mathematics. While the practical development of cryptographic protocols might not include the need to understand the whole underlying theory, much of it typically still arises. For example, one might need to implement the Elliptic Curve Pairing operation verification in a Zero-Knowledge Proof system, or RSA encryption and decryption methods in Circom circuits. In case you specifically develop the zero-knowledge systems from scratch, there is no need to explain why you need to understand the underlying theory.

This part of the book is dedicated to the mathematical background required for Cryptography. Namely, we will cover essentials specified in [Table 1](#).

Section	Topic	Key Concepts
3	Number Theory	Modular Arithmetic, Ring $\mathbb{Z}_n$ and field $\mathbb{Z}_n^\times$, Fermat's Little Theorem
4	Abstract Algebra	Groups, Subgroups, Fields, Prime field $\mathbb{F}_p$, Isomorphisms, Automorphisms
5	Polynomials	Divisibility, Lagrange Interpolation, Schwartz-Zippel Lemma
6	Linear Algebra Basics	Basic Operations over Vectors and Matrices
7	Fields Extensions	Definition of $\mathbb{F}_{p^m}$, Field Multiplicative Subgroup, Algebraic Closure

Table 1: Topics covered in Part I

This is not the Mathematics book, so we will not prove every mentioned theorem or lemma, but we do provide reasoning for some key facts. We focus on the practical side of the theory, providing examples and exercises for the reader to solve. Moreover, the solutions to the exercises can be found in [Part V](#) for the reader to check their understanding.

The reader should already be familiar with very basics of the first four topics, so they will be covered for the sake of completeness and reminding the reader of the key concepts. The last topic, Fields Extensions, is less frequent in the Mathematics curriculum, so we will cover it in more detail.

In case you feel comfortable with the aforementioned topics, feel free to skip them and move to the next part of the book. Finally, if you are unsure about your knowledge, you should open the corresponding section and check whether you can solve exercises in the end without any help.

3 Introduction to Number Theory

As you can recall from high school math, typically real-world processes are described using real numbers, denoted by $\mathbb{R}$. For example, to describe the position or the velocity of an object, you would rather use real numbers instead of fields, groups, or p-adic numbers.

However, when it comes to working with computers, real numbers become very inconvenient to work with. The primary reason is that the set $\mathbb{R}$ is uncountably infinite, so the computer can only store the approximation of a real number (as the precise interpretation of the real number would require the infinite number of bits). For instance, different programming languages might output different values for quite a straightforward operation, such as an addition $2.00001 + 2.00000$. This becomes a huge problem when dealing with cryptography, which must check *precisely* whether two quantities are equal.

This is why the cryptography must work with integer-based formats. One of the earliest and most fundamental branches of mathematics is number theory, which deals with the properties of the integer set $\mathbb{Z}$. Although, as we will see later, we will primarily need notions of *finite fields* and *groups* further, the basic concepts of number theory will be used extensively nonetheless.

3.1 Division, GCD, LCM

We start with the most basic definition of number theory — *divisibility*.

Definition 3.1. An integer a is divisible by a non-zero integer b , denoted $b \mid a$ (or b is a *divisor* of a), if and only if there exists an integer $k \in \mathbb{Z}$ such that $a = k \cdot b$.

Example 3.1. Let us consider the following examples,

1. $41 \mid 123$, since $123 = 3 \cdot 41$
2. $5 \nmid 29$, since there is no integer k such that $29 = k \cdot 5$

Let us now explore some more basic properties of divisibility.

Lemma 3.2 (Divisibility properties). For all $a, b, c \in \mathbb{Z}$:

1. $1 \mid a$ (any number is divisible by 1)
2. If $a \neq 0$, then $a \mid a$ (any non-zero integer divides itself).
3. If $a \neq 0$, then $a \mid 0$ (any non-zero integer divides 0).
4. If $b \mid a$ and $c \mid b$, then $c \mid a$ (formally called *transitivity*).
5. $b \mid a \Leftrightarrow b \cdot c \mid a \cdot c$ for any $c \neq 0$.
6. If $c \mid a$ and $c \mid b$, then $c \mid (\alpha \cdot a \pm \beta \cdot b)$, for any $\alpha, \beta \in \mathbb{Z}$

This way, if we have a and b with $b \mid a$, we can safely divide a by b . However, in the majority of cases, it so happens that a is not divisible by b , making it hard to define the division in such cases. For that reason, consider one of the central theorems of Number Theory: the *Division theorem*.

Theorem 3.3 (Division theorem). For any integers $a, b \in \mathbb{Z}$ there exist unique integers $q, r \in \mathbb{Z}$ with $0 \leq r < |b|$ such that

$$a = b \cdot q + r.$$

This theorem allows us to define two new operations. Suppose a, b, q, r are given as in the **Definition 3.3**. Then,

- **Floor Operation** ($\lfloor a/b \rfloor$ or $a \text{ div } b$) is defined as q . This operation is a standard div operation commonly used in programming languages.
- **Mod Operation** ($a \bmod b$) is defined as r . This operation is a standard mod operation commonly used in programming languages.

Remark. There is one caveat with programming languages, though. Often-times, the quotient q and remainder r obey the aforementioned condition $a = b \cdot q + r$, but with the difference that $|r| < |b|$, allowing two possible values for r , including negative.

Example 3.2. Continuing **Example 3.1**,

1. We know that $5 \nmid 29$. However, by **Definition 3.3** we can express 29 as $29 = 5 \cdot 5 + 4$.
2. Similarly, it is evident that $34 \nmid 123$, but $123 = 3 \cdot 34 + 21$.

To compare the results, we can use the following Python code:

```
def divmod(a: int, b: int) -> Tuple[int, int]:
    q = a // b
    r = a % b
    return q, r

a, b = 29, 5
q, r = divmod(a, b)
assert a == b * q + r, 'Theorem does not hold'

print(q, r) # (5, 4)
```

In division operations, it is common to check for any shared factors between two numbers. This is where the concept of the **greatest common divisor** (or **GCD** for short) comes into play.

Definition 3.4 (GCD). For any $a, b \in \mathbb{Z}$, the **greatest common divisor** $\gcd(a, b)$ is defined as an integer $d \in \mathbb{N}$ such that:

1. $d \mid a$ and $d \mid b$.
2. d is a maximal integer that satisfies the first condition.

One might right the above definition more concisely:

$$\gcd(a, b) = \max\{d \in \mathbb{N} : d \mid a \text{ and } d \mid b\}.$$

Example 3.3. Just as an example, let us find the $\gcd(a, b)$ for a few pairs of integers.

1. $\gcd(42, 28) = 14$
2. $\gcd(36, 14) = 2$
3. $\gcd(13, 17) = 1$
4. $\gcd(13, 18) = 1$

Definition 3.5. Integers $a, b \in \mathbb{Z}$ are called **coprime** if $\gcd(a, b) = 1$.

As with any fundamental concept, let us explore some basic properties of the GCD operation.

Lemma 3.6 (Greatest common divisor properties).

1. $\gcd(a, b) = b \Leftrightarrow b \mid a$
2. If $a \neq 0$, then $\gcd(a, 0) = a$
3. If there exists $\delta \in \mathbb{Z}$ such that $\delta \mid a$ and $\delta \mid b$, then $\delta \mid \gcd(a, b)$
4. If $c > 0$, then $\gcd(ac, bc) = c \cdot \gcd(a, b)$
5. Integers $a/\gcd(a, b)$ and $b/\gcd(a, b)$ are coprime.

Now, from the programming standpoint, you might ask how to implement the GCD computation in practice. Consider the following lemma which, as of now, might seem quite abstract.

Lemma 3.7. For any integers $a, b \in \mathbb{Z}$, we have $\gcd(a, b) = \gcd(b, a - b)$.

How does it help? Here is how: instead of computing $\gcd(a, b)$ directly, we reduce the problem to a simpler one, where one of the numbers is smaller. We apply this recursively until, at some point, one of the integers is zero. The other one will be the GCD result, as follows from the second property of [Definition 3.6](#). Suprisingly, this can be written in a couple of lines of code:

```
def gcd(a: int, b: int) -> int:
    # We want a to be always greater than b
    if a <= b:
        a, b = b, a
```

```

    return a if b == 0 else gcd(b, a - b)

print(gcd(42, 28)) # Output: 14
print(gcd(36, 14)) # Output: 2
print(gcd(13, 17)) # Output: 1

```

However, such an approach in the worst-case scenario still requires linear time in the size of the numbers: for example, when computing $\text{gcd}(n, 1)$. The following lemma will be extensively employed in the so-called *Euclidean algorithm*, which will reduce the complexity down to logarithmic time.

Corollary 3.8. For any $a, b \in \mathbb{Z}$, we have $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

Now, this is much better! Let us apply this idea again, but with the modulo operation instead of subtraction:

```

def gcd(a: int, b: int) -> int:
    return gcd(b, a % b) if b != 0 else a

print(gcd(42, 28)) # Output: 14
print(gcd(36, 14)) # Output: 2
print(gcd(13, 17)) # Output: 1

```

Notice that the algorithm can be easily implemented in a single line! Moreover, the complexity reduced down to logarithmic time: for computing $\text{gcd}(a, b)$ we need $O(\log \max\{a, b\})$ time.

Example 3.4. Let us find the $\text{gcd}(535, 230)$ manually. We will use the Euclidean algorithm for this purpose.

$$535 = 2 \cdot 230 + 75$$

$$230 = 3 \cdot 75 + 5$$

$$75 = 15 \cdot 5 + 0$$

Therefore, $\text{gcd}(535, 230) = 5$.

While the greatest common divisor (GCD) focuses on finding the largest shared factor between two numbers, the least common multiple (LCM) deals with finding the smallest multiple that both numbers have in common. The LCM is particularly useful when we need to synchronize cycles or work with fractions.

Definition 3.9 (LCM). For any $a, b \in \mathbb{Z}$, the **least common multiple** $\text{lcm}(a, b)$ is defined as an integer $m \in \mathbb{N}$ such that:

1. $a \mid m$ and $b \mid m$
2. m is a minimal integer that satisfies the first condition

One might right the above definition more concisely:

$$\text{lcm}(a, b) = \min\{m \in \mathbb{N} : a \mid m \text{ and } b \mid m\}.$$

Example 3.5. Let us again find, but now lcm for few pairs of numbers.

- | | |
|-------------------------------|-------------------------------|
| 1. $\text{lcm}(12, 4) = 12$ | 5. $\text{lcm}(13, 18) = 234$ |
| 2. $\text{lcm}(13, 17) = 221$ | 4. $\text{lcm}(42, 28) = 84$ |

Lemma 3.10 (Least Common Multiple Properties).

1. We consider $\text{lcm}(a, 0)$ to be undefined.
2. $\text{lcm}(a, b) = a \Leftrightarrow b \mid a$.
3. If a and b are coprime, then $\text{lcm}(a, b) = a \cdot b$.
4. Any common divisor δ of a and b satisfies $\delta \mid \text{lcm}(a, b)$.
5. For any $c > 0$, we have $\text{lcm}(a \cdot c, c \cdot b) = c \cdot \text{lcm}(a, b)$.
6. Integers $\text{lcm}(a, b)/a$ and $\text{lcm}(a, b)/b$ are coprime.

Theorem 3.11. For any $a, b \in \mathbb{N}$, we have $\text{gcd}(a, b) \cdot \text{lcm}(a, b) = ab$.

One interpretation of the above theorem is that no additional algorithm is required for $\text{lcm}(a, b)$ if we already have an algorithm for $\text{gcd}(a, b)$. Indeed, we can simply use the formula $\text{lcm}(a, b) = ab/\text{gcd}(a, b)$ with the previously computed $\text{gcd}(a, b)$.

The reasonable question is how to generalize the gcd and lcm operations to more than two arguments. For that reason, we provide the following definition.

Definition 3.12 (GCD and LCM for multiple arguments). We define the **greatest common divisor** $\text{gcd}(a_1, a_2, \dots, a_n)$ and the **least common multiple** $\text{lcm}(a_1, a_2, \dots, a_n)$ for any set of integers $a_1, a_2, \dots, a_n \in \mathbb{Z}$ as follows:

$$\text{gcd}(a_1, a_2, \dots, a_n) = \max\{d \in \mathbb{N} : d \mid a_1, d \mid a_2, \dots, d \mid a_n\}.$$

$$\text{lcm}(a_1, a_2, \dots, a_n) = \min\{m \in \mathbb{N} : a_1 \mid m, a_2 \mid m, \dots, a_n \mid m\}.$$

Theorem 3.13 (Computational Properties Of GCD and LCM For Multiple Arguments.). The following two statements are true:

- $\forall a, b \in \mathbb{N} : \text{gcd}(a, b, c) = \text{gcd}(\text{gcd}(a, b), c) = \text{gcd}(a, \text{gcd}(a, b))$.
- $\forall a, b \in \mathbb{N} : \text{lcm}(a, b, c) = \text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, \text{lcm}(a, b))$.

In conclusion, from these two theorems there is no conceptual necessity for specific algorithms for $\text{gcd}(\cdot)$ and $\text{lcm}(\cdot)$ when dealing with more than two arguments. It is worth noting that there are more effective algorithms for more

than two arguments, but such a topic is well beyond the scope of this book.

3.2 Extended Euclidean algorithm

In this section, we will introduce the Extended Euclidean Algorithm and an important lemma related to the GCD. You might have reasonable question: why do we even need an extended version? One of the primary reasons is that this algorithm will help in finding inverse modular elements, introduced in the subsequent sections.

Lemma 3.14 (Bezout identity). For any two given integers $a, b \in \mathbb{N}$ with $d = \gcd(a, b)$ there exists such $u, v \in \mathbb{Z}$ that $d = au + bv$.

Example 3.6. It is evident that $\gcd(125, 93) = 1$, so we can find such $u, v \in \mathbb{Z}$ that $1 = 125 \cdot u + 93 \cdot v$. One such example is $u = 32$ and $v = -43$ since $1 = 125 \cdot 32 + 93 \cdot (-43)$. In this case, we call the tuple $(32, -43)$ as *Bezout coefficients*. We will explain and show how to find these coefficients concretely a bit later.

Corollary 3.15 (From Bezout identity).

1. Integers u and v are of different signs (excluding the case when either of Bezout coefficients is zero).
2. *Generalization for multiple integers:* Suppose $d = \gcd(a_1, a_2, \dots, a_n)$, then there exist such integers $u_1, u_2, \dots, u_n \in \mathbb{Z}$ that $d = \sum_{i=1}^n u_i a_i$.

The integers u and v are called **Bezout coefficients**. The first corollary can be understood intuitively: if all coefficients are non-negative, the result will be much larger than necessary. Similarly, if they are non-positive, the result must be negative, but the GCD was defined to be positive. The second consequence follows from the fact that we can decompose GCD, thus sequentially derive this sequence. Also note that we can find Bezout coefficients on each Euclidean algorithm step.

Now we introduce the **extended Euclidean algorithm**. The extended Euclidean algorithm, besides calculating the GCD itself, finds the Bezout coefficients efficiently (in the polylogarithmic time). For example, as we will see, for calculating the modular inverse element, one would rather use the extended Euclidean algorithm than the naive approach, which shows whether inverse exists at all. The time complexity of the Extended Euclidean algorithm is $O(\log \max\{a, b\})$ — the same as for the classic Euclidean algorithm.

Algorithm 1: Extended Euclidean algorithm

Input : $a, b \in \mathbb{N}$. Without loss of generality, $a \geq b$

Output: $(\gcd(a, b), u, v)$

```
1  $r_0 \leftarrow a; r_1 \leftarrow b$ 
2  $u_0 \leftarrow 1; u_1 \leftarrow 0$ 
3  $v_0 \leftarrow 0; v_1 \leftarrow 1$ 
4 while  $r_{i+1} \neq 0, i = 1, 2, \dots$  do
5    $q_i \leftarrow r_{i-1} \text{ div } r_i$ 
6    $u_{i+1} \leftarrow u_{i-1} - u_i q_i$ 
7    $v_{i+1} \leftarrow v_{i-1} - v_i q_i$ 
8    $r_{i+1} \leftarrow a u_{i+1} + b v_i$ 
9 end
10 return  $(r_i, u_i, v_i)$ 
```

Although we already know how the Extended Euclidean Algorithm works, this method is still not very intuitive. Let us consider an example to better understand how it works.

Example 3.7 (Extended Euclidean algorithm example).

First, we you need to find $d = \gcd(a, b)$. Then, knowing the sequence of expansions (quotient q_i at each step), we can find the Bezout coefficients.

1. $125 = 93 \cdot 1 + 32$
2. $93 = 32 \cdot 2 + 29$
3. $32 = 29 \cdot 1 + 3$
4. $29 = 3 \cdot 9 + 2$
5. $3 = 2 \cdot 1 + 1$
6. $2 = 1 \cdot 2 + 0$

i	0	1	2	3	4	5	6
q_i	$\times$	1	2	1	9	1	2
u_i	1	0	1	-2	3	-29	32
v_i	0	1	-1	3	-4	?	?

Here, each corresponding cell is calculated with the following formula:

$$u_{i-1} - q_i u_i = u_{i+1}$$

$$v_{i-1} - q_i v_i = v_{i+1}$$

Exercise. Finish this example by filling in the missed cells marked by ?. After finding v_6 , be sure to check yourself.

3.3 Prime numbers

Prime numbers are fundamental in mathematics due to their role of the building blocks of all natural numbers: in fact, every integer $n > 1$ can be uniquely factored into the product of primes, a concept known as the *Fundamental Theorem of Arithmetic*, which we introduce in the next section. This property makes primes central to number theory and cryptography in particular, where we will use them in finite fields extensively.

Definition 3.16. Number $p \in \mathbb{N}$ is called **prime** if and only if its only two positive divisors are 1 and n .

Definition 3.17. Number $n \in \mathbb{N}$ is **composite** iff there exists an integer $a \in \mathbb{N}$, which is not 1 or n , for which $a \mid n$. In other words, it is not prime.

Example 3.8. For example 3, 17, 19, 29, 71, 997, 7817, 104729 are all examples of prime numbers. In contrast, 8, 1001, 10000, 100000 are all examples of composite numbers.

One might ask: what to do with the number 1? We consider it to be neither prime nor composite. Another logical question is whether the set of prime numbers is finite or infinite. The next theorem settles this question.

Theorem 3.18 (Euclidean theorem). If $P \subset \mathbb{N}$ is a finite set, consisting of prime numbers, then there exists a prime number p' such that $p' \notin P$.

Proof Idea. We can set $p' := \prod_{p \in P} p + 1$. Although it is intuitively clear that p' is not divisible by any prime number in P , the formal proof requires a bit more effort.

Corollary 3.19. There exists an infinite number of primes.

Lemma 3.20. For all $n \in \mathbb{N}$, where n is composite, if there exists a minimal divisor $d > 1$ of n , then d is a prime number.

Consider a scenario where we have a large number and need to determine whether it is prime. How can this be achieved? In other words, what methods exist for primality testing? Several approaches are available. The most straightforward method is brute-force division, though it is impractical for large numbers. Probabilistic tests provide a more feasible alternative, though they introduce a small probability of error. In 2003, a significant breakthrough was made with the discovery of a deterministic primality test, which placed the problem within the P complexity class.

It is also important to note that various types of prime numbers have applications in different fields. Examples include Mersenne primes, factorial primes, Euclidean primes, and Fibonacci primes, among others. As an example, consider one of the most well-known categories (which you might encounter in the modern zero-knowledge protocols): Mersenne primes.

Definition 3.21 (Mersenne primes). The prime number of the form $2^p - 1$ is called **Mersenne prime**, where p is a prime number, usually notated as following $M_p = 2^p - 1$.

Example 3.9. For example, $2^7 - 1 = 127$ is a Mersenne prime, since 127 is a prime number. However, $2^{11} - 1 = 2047$ is not a Mersenne prime, since $2047 = 23 \cdot 89$. As of 2025, only 52 such numbers were found, the largest of which is $2^{136279841} - 1$.

Lemma 3.22. If the number M_p is prime, then so is p .

Mersenne primes, are important in both number theory and cryptography. They have unique mathematical properties that make them useful for testing primality and generating large prime numbers. Mersenne primes are also crucial in the construction of efficient algorithms for error correction in coding theory and for generating random numbers in cryptographic applications, etc.

Finally, we introduce the Dirichlet's theorem, which is sometimes used for prime generation in cryptographic systems or constructing special primes.

Theorem 3.23 (Dirichlet's theorem). For any $a, b \in \mathbb{N}$ if $\gcd(a, b) = 1$, then infinite primes number form of $am + b$ exists, where $m \in \mathbb{N}$.

In other words, every infinite arithmetic progression whose first term and difference are positive integers contains an infinite number of primes.

3.4 Fundamental Theorem of Arithmetic

The Fundamental Theorem of Arithmetic states that every integer greater than 1 can be uniquely factored into primes, which is crucial for understanding the structure of numbers. It plays a key role in areas like number theory, cryptography, and simplifying calculations involving divisibility.

Example 3.10. To see this fact in action, let us decompose an integer $n = 123456$ into a product of prime numbers.

$$123456 = 2^6 \cdot 3^1 \cdot 643^1$$

Before formulating the Fundamental Theorem of Arithmetic, let us consider the auxiliary Lemma 3.24.

Lemma 3.24 (Euclidean). If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.

As an exercise and to show where Bezout coefficients are used, let us prove

both this lemma and the subsequent theorem.

Proof. ► Let $p \mid ab$, but $p \nmid a$, then $\gcd(a, p) = 1$ because there are no common divisors. In which case, by Bezout identity [Definition 3.14](#), there exist such $u, v \in \mathbb{Z}$ that $au + pv = 1$. Let's multiply the left and right sides by b : $abu + pbv = b$, but $p \mid ab$ and $p \mid pb$, therefore their sum is also divisible by $p \mid abu + pbv$. ◀

Now, we are ready to introduce the central Number Theory theorem – Fundamental Theorem of Arithmetic.

Theorem 3.25 (Fundamental theorem of arithmetic). Any integer $n > 1$ can be decomposed in the unique way into a product of prime numbers:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_t^{\alpha_t} = \prod_{j=1}^t p_j^{\alpha_j},$$

where $p_1, \dots, p_t$ are prime numbers and $\alpha_1, \dots, \alpha_t \in \mathbb{N}$.

Proof. ► We need to prove both existence and uniqueness. Since the existence is easy to prove, let us make the small exception and show it. For other proves (in particular, for the uniqueness case), see the literature provided.

Existence. Suppose the theorem statement is false and there exists n that does not have such a representation. Let n_0 be the smallest of them. If n_0 is prime, it can be represented as $p_1 = n_0$, $\alpha_1 = 1$, which satisfies the representation. Therefore, n_0 must be composite, which means $\exists a, b \in \mathbb{N}, 1 < a, b < n_0$ such that $n = ab$. Since n_0 is the smallest non-decomposable number and $a, b < n_0$, then a and b can be represented as

$$\begin{aligned} a &= p_1^{\alpha_1} \cdots p_t^{\alpha_t} \\ b &= q_1^{\beta_1} \cdots q_k^{\beta_k}. \end{aligned}$$

Therefore, $n_0 = ab = p_1^{\alpha_1} \cdots p_t^{\alpha_t} q_1^{\beta_1} \cdots q_k^{\beta_k}$. Thus, the contradiction, as n_0 is represented as the product of primes. ◀

Corollary 3.26. Suppose n is decomposed as in [Definition 3.25](#). Then,

1. If $d \mid n$, then $d = p_1^{\beta_1} p_2^{\beta_2} \cdots p_t^{\beta_t}$, where $0 \leq \beta_i \leq \alpha_i$.
2. Suppose $a = p_1^{\alpha_1} \cdots p_t^{\alpha_t}$ and $b = p_1^{\beta_1} \cdots p_t^{\beta_t}$. Then, the GCD can be evaluated as $\gcd(a, b) = \prod_{i=1}^t p_i^{\min\{\alpha_i, \beta_i\}}$.
3. Suppose $a = p_1^{\alpha_1} \cdots p_t^{\alpha_t}$ and $b = p_1^{\beta_1} \cdots p_t^{\beta_t}$. Then, the LCM can be evaluated as $\text{lcm}(a, b) = \prod_{i=1}^t p_i^{\max\{\alpha_i, \beta_i\}}$.
4. If $b \mid a$ and $c \mid a$ and $\gcd(b, c) = 1$, then $bc \mid a$.

The implications and applications of the Fundamental Theorem of Arithmetic are large and important, and the number of “obvious” ones are listed above.

Remark. Note that the problem of factorization, i.e., finding the decomposition of a number into prime factors, is in the NP class (of complexity) and is the basis of certain cryptosystems such as RSA. For stricter P/NP definitions, see Section 11.

3.5 Modular arithmetic

Now we are ready for practical applications of number theory, starting with modulo congruence. In this section, we will review the idea of congruence, learn how to use it, and explore its key properties. Understanding these properties will help simplify calculations and solve problems more efficiently in fields such as cryptography, algorithms, and number theory.

Definition 3.27. Two integers $a, b \in \mathbb{Z}$ are said to be **congruent** modulo $n \in \mathbb{N}$ iff $n \mid (a - b)$. We denote this by $a \equiv b \pmod{n}$.

Example 3.11. It easy to see that $26 \equiv 5 \pmod{7}$ since $7 \mid (26 - 5)$.

As always, we enumerate some key properties of congruence modulo n relation to better understand its nature. First, we consider the properties of reflexivity, symmetry, and transitivity to confirm that congruence modulo n is what is called an *equivalence relation*. For general equivalence relation definition, see Section 9.3.

Lemma 3.28 (Reflexivity, Symmetry, and Transitivity).

1. $a \equiv a \pmod{n}$
2. If $a \equiv b \pmod{n}$, then $b \equiv a \pmod{n}$
3. If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$

In turn, the following lemma states that we can perform addition, subtraction, and multiplication on the congruent numbers modulo n similarly to the usual arithmetic.

Lemma 3.29. Suppose we have $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$, then

1. $a \pm c \equiv b \pm d \pmod{n}$
2. $ac \equiv bd \pmod{n}$

Example 3.12. Let use see how the lemma works in practice.

1. $17 \times 26 \pmod{7} \equiv 3 \times 5 \equiv 15 \equiv 1 \pmod{7}$
2. $5 + 26 \pmod{7} \equiv 5 + 5 \equiv 10 \equiv 3 \pmod{7}$
3. $(-5) \times 13 \pmod{7} \equiv (7 - 5) \times 6 \equiv 2 \times 6 \equiv 12 \equiv 5 \pmod{7}$

It is clear that we can perform additions and multiplications modulo n as usual,

but what about division? Consider the following lemma.

Lemma 3.30. If $ca \equiv cb \pmod{n}$, $\gcd(c, n) = 1$, then $a \equiv b \pmod{n}$.

This lemma states that we can cancel out the same left and right parts respectively, but with a certain requirement: this part must be coprime to n . Consider the following example that shows why this requirement is mandatory.

Example 3.13. Let us try to simplify the following equation: $6 \equiv 2 \pmod{4}$. Suppose we divide both sides by 2, then we have the false statement $3 \equiv 1 \pmod{4}$. That being said, the requirement $\gcd(c, n) = 1$ is mandatory.

Remark. In particular, the example above shows why the division operation is fundamentally different from the regular real/rational numbers. We will see that, in fact, the arithmetic modulo n is not always what mathematicians call a **field**. You will see more details in [Section 7](#).

Now, in no particular order, we enumerate the key properties of the congruence relation modulo n , which require special attention. Note that all these properties require the rigorous proof, but, as mentioned earlier, we will not delve into the details of the proofs in this book.

Lemma 3.31 (Key Congruence Relation Properties).

1. For any $k \neq 0$ and $a \equiv b \pmod{n}$, we have $ak \equiv bk \pmod{nk}$
2. If $a \equiv b \pmod{n}$, then we can divide both sides by $\gcd(a, b, n)$:

$$a/d \equiv b/d \pmod{n/d} \text{ with } d = \gcd(a, b, n)$$
3. If $d \mid n$ and $a \equiv b \pmod{n}$, then $a \equiv b \pmod{d}$
4. If $a \equiv b \pmod{n_i}$ for $i \in [k]$, then $a \equiv b \pmod{\text{lcm}(n_0, \dots, n_{k-1})}$
5. If $a \equiv b \pmod{n}$, then $\gcd(a, n) = \gcd(b, n)$

Finally, we introduce the concept of congruence classes and residue rings, which is the key concept in Cryptographic applications. In the subsequent discussion, we use notation $n\mathbb{Z}$ to denote the set of all multiples of n .

Definition 3.32. The congruence class or residues of k modulo n is defined as a set $k + n\mathbb{Z} = \{k + nt \mid t \in \mathbb{Z}\}$, sometimes denoted as $[k]$ or $[k]_n$.

Example 3.14. For example, the congruence classes $[0]_2, [1]_2$ modulo 2 are respectively even and odd numbers:

$$\begin{aligned} [0]_2 &= \{0, \pm 2, \pm 4, \pm 6, \pm 8, \dots\}, \\ [1]_2 &= \{1, \pm 3, \pm 5, \pm 7, \pm 9, \dots\}. \end{aligned}$$

In turn, the congruence class $[3]_5$ modulo 5 is the set of numbers that give the remainder 3 when divided by 5:

$$[3]_5 = \{3, 3 \pm 5, 3 \pm 10, 3 \pm 15, \dots\}.$$

Definition 3.33. The **complete residue system modulo n** (or residue ring) is a set of integers, where every integer is congruent to a unique member of the set modulo n , usually denoted as $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$. In more formal literature, the set is often denoted as $\mathbb{Z}/n\mathbb{Z} = \{[0], [1], \dots, [n-1]\}$.

Example 3.15. $\mathbb{Z}_5$ is a complete residue system modulo 5 and consists of the following elements: $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$. Sometimes, the formal literature defines $\mathbb{Z}/5\mathbb{Z}$ to be the set of congruence classes modulo 5:

$$\mathbb{Z}/5\mathbb{Z} = \{\{5k : k \in \mathbb{Z}\}, \dots, \{5k + 4 : k \in \mathbb{Z}\}\}.$$

The aforementioned lemmas and definitions describe the properties of addition, subtraction, and multiplication. But what about division? To define the division (if such operation is valid at all), we need to consider the so-called *modular multiplicative inverse*, which is usually denoted as $a^{-1} \pmod{n}$, similarly to real/rational numbers.

Definition 3.34. A modular multiplicative inverse of an integer a modulo n is such an integer a^{-1} that satisfies $a \cdot a^{-1} \equiv a^{-1}a \equiv 1 \pmod{n}$.

Example 3.16. Let us find the modular multiplicative inverse of $a = 3$ modulo $n = 7$. We need to find such a^{-1} that satisfies $3 \cdot a^{-1} \equiv 1 \pmod{7}$. By simple brute force, we find that $3 \cdot 5 \equiv 1 \pmod{7}$, thus $3^{-1} = 5$.

Remark. The inverse (if exists) behaves similarly to the usual inverse over rationals/reals. For example, $(a^2)^{-1} = (a^{-1})^2$, which means, we can first find the inverse, then the squared value, and vice versa.

The question then is when does the number have the inverse? Consider the following theorem.

Theorem 3.35. The modular multiplicative inverse $a^{-1} \pmod{n}$ exists if and only if $\gcd(a, n) = 1$.

Based on Extended Euclidean [Algorithm 1](#) and [Definition 3.35](#), we build the algorithm that can verify an existence and find the inverse value.

Algorithm 2: Modular multiplicative inverse algorithm

Input : a, n **Output**: a^{-1}

```
1  $(d, u, v) \leftarrow \text{ExtendedEuclideanAlgorithm}(a, n)$  /* See Algorithm 1 */
2 if  $d \neq 1$  then
3   | return inverse does not exist.
4 end
5 return  $u$ 
```

Let us see the concrete implementation of the Extended Euclidean Algorithm and the Modular Multiplicative Inverse Algorithm in Python.

```
def extended_gcd(a: int, b: int) -> Tuple[int, int]:
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q, a, b = a // b, b, a % b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

def inverse(a: int, n: int) -> int:
    d, u, v = extended_gcd(a, n)
    if d != 1:
        raise ValueError('Inverse does not exist')
    return u

# Example usage
print(inverse(3, 7)) # Output: -2
print(inverse(12, 17)) # Output: -7
print(inverse(18, 24)) # Panics as gcd(18, 24) != 1
```

Let us verify manually that 12^{-1} modulo 17 was found correctly. Consider the following example.

Example 3.17. Let us find the modular multiplicative inverse of $a = 12$ modulo $n = 17$. First of all, we need to verify that the inverse exists, in other words whether $\text{gcd}(12, 17) = 1$ is true at all.

$$(1) \quad 17 = 1 \cdot 12 + 5$$

$$(2) \quad 12 = 2 \cdot 5 + 2$$

$$(3) \quad 5 = 2 \cdot 2 + 1$$

$$(4) \quad 2 = 1 \cdot 2 + 0$$

Thus, we have verified the inverse existence. In the previous example, we used the Extended Euclidean Algorithm 1 to find Bezout coefficients, but in this case, we will show another method. Let us iteratively go backwards from

equation (3) and substitute the result from the earlier step.

$$\begin{aligned} 1 &= 5 - 2 \cdot 2 = 5 - 2(12 - 2 \cdot 5) = 5 \cdot 5 - 2 \cdot 12 \\ &= 5(17 - 12 \cdot 1) - 2 \cdot 12 = 5 \cdot 17 - 7 \cdot 12. \end{aligned}$$

As you can see, we have found Bezout coefficients, $\gcd(12, 17) = 5 \cdot 17 - 7 \cdot 12$. Therefore, the inverse is -7 , which is the output of the algorithm.

Remark. Note that the complexity of this algorithm is the same as for **Algorithm 1**, which is $O(\log a \log n)$ bit operations. In real applications, typically $n > a$, so the complexity is simplified to $O(\log^2 n)$.

Definition 3.36. The **multiplicative group of integers** modulo n , denoted as $\mathbb{Z}_n^\times$, is a set of natural numbers that are coprime to n and less than n . In other words, $\mathbb{Z}_n^\times = \{a \in \mathbb{N} : \gcd(a, n) = 1\}$.

Example 3.18. For instance, $\mathbb{Z}_{11}^\times = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ since all naturals less than 11 are coprime to 11 (since 11 is prime). However, $\mathbb{Z}_{12}^\times = \{1, 5, 7, 11\}$, since integers 2, 3, 4, 6, 8, 9, 10 are not coprime to 12.

The $\mathbb{Z}_n^\times$ is a fundamental object in number theory and plays a crucial role in cryptography, forming the basis almost for every cryptographic primitive. The structure of the group $\mathbb{Z}_n^\times$ has a crucial meaning and has deep connections with algebraic structures: even the number of elements of $\mathbb{Z}_n^\times$ has the special name, which we define below.

Definition 3.37. Euler's Totient Function $\varphi(n)$ is the cardinality of the multiplication group of integers $\mathbb{Z}_n^\times$. In other words, $\varphi(n) = |\mathbb{Z}_n^\times|$.

Remark. The alternative Euler's totient function interpretation is following: $\varphi(n)$ counts all coprime integers with n in range $[1, n]$.

Remarkably, $\varphi(n)$ has a lot of curious properties, which we specify below.

Lemma 3.38 (Euler's totient function properties).

1. $\varphi(1) = 1$.
2. $\varphi(p) = p - 1$, where p is prime.
3. $\varphi(pq) = \varphi(p) \cdot \varphi(q)$, where p, q are primes.
4. $\varphi(p^\alpha) = p^\alpha - p^{\alpha-1}$, where p is prime.

With these properties, we can derive a general formula for Euler's totient function, knowing the prime factorization of n .

Corollary 3.39. For any number $n = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_t^{\alpha_t}$ general formula for Euler's totient function is:

$$\varphi(n) = \prod_{i=1}^t (p_i^{\alpha_i} - p_i^{\alpha_i-1}) = n \prod_{i=1}^t \left(1 - \frac{1}{p_i}\right).$$

Example 3.19.

- $\varphi(107) = 106$, because 107 is prime.
- $\varphi(123) = \varphi(3 \cdot 41) = \varphi(3)\varphi(41) = 2 \cdot 40 = 80$
- $\varphi(729) = \varphi(3^6) = 3^6 - 3^5 = 486$

3.6 Chinese Remainder Theorem

The Chinese Remainder Theorem is a fundamental result in number theory that provides an efficient method for solving congruence systems. It allows one to decompose a complex problem with large integers into several simpler problems with smaller modules. This decomposition reduces the required computational complexity, especially when working with large numbers, and is widely used in areas such as modular arithmetic, cryptography, and algorithmic number theory.

Theorem 3.40 (Chinese Remainder Theorem). Suppose there is the following system

$$\begin{cases} x \equiv x_1 \pmod{n_1} \\ x \equiv x_2 \pmod{n_2} \\ \dots \\ x \equiv x_t \pmod{n_t} \end{cases}$$

where the $n_1, \dots, n_t$ are pairwise coprimes and $x_1, x_2, \dots, x_t$ are fixed numbers. Then, this system has a unique solution modulo $N = n_1 n_2 \dots n_t$.

In fact, we can easily find such a solution. Let $N_i = N/n_i$. Since $n_1, \dots, n_t$ are pairwise coprimes, we have $\gcd(N_i, n_i) = 1$ for each i . By the Extended Euclidean Algorithm, there exists an integer a_i such that $a_i N_i \equiv 1 \pmod{n_i}$. Thus, the solution is given by

$$x_0 = a_1 N_1 x_1 + a_2 N_2 x_2 + \dots + a_t N_t x_t \pmod{N}.$$

This expression provides the unique solution modulo N , where $N = n_1 n_2 \dots n_t$. Besides, using this idea, we can write an efficient algorithm to compute x_0 iteratively.

Example 3.20. Let us solve the following system of congruences.

$$\begin{cases} x \equiv 1 \pmod{5} \\ x \equiv 2 \pmod{7} \\ x \equiv 3 \pmod{11} \end{cases}$$

First of all, we find $N = 5 \times 7 \times 11 = 385$, then we compute N_i .

- $N_1 = 7 \times 11 = 77$
- $N_2 = 5 \times 11 = 55$
- $N_3 = 5 \times 7 = 35$

To simplify the calculations, we give inverses for granted: $a_1 = 3, a_2 = 6, a_3 = 6$. The solution is then given by $x_0 \equiv 3 \times 77 \times 1 + 6 \times 55 \times 2 + 6 \times 35 \times 3 \equiv 1521 \equiv 81 \pmod{385}$.

However, as you can notice, the solution is not unique in general. We can increase the value with $N = 385$. Thus, the final solutions are $x = 81 + 385k \in \mathbb{Z}$.

3.7 Euler's Theorem

Now we are ready to introduce one of the fundamental results in Number Theory. Euler's Theorem was discovered by the mathematician Leonhard Euler as a generalization of Fermat's Little Theorem.

Why is it so important? Euler's Theorem is a key result in number theory and cryptographic applications, such as RSA encryption, and so on. Essentially, it helps in understanding the structure of the multiplicative group of integers modulo n and gives a foundation for other important results in number theory. Besides it provides the simplification of large exponents in modular arithmetic.

Theorem 3.41 (Euler's). $a^{\varphi(n)} \equiv 1 \pmod{n}$ for any $a \in \mathbb{Z}_n^\times$.

► To prove this fact, we prove the following auxiliary lemma.

Lemma 3.42. Suppose $a \in \mathbb{Z}_n^\times$. Denote by $a\mathbb{Z}_n^\times = \{ax : x \in \mathbb{Z}_n^\times\}$. Then, $\mathbb{Z}_n^\times = a\mathbb{Z}_n^\times$. In other words, elementwise multiplication by a permutes the elements of $\mathbb{Z}_n^\times$.

Proof. ► Our statement is equivalent to claiming that the function $f : \mathbb{Z}_n^\times \rightarrow a\mathbb{Z}_n^\times$ defined as $f(x) = ax \pmod{n}$ is a **bijection**. To show this, we need to prove the following three statements:

1. **Correctness** is obvious since ax is in $a\mathbb{Z}_n^\times$ if x is in $\mathbb{Z}_n^\times$.
2. **Injectivity:** We need to prove that if $f(x_1) = f(x_2)$, then $x_1 = x_2 \in \mathbb{Z}_n^\times$. From the function definition, we have $ax_1 \equiv ax_2 \pmod{n}$. By the following **Definition 3.31**, we can cancel a modulo n as a is coprime to n . Thus, $x_1 \equiv x_2 \pmod{n}$.
3. **Surjectivity:** We need to prove that every element $y \in a\mathbb{Z}_n^\times$ has at least one

pre-image $f^{-1}(y)$. Indeed, in this case we have $ax \equiv y \pmod{n}$. Then, set $x := a^{-1}y \pmod{n}$. This expression is well-defined as a is coprime to n and has an inverse.

Therefore, f is bijective and thus $\mathbb{Z}_n^\times = a\mathbb{Z}_n^\times$. ◀

So we know, $a\mathbb{Z}_n^\times = \mathbb{Z}_n^\times$, how do we proceed? Notice, from the set equality follows the equality of the products of all elements. Let us write this down:

$$\prod_{i=1}^{\varphi(n)} x_i \equiv \prod_{i=1}^{\varphi(n)} ax_i \pmod{n} \implies \prod_{i=1}^{\varphi(n)} x_i \equiv a^{\varphi(n)} \prod_{i=1}^{\varphi(n)} x_i \pmod{n}$$

Since all x_i are coprime to n by definition, we can cancel out the product of all x_i from both sides, leading to the conclusion of Euler's theorem. ◀

Example 3.21. Let us consider how Euler's Theorem works by computing seemingly scary expression $29^{202} \pmod{13}$. By means of **Definition 3.41** we have $29^{\varphi(n)} \equiv 1 \pmod{13}$, where $\varphi(13) = 12$. Using this, let us try to simplify the expression,

$$\begin{aligned} 29^{202} \pmod{13} &\equiv \\ &\equiv 29^{12 \times 16 + 10} \pmod{13} \\ &\equiv \cancel{(29^{12})^{16}} 29^{10} \pmod{13} \\ &\equiv 29^{10} \pmod{13} \\ &\equiv 3^{10} \pmod{13} \\ &\equiv 59049 \pmod{13} \\ &\equiv 3 \pmod{13}. \end{aligned}$$

In other words, we significantly reduced the computational complexity by using the Euler's theorem. Now, let us consider one essential corollary of Euler's theorem, which is used in finite field arithmetic.

Corollary 3.43. If p is a prime number and a is not divisible by p , then $a^{p-1} \equiv 1 \pmod{p}$.

This result is commonly known as Fermat's Little Theorem. However, to maintain generality, we state the theorem in a more general form below.

Theorem 3.44 (Fermat's Little Theorem.). Let p be a prime number. For any integer a , the following holds: $a^p \equiv a \pmod{p}$.

The proof of Fermat's Little Theorem is a direct consequence of Euler's theorem. We can consider Euler's Theorem as a generalization of Fermat's Little Theorem. Moreover, this fact allows to find the modular multiplicative inverse of an integer a modulo prime number faster than using the Extended Euclidean Algorithm.

Corollary 3.45. For any integer a and prime number p , the modular multiplicative inverse $a^{-1} \pmod{p}$ is $a^{p-2} \pmod{p}$.

Proof. Notice that $a \cdot a^{p-2} = a^{p-1} \equiv 1 \pmod{p}$ from Definition 3.43. Note that computing $a^{p-2} \pmod{p}$ requires less than $2 \log_2 p$ modular operations using the fast exponentiation algorithm. In Python, implementation using in-built pow function looks extremely simple:

```
def inverse(a: int, p: int) -> int:
    return pow(a, p-2, p)

print(inverse(3, 5))    # Output: 2
print(inverse(7, 39))   # Output: 7
print(inverse(8, 17))   # Output: 15
```

3.8 Exercises

Exercise 1. Which of the following statements is **true**?

- a) Prime factorization of 30 is $30 = 5 \cdot 6$.
- b) Number $2^{10} + 1$ is prime.
- c) Suppose $n \mid m$. Then m^2/n might not be an integer for some n, m .
- d) Any natural number n equals to the product of its divisors.
- e) Any prime number p equals to the product of its divisors.

Exercise 2. Which of the following statements is **false**?

- a) $\gcd(p, q) = 1$ for any two prime numbers p and q .
- b) $\text{lcm}(p, q) = pq$ for any two primes p and q .
- c) $\gcd(n, m) = \gcd(m, n)$ for any integers n, m .
- d) $\text{lcm}(n+1, n-1) = n^2 - 1$ for any integer n .

Exercise 3. What the inverse element $5^{-1} \pmod{11}$?

- a) 5
- b) 7
- c) 9
- d) 10

Exercise 4. What element is not contained in $\mathbb{Z}_{21}^\times$?

- a) 2
- b) 5
- c) 15
- d) 16

Exercise 5. What sum of all integers n satisfying $(n+1) \mid (n^2+1)$?

- a) 1
- b) 2
- c) 3
- d) 4

Exercise 6. In the quadratic polynomial $f(x) = (x-p)^2 - p(x-q) + x^2 - qx$, p and q are *distinct* prime numbers both less than 10. This polynomial can be rewritten as $f(x) = 2(x-r_1)(x-r_2)$ where r_1 and r_2 are the roots of f . If both r_1 and r_2 are prime numbers, what is their sum $r_1 + r_2$?

a) 7

b) 8

c) 10

d) 11

Exercises 7-9. Tau Function

Define function $\tau(n) := \{\# \text{ of positive divisors of } n\}$. For example, the number 12 has 6 divisors: 1, 2, 3, 4, 6, 12, so $\tau(12) = 6$. In fact, following **Definition 3.25** notation, if $n = \prod_{j=1}^r p_j^{\alpha_j}$, then the following holds: $\tau(n) = \prod_{j=1}^r (\alpha_j + 1)$. For example, since $12 = 2^2 \cdot 3^1$ we have $\alpha_1 = 2, \alpha_2 = 1$ and therefore $\tau(12) = (2 + 1) \cdot (1 + 1) = 6$.

Exercise 7. Suppose that p is an arbitrary prime. What is $\tau(p^{100})$?

a) 100

b) 101

c) 102

d) 99

Exercise 8. Suppose for some integer n we have $\tau(n) = 7$. What is $\tau(n^2)$?

a) 2

b) 6

c) 12

d) 13

Exercise 9*. What is the smallest integer m , greater than n^2 for some $n \in \mathbb{N}$, for which $\tau(m)$ is odd?

a) $n^2 + n$ b) $(n + 1)^2$ c) $n^2 + 1$ d) n^3

4 Introduction to Abstract Algebra

In the previous section, we introduced the fundamentals of Number Theory. Now, we will move on to abstract algebra, which forms the core of modern cryptography. But you might wonder: why should we study abstract algebra?

Consider the set $\mathbb{Z}_n$. Naturally, we want to define certain arithmetical operations over $\mathbb{Z}_n$. For example, having $3, 6 \in \mathbb{Z}_9$ I would like to know what it means to add them. Naturally, one would say that $3 + 6 = 0$ in $\mathbb{Z}_9$ since $3 + 6$ over $\mathbb{Z}$ gives 9, which is congruent to 0 modulo 9. However, while such a structure is straightforward for such a set, we can define operations over much more difficult sets such as elliptic curve points, which are fundamental blocks for many Cryptography protocols.

In a way, abstract algebra gives *interfaces* to objects, just like in programming languages, where interfaces specify certain common patterns observed in different instances of this interface.

Let us explore this concept on the previously defined set $\mathbb{Z}_n$. We present the following lemma to show the properties of modular arithmetic.

Lemma 4.1. Suppose $a, b, c \in \mathbb{Z}_n$, then we have the following modular addition properties,

1. *Addition associative:* $a + (b + c) \equiv (a + b) + c \pmod{n}$. The order in which we compute pairwise sums does not matter.
2. *Addition identity:* $a + 0 \equiv a \pmod{n}$. We always have an element which behaves like “zero” in the addition operation (or like “one” when multiplying).
3. *Additive inverse:* $a + (-a) \equiv 1 \pmod{n}$. For any element a we always have an element $-a$ which “zeroes out” an element a .
4. *Addition commutative:* $a + b \equiv b + a \pmod{n}$. The order of elements in the addition operation does not matter.

Remark. All these properties also hold for multiplication operation, but with one drastic exception: the existence of the inverse element. So in other words, the multiplicative inverse property does not hold in $\mathbb{Z}_{16}$.

In abstract algebra, we explore algebraic structures defined by abstract operations. These structures possess specific properties that help us understand the behavior of their elements. By examining these properties, we can derive general principles and insights applicable to any mathematical systems.

4.1 Groups

Let us jump straight to the most fundamental algebraic structure – the group. Based on the introduction above, based on the set $\mathbb{G}$, we want some operation $\oplus$ to have certain properties, which are specified in the following definition.

Definition 4.2. A **group**, denoted by $(\mathbb{G}, \oplus)$, is a set with a binary operation $\oplus$, obeying the following rules:

1. **Closure:** Binary operations always output an element from $\mathbb{G}$, that is $\forall a, b \in \mathbb{G} : a \oplus b \in \mathbb{G}$.
2. **Associativity:** $\forall a, b, c \in \mathbb{G} : (a \oplus b) \oplus c = a \oplus (b \oplus c)$.
3. **Identity element:** There exists a so-called identity element $e \in \mathbb{G}$ such that $\forall a \in \mathbb{G} : e \oplus a = a \oplus e = a$.
4. **Inverse element:** $\forall a \in \mathbb{G} \exists b \in \mathbb{G} : a \oplus b = b \oplus a = e$. We commonly denote the inverse element by $(\ominus a)$ or a^{-1} .

Quite confusing at first glance, right? The best way to grasp this concept is to consider a couple of examples.

Example 4.1. A group of integers with the regular addition $(\mathbb{Z}, +)$ (also called the *additive group of integers*) is a group. Indeed, an identity element is $e_{\mathbb{Z}} = 0$, associativity obviously holds, and an inverse for each element $a \in \mathbb{Z}$ is $(\ominus a) := -a \in \mathbb{Z}$.

Remark. We use the term **additive group** when we mean that the binary operation is addition $+$, while **multiplicative group** means that we are multiplying two numbers via $\times^a$.

^aIn this section, regard $\cdot$ and $\times$ as the same operation of multiplication.

Example 4.2. The multiplicative group of positive real numbers $(\mathbb{R}_{>0}, \times)$ is a group for similar reasons. An identity element is $e_{\mathbb{R}_{>0}} = 1$, while the inverse for $a \in \mathbb{R}_{>0}$ is defined as $\frac{1}{a}$.

Example 4.3. The additive set of natural numbers $(\mathbb{N}, +)$ is not a group. Although operation of addition is closed, there is no identity element nor inverse element for, say, 2 or 10.

Exercise. Prove that for any group $\mathbb{G} = \{g, e\}$ with two elements and identity element e it holds that $g^2 = e$.

One might ask a reasonable question: suppose you pick $a, b \in \mathbb{G}$. Is $a \oplus b$ the same as $b \oplus a$? Unfortunately, for some groups, this is not true.

For this reason, it makes sense to give a special name to a group in which the operation is commutative (meaning, we can swap the elements).

Definition 4.3. A group $(\mathbb{G}, \oplus)$ is called **abelian** if $\forall a, b \in \mathbb{G} : a \oplus b = b \oplus a$.

Example 4.4. The additive group of integers $(\mathbb{Z}, +)$ is an abelian group. Indeed, $a + b = b + a$ for any $a, b \in \mathbb{Z}$.

Example 4.5. The set of 2×2 matrices with real entries and determinant 1 (denoted by $\text{SL}(2, \mathbb{R})$) is a group with respect to matrix multiplication. However, this group is not abelian! Take

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Then, it is easy to verify that

$$AB = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \quad BA = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix},$$

so clearly $AB \neq BA$ – the elements of $\text{SL}(2, \mathbb{R})$ do not commute.

Remark. Further, we will write ab instead of $a \times b$ and a^{-1} instead of $\ominus a$ for the sake of simplicity (and because it is more common in the literature). As mentioned before, it is usually called the *multiplicative notation*.

Finally, for cryptography it is important to know the number of elements in a group. This number is called the *order* of the group.

Definition 4.4. The **order** of a finite group $\mathbb{G}$ is the number of elements in the group. We denote the order of a group as $|\mathbb{G}|$.

Example 4.6. Integers modulo 13, denoted by $\mathbb{Z}_{13}$, is a group with respect to addition modulo 13 (e.g., $5 + 12 = 4$ in $\mathbb{Z}_{13}$). The order of this group is 13.

Despite the aforementioned definitions, many things are not generally obvious. For example, one might ask whether the identity element is unique. Or, whether the inverse element is unique for each group element. For that reason, we formulate the following lemma.

Lemma 4.5. Suppose $\mathbb{G}$ is a group. Then, the following statements hold:

1. The identity element is unique.
2. The inverse element is unique for each element: $\forall a \in \mathbb{G} \exists! a^{-1} \in \mathbb{G} : aa^{-1} = a^{-1}a = e$.
3. For all $a, b \in \mathbb{G}$ there is a unique $x \in \mathbb{G}$ such that $ax = b$.
4. If $ab = ac$ then $b = c$. Similarly, if $xy = zy$ then $x = z$.

Since this guide is not a textbook on abstract algebra, we will not prove all the statements. However, we will prove the first and second one to show the nature

of the proofs in abstract algebra.

Proof (first statement). ► Suppose we have two identity elements $e_1, e_2 \in \mathbb{G}$. Consider their product $e_1 e_2$. By the identity element definition, we know that $e_1 e_2 = e_1$ and $e_1 e_2 = e_2$. It follows immediately that $e_1 = e_2$. ◀

Proof (second statement). ► Take some element $g \in \mathbb{G}$ and suppose $a, b \in \mathbb{G}$ are both inverses of g . By the inverse element definition, we have

$$ag = ga = e, \quad bg = gb = e.$$

Now, notice the following:

$$a = ae = a(gb) = (ag)b = eb = b$$

Thus, we have proven that $a = b$. ◀

Exercise. Prove the third and fourth statements.

4.2 Subgroups

When we are finally comfortable with the concept of a group, we can move on to the concept of a *subgroup*.

Suppose we have a group $(\mathbb{G}, \oplus)$. Suppose one takes the subset $\mathbb{H} \subset \mathbb{G}$. Of course, since all elements in $\mathbb{H}$ are still elements in $\mathbb{G}$, we can conduct operations between them via $\oplus$. The natural question to ask is whether $\mathbb{H}$ is a group itself. We say that $\mathbb{H}$ is a **subgroup** of $\mathbb{G}$ if this is indeed the case.

Definition 4.6. A subset $\mathbb{H} \subset \mathbb{G}$ is called a **subgroup** of $\mathbb{G}$ if $\mathbb{H}$ is a group with respect to the same operation $\oplus$. We denote this as $\mathbb{H} \leq \mathbb{G}$.

Example 4.7. Of course, not every subset of $\mathbb{G}$ is a subgroup. Take $(\mathbb{Z}, +)$. If we cut, say, 3 out of $\mathbb{Z}$ (so we get $\mathbb{H} = \mathbb{Z} \setminus \{3\}$), then $\mathbb{H}$ is not a subgroup of $\mathbb{Z}$ since an element -3 does not have an inverse in $\mathbb{H}$. Moreover, it is not closed: take $1, 2 \in \mathbb{H}$. In this case, $1 + 2 = 3 \notin \mathbb{H}$.

Example 4.8. Now, let us define some valid subgroup of $\mathbb{Z}$. Take $\mathbb{H} = 3\mathbb{Z}$. This is a subgroup of $\mathbb{Z}$, since it is closed under addition, has an identity element 0, and has an inverse for each element $3k$ (namely, $-3k$). That being said, $\mathbb{H} = 3\mathbb{Z} \leq \mathbb{Z}$.

4.3 Cyclic Groups

Cyclic groups are the most interesting groups in cryptography because of their simple yet powerful structure. In a cyclic group, every element can be created by repeatedly applying the group operation to one element, called the *generator*. This makes cyclic groups useful for many cryptographic algorithms, allowing for secure and efficient operations.

However, before diving deeper, we need to know how to multiply (add) elements multiple times. Traditionally, cyclic groups use the multiplicative notation, so we stick to it. Suppose we have a group $(\mathbb{G}, \times)$ and $g \in \mathbb{G}$. Then, for positive n we define g^n to be the result of multiplying g by itself n times. For $n = 0$ we assume $g^0 = 1$ with $1 \in \mathbb{G}$ being the identity element. Finally, for negative n we define $g^{-n} = (g^{-1})^n$. Now, let us consider the first definition.

Definition 4.7. Let $\mathbb{G}$ be a group and $g \in \mathbb{G}$, then **cyclic subgroup** generated by g is $\langle g \rangle = \{g^n : n \in \mathbb{Z}\} = \{\dots, g^{-3}, g^{-2}, g^{-1}, e, g, g^2, g^3, \dots\}$.

Example 4.9. Let $\mathbb{Z}_{12}$ be the group of integers modulo 12. Consider element $2 \in \mathbb{Z}_{12}$ then the group generated by 2 is

$$\langle 2 \rangle = \{2, 4, 6, 8, 10, 0\}$$

Note that the set $\langle g \rangle = \{g^n : n \in \mathbb{Z}\}$ is indeed a subgroup of $\mathbb{G}$. Indeed, it is closed under multiplication, has an identity element $e \in \langle g \rangle$ (corresponding to g^0), and has an inverse for each element (since for each $g^n \in \langle g \rangle$ the inverse is defined as $g^{-n} \in \langle g \rangle$).

For finite groups $\mathbb{G}$ we naturally expect $\langle g \rangle$ to have a finite size. Such size is what we call the *order* of the element g .

Definition 4.8. Let $\mathbb{G}$ be a group and $g \in \mathbb{G}$. The minimal natural number n for which $g^n = 1$ is called the **order** of element g and denote it by $\text{ord}(g)$. If such integer does not exist, we assume that $\text{ord}(g) = \infty$.

Example 4.10. Suppose we have a group $(\mathbb{Z}_9, +)$. Let us find the order of element $a = 3$. It is easy to notice that $3 \cdot 3 \equiv 0 \pmod{9}$, thus $\text{ord}(3) = 3$.

Example 4.11. Let $(\mathbb{Z}, +)$ be a group. Any non-zero integer $a \in \mathbb{Z}$ has an infinite order (while 0 has an order of 1).

Finally, we can define what it means for a group to be cyclic.

Definition 4.9. We say that a group $\mathbb{G}$ is **cyclic** if there exists an element $g \in \mathbb{G}$ such that $\mathbb{G}$ is generated by g , that is, $\mathbb{G} = \langle g \rangle$. In this case, g is a generator of $\mathbb{G}$.

Example 4.12. The group of integers $(\mathbb{Z}, +)$ is an infinite cyclic group. Indeed, it is generated by 1.

Now it is time to discuss some interesting properties of cyclic groups and their subgroups. We start with the following theorem.

Theorem 4.10. Every cyclic group is abelian.

Reasoning. Suppose the generator of the group is g . Then, for any $a, b \in \mathbb{G}$ we have

$$ab = g^m g^n = g^{m+n} = g^{n+m} = g^n g^m = ba.$$

Cryptographic protocols typically rely on the properties of cyclic groups, so it turns out that they usually work with abelian groups by default. Common cryptographic groups, like those based on elliptic curves or modular arithmetic, are cyclic and, therefore, abelian.

Lemma 4.11. Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of order n . Then $g^m = e$ holds if and only if $m \mid n$.

Theorem 4.12. Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of order n . If $h = g^m$, then the order of h is $n/\gcd(m, n)$.

Consider this theorem as a generalization of [Definition 4.11](#). By using the above theorem, we can derive a criterion which the generators of $\mathbb{Z}_n$ must adhere to.

Corollary 4.13. The generators of group $\mathbb{Z}_n$ are the integers from $\mathbb{Z}_n^\times$.

It is also easy to see that the order of any group generator is the same as the group it generates.

Example 4.13. Let us examine how many elements can generate $(\mathbb{Z}_{12}, +)$. By theorem above each element of $\mathbb{Z}_{12}^\times = \{1, 5, 7, 11\}$ generates $\mathbb{Z}_{12}$. For example, let us consider $\langle 5 \rangle$:

• $5^1 = 0 + 5 = 5$	• $5^5 = 8 + 5 = 1$	• $5^9 = 4 + 5 = 9$
• $5^2 = 5 + 5 = 10$	• $5^6 = 6 + 5 = 6$	• $5^{10} = 2 + 5 = 2$
• $5^3 = 10 + 5 = 3$	• $5^7 = 11 + 5 = 11$	• $5^{11} = 7 + 5 = 7$
• $5^4 = 3 + 5 = 8$	• $5^8 = 4 + 5 = 4$	• $5^{12} = 7 + 5 = 0$

As we see, all elements of $\mathbb{G}$ are generated by the element 5. However, does it hold for any other integer from $\mathbb{Z}_n \setminus \mathbb{Z}_n^\times$? Consider $\langle 3 \rangle$ as an example:

• $3^1 = 3$	• $3^2 = 6$	• $3^3 = 9$	• $3^4 = 0$
-------------	-------------	-------------	-------------

While this is a valid subgroup, it does not generate the whole group $(\mathbb{Z}_{12}, +)$.

Based on the previous example, you may have a reasonable question about the types of subgroups of cyclic groups. For example, can we find some non-cyclic subgroup of $\mathbb{Z}_{12}$? Turns out that this is not possible due to the subsequent

theorem.

Theorem 4.14. Every subgroup of cyclic group is cyclic.

4.4 Cosets and Lagrange's Theorem*

Lagrange's Theorem is one of the most important results in finite group theory, considering all the consequences. Central to understanding Lagrange's Theorem is the notion of a *coset*. While this concept is essential for Cryptography, feel free to skip this section if feel uncomfortable with the material.

Definition 4.15. Let $\mathbb{G}$ be a group and $\mathbb{H}$ a subgroup of $\mathbb{G}$. **Left coset** with representative $g \in \mathbb{G}$ is defined as $g\mathbb{H} = \{gh : h \in \mathbb{H}\}$. Similarly, the **right coset** with representative $g \in \mathbb{G}$ is defined as $\mathbb{H}g = \{hg : h \in \mathbb{H}\}$.

Example 4.14. Suppose $\mathbb{H}$ be subgroup of $\mathbb{Z}_8$ with elements $\{0, 4\}$. Let us find all the cosets

$$0 + \mathbb{H} = 4 + \mathbb{H} = \{0, 4\}$$

$$1 + \mathbb{H} = 5 + \mathbb{H} = \{1, 5\}$$

$$2 + \mathbb{H} = 6 + \mathbb{H} = \{0, 6\}$$

$$3 + \mathbb{H} = 7 + \mathbb{H} = \{3, 7\}$$

Note that in a commutative group, left and right cosets are always identical, so for instance $4 + \mathbb{H} = \mathbb{H} + 4$.

Lemma 4.16. For any group element $g \in \mathbb{G}$ it holds that $|g\mathbb{H}| = |\mathbb{H}|$.

Definition 4.17. Let $\mathbb{G}$ be a group and $\mathbb{H}$ be a subgroup of $\mathbb{G}$. The **index** of subgroup $\mathbb{H}$ in group $\mathbb{G}$ is the number of left cosets of $\mathbb{H}$ in $\mathbb{G}$. We will denote this index by $[\mathbb{G} : \mathbb{H}]$.

Example 4.15. Let $\mathbb{G} = \mathbb{Z}_8$ and $\mathbb{H} = 4\mathbb{Z}_8 = \{0, 4\}$. Then, $[\mathbb{G} : \mathbb{H}] = 4$. Moreover, for every $m \in \mathbb{N}$ it holds that $[\mathbb{Z} : m\mathbb{Z}] = m$.

Finally, we are ready for Lagrange's Theorem. This is a fundamental result in the group theory, crucial for understanding the structure of finite groups.

Theorem 4.18 (Lagrange). If $\mathbb{H}$ is a subgroup of an any finite group $\mathbb{G}$, then $|\mathbb{G}| = [\mathbb{G} : \mathbb{H}] |\mathbb{H}|$.

Proof. ► The group $\mathbb{G}$ is partitioned into $[\mathbb{G} : \mathbb{H}]$ distinct left cosets. Each

left coset has $|\mathbb{H}|$ elements, therefore $|\mathbb{G}| = [\mathbb{G} : \mathbb{H}] |\mathbb{H}|$. ◀.

Corollary 4.19. Suppose $\mathbb{H}$ is subgroup of a finite group $\mathbb{G}$, then $|\mathbb{H}| \mid |\mathbb{G}|$.

Proof. ▶ By Lagrange's theorem, the order of the group $\mathbb{G}$ decomposes as the product of the index $[\mathbb{G} : \mathbb{H}]$ and the order of the subgroup $\mathbb{H}$. Therefore, by the definition of divisibility (see Definition 3.1), it follows directly that $|\mathbb{H}|$ divides $|\mathbb{G}|$. ◀.

Corollary 4.20. Let $\mathbb{G}$ be a finite group and $g \in \mathbb{G}$, then $\text{ord}(g) \mid |\mathbb{G}|$. In other words, the order of g must divide the numbers of elements in $\mathbb{G}$.

Additionally, consider the following fact which seems to be obvious but required aforementioned theorems to be easily proven.

Corollary 4.21. Let $\mathbb{G}$ be a group of order n and $g \in \mathbb{G}$. Then, $g^n = e$.

Example 4.16. Let us take the multiplicative group $\mathbb{Z}_{17}^\times$. As we already know, the order of this group is $\varphi(17) = 16$. So, for any element $x \in \mathbb{Z}_{17}^\times$ it holds that $x^{16} = 1$. Note that this fact is also a direct consequence of the Euler's theorem: $x^{\varphi(n)} \equiv 1 \pmod{n}$.

Based on the example above, one might notice that Definition 4.21 serves as a generalization of Euler's Theorem for arbitrary finite group.

4.5 Isomorphisms and Endomorphisms

Finally, we will define the concept of isomorphisms and endomorphisms. These are important concepts in the world of cryptography, since they allow us to compare different groups. Namely, suppose we have two groups $(\mathbb{G}, \oplus)$ and $(\mathbb{H}, \odot)$. Is there any way to state that these two groups are the same? The answer is yes, and this is done via isomorphisms.

Definition 4.22. A function $\varphi : \mathbb{G} \rightarrow \mathbb{H}$ is called an **homomorphism** if it is a function that preserves the group operation, that is,

$$\forall a, b \in \mathbb{G} : \varphi(a \oplus b) = \varphi(a) \odot \varphi(b).$$

Definition 4.23. An **isomorphism** is a bijective homomorphism.

Definition 4.24. If there exists an isomorphism between two groups $\mathbb{G}$ and $\mathbb{H}$, we say that these groups are isomorphic and write $\mathbb{G} \cong \mathbb{H}$.

Example 4.17. Consider the group of integers $(\mathbb{Z}, +)$ and the group of integers modulo 12 $(\mathbb{Z}_{12}, +)$. The function $\varphi : \mathbb{Z} \rightarrow \mathbb{Z}_{12}$ defined as $\varphi(x) = x \bmod 12$ is a homomorphism. Indeed:

$$\varphi(a + b) = (a + b) \bmod 12 = (a \bmod 12) + (b \bmod 12) = \varphi(a) + \varphi(b).$$

However, this function is not an isomorphism, since it is not bijective. For example, $\varphi(0) = \varphi(12) = 0$.

Example 4.18. Additive group of reals $(\mathbb{R}, +)$ and the multiplicative group of positive reals $(\mathbb{R}_{>0}, \times)$ are isomorphic. The function $\varphi : \mathbb{R} \rightarrow \mathbb{R}_{>0}$ defined as $\varphi(x) = e^x$ is an isomorphism. Indeed:

$$\varphi(a + b) = e^{a+b} = e^a \cdot e^b = \varphi(a) \cdot \varphi(b).$$

Thus, φ is a homomorphism. It is also injective since $e^x = e^y \implies x = y$. Finally, it is obviously onto. This means $(\mathbb{R}, +) \cong (\mathbb{R}_{>0}, \times)$.

Example 4.19. All groups of order 2 are isomorphic to $\mathbb{Z}_2$. Indeed, let $\mathbb{G} = \{g, e\}$ – any group of order 2, and define $\varphi : \mathbb{Z}_2 \rightarrow \mathbb{G}$ as $\varphi(0) = e$ and $\varphi(1) = g$. This is an isomorphism.

A generalization of the above example is the following interesting theorem.

Theorem 4.25. Suppose $\mathbb{G} = \langle g \rangle$ is a finite cyclic group and $|G| = n \in \mathbb{N}$. Then, $\mathbb{G} \cong \mathbb{Z}_n$.

Idea of the proof. Define a function $\varphi : \mathbb{Z}_n \rightarrow \mathbb{G}$ as $m \mapsto g^m$. One can prove that this is an isomorphism.

Here, it is quite evident that isomorphism tells us that the groups have the same structure. Moreover, it is correct to say that if $\mathbb{G} \equiv \mathbb{H}$, then $\mathbb{G}$ and $\mathbb{H}$ are *equivalent* since $\cong$ is an equivalence relation.

Exercise (*). Prove that $\cong$ is an equivalence relation.

Finally, we will define the concept of an endomorphism and automorphism to finish the section.

Definition 4.26. An **endomorphism** is a function φ which maps set X to itself ($\varphi : X \rightarrow X$).

Definition 4.27. An **automorphism** is an isomorphic endomorphism.

Example 4.20. Given a group $\mathbb{G}$, fixate $a \in \mathbb{G}$. The map $\varphi : x \mapsto axa^{-1}$ is an automorphism.

Last two definitions are especially frequently used in Elliptic Curves theory.

4.6 Fields

Although typically one introduces rings before fields, we believe that for the basic understanding, it is better to start with fields.

Notice that when dealing with groups, we had a single operation $\oplus$, which, depending on the context, is either interpreted as addition or multiplication. However, fields allow to extend this concept a little bit further by introducing a new operation, say, $\odot$, which, combined with $\oplus$, allows us to perform the basic arithmetic.

This is very similar to the real or rational numbers, for example. We can add, subtract, multiply, and divide them. This is exactly what fields are about, but in a more abstract way. That being said, let us see the definition.

Definition 4.28. A **field** is a set $\mathbb{F}$ with two operations $\oplus$ and $\odot$ such that:

1. $(\mathbb{F}, \oplus)$ is an abelian group with identity $e_{\oplus}$.
2. $(\mathbb{F} \setminus \{e_{\oplus}\}, \odot)$ is an abelian group.
3. The **distributive law** holds:

$$\forall a, b, c \in \mathbb{F} : a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c).$$

What this definition states is that we can perform the following operations:

1. Addition: $a \oplus b$, inherited from group structure $(\mathbb{F}, \oplus)$.
2. Subtraction: $a \oplus (\ominus b)$, inherited from group structure $(\mathbb{F}, \oplus)$.
3. Multiplication: $a \odot b$, inherited from group structure $(\mathbb{F} \setminus \{e_{\oplus}\}, \odot)$.
4. Division: $a \odot b^{-1}$, except for $b = 0$, inherited from group structure $(\mathbb{F} \setminus \{e_{\oplus}\}, \odot)$.

Example 4.21. The set of real numbers $(\mathbb{R}, +, \times)$ is obviously a field.

Example 4.22. The set of complex numbers $(\mathbb{C}, +, \times)$ is also a field. Indeed, let us see how we can perform operations. Suppose we are given $z = a_0 + a_1 i$ and $w = b_0 + b_1 i$ with $i^2 + 1 = 0$. In this case:

1. Addition: $z + w = (a_0 + b_0) + (a_1 + b_1)i$.
2. Subtraction: $z - w = (a_0 - b_0) + (a_1 - b_1)i$.
3. Multiplication: $z \cdot w = (a_0 b_0 - a_1 b_1) + (a_0 b_1 + a_1 b_0)i$.
4. Division: $z/w = \frac{a_0 b_0 + a_1 b_1}{b_0^2 + b_1^2} + \frac{a_1 b_0 - a_0 b_1}{b_0^2 + b_1^2} i$.

Interestingly though, it is very difficult to come up with some more complicated, non-trivial examples. For that reason, we will simply move to the most central field used in cryptography – finite fields.

4.7 Finite Fields

Notice that $(\mathbb{Z}, +, \times)$ does not form a field since division is not closed. Similarly, the tuple $(\mathbb{Z}_n, +, \times)$ also does not always form a field since the multiplicative inverses are defined only for coprime numbers. This motivates us to consider the set $\mathbb{Z}_p$ with prime p in which every integer has a well-defined inverse modulo p . Such fields are called *prime fields*.

Definition 4.29. The **Prime field** of order p is a set $\{0, 1, \dots, p-2, p-1\}$, in which operations are defined “modulo p ” (see details above). Typically, finite fields are denoted as $\mathbb{F}_p$ or $\text{GF}(p)$.

Finite fields is the core object in cryptography. Instead of real numbers or pure integers, we will almost always use finite fields.

Remark. In many cases, one might encounter both $\mathbb{F}_p$ and $\mathbb{Z}_p$ notations. The difference is the following: when one refers to $\mathbb{Z}_p$, it is typically assumed that the operations are performed in the ring^a of integers modulo p (meaning, we need only addition, subtraction, and multiplication in the protocol), while division is of little interest. When one refers to $\mathbb{F}_p$, it is typically assumed that we need full arithmetic (including division) for the protocol.

^aWe have not defined as of now what ring is, but, roughly speaking, this is a field without multiplicative inverses (such as $\mathbb{Z}_n$ for composite n)

Example 4.23. Consider $9, 14 \in \mathbb{F}_{17}$. Some examples of calculations:

1. $9 + 14 = 6$.
2. $9 - 14 = 12$.
3. $9 \times 14 = 7$.
4. $14^{-1} = 11$ since $14 \cdot 11 = 154 \equiv 1 \pmod{17}$.

4.8 Some Fun: Group Implementation in Rust

In programming, we can think of a group as an interface, having a single binary operation defined, that obeys the rules of closure, associativity, identity element, and inverse element.

For that reason, we might even code a group in Rust! We will also write a simple test to check whether the group is valid and whether the group is abelian.

Trait for Group. First, we define a trait for a group. We will define a group as a trait with the following methods:

```
/// Trait that represents a group.
pub trait Group: Sized {
    /// Checks whether the two elements are equal.
    fn eq(&self, other: &Self) -> bool;
```

```

    /// Returns the identity element of the group.
    fn identity() -> Self;
    /// Adds two elements of the group.
    fn add(&self, a: &Self) -> Self;
    /// Returns the negative of the element.
    fn negate(&self) -> Self;
    /// Subtracts two elements of the group.
    fn sub(&self, a: &Self) -> Self {
        self.add(&a.negate())
    }
}

```

Checking group validity. Now observe the following: we get closure for free, since the compiler will check whether the return type of the operation is the same as the type of the group. However, there is no guarantee that associativity holds, and our identity element is at all valid. For that reason, we need to somehow additionally check the validity of implementation.

We propose to do the following: we will randomly sample three elements from the group $a, b, c \leftarrow \mathbb{G}$ and check our three properties:

1. $a \oplus (b \oplus c) \stackrel{?}{=} (a \oplus b) \oplus c.$
2. $a \oplus e \stackrel{?}{=} e \oplus a \stackrel{?}{=} a.$
3. $a \oplus (\ominus a) \stackrel{?}{=} (\ominus a) \oplus a \stackrel{?}{=} e.$

Additionally, if we want to verify whether the group is abelian, we can check whether $a \oplus b \stackrel{?}{=} b \oplus a.$

For that reason, for the check, we require the group to be samplable (i.e. we can randomly sample elements from the group):

```

/// Trait for sampling a random element from a group.
pub trait Samplable {
    /// Returns a random element from the group.
    fn sample() -> Self;
}

```

And now, our test looks as follows:

```

/// Number of tests to check the group properties.
const TESTS_NUMBER: usize = 100;

/// Asserts that the given group G is valid.
/// A group is valid if the following properties hold:
/// 1. Associativity: (a + b) + c = a + (b + c)
/// 2. Identity: a + e = a = e + a
/// 3. Inverse: a + (-a) = e = (-a) + a
pub fn assert_group_valid<G>()
where
    G: Group + Samplable,

```

```

{
  for _ in 0..TESTS_NUMBER {
    // Take random three elements
    let a = G::sample();
    let b = G::sample();
    let c = G::sample();

    // Check whether associativity holds
    let ab_c = a.add(&b).add(&c);
    let a_bc = a.add(&b.add(&c));
    let associativity_holds = ab_c.eq(&a_bc);
    assert!(associativity_holds, "Associativity does
    ↪ not hold for the given group");

    // Check whether identity element is valid
    let e = G::identity();
    let ae = a.add(&e);
    let ea = e.add(&a);
    let identity_holds = ae.eq(&a) && ea.eq(&a);
    assert!(identity_holds, "Identity element does
    ↪ not hold for the given group");

    // Check whether inverse element is valid
    let a_neg = a.negate();
    let a_neg_add_a = a_neg.add(&a);
    let a_add_a_neg = a.add(&a_neg);
    let inverse_holds = a_neg_add_a.eq(&e) &&
    ↪ a_add_a_neg.eq(&e);
    assert!(inverse_holds, "Inverse element does not
    ↪ hold for the given group");
  }
}

/// Asserts that the given group G is abelian.
/// A group is an abelian group if the following
  ↪ property holds:
///  $a + b = b + a$  for all  $a, b$  in  $G$  (commutativity)
pub fn assert_group_abelian<G>()
where
  G: Group + Samplable,
{
  for _ in 0..TESTS_NUMBER {
    assert_group_valid::<G>();

    // Take two random elements
    let a = G::sample();
    let b = G::sample();

```

```

        // Check whether commutativity holds
        let ab = a.add(&b);
        let ba = b.add(&a);
        assert!(ab.eq(&ba), "Commutativity does not hold
        ↪ for the given group");
    }
}

```

Testing the group ($\mathbb{Z}, +$). And now, we can define a group for integers and check whether it is valid and abelian:

```

use crate::group::{Group, Samplable};
use rand::Rng;

/// Implementing group for Rotation3<f32>
impl Group for i64 {
    fn eq(&self, other: &Self) -> bool {
        self == other
    }

    fn identity() -> Self {
        0i64
    }

    fn add(&self, a: &Self) -> Self {
        self + a
    }

    fn negate(&self) -> Self {
        -self
    }
}

impl Samplable for i64 {
    fn sample() -> Self {
        let mut gen = rand::thread_rng();

        // To prevent overflow, we choose a smaller
        ↪ range for i64
        let min = i64::MIN / 3;
        let max = i64::MAX / 3;
        gen.gen_range(min..max)
    }
}

```

Just a small note: since we cannot generate infinite integers, we restrict the range of integers to prevent overflow. So, for the sake of simplicity, we divide the range of integers by 3, in which overflow never occurs.

And now, the moment of truth! Let us define some tests and run them:

```
#[cfg(test)]
mod tests {
    use super::*;
    use group::*;

    #[test]
    fn test_integers_are_group() {
        assert_group_valid::<i64>()
    }

    #[test]
    fn test_integers_are_abelian() {
        assert_group_abelian::<i64>();
    }
}
```

Both tests pass! Now let us consider something a bit trickier.

Testing the group $SO(3)$. We can define a group for 3×3 rotation matrices. Of course, composition of two rotation is not commutative, so we expect the abelian test to fail. However, the group is still valid! For example, there is an identity rotation matrix E , and for each rotation matrix $A \in SO(3)$, there exists a rotation matrix $A^{-1} \in SO(3)$ such that $AA^{-1} = A^{-1}A = E$. Finally, the associativity holds as well.

We will use the `nalgebra` library for this purpose, which contains the implementation of rotation matrices. So our implementation can look as follows:

```
/// A threshold below which two floating point numbers
    ↪ are considered equal.
const EPSILON: f32 = 1e-6;

/// Implementing group for Rotation3<f32>
impl Group for Rotation3<f32> {
    fn eq(&self, other: &Self) -> bool {
        // Checking whether the norm of a difference is
        ↪ small
        let difference = self.matrix() - other.matrix();
        difference.norm_squared() < EPSILON
    }

    fn identity() -> Self {
        Rotation3::identity()
    }

    fn add(&self, a: &Self) -> Self {
        self * a
    }
}
```

```

    fn negate(&self) -> Self {
        self.inverse()
    }
}

impl Samplable for Rotation3<f32> {
    fn sample() -> Self {
        let mut gen = rand::thread_rng();

        // Pick three random angles
        let roll = gen.gen_range(0.0..1.0);
        let pitch = gen.gen_range(0.0..1.0);
        let yaw = gen.gen_range(0.0..1.0);

        Rotation3::from_euler_angles(roll, pitch, yaw)
    }
}

```

Here, there are two tricky moments:

1. We cannot compare floating point numbers directly, since they might differ by a small amount. For that reason, we define a small threshold ε . We say that two matrices are equal iff the norm² of their difference is less than ε .
2. To generate a random rotation matrix, we generate three random angles and create a rotation matrix from these angles.

Acknowledgements

This section was greatly inspired by [Michael Penn's "Abstract Algebra" Youtube series](#) and [MIT Modern Algebra Class \(18.703\)](#). For those interested to dive deeper into the topic, we highly recommend these resources.

4.9 Exercises

Exercise 1. Define $X := \{(x, y) \in \mathbb{Q}^2 : xy = 1\}$. Oleksandr claims that:

1. $X \cap \mathbb{N}^2 = \{(1, 1)\}$.
2. $|X \cap \mathbb{Z}^2| = 2|X \cap \mathbb{N}^2|$.
3. $(X, \odot)$ is a group for $(x_1, y_1) \odot (x_2, y_2) = (x_1x_2, y_1y_2)$.

Which statements are **true**?

- a) Only 1.
- b) Only 1 and 2.
- c) Only 1 and 3.

²one can think of norm as being the measure of “distance” between two objects. Similarly, we can define norm not only on matrices, but on vectors as well.

- d) Only 2 and 3.
- e) All statements are correct.

Exercise 2. Does $(\mathbb{Z}, \oplus)$ with operation $a \oplus b = a + b - 1$ define a group?

- a) Yes, and this group is abelian.
- b) Yes, but this group is not abelian.
- c) No, since the associativity property does not hold.
- d) No, since there is no identity element in this group.
- e) No, since there is no inverse for some elements in this group.

Exercise 3. Consider the Cartesian plane $\mathbb{R}^2$, where two coordinates are real numbers. For two points A, B define the operation $\oplus$ as follows: $A \oplus B$ is the midpoint on segment AB . Does $(\mathbb{R}^2, \oplus)$ define a group?

- a) Yes, and this group is abelian.
- b) Yes, but this group is not abelian.
- c) No, since the associativity property does not hold and there is no identity element in this group.
- d) No, since the associativity property does not hold, but we might define an identity element nonetheless.

Exercise 4. Let $\mathbb{G}$ be the set of 2×2 matrices with integer entries and a non-zero determinant. Does $(\mathbb{G}, \times)$ form a group?

- a) Yes, and this group is abelian.
- b) Yes, but this group is not abelian.
- c) No, since the associativity property does not hold.
- d) No, since there is no identity element in this group.
- e) No, since there is no inverse for some elements in this group.

Exercise 5*. Let $(\mathbb{G}, \times)$ be an abelian group and Ω be any set. Let $\mathcal{F}$ be the set of functions $f : \Omega \rightarrow \mathbb{G}$ with domain Ω and range $\mathbb{G}$. Define the operation $\star$ on $\mathcal{F}$ by $(f \star g)(\omega) = f(\omega) \times g(\omega)$. Is $(\mathcal{F}, \star)$ a group?

- a) Yes, and this group is abelian.
- b) Yes, but this group is not abelian.
- c) No, since the associativity property does not hold.
- d) No, since there is no identity element in this group.
- e) No, since there is no inverse for some elements in this group.

Exercise 6. Which of the following is a **valid** automorphism $f : X \rightarrow X$?

- a) $X = [0, 1], f : x \mapsto x^2$.
- b) $X = [0, 1], f : x \mapsto x + 1$.
- c) $X = \mathbb{R}_{>0}, f : x \mapsto (x - 1)^3$.
- d) $X = \mathbb{Q}_{>0}, f : x \mapsto \sqrt{x}$.

Exercise 7*. Denote by $\text{GL}(2, \mathbb{R})$ a set of 2×2 invertible matrices with real entries. Define two functions $\varphi : \text{GL}(2, \mathbb{R}) \rightarrow \mathbb{R}$:

$$\varphi_1 \left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} \right) = ad - bc, \quad \varphi_2 \left(\begin{bmatrix} a & b \\ c & d \end{bmatrix} \right) = a + d$$

Den claims the following:

1. φ_1 is a group homomorphism between multiplicative groups $(\text{GL}(2, \mathbb{R}), \times)$ and $(\mathbb{R}, \times)$.
2. φ_2 is a group homomorphism between additive groups $(\text{GL}(2, \mathbb{R}), +)$ and $(\mathbb{R}, +)$.

Which of the following is **true**?

- a) Only statement 1 is correct.
- b) Only statement 2 is correct.
- c) Both statements 1 and 2 are correct.
- d) None of the statements is correct.

Exercise 8*. Introduce the set $\mathbb{M}$ as follows:

$$\mathbb{M} = \left\{ \begin{bmatrix} a & -b \\ b & a \end{bmatrix} : a, b \in \mathbb{R}, a^2 + b^2 \neq 0 \right\}$$

Suppose the function $\psi : \mathbb{C} \setminus \{0\} \rightarrow (\mathbb{M}, \times)$ is given by:

$$\psi(a + bi) = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

Is the function ψ an isomorphism?

- a) Yes, ψ is an isomorphism.
- b) No, ψ is not an isomorphism, but it is a homomorphism.
- c) ψ is neither an isomorphism nor a homomorphism.

Exercise 9*. Prove that a group of order 9 is abelian.

5 Polynomials

5.1 Basic Definition

Polynomials are intensively used in almost all areas of cryptography. In our particular case, polynomials will encode the information about statements we will need to prove. That being said, let us define what polynomial is.

Definition 5.1. A **polynomial** $f(x)$ is a function of the form

$$p(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n = \sum_{k=0}^n c_kx^k,$$

where $c_0, c_1, \dots, c_n$ are coefficients of the polynomial.

Notice that for now we did not specify what are c_i 's. We are interested in the case where $c_i \in \mathbb{F}$, where $\mathbb{F}$ is a field.

Definition 5.2. A set of polynomials depending on x with coefficients in a field $\mathbb{F}$ is denoted as $\mathbb{F}[x]$, that is

$$\mathbb{F}[x] = \left\{ p(x) = \sum_{k=0}^n c_kx^k : c_k \in \mathbb{F}, k = 0, \dots, n \right\}.$$

Definition 5.3. Evaluation of a polynomial $p(x) \in \mathbb{F}[x]$ at point $x_0 \in \mathbb{F}$ is simply finding the value of $p(x_0) \in \mathbb{F}$.

Example 5.1. Consider the finite field $\mathbb{F}_3$. Then, some examples of polynomials from $\mathbb{F}_3[x]$ are listed below:

1. $p(x) = 1 + x + 2x^2$.
2. $q(x) = 1 + x^2 + x^3$.
3. $r(x) = 2x^3$.

If we were to evaluate these polynomials at $1 \in \mathbb{F}_3$, we would get:

1. $p(1) = 1 + 1 + 2 \cdot 1 \bmod 3 = 1$.
2. $q(1) = 1 + 1 + 1 \bmod 3 = 0$.
3. $r(1) = 2 \cdot 1 = 2$.

Definition 5.4. The **degree** of a polynomial $p(x) = c_0 + c_1x + c_2x^2 + \dots$ is the largest $k \in \mathbb{Z}_{\geq 0}$ such that $c_k \neq 0$. We denote the degree of a polynomial as $\deg p$. We also denote by $\mathbb{F}^{(\leq m)}[x]$ a set of polynomials of degree at most m .

Example 5.2. The degree of the polynomial $p(x) = 1 + 2x + 3x^2$ is 2, so $p(x) \in \mathbb{F}_5^{(\leq 2)}[x]$.

Theorem 5.5. For any two polynomials $p, q \in \mathbb{F}[x]$ and $n = \deg p, m = \deg q$, the following two statements are true:

1. $\deg(pq) = n + m$.
2. $\deg(p + q) = \max\{n, m\}$ if $n \neq m$ and $\deg(p + q) \leq m$ for $m = n$.

5.2 Roots and divisibility

Definition 5.6. Let $p(x) \in \mathbb{F}[x]$ be a polynomial of degree $\deg p \geq 1$. A field element $x_0 \in \mathbb{F}$ is called a root of $p(x)$ if $p(x_0) = 0$.

Example 5.3. Consider the polynomial $p(x) = 1 + x + x^2 \in \mathbb{F}_3[x]$. Then, $x_0 = 1$ is a root of $p(x)$ since $p(x_0) = 1 + 1 + 1 \bmod 3 = 0$.

One of the fundamental theorems of polynomials is following.

Theorem 5.7. Let $p(x) \in \mathbb{F}[x]$, $\deg p \geq 1$. Then, $x_0 \in \mathbb{F}$ is a root of $p(x)$ if and only if there exists a polynomial $q(x)$ (with $\deg q = n - 1$) such that

$$p(x) = (x - x_0)q(x)$$

Example 5.4. Note that $x_0 = 1$ is a root of $p(x) = x^2 + 2$. Indeed, we can write $p(x) = (x - 1)(x - 2)$, so here $q(x) = x - 2$.

Also, this might not be obvious, but we can also divide polynomials in the same way as we divide integers. The result of division is not always a polynomial, so we also get a remainder.

Theorem 5.8. Given $f, g \in \mathbb{F}[x]$ with $g \neq 0$, there are unique polynomials $p, q \in \mathbb{F}[x]$ such that

$$f = q \cdot g + r, \quad 0 \leq \deg r < \deg g$$

Example 5.5. Consider $f(x) = x^3 + 2$ and $g(x) = x + 1$ over $\mathbb{R}$. Then, we can write $f(x) = (x^2 - x + 1)g(x) + 1$, so the remainder of the division is 1. Typically, we denote this as:

$$f \operatorname{div} g = x^2 - x + 1, \quad f \bmod g = 1.$$

The notation is pretty similar to one used in integer division.

Similarly, one can define gcd, lcm, and other number field theory operations for polynomials. However, we will not go into details here, besides mentioning the divisibility.

Definition 5.9. A polynomial $f(x) \in \mathbb{F}[x]$ is called **divisible** by $g(x) \in \mathbb{F}[x]$ (or, g **divides** f , written as $g \mid f$) if there exists a polynomial $h(x) \in \mathbb{F}[x]$ such that $f = gh$.

Theorem 5.10. If $x_0 \in \mathbb{F}$ is a root of $p(x) \in \mathbb{F}[x]$, then $(x - x_0) \mid p(x)$.

Definition 5.11. A polynomial $f(x) \in \mathbb{F}[x]$ is said to be **irreducible** in $\mathbb{F}$ if there are no polynomials $g, h \in \mathbb{F}[x]$ both of degree more than 1 such that $f = gh$.

Example 5.6. A polynomial $f(x) = x^2 + 16$ is irreducible in $\mathbb{R}$. In turn, $f(x) = x^2 - 2$ is not irreducible since $f(x) = (x - \sqrt{2})(x + \sqrt{2})$.

Example 5.7. There are no polynomials over complex numbers $\mathbb{C}$ with degree more than 2 that are irreducible. This follows from the *fundamental theorem of algebra*.

5.3 Interpolation

Now, let us ask the question: what defines the polynomial? Well, given expression $p(x) = \sum_{k=0}^n c_k x^k$ one can easily say: “hey, I need to know the coefficients $\{c_k\}_{k=0}^n$ ”.

Indeed, each polynomial of degree n is uniquely determined by the vector of its coefficients $(c_0, c_1, \dots, c_n) \in \mathbb{F}^{n+1}$. However, that is not the only way to define a polynomial.

Suppose I tell you that $p(x) = ax + b$ – just a simple linear function over $\mathbb{R}$. Suppose I tell you that $p(x)$ intercepts $(0, 0)$ and $(1, 2)$. Then, you can easily say that $p(x) = 2x$.

The more general question is: suppose $\deg p = n$, how many points do I need to define the polynomial $p(x)$ uniquely? The answer is $n + 1$ distinct points. This is the idea behind the interpolation: the polynomial is uniquely defined by $n + 1$ distinct points on the plane. An example is depicted in Figure 5.1. Now, let us see how we can interpolate the polynomial practically.

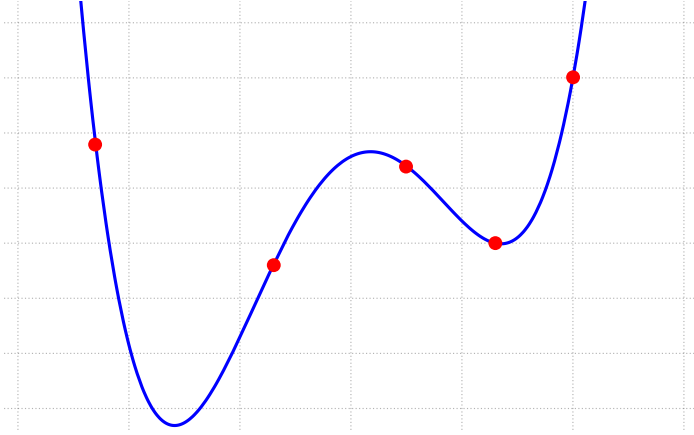


Figure 5.1: 5 points on the plane uniquely define the polynomial of degree 4.

Theorem 5.12. Given a set of points $\{(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)\} \subset \mathbb{F} \times \mathbb{F}$, there is a unique polynomial $L(x)$ of degree n such that $L(x_i) = y_i$ for all $i = 0, \dots, n$. This polynomial is called the **Lagrange interpolation polynomial** and can be found through the following formula:

$$L(x) = \sum_{i=0}^n y_i \ell_i(x), \quad \ell_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}.$$

Lemma 5.13. The polynomials $\{\ell_i\}_{i=1}^n$, in fact, have quite an interesting property:

$$\ell_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases},$$

where δ_{ij} is the Kronecker delta. Moreover, $\{\ell_i\}_{i=1}^n$ form a basis of $\mathbb{F}^{(\leq n)}[x]$: for any polynomial $p(x) \in \mathbb{F}^{(\leq n)}[x]$ there exist unique coefficients $\alpha_0, \dots, \alpha_n \in \mathbb{F}$ such that

$$p(x) = \sum_{i=0}^n \alpha_i \ell_i(x).$$

Example 5.8. Suppose we have points $(0, 1)$ and $(1, 2)$. Then, the Lagrange interpolation polynomial is

$$L(x) = 1 \cdot \frac{x-1}{0-1} + 2 \cdot \frac{x-0}{1-0} = (-1) \cdot (x-1) + 2 \cdot x = x+1$$

5.4 Schwartz-Zippel Lemma

Schwartz-Zippel Lemma is central for most modern zero-knowledge protocols. Its simple interpretation is the following: if we want to check whether two polynomials are equal, we can do it by checking whether they are equal at a random point. The probability of two different polynomials being equal at a random point is very low for sufficiently large finite fields $\mathbb{F}$. However, let us formulate the lemma more rigorously. Let us start with the single-variable case, which is easiest to understand.

Lemma 5.14 (Single Variable Schwartz-Zippel Lemma). Let $\mathbb{F}$ be a finite field and let $f \in \mathbb{F}[x]$ be a polynomial. Then,

$$\Pr_{x \leftarrow \mathbb{F}} [f(x) = 0] \leq \frac{\deg f}{|\mathbb{F}|}.$$

Proof Idea. The proof is quite simple. The polynomial of degree up to $n := \deg f$ can have up to n roots. Therefore, when we pick the point x uniformly at random from the field $\mathbb{F}$, there is at most $n/|\mathbb{F}|$ chance to pick the root of the polynomial.

However, the lemma can be generalized to the multi-variable case, which is more difficult to prove. The generalization is as follows.

Lemma 5.15 (Multivariable Schwartz-Zippel). Let $\mathbb{F}$ be a finite field and $\mathbb{S} \subseteq \mathbb{F}^n$ be a finite n -dimensional subspace. Let $f : \mathbb{S} \rightarrow \mathbb{F}$ be a polynomial in n variables. Then,

$$\Pr_{\mathbf{x} \leftarrow \mathbb{S}} [f(\mathbf{x}) = 0] \leq \frac{\deg f}{|\mathbb{S}|}.$$

Example 5.9. Suppose we are working over the Mersenne prime field of order $p = 2^{31} - 1$ with a polynomial $f(x_1, x_2) = x_1^{50} + 3x_1x_2 + x_2^5$. If we pick (x_1, x_2) at random from $\mathbb{F}_p^2$, the probability of $f(x_1, x_2) = 0$ is at most $50/(2^{31} - 1)$, providing approximately 25 bits of security.

5.5 Some Fun: Shamir's Secret Sharing

Shamir's Secret Sharing, also known as (t, n) -threshold scheme, is one of the protocols exploiting Lagrange Interpolation.

But first, let us define what secret sharing is. Suppose we have a secret data α , which is represented as an element from some finite set F . We divide this secret into n pieces $\alpha_1, \dots, \alpha_n \in F$ in such a way:

1. Knowledge of any t shares can reconstruct the secret α .
2. Knowledge of any number of shares below t cannot be used to reconstruct the secret α .

Now, let us define the sharing scheme.

Definition 5.16. Secret Sharing scheme is a pair of efficient algorithms (Gen, Comb) which work as follows:

- Gen(α, t, n): probabilistic sharing algorithm that yields n shards $(\alpha_1, \dots, \alpha_t)$ for which t shards are needed to reconstruct the secret α .
- Comb($\mathcal{I}, \{\alpha_i\}_{i \in \mathcal{I}}$): deterministic reconstruction algorithm that reconstructs the secret α from the shards $\mathcal{I} \subset \{1, \dots, n\}$ of size t .

Here, we require the **correctness**: for every $\alpha \in F$, for every possible output $(\alpha_1, \dots, \alpha_n) \leftarrow \text{Gen}(\alpha, t, n)$, and any t -size subset $\mathcal{I}$ of $\{1, \dots, n\}$ we have

$$\text{Comb}(\mathcal{I}, \{\alpha_i\}_{i \in \mathcal{I}}) = \alpha.$$

Now, Shamir's protocol is one of the most famous secret sharing schemes. It works as follows: our finite set is $\mathbb{F}_q$ for some large prime q . Then, algorithms in the protocol are defined as follows:

- Gen(α, k, n): choose random $k_1, \dots, k_{t-1} \leftarrow \mathbb{F}_q$ and define polynomial $\omega(x) := \alpha + \sum_{j=1}^{t-1} k_j x^j \in \mathbb{F}_q^{\leq(t-1)}[x]$. Then, compute $\alpha_i \leftarrow \omega(i) \in \mathbb{F}_q$ for each i and return $(\alpha_1, \dots, \alpha_n)$.
- Comb($\mathcal{I}, \{\alpha_i\}_{i \in \mathcal{I}}$): reconstruct the polynomial $\omega(x)$ using Lagrange interpolation and return $\omega(0) = \alpha$.

The combination function is possible since, having t points $\{i, \alpha_i\}_{i \in \mathcal{I}}$ with $\omega(i) = \alpha_i$, we can fully reconstruct the polynomial $\omega(x)$ and then evaluate it at 0 to get α . Instead, suppose we have only $t - 1$ (or less) pairs $\{i, \alpha_i\}_{i \in \mathcal{I}}$. Then, there are many polynomials $\omega(x)$ that pass through these points (in fact, if we were in the field of real numbers, this number would be infinite), and thus the secret α is not uniquely determined. The intuition behind Shamir's Secret Sharing protocol is illustrated in Figure 5.2: for $t = 3$, having only two blue points, we cannot reconstruct the polynomial, while knowing the third red point allows us to uniquely determine the polynomial and thus get value at 0.

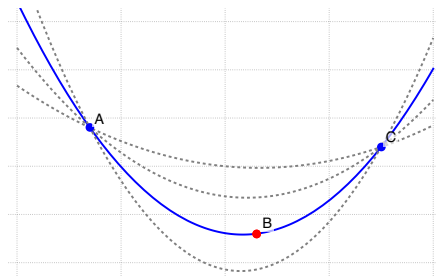


Figure 5.2: Illustration of Shamir's Secret Sharing

5.6 Exercises

Exercise 1. Suppose for three polynomials $p, q, r \in \mathbb{F}[x]$ we have $\deg p = 3$, $\deg q = 4$, $\deg r = 5$. Which of the following is true for $n := \deg\{(p - q)r\}$?

- a) $n = 9$.
- b) n might be less than 9.
- c) $n = 20$.
- d) n is less than $\deg\{qr\}$.

Exercise 2. Define the polynomial over $\mathbb{F}_5$: $f(x) := 4x^2 + 2$. Which of the following is the root of $f(x)$?

- a) 2
- b) 3
- c) 4
- d) f has no roots.

Exercise 3. Quadratic polynomial $p(x) = ax^2 + bx + c \in \mathbb{R}[x]$ has zeros at 1 and 2 and $p(0) = 2$. Find the value of $a + b + c$.

- a) 0
- b) -1
- c) 1
- d) Not enough information is provided.

Exercise 4*. Consider two polynomials:

$$P_n(x) = x^{2n} + x^n - 2, \quad Q(x) = x^2 + 1$$

For which values of n does polynomial $Q(x)$ divides $P_n(x)$

- a) For all even n .
- b) For all odd n .
- c) For all n which are divisible by 4.
- d) Just for $n = 4$.
- e) It does not work for any natural number n .

Exercise 5. Suppose $f \in \mathbb{F}_p[x]$ is a d -degree polynomial with d **distinct** roots in $\mathbb{F}_p$. Suppose Alice wants to fool Bob by claiming that f is not a zero polynomial. The protocol consists of n rounds, where on each round Bob samples the random scalar $\tau \leftarrow \mathbb{F}_p$ and checks the value of $f(\tau)$. What is the probability that Alice successfully fools Bob during the protocol?

- a) Exactly $(d/p)^n$
- b) Strictly less than $(d/p)^n$
- c) Exactly nd/p .
- d) Strictly less than d/np .

Exercise 6. Suppose $f \in \mathbb{F}_p[x_1, \dots, x_n]$ is an n -variable polynomial. Carol randomly samples the point $\mathbf{x} \leftarrow \mathbb{S}$ where $\mathbb{S} \subseteq \mathbb{F}_p^n$ is a set of points $(x_1, \dots, x_n)$ such that $\sum_{i=1}^n x_i = 1$ over $\mathbb{F}_p$. According to the Schwartz-Zippel Lemma, what is the tightest upper bound for $\Pr[f(\mathbf{x}) = 0]$ among specified below?

- a) $\deg f / p^n$
- b) $\deg f / p^{n-1}$
- c) $\deg f / p^{n-2}$
- d) $\deg f / p$

6 Linear Algebra Basics

Although linear algebra is out of the main scope of this course, familiarity with its basic definitions and concepts is still essential (especially when considering Linear PCPs further from [Section 13](#)). In this section, we provide a brief review of the key principles that are used throughout the course. You can skip this section without any doubts if you are already familiar with that, and come back to it whenever you need to refresh your memory.

6.1 Vector Space

Similarly to group theory working with *groups*, the linear algebra also has a special designated primitive – **vector space**. If previously we were working with the (finite) field $\mathbb{F}$, now we will work with the vector space V over this field. In many practical applications, vector space is formed by **vectors** consisting of a finite fixed collection of elements from the field $\mathbb{F}$. For example, the vector space might be simply $\mathbb{F}^n$: the set of all n -tuples $(x_1, x_2, \dots, x_n)$ of elements from $\mathbb{F}$. Yet, let us give a bit more general definition.

Definition 6.1. A **vector space** V over the field $\mathbb{F}$ is an abelian group for addition $+$ together with a scalar multiplication operation $\cdot$ from $\mathbb{F} \times V$ to V , sending $(\lambda, x) \mapsto \lambda x$ and such that for any $\mathbf{v}, \mathbf{u} \in V$ and $\lambda, \mu \in \mathbb{F}$ we have:

- $\lambda(\mathbf{u} + \mathbf{v}) = \lambda\mathbf{u} + \lambda\mathbf{v}$
- $(\lambda\mu)\mathbf{v} = \lambda(\mu\mathbf{v})$
- $(\lambda + \mu)\mathbf{v} = \lambda\mathbf{v} + \mu\mathbf{v}$
- $1\mathbf{v} = \mathbf{v}$

Any element $\mathbf{v} \in V$ is called a **vector**, and any element $\lambda \in \mathbb{F}$ is called a **scalar**. We also mark vector elements in boldface.

Example 6.1. For example, $V = \mathbb{F}^n$ with operations defined as:

$$\lambda \cdot (x_1, x_2, \dots, x_n) = (\lambda x_1, \lambda x_2, \dots, \lambda x_n)$$

$(x_1, x_2, \dots, x_n) + (y_1, y_2, \dots, y_n) = (x_1 + y_1, x_2 + y_2, \dots, x_n + y_n)$ is a vector space. Similarly, the following three sets V_1, V_2, V_3 with operations defined above are also valid vector spaces:

$$V_1 = \{(x_1, x_2, \dots, x_n) \in \mathbb{F}^n : x_1 = 0\}$$

$$V_2 = \{(x_1, x_2, \dots, x_n) \in \mathbb{F}^n : x_1 - x_3 = 0\}$$

$$V_3 = \{(x_1, x_2, \dots, x_n) \in \mathbb{F}^n : x_1 + x_2 + \dots + x_n = 0\}$$

Reasoning. Let us see why, for example, V_3 is a valid vector space. Since all operations are inherited from $\mathbb{F}^n$, we only need to check that V_3 is closed under addition and scalar multiplication. Suppose $\mathbf{v}_1 = (x_1, \dots, x_n)$ and $\mathbf{v}_2 = (y_1, \dots, y_n) \in V_3$. Then:

$$\mathbf{v}_1 + \mathbf{v}_2 = (x_1 + y_1, \dots, x_n + y_n)$$

Note that $\sum_{i=1}^n x_i + y_i = (\sum_{i=1}^n x_i) + (\sum_{i=1}^n y_i) = 0 + 0 = 0$, so the sum is indeed closed.

We will also need to define one very important notion: the linear dependence and independence of vectors.

Definition 6.2. A set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k\}$ in a vector space V over a field $\mathbb{F}$ is said to be **linearly independent** if the only scalars $\lambda_1, \lambda_2, \dots, \lambda_k \in \mathbb{F}$ that satisfy the equation

$$\lambda_1 \mathbf{v}_1 + \lambda_2 \mathbf{v}_2 + \dots + \lambda_k \mathbf{v}_k = \mathbf{0}$$

are $\lambda_1 = \lambda_2 = \dots = \lambda_k = 0$. If there exist scalars, not all zero, that satisfy this equation, then the vectors are said to be **linearly dependent**.

Example 6.2. Consider the vectors $\mathbf{v}_1 = (1, 0, 0)$, $\mathbf{v}_2 = (0, 1, 0)$, and $\mathbf{v}_3 = (0, 0, 1)$ in $\mathbb{R}^3$. These vectors are linearly independent because the only solution to

$$\lambda_1 \mathbf{v}_1 + \lambda_2 \mathbf{v}_2 + \lambda_3 \mathbf{v}_3 = \mathbf{0}$$

is $\lambda_1 = \lambda_2 = \lambda_3 = 0$.

Example 6.3. Consider the vectors $\mathbf{u}_1 = (1, 2, 3)$, $\mathbf{u}_2 = (2, 4, 6)$ in $\mathbb{R}^3$. These vectors are linearly dependent because $\mathbf{u}_2 = 2\mathbf{u}_1$, so the equation

$$\lambda_1 \mathbf{u}_1 + \lambda_2 \mathbf{u}_2 = \mathbf{0}$$

has non-trivial solutions, such as $\lambda_1 = 2$ and $\lambda_2 = -1$.

6.2 Matrix

Besides vectors, frequently we are working with **matrices**. In the most basic sense, the matrix is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns. For example, the matrix A with m rows and n columns, consisting of elements from the finite field $\mathbb{F}$ is denoted as $A \in \mathbb{F}^{m \times n}$. Additionally, we use notation $A = \{a_{i,j}\}_{i,j=1}^{m \times n}$ to denote the square matrix A of size $m \times n$ with elements $a_{i,j}$. Now, let us define operations on matrices.

Definition 6.3. Let A, B be two matrices over the field $\mathbb{F}$. The following operations are defined:

- **Matrix addition/subtraction:** $A \pm B = \{a_{i,j} \pm b_{i,j}\}_{i,j=1}^{m \times n}$. The matrices A and B must have the same size $m \times n$.
- **Scalar multiplication:** $\lambda A = \{\lambda a_{i,j}\}_{1 \leq i,j \leq n}$ for any $\lambda \in \mathbb{F}$.
- **Matrix multiplication:** $C = AB$ is a matrix $C \in \mathbb{F}^{m \times p}$ with elements $c_{i,j} = \sum_{\ell=1}^n a_{i,\ell} b_{\ell,j}$. The number of columns in A must be equal to the number of rows in B , that is $A \in \mathbb{F}^{m \times n}$ and $B \in \mathbb{F}^{n \times p}$.

Example 6.4. Suppose $\mathbb{F} = \mathbb{R}$. Then, consider

$$A = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 3}, \quad B = \begin{bmatrix} 2 & 1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}$$

We cannot add A and B since they have different sizes. However, we can multiply them:

$$AB = \begin{bmatrix} 5 & 6 \\ 7 & 9 \end{bmatrix}, \quad BA = \begin{bmatrix} 4 & 4 & 5 \\ 7 & 7 & 5 \\ 3 & 3 & 3 \end{bmatrix}$$

To see why, for example, the upper left element of AB is 5, we can calculate it as $\sum_{\ell=1}^3 a_{1,\ell}b_{\ell,1} = 1 \times 2 + 1 \times 1 + 2 \times 1 = 5$.

Remark. Now, we add a very important remark. It just so happens that when working with vectors, we usually assume that they are **column vectors**. This means that the vector $v = (v_1, v_2, \dots, v_n)$ is represented as a matrix:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

This is a common convention in linear algebra, and we will use it in the following sections.

Sometimes we may derive some rows or columns of the matrix from the others. There is a very important notion related to this.

Definition 6.4. The **rank** of a matrix $A \in \mathbb{F}^{m \times n}$ is the maximum number of linearly independent rows or columns in A . This is also known as the **row rank** or **column rank** of the matrix.

Remark. The row rank and column rank of a matrix are always equal.

Example 6.5. Consider the matrix $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$. To determine the rank of A , we can perform certain linear rows (or columns) permutations to show that the rows (or columns) are linearly independent vectors. For example, we can perform the following operations:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \xrightarrow{R_2 \leftarrow R_2 - 3R_1} \begin{bmatrix} 1 & 2 \\ 0 & -2 \end{bmatrix} \xrightarrow{R_2 \leftarrow -\frac{1}{2}R_2} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \xrightarrow{R_1 \leftarrow R_1 - 2R_2} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Since we can clearly see that the rows (and columns) are linearly independent, the rank of A is 2.

Example 6.6. Consider the matrix $B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. Let's perform the following operations:

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \xrightarrow{R_2 \leftarrow R_2 - 2R_1} \begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix}$$

Here we can see that the second row is a linear combination of the first row, so the rank of B is 1.

One important operation we will be frequently working with is the **transpose** of the matrix. The transpose of a matrix is an operator that flips a matrix over its diagonal, that is, it switches the row and column indices of the matrix by producing another matrix denoted as $A^\top$.

Definition 6.5 (Transposition). Given a matrix $A \in \mathbb{F}^{m \times n}$, the **transpose** of A is a matrix $A^\top \in \mathbb{F}^{n \times m}$ with elements $A_{ij}^\top = A_{ji}$.

Example 6.7. For example, consider the square matrix $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$. Then, the transpose of A is $A^\top = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$.

Finally, it just happens that we can construct matrix from the vectors. Therefore, let us introduce the corresponding notation.

Definition 6.6 (Composing Matrix from vectors). Suppose we are given n vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n \in \mathbb{F}^m$. Then, we might define matrix A as a matrix with columns $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ as follows:

$$A = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \dots \quad \mathbf{v}_n] = \begin{bmatrix} v_{1,1} & v_{2,1} & \dots & v_{n,1} \\ v_{1,2} & v_{2,2} & \dots & v_{n,2} \\ \vdots & \vdots & \ddots & \vdots \\ v_{1,m} & v_{2,m} & \dots & v_{n,m} \end{bmatrix}$$

Alternatively, vectors might be represented as rows, and the matrix A might be defined as a matrix with rows $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$:

$$A = \begin{bmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_n^\top \end{bmatrix} = \begin{bmatrix} v_{1,1} & v_{1,2} & \dots & v_{1,m} \\ v_{2,1} & v_{2,2} & \dots & v_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ v_{n,1} & v_{n,2} & \dots & v_{n,m} \end{bmatrix}$$

Example 6.8. For example, consider the vectors $\mathbf{v}_1 = (1, 2, 3)$ and $\mathbf{v}_2 = (4, 5, 6)$. Then, the matrix A with columns $\mathbf{v}_1$ and $\mathbf{v}_2$ is:

$$A = [\mathbf{v}_1 \quad \mathbf{v}_2] = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Similarly, the matrix B with rows $\mathbf{v}_1$ and $\mathbf{v}_2$ is:

$$B = \begin{bmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

6.3 Inner Product

Definition 6.7. Consider the vector space $\mathbb{F}^n$. The **inner product** is a function $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^n \rightarrow \mathbb{F}$ satisfying the following conditions for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{F}^n$:

- $\langle \mathbf{u} + \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{w} \rangle + \langle \mathbf{v}, \mathbf{w} \rangle$.
- $\langle \mathbf{u}, \mathbf{v} + \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{v} \rangle + \langle \mathbf{u}, \mathbf{w} \rangle$.
- $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ for all $\mathbf{u} \in \mathbb{F}^n$ iff $\mathbf{v} = \mathbf{0}$.
- $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ for all $\mathbf{v} \in \mathbb{F}^n$ iff $\mathbf{u} = \mathbf{0}$.

Plenty of functions can be built that satisfy the inner product definition, we will use the one that is usually called **dot product**.

Definition 6.8. Consider the vector space $\mathbb{F}^n$. The **dot product** on $\mathbb{F}^n$ is a function $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{F}$, defined for every $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$ as follows:

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^n u_i v_i$$

Alternatively, the dot product can also be denoted using the dot notation as $\mathbf{u} \cdot \mathbf{v}$. That is why it is called the “dot” product.

Example 6.9. Let $\mathbf{u}, \mathbf{v}$ are vectors over the real number $\mathbb{R}$, where $\mathbf{u} = (1, 2, 3)$, and $\mathbf{v} = (2, 4, 3)$. Then:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{i=1}^3 u_i v_i = 2 \cdot 1 + 2 \cdot 4 + 3 \cdot 3 = 2 + 8 + 9 = 19$$

6.4 Hadamard Product

Yet another product we are going to use is the **Hadamard product**. Let us see how it works.

Definition 6.9. Suppose $A, B \in \mathbb{F}^{m \times n}$. The **Hadamard product** $A \odot B$ gives a matrix C such that $C_{i,j} = A_{i,j}B_{i,j}$. Essentially, we multiply elements elementwise.

Example 6.10. Consider $A = \begin{bmatrix} 1 & 1 & 2 \\ 3 & 0 & 3 \end{bmatrix}, B = \begin{bmatrix} 3 & 2 & 1 \\ 0 & 2 & 1 \end{bmatrix}$. Then, the Hadamard product is:

$$A \odot B = \begin{bmatrix} 1 \cdot 3 & 1 \cdot 2 & 2 \cdot 1 \\ 3 \cdot 0 & 0 \cdot 2 & 3 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 2 \\ 0 & 0 & 3 \end{bmatrix}$$

6.5 Outer Product

The final product we want to introduce is the **outer product**.

Definition 6.10. Given two vectors $\mathbf{u} \in \mathbb{F}^n, \mathbf{v} \in \mathbb{F}^m$ the **outer product** is a matrix whose entries are all products of an element in the first vector with an element in the second vector:

$$\mathbf{u} \otimes \mathbf{v} := \mathbf{u}\mathbf{v}^\top = \begin{bmatrix} u_1v_1 & u_1v_2 & \cdots & u_1v_n \\ u_2v_1 & u_2v_2 & \cdots & u_2v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_mv_1 & u_mv_2 & \cdots & u_mv_n \end{bmatrix}$$

Lemma 6.11 (Properties of outer product). For any scalar $c \in \mathbb{F}$ and $(\mathbf{u}, \mathbf{v}, \mathbf{w}) \in \mathbb{F}^n \times \mathbb{F}^m \times \mathbb{F}^p$:

- Transpose: $(\mathbf{u} \otimes \mathbf{v}) = (\mathbf{v} \otimes \mathbf{u})^\top$
- Distributivity: $\mathbf{u} \otimes (\mathbf{v} + \mathbf{w}) = \mathbf{u} \otimes \mathbf{v} + \mathbf{u} \otimes \mathbf{w}$
- Scalar Multiplication: $c(\mathbf{v} \otimes \mathbf{u}) = (c\mathbf{v}) \otimes \mathbf{u} = \mathbf{v} \otimes (c\mathbf{u})$
- Rank: the outer product $\mathbf{u} \otimes \mathbf{v}$ is a rank-1 matrix if $\mathbf{u}$ and $\mathbf{v}$ are non-zero vectors

Example 6.11. Let $\mathbf{u}, \mathbf{v}$ are vectors over the real number $\mathbb{R}$, where $\mathbf{u} = (1, 2, 3)$ and $\mathbf{v} = (2, 4, 3)$. Then:

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^\top = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 2 & 4 & 3 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 & 1 \cdot 4 & 1 \cdot 3 \\ 2 \cdot 2 & 2 \cdot 4 & 2 \cdot 3 \\ 3 \cdot 2 & 3 \cdot 4 & 3 \cdot 3 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 3 \\ 4 & 8 & 6 \\ 6 & 12 & 9 \end{bmatrix}$$

Additionally, as we can see the rows number 2 and 3 in the result matrix can be represented as a linear combination of the first row, specifically by multiplying it by 2 and 3, respectively. The same property applies to the columns. This demonstrates the property of the outer product, that the resulting matrix has a rank of 1.

7 Fields Extension

7.1 General Definition

Previously, our discussion revolved around the finite field $\mathbb{F}_p$ for a prime p . However, many protocols need more than just a prime field. For example, elliptic curve pairings and certain STARK constructions require extending $\mathbb{F}_p$ to, in a sense, the analogous of complex numbers.

From school and, possibly, university, you might remember how complex numbers $\mathbb{C}$ are constructed. You take two real numbers, say, $x, y \in \mathbb{R}$, introduce a new symbol i satisfying $i^2 = -1$, and define the complex number as $z = x + iy$. In certain cases, one might encounter a bit more rigorous and abstract definition of complex numbers as the set of pairs $(x, y) \in \mathbb{R}^2$ where addition is naturally defined as $(x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2)$, and the multiplication is:

$$(x_1, y_1) \cdot (x_2, y_2) = (x_1x_2 - y_1y_2, x_1y_2 + x_2y_1)^3.$$

In spite of what interpretation you have seen, the complex number is just a tuple of two real numbers that satisfy a bit different rules of multiplication (since addition is typically defined in the same way). What is even more important to us, is that $\mathbb{C}$ is our first example of the so-called **field extension** of $\mathbb{R}$.

Formally, definition of the field extension is straightforward:

Definition 7.1. Let $\mathbb{F}$ be a field and $\mathbb{K}$ be another field. We say that $\mathbb{K}$ is an **extension** of $\mathbb{F}$ if $\mathbb{F} \subset \mathbb{K}$ and we denote it as $\mathbb{K}/\mathbb{F}$.

Despite just a simplicity of the definition, the field extensions are a very powerful tool in mathematics. But first, let us consider a few non-trivial examples of field extensions.

Example 7.1. Denote by $\mathbb{Q}(\sqrt{2}) = \{x + y\sqrt{2} : x, y \in \mathbb{Q}\}$. This is a field extension of $\mathbb{Q}$. It is obvious that $\mathbb{Q} \subset \mathbb{Q}(\sqrt{2})$, but why is $\mathbb{Q}(\sqrt{2})$ a field? Addition and multiplication operations are obviously closed:

$$\begin{aligned} (x_1 + y_1\sqrt{2}) + (x_2 + y_2\sqrt{2}) &= (x_1 + x_2) + (y_1 + y_2)\sqrt{2}, \\ (x_1 + y_1\sqrt{2}) \cdot (x_2 + y_2\sqrt{2}) &= (x_1x_2 + 2y_1y_2) + (x_1y_2 + x_2y_1)\sqrt{2}. \end{aligned}$$

But what about the inverse element? Well, here is the trick:

$$\begin{aligned} \frac{1}{x + y\sqrt{2}} &= \frac{x - y\sqrt{2}}{(x + y\sqrt{2})(x - y\sqrt{2})} = \frac{x - y\sqrt{2}}{x^2 - 2y^2} = \\ &= \frac{x}{x^2 - 2y^2} - \frac{y}{x^2 - 2y^2}\sqrt{2} \in \mathbb{Q}(\sqrt{2}). \end{aligned}$$

³Notice that $(x_1 + iy_1)(x_2 + iy_2) = x_1x_2 + iy_2x_1 + iy_1x_2 + i^2y_1y_2 = (x_1x_2 - y_1y_2) + (x_1y_2 + x_2y_1)i$.

Example 7.2. Consider $\mathbb{Q}(\sqrt{2}, i) = \{a + bi : a, b \in \mathbb{Q}(\sqrt{2})\}$ where $i^2 = -1$. This is a field extension of $\mathbb{Q}(\sqrt{2})$ and, consequently, of $\mathbb{Q}$. The representation of the element is:

$$(a + b\sqrt{2}) + (c + d\sqrt{2})i = a + b\sqrt{2} + ci + d\sqrt{2}i$$

Showing that this is a field is a bit more tedious, but still straightforward. Suppose we take $\alpha + \beta i \in \mathbb{Q}(\sqrt{2}, i)$ with $\alpha, \beta \in \mathbb{Q}(\sqrt{2})$. Then:

$$\frac{1}{\alpha + \beta i} = \frac{\alpha - \beta i}{\alpha^2 + \beta^2} = \frac{\alpha}{\alpha^2 + \beta^2} - \frac{\beta}{\alpha^2 + \beta^2}i$$

Since $\mathbb{Q}(\sqrt{2})$ is a field, both $\frac{\alpha}{\alpha^2 + \beta^2}$ and $\frac{\beta}{\alpha^2 + \beta^2}$ are in $\mathbb{Q}(\sqrt{2})$, and, consequently, $\mathbb{Q}(\sqrt{2}, i)$ is a field as well.

Remark. Notice that basically, $\mathbb{Q}(\sqrt{2}, i)$ is just a linear combination of $\{1, \sqrt{2}, i, \sqrt{2}i\}$. This has a very important implication: $\mathbb{Q}(\sqrt{2}, i)$ is a four-dimensional vector space over $\mathbb{Q}$, where elements $\{1, \sqrt{2}, i, \sqrt{2}i\}$ naturally form **basis**. We are not going to use it implicitly, but this observation might make further discussion a bit more intuitive.

Remark. One might have defined $\mathbb{Q}(\sqrt{2}, i) = \{x + \sqrt{2}y : x, y \in \mathbb{Q}(i)\}$ instead. Indeed, $\mathbb{Q}(\sqrt{2})(i) = \mathbb{Q}(i)(\sqrt{2}) = \mathbb{Q}(\sqrt{2}, i)$.

7.2 Polynomial Quotient Ring

Now, we present a more general way to construct field extensions. Notice that when constructing $\mathbb{C}$, we used the magical element i that satisfies $i^2 = -1$. But here is another way how to think of it.

Consider the set of polynomials $\mathbb{R}[x]$, then I pick $p(x) := x^2 + 1 \in \mathbb{R}[x]$ and ask you to find roots of $p(x)$. Of course, you would claim “hey, this equation has no solutions over $\mathbb{R}$ ” and that is totally true. That is why mathematicians introduced a new element i that we formally called the root of $x^2 + 1$. Note however, that i is not a number in the traditional sense, but rather a fictional symbol that we artificially introduced to satisfy the equation.

Now, could we have picked another polynomial, say, $q(x) = x^2 + 4$? Sure! As long as its roots cannot be found in $\mathbb{R}$, we are good to go.

Example 7.3. Suppose β is the root of $q(x) := x^2 + 4$. Then we could have defined complex numbers as a set of $x + y\beta$ for $x, y \in \mathbb{R}$. In this case, multiplication, for example, would be defined a bit differently than in the case of $\mathbb{C}$:

$$(x_1 + y_1\beta) \cdot (x_2 + y_2\beta) = (x_1x_2 - 4y_1y_2) + (x_1y_2 + x_2y_1)\beta.$$

We shifted to the polynomial consideration for a reason: now, instead of considering the complex number $\mathbb{C}$ as “some” tuple of real numbers (c_0, c_1) , now

let us view it as a polynomial⁴ $c_0 + c_1X$ modulo polynomial $X^2 + 1$.

Example 7.4. Indeed, take, for example, $p_1(X) := 1 + 2X$ and $p_2(X) := 2 + 3X$. Addition is performed as we are used to:

$$p_1 + p_2 = (1 + 2X) + (2 + 3X) = 3 + 5X,$$

but multiplication is a bit different:

$$p_1 p_2 = (1 + 2X) \cdot (2 + 3X) = 2 + 3X + 4X + 6X^2 = 6X^2 + 7X + 2.$$

Well, and what next? Recall that we are doing arithmetic modulo $X^2 + 1$ and for that reason, we divide the polynomial by $X^2 + 1$:

$$6X^2 + 7X + 2 = 6(X^2 + 1) + 7X - 4 \implies (6X^2 + 7X + 2) \bmod (X^2 + 1) = 7X - 4,$$

meaning that $p_1 p_2 = 7X - 4$. Oh wow, hold on! Let us come back to our regular complex number representation and multiply $(1 + 2i)(2 + 3i)$. We get $2 + 3i + 4i + 6i^2 = -4 + 7i$. That is exactly the same result if we change X to i above! In fact, what we have observed is the fact that our polynomial quotient ring $\mathbb{R}[X]/(X^2 + 1)$ is isomorphic to $\mathbb{C}$.

So, let us generalize this observation to any field $\mathbb{F}$ and any irreducible polynomial $\mu(x) \in \mathbb{F}[x]$.

Theorem 7.2. Let $\mathbb{F}$ be a field and $\mu(x)$ — irreducible polynomial over $\mathbb{F}$ (sometimes called a **reduction polynomial**). Consider a set of polynomials over $\mathbb{F}[x]$ modulo $\mu(x)$, formally denoted as $\mathbb{F}[x]/(\mu(x))$. Then, $\mathbb{F}[x]/(\mu(x))$ is a field.

Example 7.5. As we considered above, let $\mathbb{F} = \mathbb{R}$, $\mu(x) = x^2 + 1$, then $\mathbb{R}[X]/(X^2 + 1)$ (a set of polynomials modulo $X^2 + 1$) is a field.

Example 7.6. Suppose $\mathbb{F} = \mathbb{Q}$ and $\mu(x) := x^2 - 2$. Then, $\mathbb{Q}[X]/(X^2 - 2)$ is a field isomorphic to $\mathbb{Q}(\sqrt{2})$, considered above.

Example 7.7. Suppose $\mathbb{F} = \mathbb{Q}$ and $\mu(x) := (x^2 + 1)(x^2 - 2) = x^4 - x^2 - 2$. Then, $\mathbb{Q}[X]/(x^4 - x^2 - 2)$ is a field isomorphic to $\mathbb{Q}(\sqrt{2}, i)$.

Remark. Although we have not defined the isomorphism between two rings/fields, it is defined similarly to group isomorphism. Suppose we have fields $(\mathbb{F}, +, \times)$ and $(\mathbb{K}, \oplus, \otimes)$. Bijective function $\phi : \mathbb{F} \rightarrow \mathbb{K}$ is called an isomorphism if it preserves additive and multiplicative structures, that is for all $a, b \in \mathbb{F}$:

⁴Here, we use X to represent the polynomial variable to avoid confusion with the notation $x + yi$.

$$\begin{aligned}\phi(a + b) &= \phi(a) \oplus \phi(b), \\ \phi(a \times b) &= \phi(a) \otimes \phi(b).\end{aligned}$$

This theorem (aka definition) corresponds to viewing complex numbers as a polynomial quotient ring $\mathbb{R}[X]/(X^2 + 1)$. But, we can give a theorem (aka definition) for our classical representation via magical root i of $x^2 + 1$.

Theorem 7.3. Let $\mathbb{F}$ be a field and $\mu \in \mathbb{F}[X]$ is an irreducible polynomial of degree n and let $\mathbb{K} := \mathbb{F}[X]/(\mu(X))$. Let $\theta \in \mathbb{K}$ be the root of μ over $\mathbb{K}$. Then,

$$\mathbb{K} = \{c_0 + c_1\theta + \cdots + c_{n-1}\theta^{n-1} : c_0, \dots, c_{n-1} \in \mathbb{F}\}$$

Although this definition is quite useful, we will mostly rely on the polynomial quotient ring definition. Let us define the **prime field extension**.

Definition 7.4. Suppose p is prime and $m \geq 2$. Let $\mu \in \mathbb{F}_p[X]$ be an irreducible polynomial of degree m . Then, elements of $\mathbb{F}_{p^m}$ are polynomials in $\mathbb{F}_p^{(\leq m)}[X]$. In other words,

$$\mathbb{F}_{p^m} = \{c_0 + c_1X + \cdots + c_{m-1}X^{m-1} : c_0, \dots, c_{m-1} \in \mathbb{F}_p\},$$

where all operations are performed modulo $\mu(X)$.

Again, let us consider a few examples.

Example 7.8. Consider the $\mathbb{F}_{2^4}$. Then, there are 16 elements in this set:

$$\begin{aligned}0, 1, X, X + 1, \\ X^2, X^2 + 1, X^2 + X, X^2 + X + 1, \\ X^3, X^3 + 1, X^3 + X, X^3 + X + 1, \\ X^3 + X^2, X^3 + X^2 + 1, X^3 + X^2 + X, X^3 + X^2 + X + 1.\end{aligned}$$

One might choose the following reduction polynomial: $\mu(X) = X^4 + X + 1$ (of degree 4). Then, operations are performed in the following manner:

- Addition: $(X^3 + X^2 + 1) + (X^2 + X + 1) = X^3 + X$.
- Subtraction: $(X^3 + X^2 + 1) - (X^2 + X + 1) = X^3 + X$.
- Multiplication: $(X^3 + X^2 + 1) \cdot (X^2 + X + 1) = X^2 + 1$ since:
 $(X^3 + X^2 + 1) \cdot (X^2 + X + 1) = X^5 + X + 1 \bmod (X^4 + X + 1) = X^2 + 1$
- Inversion: $(X^3 + X^2 + 1)^{-1} = X^2$ since $(X^3 + X^2 + 1) \cdot X^2 \bmod (X^4 + X + 1) = 1$.

Now, in the subsequent sections, we would need to extend $\mathbb{F}_p$ at least to $\mathbb{F}_{p^2}$. A convenient choice, similarly to the complex numbers, is to take $\mu(X) = X^2 + 1$. However, in contrast to $\mathbb{R}$, equation $X^2 = -1 \pmod{p}$ might have solutions

over certain prime numbers p . Thus, we consider proposition below.

Proposition 7.5. Let p be an odd prime. Then $X^2 + 1$ is irreducible in $\mathbb{F}_p[X]$ if and only if $p \equiv 3 \pmod{4}$.

Corollary 7.6. $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 + 1)$ is a valid prime field extension for odd primes p satisfying $p \equiv 3 \pmod{4}$. In this case, extended elements are of the form $c_0 + c_1u$ where $c_0, c_1 \in \mathbb{F}_p$ and $u^2 = -1$.

7.3 Multiplicative Group of a Finite Field

The non-zero elements of $\mathbb{F}_p$, denoted as $\mathbb{F}_p^\times$, form a multiplicative cyclic group. In other words, there exist elements $g \in \mathbb{F}_p^\times$, called *generators*, such that

$$\mathbb{F}_p^\times = \{g^k : 0 \leq k \leq p-2\}$$

The order of $x \in \mathbb{F}_p^\times$ is the smallest positive integer r such that $x^r = 1$. It is also not difficult to show that $r \mid (p-1)$.

Definition 7.7. $\omega \in \mathbb{F}$ is the *primitive root* in the finite field $\mathbb{F}$ if $\langle \omega \rangle = \mathbb{F}^\times$.

Example 7.9. $\omega = 3$ is the primitive root of $\mathbb{F}_7$. Indeed,

$$3^1 = 3, 3^2 = 2, 3^3 = 6, 3^4 = 4, 3^5 = 5, 3^6 = 1.$$

So clearly $\langle \omega \rangle = 7$.

In STARKs (and in optimizing operations) for DFT (Discrete Fourier Transform) we would need the so-called n th primitive roots of unity.

Example 7.10. For those who studied complex numbers a bit (it is totally OK if you did not, so you might skip this example), recall an equation $\zeta^n = 1$ over $\mathbb{C}$. The solutions are $\zeta_k = \cos\left(\frac{2\pi k}{n}\right) + i \sin\left(\frac{2\pi k}{n}\right)$ for $k \in \{0, 1, \dots, n-1\}$, so one has exactly n solutions (in contrast to $x^n = 1$ over $\mathbb{R}$ where there are at most 2 solutions^a). For any solution ζ_k , it is true that $\zeta_k^n = 1$, but if one were to consider the subgroup generated by ζ_k (that is, $\{1, \zeta_k, \zeta_k^2, \dots\}$), then not necessarily $\langle \zeta_k \rangle$ would enumerate all the roots of unity $\{\zeta_j\}_{j=0}^{n-1}$. For that reason, we call ζ_k the n th primitive root of unity if $\langle \zeta_k \rangle$ enumerates all roots of unity. One can show that this is the case if and only if $\gcd(k, n) = 1$. This is always the case for $k = 1$, so commonly mathematicians use ζ_n to denote an expression $\cos \frac{2\pi}{n} + i \sin \frac{2\pi}{n} = e^{2\pi i/n}$.

^aThink why.

Yet, let us give the broader definition, including the finite fields case.

Definition 7.8. ω is the n th primitive root of unity if $\omega^n = 1$ and $\omega^k \neq 1$ for all $1 \leq k < n$.

Note that such ω exists if and only if $n \mid (p - 1)$.

7.4 Algebraic Closure

Consider the following interesting question: suppose we have a field $\mathbb{F}$. Is there an extension $\mathbb{K}/\mathbb{F}$ such that $\mathbb{K}$ contains all roots of any polynomial in $\mathbb{F}[X]$? The answer is yes, and such a field is called the **algebraic closure** of $\mathbb{F}$, although not always this algebraic closure has a nice form. But first, let us define what it means for field $\mathbb{F}$ to be algebraically closed.

Definition 7.9. A field $\mathbb{F}$ is called **algebraically closed** if every non-constant polynomial $p(x) \in \mathbb{F}[X]$ has a root in $\mathbb{F}$.

Example 7.11. $\mathbb{R}$ is not algebraically closed since $X^2 + 1$ has no roots in $\mathbb{R}$. However, $\mathbb{C}$ is algebraically closed, which follows from the fundamental theorem of algebra. Since $\mathbb{C}$ is a field extension of $\mathbb{R}$, it is also an algebraic closure of $\mathbb{R}$. This is commonly denoted as $\overline{\mathbb{R}} = \mathbb{C}$.

Definition 7.10. A field $\mathbb{K}$ is called an **algebraic closure** of $\mathbb{F}$ if $\mathbb{K}/\mathbb{F}$ is algebraically closed. This is denoted as $\overline{\mathbb{F}} = \mathbb{K}$.

Since we are doing cryptography and not mathematics, we are interested in the algebraic closure of $\mathbb{F}_p$. Well, I have two news for you (as always, one is good and one is bad). The good news is that any finite field $\mathbb{F}_{p^m}$ has an algebraic closure. The bad news is that it does not have a form $\mathbb{F}_{p^k}$ for $k > m$ and there are infinitely many elements in it (so in other words, the algebraic closure of a finite field is not finite). This is due to the following theorem.

Theorem 7.11. No finite field $\mathbb{F}$ is algebraically closed.

Proof. Suppose $f_1, f_2, \dots, f_n \in \mathbb{F}$ are all elements of $\mathbb{F}$. Consider the following polynomial:

$$p(x) = \prod_{i=1}^n (x - f_i) + 1 = (x - f_1)(x - f_2) \cdots (x - f_n) + 1.$$

Clearly, $p(x)$ is a non-constant polynomial and has no roots in $\mathbb{F}$, since for any $f \in \mathbb{F}$, one has $p(f) = 1$. ■

But what form does the $\overline{\mathbb{F}_p}$ have? Well, it is a union of all $\mathbb{F}_{p^k}$ for $k \geq 1$. This is formally written as:

$$\overline{\mathbb{F}_p} = \bigcup_{k \in \mathbb{N}} \mathbb{F}_{p^k}.$$

Remark. But this definition is super counter-intuitive! So here how we usually interpret it. Suppose I tell you that polynomial $q(x)$ has a root in $\overline{\mathbb{F}}_p$. What that means is that there exists some extension $\mathbb{F}_{p^m}$ such that for some $\alpha \in \mathbb{F}_{p^m}$, $q(\alpha) = 0$. We do not know how large this m is, but we know that it exists. For that reason, $\overline{\mathbb{F}}_p$ is defined as an infinite union of all possible field extensions.

7.5 Exercises

Exercise 1. Oleksandr decided to build $\mathbb{F}_{49}$ as $\mathbb{F}_7[i]/(i^2 + 1)$. Compute $(3 + i)(4 + i)$.

- a) $6 + i$. b) 6. c) $4 + i$. d) 4. e) $2 + 4i$.

Exercise 2. Oleksandr came up with yet another extension $\mathbb{F}_{p^2} = \mathbb{F}_p[i]/(i^2 + 2)$. He asked interns to calculate $2/i$. Based on five answers given below, help Oleksandr to find the correct one.

- a) 1. c) $(p - 3)i$. e) $p - 1$.
b) $p - 2$. d) $(p - 1)i$.

Exercise 3*. After endless tries, Oleksandr has finally found the perfect field extension: $\mathbb{F}_{p^2} := \mathbb{F}_p[v]/(v^2 + v + 1)$. However, Oleksandr became very frustrated since not for any p this would be a valid field extension. For which of the following values p such construction would **not** be a valid field extension? Use the fact that equation $\omega^3 = 1$ over $\mathbb{F}_p$ has non-trivial solutions (meaning, two others except for $\omega = 1$) if $p \equiv 1 \pmod{3}$. You can assume that listed numbers are primes.

- a) 8431. b) 9173. c) 9419. d) 6947.

Exercises 4-9. Tower of Extensions

You are given the passage explaining the topic of tower of extensions. The text has gaps that you need to fill in with the correct statement among the provided choices.

This question demonstrates the concept of the so-called **tower of extensions**. Suppose we want to build an extension field $\mathbb{F}_{p^4}$. Of course, we can find some irreducible polynomial $p(X)$ of degree 4 over $\mathbb{F}_p$ and build $\mathbb{F}_{p^4}$ as $\mathbb{F}_p[X]/(p(X))$. However, this method is very inconvenient since implementing the full 4-degree polynomial arithmetic is inconvenient. Moreover, if we were to implement arithmetic over, say, $\mathbb{F}_{p^{24}}$, that would make the matters worse. For this reason, we will build $\mathbb{F}_{p^4}$ as $\mathbb{F}_{p^2}[j]/(q(j))$ where $q(j)$ is an irreducible polynomial of degree 2 over $\mathbb{F}_{p^2}$, which itself is represented as $\mathbb{F}_p[i]/(r(i))$ for some suitable irreducible quadratic polynomial $r(i)$. This way, we can first implement $\mathbb{F}_{p^2}$, then $\mathbb{F}_{p^4}$, relying on the implementation of $\mathbb{F}_{p^2}$.

For illustration purposes, let us pick $p := 5$. As noted above, we want to build $\mathbb{F}_{5^2}$ first. A valid way to represent $\mathbb{F}_{5^2}$ would be to set $\mathbb{F}_{5^2} := \boxed{4}$. Given this representation, the zero of a linear polynomial $f(x) = ix - (i + 3)$, defined over $\mathbb{F}_{5^2}$, is $\boxed{5}$.

Now, assume that we represent $\mathbb{F}_{5^4}$ as $\mathbb{F}_{5^2}[j]/(j^2 - \xi)$ for $\xi = i + 1$. Given such representation, the value of j^4 is $\boxed{6}$. Finally, given $c_0 + c_1j \in \mathbb{F}_{5^4}$ we call $c_0 \in \mathbb{F}_{5^2}$ a **real part**, while $c_1 \in \mathbb{F}_{5^2}$ an **imaginary part**. For example, the imaginary part of number $j^3 + 2i^2\xi$ is $\boxed{7}$, while the real part of $(a_0 + a_1j)b_1j$ is $\boxed{8}$. Similarly to complex numbers, it motivates us to define the number's **conjugate**: for $z = c_0 + c_1j$, define the conjugate as $\bar{z} := c_0 - c_1j$. The expression $z\bar{z}$ is then $\boxed{9}$.

Exercise 4.

- a) $\mathbb{F}_5[i]/(i^2 + 1)$
- b) $\mathbb{F}_5[i]/(i^2 + 2)$
- c) $\mathbb{F}_5[i]/(i^2 + 4)$
- d) $\mathbb{F}_5[i]/(i^2 + 2i + 1)$
- e) $\mathbb{F}_5[i]/(i^2 + 4i + 4)$

Exercise 7.

- a) equal to zero.
- b) equal to one.
- c) equal to the real part.
- d) $2(1 + i)$
- e) -4

Exercise 5.

- a) $1 + i$
- b) $1 + 2i$
- c) $1 + 4i$
- d) $2 + 3i$
- e) $3 + i$

Exercise 8.

- a) a_1b_1
- b) $a_1b_1\xi$
- c) a_0b_1
- d) $a_0b_1\xi$
- e) a_0a_1

Exercise 6.

- a) $4 + 2i$
- b) $4i$
- c) 1
- d) $1 + 2i$
- e) $2 + 4i$

Exercise 9.

- a) $c_0^2 + c_1^2$
- b) $c_0^2 - c_1^2\xi$
- c) $c_0^2 + c_1^2\xi^2$
- d) $(c_0^2 + c_1^2\xi)j$
- e) $(c_0^2 - c_1^2)j$

Part II

Cryptography Tools

With all Mathematics preliminaries covered, we can now move to the next part of the book. This part is dedicated to the mathematical background required for Cryptography. Namely, we will cover essentials specified in [Table 2](#).

Section	Topic	Key Concepts
8	Security Analysis	Advantage, negligible function, DL/CD-H/DDH Assumptions
9	Elliptic Curves, ecpairing	Group of points on Elliptic Curve, Projective Coordinates, elliptic curve pairing
10	Commitment Schemes	Hash-based Commitments, Pedersen Commitments, KZG Commitments

Table 2: Topics covered in Part II

In the [Section 9](#), we consider the core object in the majority of modern zk-SNARK systems, the Elliptic Curves. Besides considering the definition and basics, we also describe the projective coordinates and the elliptic curve pairing operation. In the [Section 10](#), we consider the commitment schemes: hash-based, Pedersen, and KZG commitments. In the [Section 8](#), we consider the basics of security analysis, which, from the practical standpoint, is needed to read cryptographic papers and understand which security assumptions are used in the protocols and how authors typically describe them.

8 Basics of Security Analysis

In many cases, technical papers include the analysis on the key question: “How secure is this cryptographic algorithm?” or rather “Why this cryptographic algorithm is secure?”. In this section, we will shortly describe the notation and typical construction for justifying the security of cryptographic algorithms.

Typically, the cryptographic security is defined in a form of a game between the adversary (who we call $\mathcal{A}$) and the challenger (who we call $\mathcal{Ch}$). The adversary is trying to break the security of the cryptographic algorithm using arbitrary (but still efficient) protocol, while the challenger is following a simple, fixed protocol. The game is played in a form of a challenge, where the adversary is given some information and is asked to perform some task. The security of the cryptographic algorithm is defined based on the probability of the adversary to win the game.

8.1 Cipher Semantic Security

Let us get into specifics. Suppose that we want to specify that the encryption scheme is secure. Recall that cipher $\mathcal{E} = (E, D)$ over the space $(\mathcal{K}, \mathcal{M}, \mathcal{C})$ (here, $\mathcal{K}$ is the space containing all possible keys, $\mathcal{M}$ – all possible messages and $\mathcal{C}$ – all possible ciphers) consists of two efficiently computable methods:

- $E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$ – encryption method, that based on the provided message $m \in \mathcal{M}$ and key $k \in \mathcal{K}$ outputs the cipher $c = E(k, m) \in \mathcal{C}$.
- $D : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$ – decryption method, that based on the provided cipher $c \in \mathcal{C}$ and key $k \in \mathcal{K}$ outputs the message $m = D(k, c) \in \mathcal{M}$.

Of course, we require the **correctness**:

$$(\forall k \in \mathcal{K}) (\forall m \in \mathcal{M}) : \{D(k, E(k, m)) = m\}$$

Now let us play the following game between adversary $\mathcal{A}$ and challenger $\mathcal{Ch}$:

1. $\mathcal{A}$ picks any two messages $m_0, m_1 \in \mathcal{M}$ on his choice.
2. $\mathcal{Ch}$ picks a random key $k \leftarrow \$ \mathcal{K}$ and random bit $b \leftarrow \$ \{0, 1\}$ and sends the cipher $c = E(k, m_b)$ to $\mathcal{A}$.
3. $\mathcal{A}$ is trying to guess the bit b by using the cipher c .
4. $\mathcal{A}$ outputs the guess $\hat{b}$.

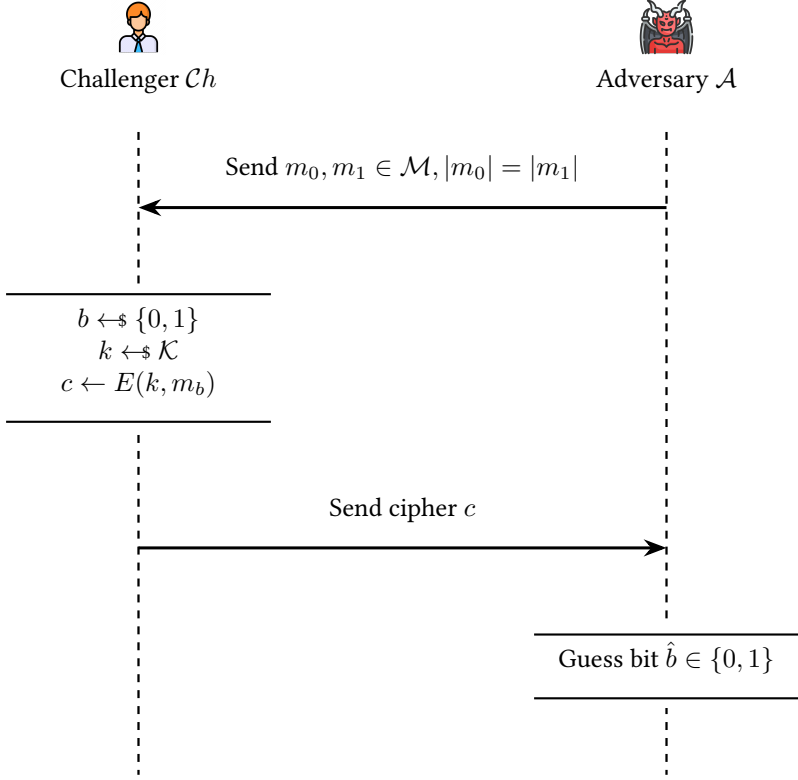


Figure 8.1: The game between the adversary $\mathcal{A}$ and the challenger $\mathcal{Ch}$ for defining the semantic security.

Now, what should happen if our encryption scheme is secure? The adversary should not be able to guess the bit b with a probability significantly higher than $1/2$ (a random guess). Formally, define the **advantage** of the adversary $\mathcal{A}$ as:

$$\text{SSAdv}[\mathcal{E}, \mathcal{A}] := \left| \Pr[\hat{b} = b] - \frac{1}{2} \right|$$

We say that the encryption scheme is **semantically secure**⁵ if for any efficient adversary $\mathcal{A}$ the advantage $\text{SSAdv}[\mathcal{A}]$ is negligible. In other words, the adversary cannot guess the bit b with a probability significantly higher than $1/2$.

Now, what negligible means? Let us give the formal definition!

Definition 8.1. A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is called **negligible** if for all $c \in \mathbb{R}_{>0}$ there exists $n_c \in \mathbb{N}$ such that for any $n \geq n_c$ we have $|f(n)| < 1/n^c$.

⁵This version of definition is called a **bit-guessing** version.

The alternative definition, which is probably easier to interpret, is the following.

Theorem 8.2. A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is **negligible** if and only if for any $c \in \mathbb{R}_{>0}$, we have

$$\lim_{n \rightarrow \infty} f(n)n^c = 0$$

Example 8.1. The function $f(n) = 2^{-n}$ is negligible since for any $c \in \mathbb{R}_{>0}$ we have

$$\lim_{n \rightarrow \infty} 2^{-n}n^c = 0$$

The function $g(n) = \frac{1}{n!}$ is also negligible for similar reasons.

Example 8.2. The function $h(n) = \frac{1}{n}$ is not negligible since for $c = 1$ we have

$$\lim_{n \rightarrow \infty} \frac{1}{n} \times n = 1 \neq 0$$

Well, that is weird. For some reason we are considering a function the depends on some natural number n , but what is this number?

Typically, when defining the security of the cryptographic algorithm, we are considering the security parameter λ (e.g., the length of the key). The function is negligible if the probability of the adversary to break the security of the cryptographic algorithm is decreasing with the increasing of the security parameter λ . Moreover, we require that the probability of the adversary to break the security of the cryptographic algorithm is decreasing faster than any polynomial function of the security parameter λ .

So all in all, we can define the semantic security as follows.

Definition 8.3. The encryption scheme $\mathcal{E}$ with a security paramter $\lambda \in \mathbb{N}$ is **semantically secure** if for any efficient adversary $\mathcal{A}$ we have:

$$\left| \Pr \left[b = \hat{b} \mid \begin{array}{l} m_0, m_1 \leftarrow \mathcal{M}, k \leftarrow \mathcal{K}, b \leftarrow \{0, 1\} \\ c \leftarrow E(k, m_b) \\ \hat{b} \leftarrow \mathcal{A}(c) \end{array} \right] - \frac{1}{2} \right| < \text{negl}(\lambda)$$

Do not be afraid of such complex notation, it is quite simple. Notation $\Pr[A \mid B]$ means “the probability of A , given that B occurred”. So our inner probability is read as “the probability that the guessed bit $\hat{b}$ equals b given the setup on the right”. Then, on the right we define the setup: first we generate two messages $m_0, m_1 \in \mathcal{M}$, then we choose a random bit b and a key k , cipher the message m_b , send it to the adversary and the adversary, based on provided cipher,

gives $\hat{b}$ as an output. We then claim that the probability of the adversary to guess the bit b is close to $1/2$.

Let us see some more examples of how to define the security of certain cryptographic objects.

8.2 Message recovery attacks

Essentially, message recovery attacks [1, chapter 2] are types of attacks that, from given ciphertext, recover the message with significantly better probability than random guessing, which is obviously $1/|\mathcal{M}|$. Of course, any reasonable notion of security should rule out such an attack, and indeed, semantic security does.

Although this might seem intuitively obvious, we provide a formal proof to clarify the reasoning. One of the reasons for doing this is to thoroughly demonstrate the concept of a *security reduction*, which is the primary method used to assess the security of systems.

Let us play the following attack game message recovery between adversary $\mathcal{A}$ and challenger $\mathcal{Ch}$. For a given cipher $\mathcal{E} = (E, D)$, defined over $\mathcal{K}, \mathcal{M}, \mathcal{C}$, and for a given adversary $\mathcal{A}$, the attack game proceeds as follows:

- The challenger computes $m \leftarrow \$ \mathcal{M}$, $k \leftarrow \$ \mathcal{K}$, $c \leftarrow \$ E(k, m)$ and sends c to the adversary.
- the adversary output a message $\hat{m} \in \mathcal{M}$.

We say that $\mathcal{A}$ wins the game in this case, and we define $\mathcal{A}$'s message recovery advantage with respect to $\mathcal{E}$ as:

$$\text{MRadv}[\mathcal{E}, \mathcal{A}] := \left| \Pr[\hat{m} = m] - \frac{1}{|\mathcal{M}|} \right|.$$

Definition 8.4 (Security against message recovery). A cipher $\mathcal{E}$ is secure against message recovery if for all efficient adversaries $\mathcal{A}$, the value $\text{MRadv}[\mathcal{E}, \mathcal{A}]$ is negligible.

Theorem 8.5. Let $\mathcal{E} = (E, D)$ be a cipher defined over $(\mathcal{K}, \mathcal{M}, \mathcal{C})$. If $\mathcal{E}$ is semantically secure then $\mathcal{E}$ is secure against message recovery.

► Let us prove by assuming the opposite statement. In other words, if $\mathcal{E}$ is not secure against message recovery, then $\mathcal{E}$ is not semantically secure.

Assume that $\mathcal{E}$ is not secure against message recovery. We have an efficient adversary $\mathcal{A}$ who has a significant advantage in the message recovery game and an efficient adversary $\mathcal{B}$ who is trying to compromise $\mathcal{E}$. The proof idea is to use an efficient adversary $\mathcal{A}$ as a "black box" (or oracle machine). We construct $\mathcal{B}$ as follows:

1. Adversary $\mathcal{B}$ generates $m_0, m_1 \in \mathcal{M}$ and sends them to the semantic security challenger.
2. The semantic security challenger returns ciphertext c to adversary $\mathcal{B}$. Since $\mathcal{B}$ can use the message recovery oracle, it sends c to $\mathcal{A}$.
3. $\mathcal{A}$ returns $\hat{m}$. If $\hat{m} = m_0$, then $b = 0$ otherwise, $b = 1$.
4. Sends b to the semantic security challenger.

Intuitively, this implies that $\text{SSadv}[\mathcal{E}, \mathcal{B}]$ is exactly equal to $\text{MRadv}[\mathcal{E}, \mathcal{A}]$. ◀

This means that if an adversary is capable of successfully winning the message recovery game, then they can also effectively win the semantic security game. In essence, security against message recovery is a stronger property than semantic security.

Our motivation for demonstrating proof is to illustrate in detail the notion of a security reduction, which is the main technique used to reason about the security of systems.

Security reduction is typically applied to certain computational problems. To be more precise, we will now explain how security reduction operates in general.

Definition 8.6 (Security Reduction).

1. Let $\mathcal{A}$ be an efficient adversary who is trying to compromise the cryptosystem $\mathcal{E}$.
2. We construct an efficient algorithm $\mathcal{A}'$, which we call the **reduction**, that will solve the computationally hard problem $\mathcal{X}$ while using an oracle access to $\mathcal{A}$. For any input x of the problem $\mathcal{X}$, the algorithm $\mathcal{A}'$ generates the input of the cryptosystem $\mathcal{E}$ for $\mathcal{A}$. Once it wins the corresponding experiment, $\mathcal{A}'$ solves $\mathcal{X}$ for input x with a certain non-negligible probability $\mu(n)$. Typically, such probability turns out to be bounded below by $1/n^c$, $c \in \mathbb{N}$.
3. Thus, the algorithm $\mathcal{A}'$ solves $\mathcal{X}$ with the non-negligible probability $\varepsilon(n)\mu(n)$ (or typically $\varepsilon(n)/n^c$). This quantity is non-negligible if $\varepsilon(n)$ is assumed to be non-negligible.
4. Since the problem $\mathcal{X}$ is computationally hard, we get a contradiction. It follows that no attacker $\mathcal{A}$ can effectively crack $\mathcal{E}$, i.e., this cryptosystem is computationally strong.

8.3 Discrete Logarithm Assumption

Now, let us define the fundamental assumption used in cryptography formally: the **Discrete Logarithm Assumption** (DL) [1, chapter 16].

Definition 8.7. Assume that $\mathbb{G}$ is a cyclic group of prime order r generated by $g \in \mathbb{G}$. Define the following game:

1. Both challenger $\mathcal{Ch}$ and adversary $\mathcal{A}$ take a description $\mathbb{G}$ as an input:

- order r and generator $g \in \mathbb{G}$.
- 2. $\mathcal{Ch}$ computes $\alpha \leftarrow \mathbb{Z}_r$, $u \leftarrow g^\alpha$ and sends $u \in \mathbb{G}$ to $\mathcal{A}$.
- 3. The adversary $\mathcal{A}$ outputs $\hat{\alpha} \in \mathbb{Z}_r$.

We define $\mathcal{A}$'s **advantage in solving the discrete logarithm problem in $\mathbb{G}$** , denoted as $\text{DLadv}[\mathcal{A}, \mathbb{G}]$, as the probability that $\hat{\alpha} = \alpha$.

Definition 8.8. The **Discrete Logarithm Assumption** holds in the group $\mathbb{G}$ if for any efficient adversary $\mathcal{A}$ the advantage $\text{DLadv}[\mathcal{A}, \mathbb{G}]$ is negligible.

Informally, this assumption means that given u , it is very hard to find α such that $u = g^\alpha$. But now we can write down this formally!

8.4 Computational Diffie-Hellman

Another fundamental problem in cryptography is the **Computational Diffie-Hellman (CDH)** problem [1, chapter 16]. It states that given g^α, g^β it is hard to find $g^{\alpha\beta}$. This property is frequently used in the construction of cryptographic protocols such as the Diffie-Hellman key exchange.

Let us define this problem formally.

Definition 8.9. Let $\mathbb{G}$ be a cyclic group of prime order r generated by $g \in \mathbb{G}$. Define the following game:

- 1. Both challenger $\mathcal{Ch}$ and adversary $\mathcal{A}$ take a description $\mathbb{G}$ as an input: order r and generator $g \in \mathbb{G}$.
- 2. $\mathcal{Ch}$ computes $\alpha, \beta \leftarrow \mathbb{Z}_r$, $u \leftarrow g^\alpha$, $v \leftarrow g^\beta$, $w \leftarrow g^{\alpha\beta}$ and sends $u, v \in \mathbb{G}$ to $\mathcal{A}$.
- 3. The adversary $\mathcal{A}$ outputs $\hat{w} \in \mathbb{G}$.

We define $\mathcal{A}$'s **advantage in solving the computational Diffie-Hellman problem in $\mathbb{G}$** , denoted as $\text{CDHadv}[\mathcal{A}, \mathbb{G}]$, as the probability that $\hat{w} = w$.

Definition 8.10. The **Computational Diffie-Hellman Assumption** holds in the group $\mathbb{G}$ if for any efficient adversary $\mathcal{A}$ the advantage $\text{CDHadv}[\mathcal{A}, \mathbb{G}]$ is negligible.

8.5 Decisional Diffie-Hellman

Now, we loosen the requirements a bit. The **Decisional Diffie-Hellman (DDH)** problem [1, chapter 16] states that given $g^\alpha, g^\beta, g^{\alpha\beta}$ it is “hard” to distinguish $g^{\alpha\beta}$ from a random element in $\mathbb{G}$. Formally, we define this problem as follows.

Definition 8.11. Let $\mathbb{G}$ be a cyclic group of prime order r generated by $g \in \mathbb{G}$. Define the following game:

1. Both challenger $\mathcal{Ch}$ and adversary $\mathcal{A}$ take a description $\mathbb{G}$ as an input: order r and generator $g \in \mathbb{G}$.
2. $\mathcal{Ch}$ computes $\alpha, \beta, \gamma \leftarrow \mathbb{Z}_r, u \leftarrow g^\alpha, v \leftarrow g^\beta, w_0 \leftarrow g^{\alpha\beta}, w_1 \leftarrow g^\gamma$. Then, $\mathcal{Ch}$ flips a coin $b \leftarrow \{0, 1\}$ and sends u, v, w_b to $\mathcal{A}$.
3. The adversary $\mathcal{A}$ outputs the predicted bit $\hat{b} \in \{0, 1\}$.

We define $\mathcal{A}$'s **advantage in solving the Decisional Diffie-Hellman problem in $\mathbb{G}$** , denoted as $\text{DDHadv}[\mathcal{A}, \mathbb{G}]$, as

$$\text{DDHadv}[\mathcal{A}, \mathbb{G}] := \left| \Pr[b = \hat{b}] - \frac{1}{2} \right|$$

Now, let us break this assumption for some quite generic group! Consider the following example.

Theorem 8.12. Suppose that $\mathbb{G}$ is a cyclic group of an even order. Then, the Decision Diffie-Hellman Assumption does not hold in $\mathbb{G}$. In fact, there is an efficient adversary $\mathcal{A}$ that can distinguish $g^{\alpha\beta}$ from a random element in $\mathbb{G}$ with an advantage $1/4$.

Proof. If $|\mathbb{G}| = 2n$ for $n \in \mathbb{N}$, it means that we can split the group into two subgroups of order n , say, $\mathbb{G}_1$ and $\mathbb{G}_2$. The first subgroup consists of elements in a form g^{2k} , while the second subgroup consists of elements in a form g^{2k+1} .

Now, if we could efficiently determine, based on group element $g \in \mathbb{G}$, whether $g \in \mathbb{G}_1$ or $g \in \mathbb{G}_2$, we essentially could solve the problem. Fortunately, there is such a method! Consider the following lemma.

Lemma 8.13. Suppose $u = g^\alpha$. Then, α is even if and only if $u^n = 1$.

Proof. If α is even, then $\alpha = 2\alpha'$ and thus

$$u^n = (g^{2\alpha'})^n = g^{2n\alpha'} = (g^{2n})^{\alpha'} = 1^{\alpha'} = 1$$

Conversely, if $u^n = 1$ then $u^{\alpha n} = 1$, meaning that $2n \mid \alpha n$, implying that α is even. Lemma is proven.

Now, we can construct our adversary $\mathcal{A}$ as follows. Suppose $\mathcal{A}$ is given (u, v, w) . Then,

1. Based on u , get the parity of α , say $p_\alpha \in \{\text{even}, \text{odd}\}$.
2. Based on v , get the parity of β , say $p_\beta \in \{\text{even}, \text{odd}\}$.
3. Based on w , get the parity of γ , say $p_\gamma \in \{\text{even}, \text{odd}\}$.
4. Calculate $p'_\gamma \in \{\text{even}, \text{odd}\}$ — parity of $\alpha\beta$.
5. Return $\hat{b} = 0$ if $p'_\gamma = p_\gamma$, and $\hat{b} = 1$, otherwise.

Suppose γ is indeed $\alpha \times \beta$. Then, condition $p'_\gamma = p_\gamma$ will always hold. If γ is a random element, then the probability that $p'_\gamma = p_\gamma$ is $1/2$. Therefore, the probability that $\mathcal{A}$ will guess the bit b correctly is $3/4$, and the advantage is $1/4$ therefore. ■

Why is this necessary? Typically, it is impossible to prove the predicate “for every efficient adversary $\mathcal{A}$ this probability is negligible” and therefore we need to make assumptions, such as the Discrete Logarithm Assumption or the Computational Diffie-Hellman Assumption. In turn, proving the statement “if X is secure then Y is also secure” is manageable and does not require solving any fundamental problems. So, for example, knowing that the probability of the adversary to break the Diffie-Hellman assumption is negligible, we can prove that the Diffie-Hellman key exchange is secure.

8.6 Exercises

Exercise 1. Which of the following functions is **negligible**?

- a) $\mu(\lambda) = \lambda^{-2^{10}}$.
- b) $\mu(\lambda) = 5^{-\log \lambda}$.
- c) $\mu(\lambda)$ is λ^{-1000} for all odd λ and $2^{-\lambda}$ for all even λ .
- d) $\mu(\lambda) = \lambda^{100} e^{-\sqrt{\lambda}}$.

Exercise 2. Let $\mathcal{M} = \mathcal{C} = \mathcal{K} = \mathbb{Z}_p^\times$ for a prime p . Define the encryption scheme $\mathcal{E} = (E, D)$ as follows:

$$E(k, m) = k \cdot m \pmod{p}, \quad D(k, c) = k^{-1} \cdot c \pmod{p}$$

Is the scheme $\mathcal{E}$ perfectly secure? (*Perfect security means that adversary's advantage is exactly 0*).

- a) Yes.
- b) No, because only the one-time pad can be perfectly secret.
- c) No, because $|\mathcal{K}|$ must be greater than $|\mathcal{M}|$.
- d) No, because there is a simple strategy for an adversary to find the message given the ciphertext.

Exercise 3. For an encryption scheme $\mathcal{E}$, an adversary can guess the last bit of a message m from its ciphertext c with probability $\frac{1}{2} + \varepsilon$ via a function $f(c)$. The adversary plays the IND-CPA game by choosing m_0 with last bit 0 and m_1 with last bit 1. Upon receiving ciphertext c_b , the adversary returns $\hat{b} = f(c_b)$. What is the adversary's advantage, defined as $\text{SSadv}[\mathcal{A}, \mathcal{E}] := 2 \left| \Pr[\hat{b} = b] - \frac{1}{2} \right|$?

- a) 2ε .
- b) $\frac{1}{2} + \varepsilon$.
- c) ε^2 .
- d) $1 - 2\varepsilon$.

Exercise 4. Let $\mathcal{E} = (E, D)$ be a semantically secure encryption scheme where $\mathcal{M} = \mathcal{C} = \{0, 1\}^\lambda$. Which of the following schemes are semantically secure? (*Multiple options may be correct.*)

Note: $\text{parity}(m)$ is 1 if the number of ones in m is odd, and 0 otherwise. $\parallel$ is concatenation, and $\text{reverse}(m)$ reverses the bit string m .

- a) $E'(k, m) = 0 \parallel E(k, m)$.
- b) $E'(k, m) = E(k, m) \parallel \text{parity}(m)$.
- c) $E'(k, m) = \text{reverse}(E(k, m))$.
- d) $E'(k, m) = E(k, \text{reverse}(m))$.
- e) $E'(k, m) = E(k, m) \parallel k$.
- f) $E'((k, k'), m) = E(k, m) \parallel E(k', m)$.

Exercise 5. Let $G(s)$ be a secure pseudorandom generator (PRG) that generates λ bits. Which of the following generators are secure? (*Multiple options may be correct.*)

- a) $G'(s) = G(s) \parallel 0$.
- b) $G'(s_1 \parallel s_2) = G(s_1) \oplus G(s_2)$.
- c) $G'(s) = G(s) \oplus 1^\lambda$, where 1^λ is a string of λ ones.
- d) $G'(s)$ is the first $\lambda/2$ bits of $G(s)$.
- e) $G'(s) = G(s) \parallel G(s)$.
- f) $G'(s_1 \parallel s_2) = (s_1, G(s_2))$.
- g) $G'(s) = G(0)$.

Exercises 6-8. PRG Security and Statistical Tests

Let $G : \mathcal{K} \rightarrow \{0, 1\}^\lambda$ be a pseudorandom generator. We want to test if G is secure using a function $\mathcal{A} : \{0, 1\}^\lambda \rightarrow \{0, 1\}$, which we call a *statistical test*. The advantage of $\mathcal{A}$ is defined as:

$$\text{PRGadv}[\mathcal{A}, G] = \left| \Pr_{k \leftarrow \mathcal{K}}[\mathcal{A}(G(k)) = 1] - \Pr_{r \leftarrow \mathcal{S}\{0, 1\}^\lambda}[\mathcal{A}(r) = 1] \right|$$

An alternative definition for the security of G is that $\text{PRGadv}[\mathcal{A}, G]$ is negligible for all efficient algorithms $\mathcal{A}$.

Exercise 6. Consider two trivial statistical tests: (a) $\mathcal{A}_0(x) = 0$ for all x and (b) $\mathcal{A}_1(x) = 1$ for all x . Which of the following statements is true?

- a) $\text{PRGadv}[\mathcal{A}_0, G] = \text{PRGadv}[\mathcal{A}_1, G] = 0$.
- b) $\text{PRGadv}[\mathcal{A}_0, G] = \text{PRGadv}[\mathcal{A}_1, G] = 1$.
- c) $\text{PRGadv}[\mathcal{A}_0, G] = \text{PRGadv}[\mathcal{A}_1, G] = \frac{1}{2}$.
- d) $\text{PRGadv}[\mathcal{A}_0, G] = 0, \text{PRGadv}[\mathcal{A}_1, G] = 1$.

Exercise 7. Let a PRG $G : \mathcal{K} \rightarrow \{0, 1\}^\lambda$ satisfy the condition that the last

bit of $G(k)$ is one for $2/3$ of the keys in $\mathcal{K}$. Consider the statistical test $\mathcal{A}(x)$ which outputs 1 if the last bit of x is one, and 0 otherwise. What is the advantage $\text{PRGadv}[\mathcal{A}, G]$?

- a) 1. b) $1/2$. c) $1/3$. d) $2/3$. e) $1/6$.

Exercise 8. Let $G : \mathcal{K} \rightarrow \{0, 1\}^\lambda$ be a secure PRG. Define a new generator $G' : \mathcal{K} \times \mathcal{K} \rightarrow \{0, 1\}^\lambda$ as:

$$G'(k_1, k_2) = G(k_1) \wedge G(k_2), \quad \text{where } \wedge \text{ is bitwise AND.}$$

Consider the statistical test $\mathcal{A}(x) = \{\text{the first bit of } x\}$. What is the advantage $\text{PRGadv}[\mathcal{A}, G']$?

- a) G' is secure.
b) $1/2$.
c) $1/3$.
d) $1/4$.
e) 1.

9 Elliptic Curves and Pairing

9.1 Classical Definition

Probably, there is no need to explain the importance of elliptic curves. Essentially, the main group being used for cryptographic protocols is the group of points on an elliptic curve. If elliptic curve is “good enough”, then the discrete logarithm problem assumption, Diffie-Hellman assumption and other core cryptographic assumptions hold. Moreover, this group does not require a large field size, which is a huge advantage for many cryptographic protocols.

So, let us formally define what an elliptic curve is. Further assume that, when speaking of the finite field $\mathbb{F}_p$, the underlying prime number is greater than 3.⁶ The definition is the following.

Definition 9.1. Suppose that $\mathbb{K}$ is a field. An **elliptic curve** E over $\mathbb{K}$ is defined as a set of points $(x, y) \in \mathbb{K}^2$:

$$y^2 = x^3 + ax + b,$$

called a **Short Weierstrass equation**, where $a, b \in \mathbb{K}$ and $4a^3 + 27b^2 \neq 0$. We denote $E/\mathbb{K}$ to denote the elliptic curve over field $\mathbb{K}$.

Remark. One might wonder why $4a^3 + 27b^2 \neq 0$. This is due to the fact that the curve $y^2 = x^3 + ax + b$ might have certain degeneracies and special points, which are not desirable for us. So we require this condition to make $E/\mathbb{K}$ “good”.

Definition 9.2. We say that $P = (x_P, y_P) \in \mathbb{A}^2(\mathbb{K})$ is the **affine representation** of the point on the elliptic curve $E/\mathbb{K}$ if it satisfies the equation $y_P^2 = x_P^3 + ax_P + b$.

Example 9.1. Consider the curve $E/\mathbb{Q} : y^2 = x^3 - x + 9$. This is an elliptic curve. Consider $P = (0, 3), Q = (-1, -3) \in \mathbb{A}^2(\mathbb{Q})$: both are valid affine points on the curve. See [Figure 9.1](#).

Typically, our elliptic curve is defined over a finite field $\mathbb{F}_p$, so we are interested in this particular case.

Remark. Although, in many cases one might encounter the definition where an elliptic curve E is defined over the algebraic closure of $\mathbb{F}_p$, that is $E/\overline{\mathbb{F}_p}$. This is typically important when considering elliptic curve pairings. However, for the sake of simplicity, we will consider elliptic curves over $\mathbb{F}_p$ and

⁶Note that, for example, for $\mathbb{F}_{2^n}$ equation of elliptic curve is very different, but usually we do not deal with binary field elements.

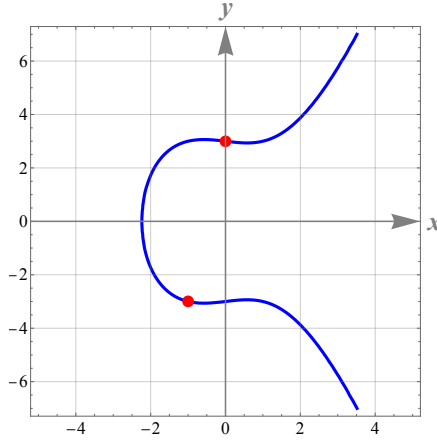


Figure 9.1: Elliptic curve $E/\mathbb{Q} : y^2 = x^3 - x + 9$ with points $P = (0, 3), Q = (-1, -3)$ depicted on it.

corresponding finite extensions $\mathbb{F}_{p^m}$ as of now.

Remark. It is easy to see that if $(x, y) \in E/\mathbb{K}$, then $(x, -y) \in E/\mathbb{K}$. We will use this fact intensively further.

Now, elliptic curves are useless without any operation defined on top of them. But as will be seen later, it is quite unclear how to define the identity element. For that reason, we introduce a bit different definition of a set of points on the curve.

Definition 9.3. The set of points on the curve, denoted as $E_{a,b}(\mathbb{K})$, is defined as:

$$E_{a,b}(\mathbb{K}) = \{(x, y) \in \mathbb{A}^2(\mathbb{K}) : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

where $\mathcal{O}$ is the so-called **point at infinity**.

Remark. The difference between $E(\mathbb{K})$ and $E/\mathbb{K}$ is that the former includes the point at infinity, while the latter does not. We also omit the index a, b , so instead of $E_{a,b}(\mathbb{K})$ we write simply $E(\mathbb{K})$.

Now, the reason we introduced the point at infinity $\mathcal{O}$ is because it allows us to define the group binary operation $\oplus$ on the elliptic curve. The operation is sometimes called the **chord-tangent law**. Let us define it.

Definition 9.4. Consider the curve $E(\mathbb{F}_{p^m})$. We define $\mathcal{O}$ as the identity element of the group. That is, for all points P , we set $P \oplus \mathcal{O} = \mathcal{O} \oplus P = P$. For any other non-identity elements $P = (x_P, y_P), Q = (x_Q, y_Q) \in E(\mathbb{F}_{p^m})$, define the $P \oplus Q = (x_R, y_R)$ as follows:

1. If $x_P \neq x_Q$, use the **chord method**. Define $\lambda := \frac{y_P - y_Q}{x_P - x_Q}$ — the slope

between P and Q . Set the resultant coordinates as:

$$x_R := \lambda^2 - x_P - x_Q, \quad y_R := \lambda(x_P - x_R) - y_P.$$

2. If $x_P = x_Q \wedge y_P = y_Q$ (that is, $P = Q$), use the **tangent method**.

Define the slope of the tangent at P as $\lambda := \frac{3x_P^2 + a}{2y_P}$ and set

$$x_R := \lambda^2 - 2x_P, \quad y_R := \lambda(x_P - x_R) - y_P.$$

3. Otherwise, define $P \oplus Q := \mathcal{O}$.

The aforementioned definition is illustrated in the Figure below⁷.

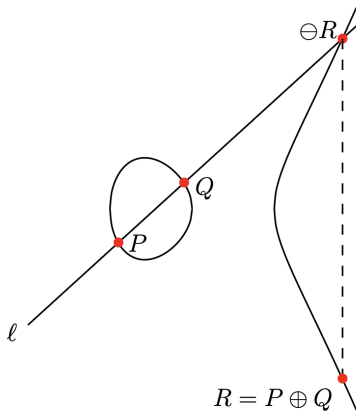


Figure 2.5: Elliptic curve addition.

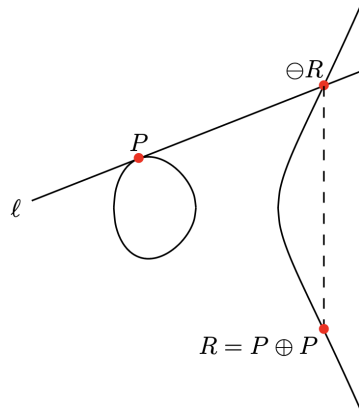


Figure 2.6: Elliptic curve doubling.

Example 9.2. Consider $E/\mathbb{R} : y^2 = x^3 - 2x$. The points $(-1, -1)$, $(0, 0)$, $(2, 2)$ are all on E and also on the line $\ell : y = x$. Therefore, $(-1, 1) \oplus (0, 0) = (2, -2)$ or, similarly, $(2, 2) \oplus (-1, -1) = (0, 0)$.

Now, let us compute $[2](-1, -1)$. Calculate the tangent slope as $\lambda := \frac{3 \cdot (-1)^2 - 2}{2 \cdot (-1)} = -\frac{1}{2}$. Thus, the tangent line has an equation $\ell' : y = -\frac{1}{2}x + c$.

Substituting $(-1, -1)$ into the equation, we get $c = -\frac{3}{2}$. Therefore, the equation of the tangent line is $y = -\frac{1}{2}x - \frac{3}{2}$. The intersection of the curve and the line is $(\frac{9}{4}, -\frac{21}{8})$, yielding $[2](-1, -1) = (\frac{9}{4}, -\frac{21}{8})$.

The whole illustration is depicted in Figure 9.2.

⁷Illustration taken from “Pairing for Beginners”

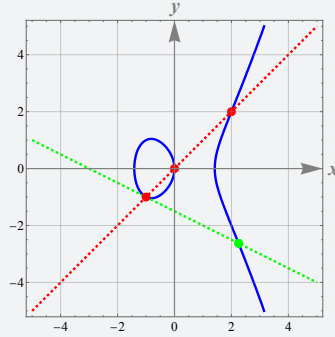


Figure 9.2: Illustration of the group law on the elliptic curve $E/\mathbb{R} : y^2 = x^3 - 2x$. In **red** we marked points lying on the line $\ell : y = x$. In **dashed red**, we marked the line ℓ , while in **dashed green** — the tangent line ℓ' at $(-1, -1)$, which is used to calculate $[2](-1, 1)$.

Theorem 9.5. $(E(\mathbb{F}_{p^m}), \oplus)$ forms an abelian group.

Proof Sketch. The identity element is $\mathcal{O}$. Every point $\mathcal{O} \neq P = (x_P, y_P) \in E(\mathbb{F}_{p^m})$ has an additive inverse: indeed, $\ominus P := (x_P, -y_P)$. Finally, a bit of algebra might show that the operation is associative. It is also clearly commutative: even geometrically it is evident, that the result of $P \oplus Q$ does not depend on the order of P and Q (“drawing a line between P and Q ” and “drawing a line between Q and P ” are equivalent statements). ■

Now, let us talk a bit about the group order. The group order is the number of elements in the group. For elliptic curves, the group order is typically denoted as r or n , but we are going to use r . Also, the following theorem is quite important.

Theorem 9.6. Define $r := |E(\mathbb{F}_{p^m})|$. Then, $r = p^m + 1 - t$ for some integer $|t| \leq 2\sqrt{p^m}$. A bit more intuitive explanation: the number of points on the curve is close to p^m . This theorem is commonly called the **Hasse’s Theorem**, and the value t is called the **Trace of Frobenius**.

Remark. In fact, $r = |E(\mathbb{F}_{p^m})|$ can be computed in polylogarithmic time $O(\log^\gamma(p^m))$ for some $\gamma > 1$. That being said, the number of points can be computed efficiently even for fairly large primes p .

Finally, let us define the scalar multiplication operation.

Definition 9.7. Let $P \in E(\mathbb{F}_{p^m})$ and $\alpha \in \mathbb{Z}_r$. Define the scalar multiplication $[\alpha]P$ as:

$$[\alpha]P = \underbrace{P \oplus P \oplus \cdots \oplus P}_{\alpha \text{ times}}.$$

Question. Why do we restrict α to $\mathbb{Z}_r$ and not to $\mathbb{Z}$?

9.2 Discrete Logarithm Problem on Elliptic Curves

Finally, as defined in the previous section, the **discrete logarithm** problem on the elliptic curve is the following: typically, $E(\mathbb{F}_p)$ is cyclic, meaning there exist some point $G \in E(\mathbb{F}_p)$, called the **generator**, such that $\langle G \rangle = E(\mathbb{F}_p)$. Given $P \in E(\mathbb{F}_p)$, the problem consists in finding such a scalar $\alpha \in \mathbb{Z}_r$ such that $[\alpha]G = P$.

Now, if the curve is “good”, then the discrete logarithm problem is hard. In fact, the best-known algorithms have a complexity $O(\sqrt{r})$. However, there are certain cases when the discrete log problem is much easier.

1. If r is composite, and all its prime factors are less than some bound $r_{\max}$, then the discrete log problem can be solved in $O(\sqrt{r_{\max}})$. For this very reason, typically r is prime.
2. If $|E(\mathbb{F}_p)| = p$, then the discrete logarithm can be solved in polynomial time. These curves are called **anomalous curves**.
3. Suppose that there is some small integer $\tau > 0$ such that $r \mid (p^\tau - 1)$. The discrete log in that case reduces to the discrete log in the finite field $\mathbb{F}_{p^\tau}$, which is typically not hard for small enough τ .

9.3 Relations

Before delving into the projective coordinates and further zero-knowledge topics, let us first discuss the concept of relations, which will be intensively used from now on. Now, what is a relation? The definition is incredibly concise.

Definition 9.8. Let $\mathcal{X}, \mathcal{Y}$ be some sets. Then, $\mathcal{R}$ is a **relation** if

$$\mathcal{R} \subset \mathcal{X} \times \mathcal{Y} = \{(x, y) : x \in \mathcal{X}, y \in \mathcal{Y}\}$$

Interpretation is approximately the following: suppose we have sets $\mathcal{X}$ and $\mathcal{Y}$. Then, relation $\mathcal{R}$ gives a set of pairs (x, y) , telling that $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ are *related*.

Example 9.3. Let $\mathcal{X} = \{\text{Oleksandr, Phat, Anton}\}$ and $\mathcal{Y} = \{\text{Backend, Frontend, Research}\}$. Define the following relation of “person x works in field y ”:

$\mathcal{R} = \{(\text{Oleksandr, Research}), (\text{Phat, Frontend}), (\text{Anton, Backend})\}$
Obviously, $\mathcal{R} \subset \mathcal{X} \times \mathcal{Y}$, so $\mathcal{R}$ is a relation.

Remark. There are many ways to express that $(x, y) \in \mathcal{R}$. Most common are $x\mathcal{R}y$ and $x \sim y$. Also, sometimes, one might encounter relation definition as a boolean function $\mathcal{R} : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$, where $\mathcal{R}(x, y)$ is 1 if (x, y) is in the relation, and 0 otherwise.

Further, we will use notation $x \sim y$ to denote that $(x, y) \in \mathcal{R}$.

Example 9.4. Let E be a cyclic group of points on the Elliptic Curve of order $r \geq 2$ with a generator $\langle G \rangle = E$. Let $\mathcal{X} = \mathbb{Z}_r$ and $\mathcal{Y} = E$. Define a relation $\mathcal{R} \subset \mathcal{X} \times \mathcal{Y}$ by:

$$\mathcal{R} = \{(\alpha, P) \in \mathbb{Z}_r \times E : [\alpha]G = P\}$$

Essentially, such a relation is a set of secret keys α and corresponding public keys P . In this case, for example, $0\mathcal{R}\mathcal{O}$ and $1\mathcal{R}G$ or $0 \sim \mathcal{O}$ and $1 \sim G$.

Remark. When we say that $\sim$ is a relation on a set $\mathcal{X}$, we mean that $\sim$ is a relation $\mathcal{R}$ on the following Cartesian product: $\mathcal{R} \subset \mathcal{X} \times \mathcal{X}$.

Remark. The provided example is relevant in most cases (ecdsa, eddsa, schnorr signatures etc.). But for some algorithms, the relation between secret key α and public key P can be defined as:

$$\mathcal{R} = \{(\alpha, P) \in \mathbb{Z}_r \times E : \ominus[\alpha]G = P\}$$

for DSTU 4145 standard or even:

$$\mathcal{R} = \{(\alpha, P) \in \mathbb{Z}_r \times E : [\alpha^{-1}]G = P\}$$

for twisted ElGamal algorithm.

Now, let us formally define the term **equivalence relation**.

Definition 9.9. Let $\mathcal{X}$ be a set. A relation $\sim$ on $\mathcal{X}$ is called an **equivalence relation** if it satisfies the following properties:

1. **Reflexivity:** $x \sim x$ for all $x \in \mathcal{X}$.
2. **Symmetry:** If $x \sim y$, then $y \sim x$ for all $x, y \in \mathcal{X}$.
3. **Transitivity:** If $x \sim y$ and $y \sim z$, then $x \sim z$ for all $x, y, z \in \mathcal{X}$.

Example 9.5. Let $\mathcal{X}$ be the set of all people. Define a relation $\sim$ on $\mathcal{X}$ by $x \sim y$ if $x, y \in \mathcal{X}$ have the same birthday. Then $\sim$ is an equivalence relation on $\mathcal{X}$. Let us demonstrate that:

1. **Reflexivity:** $x \sim x$ since x has the same birthday as x .
2. **Symmetry:** If $x \sim y$, then $y \sim x$ since x has the same birthday as y .
3. **Transitivity:** If $x \sim y$ and $y \sim z$, then $x \sim z$ since x has the same birthday as y and y has the same birthday as z .

Example 9.6. Suppose $\mathcal{X} = \mathbb{Z}$ and n is some fixed integer. Let $a \sim b$ mean that $a \equiv b \pmod{n}$. It is easy to verify that $\sim$ is an equivalence relation:

1. **Reflexivity:** $a \equiv a \pmod{n}$, so $a \sim a$.
2. **Symmetry:** If $a \equiv b \pmod{n}$, then $b \equiv a \pmod{n}$, so $b \sim a$.
3. **Transitivity:** If $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$, then $a \equiv c \pmod{n}$. It is not that obvious, so we can prove it: from the first equality we have $\exists q \in \mathbb{Z} : a - b = nq$. From the second, $\exists r \in \mathbb{Z} : b - c = nr$. Adding both we get $(a - b) + (b - c) = n(r + q)$ or, equivalently, $a - c = n(r + q)$, meaning $a \equiv c \pmod{n}$.

The example below is less obvious with a bit more difficult proof, which we will skip. Yet, it is quite curious, so here it is.

Example 9.7. Let $\mathcal{G}$ be the set of all possible groups. Define a relation $\sim$ on $\mathcal{G}$ by $\mathbb{G} \sim \mathbb{H}$ if $\mathbb{G} \cong \mathbb{H}$ (in other words, $\mathbb{G}$ and $\mathbb{H}$ are isomorphic). Then $\sim$ is an equivalence relation.

Now, suppose I give you a set $\mathcal{X}$ with some equivalence relation $\sim$ (say, $\mathcal{X} = \mathbb{Z}$ and $a \equiv b \pmod{n}$). Notice that you can find some subset $\mathcal{X}' \subset \mathcal{X}$ in which all elements are equivalent (and any other element from $\mathcal{X} \setminus \mathcal{X}'$ is not). In the case of modulo relation above, $\mathcal{X}'$ could be the set of all integers that are congruent to 1 modulo n , so $\mathcal{X}' = \{\dots, -n + 1, 1, n + 1, 2n + 1, \dots\}$. This way, we can partition the set $\mathcal{X}$ into disjoint subsets, where all elements in each subset are equivalent. Such subsets are called **equivalence classes**. Now, let us give a formal definition.

Definition 9.10. Let $\mathcal{X}$ be a set and $\sim$ be an equivalence relation on $\mathcal{X}$. For any $x \in \mathcal{X}$, the **equivalence class** of x is the set

$$[x] = \{y \in \mathcal{X} : x \sim y\}$$

The **set of all equivalence classes** is denoted by $\mathcal{X}/\sim$ (or, if the relation $\mathcal{R}$ is given explicitly, then $\mathcal{X}/\mathcal{R}$), which is read as “ $\mathcal{X}$ modulo relation $\sim$ ”.

Example 9.8. Let $\mathcal{X} = \mathbb{Z}$ and n be some fixed integer. Define $\sim$ on $\mathcal{X}$ by $x \sim y$ if $x \equiv y \pmod{n}$. Then the equivalence class of x is the set

$$[x] = \{y \in \mathbb{Z} : x \equiv y \pmod{n}\}$$

For example, $[0] = \{\dots, -2n, -n, 0, n, 2n, \dots\}$ while $[1] = \{\dots, -2n + 1, -n + 1, 1, n + 1, 2n + 1, \dots\}$.

Now, as we have said before, a set of all equivalence classes form a partition of the set $\mathcal{X}$. This means that any element $x \in \mathcal{X}$ belongs to exactly one equivalence class. This is a very important property, which we will use in the next section.

Formally, we have the following lemma.

Lemma 9.11. Let $\mathcal{X}$ be a set and $\sim$ be an equivalence relation on $\mathcal{X}$. Then,

1. For each $x \in \mathcal{X}$, $x \in [x]$ (quite obvious, follows from reflexivity).
2. For each $x, y \in \mathcal{X}$, $x \sim y$ if and only if $[x] = [y]$.
3. For each $x, y \in \mathcal{X}$, either $[x] = [y]$ or $[x] \cap [y] = \emptyset$.

Example 9.9. Let $n \in \mathbb{N}$ and, again, $\mathcal{X} = \mathbb{Z}$ with a “modulo n ” equivalence relation $\mathcal{R}_n$. Define the equivalence class of x by $[x]_n = \{y \in \mathbb{Z} : x \equiv y \pmod{n}\}$. Then,

$$\mathbb{Z}/\mathcal{R}_n = \{[0]_n, [1]_n, [2]_n, \dots, [n-2]_n, [n-1]_n\}$$

forms a partition of $\mathbb{Z}$, that is

$$\bigcup_{i=0}^{n-1} [i]_n = \mathbb{Z},$$

and for all $i, j \in \{0, 1, \dots, n-1\}$, if $i \neq j$, then $[i]_n \cap [j]_n = \emptyset$. Commonly, we denote the set of all equivalence classes as $\mathbb{Z}/n\mathbb{Z}$ or, as we got used to, $\mathbb{Z}_n$. Moreover, we can naturally define the addition as:

$$[x]_n + [y]_n = [x + y]_n$$

Then, the set $(\mathbb{Z}/n\mathbb{Z}, +)$ with the defined addition is a group.

The primary reason we considered equivalence relations is that we will define the projective space as a set of equivalence classes. Besides this, when defining proofs of knowledge, argument of knowledge and zero-knowledge protocols, we will use the concept of relations and equivalence relations intensively.

9.4 Elliptic Curve in Projective Coordinates

9.4.1 Projective Space

Recall that we defined the elliptic curve as

$$E(\overline{\mathbb{F}}_p) := \{(x, y) \in \mathbb{A}^2(\overline{\mathbb{F}}_p) : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

The above definition is the definition of the elliptic curve in *the affine space*. However, notice that in this case we need to append a somewhat artificial point $\mathcal{O}$ to the curve. This is done to make the curve a group since without this point it is unclear how to define addition of two, say, negative points on the curve (since the resultant vertical line does not intersect the curve at any other point). The way to unify all the points $E/\overline{\mathbb{F}}_p$ with this magical point at infinity $\mathcal{O}$ is to use the **projective space**.

Essentially, instead of working with points in affine n -space (in our case, with two-dimensional points $\mathbb{A}^2(\mathbb{K})$), we work with lines that pass through the origin

in $(n + 1)$ -dimensional space (in our case, 3-dimensional space $\mathbb{A}^3(\mathbb{K})$). We say that two points from this $(n + 1)$ -dimensional space are **equivalent** if they lie on the same line that passes through the origin (we will show the illustration a bit later).

It seems strange that we need to work with 3-dimensional space to describe 2-dimensional points, but this is the way to unify all the points on the curve. Because, in this case, the point at infinity is represented by a set of points on the line that passes through the origin and is parallel to the y -axis. We will get to understanding how to interpret that. Moreover, by defining operations on the projective space, we can make the operations on the curve more efficient.

Now, to the formal definition.

Definition 9.12. Projective coordinate, denoted as $\mathbb{P}^2(\mathbb{K})$ (or sometimes simply $\mathbb{K}\mathbb{P}^2$) is a triple of elements $(X : Y : Z)$ from $\mathbb{A}^3(\mathbb{K}) \setminus \{0\}$ modulo the equivalence relation^a:

$(X_1 : Y_1 : Z_1) \sim (X_2 : Y_2 : Z_2)$ if and only if exists such $\lambda \in \mathbb{K}$ that the following holds $(X_1 : Y_1 : Z_1) = (\lambda X_2 : \lambda Y_2 : \lambda Z_2)$

^aAlthough we specify the definition for $n = 2$, the definition can be generalized to any $\mathbb{P}^n(\mathbb{K})$.

This definition on itself might be a bit too abstract, so let us consider the concrete example for projective space $\mathbb{P}^2(\mathbb{R})$.

Example 9.10. Consider the projective space $\mathbb{P}^2(\mathbb{R})$. Then, two points $(x_1, y_1, z_1), (x_2, y_2, z_2) \in \mathbb{R}^3$ are equivalent if there exists $\lambda \in \mathbb{R}$ such that $(x_1, y_1, z_1) = (\lambda x_2, \lambda y_2, \lambda z_2)$. For example, $(1, 2, 3) \sim (2, 4, 6)$ since $(1, 2, 3) = 0.5(2, 4, 6)$.

Example 9.11. Now, how to geometrically interpret $\mathbb{P}^2(\mathbb{R})$? Consider the Figure below.

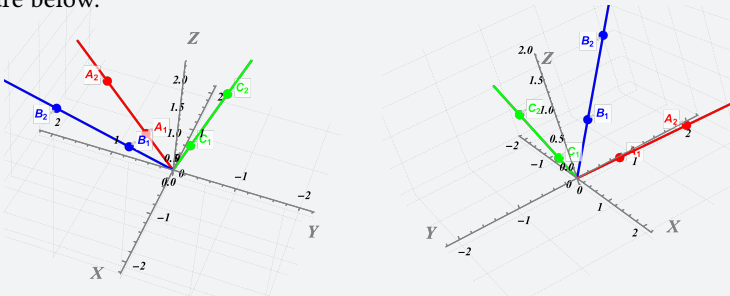


Illustration: Geometric interpretation of $\mathbb{P}^2(\mathbb{R})$, the same scene from different perspectives. The **red** line is represented by equation $(2t, 3t, t)$, **blue** line by $(-2t, 3t, 3t)$, and **green** line is represented by $(t, -2t, 5t)$ for parameter $t \in \mathbb{R}$.

Here, the figure demonstrates three equivalence classes, being a set of points on the **red**, **blue**, and **green** lines (except for the origin).

The reason why geometrically the set of equivalence classes lie on the same line that passes through the origin is following: suppose we have a point $\vec{v}_0 = (x_0, y_0, z_0) \in \mathbb{R}^3$, represented as a vector. Then, the set of all points that are equivalent to (x_0, y_0, z_0) is the set of all points $(\lambda x_0, \lambda y_0, \lambda z_0) = \lambda \vec{v}_0$ for $\lambda \in \mathbb{R} \setminus \{0\}$. So $\vec{v}_0$ is the representative of equivalence class $[\vec{v}_0] = \{\lambda \vec{v}_0 : \lambda \in \mathbb{R}, \lambda \neq 0\}$. Now notice, that this is a parametric equation of a line that passes through the origin and the point $\vec{v}_0$: notice that for $\lambda = 0$ (if we assume that expression is also defined for zero λ) we have the origin $\vec{0}$, while for $\lambda = 1$ we have the point $\vec{v}_0$. Then, any other values of λ in-between $[0, 1]$ or outside define the set of points lying on the same line.

Now, projective coordinates are not that useful unless we can come back to the affine space. This is done by defining the map $\phi : \mathbb{P}^2(\mathbb{K}) \rightarrow \mathbb{A}^2(\mathbb{K})$ as follows: $\phi : (X : Y : Z) \mapsto (X/Z, Y/Z)$. If, in turn, we want to go from the affine space to the projective space, we can define the map $\psi : \mathbb{A}^2(\mathbb{K}) \rightarrow \mathbb{P}^2(\mathbb{K})$ as follows: $\psi : (x, y) \mapsto (x : y : 1)$. Geometrically, map ϕ means that we take a point $(X : Y : Z)$ and project it onto the plane $Z = 1$.

Example 9.12. Again, consider three lines from the previous example. Now, we additionally draw a plane $\pi : z = 1$ in our 3-dimensional space (see Illustration below).

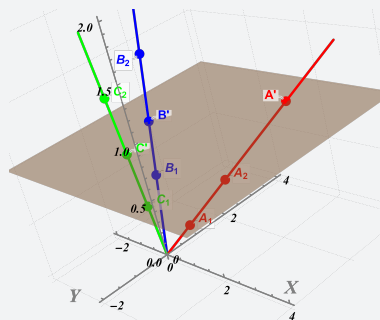


Illustration: Geometric interpretation of converting projective to affine form.

By using the map $(X : Y : Z) \mapsto (X/Z, Y/Z)$, all points on the line get mapped to the intersection of the line with the plane $\pi : z = 1$. This way, for example, points on the **red line** ℓ_{red} get mapped to the point $A' = (2, 3, 1)$, corresponding to $(2, 3)$ in affine coordinates. So, for example, point $(6, 9, 3) \in \ell_{\text{red}}$, lying on the same line, gets mapped to $(6/3, 9/3) = (2, 3)$.

Similarly, all **blue line** points get mapped to the point $B' = (-2/3, 1, 1)$, while all **green line** points get mapped to the point $C' = (0.2, -0.4, 1)^a$.

^aOne can verify that based on the equations provided from the previous example

9.4.2 Elliptic Curve Equation in Projective Form

Now, quite an interesting question is following: how to represent (basically, rewrite) the “affine” elliptic curve equation⁸

$$E_{\mathbb{A}}(\overline{\mathbb{F}}_p) : y^2 = x^3 + ax + b, \quad a, b \in \overline{\mathbb{F}}_p$$

in the projective form? Since currently, we defined the curve as the 2D curve, but now we are working in 3D space! The answer is following: recall that if $(X : Y : Z) \in \mathbb{P}^2(\overline{\mathbb{F}}_p)$ lies on the curve, so does the point $(X/Z, Y/Z)$. The condition on the latter point to lie on $E_{\mathbb{A}}(\overline{\mathbb{F}}_p)$ is following:

$$\left(\frac{Y}{Z}\right)^2 = \left(\frac{X}{Z}\right)^3 + a \cdot \frac{X}{Z} + b$$

But now multiply both sides by Z^3 to get rid of the fractions:

$$E_{\mathbb{P}}(\overline{\mathbb{F}}_p) : Y^2Z = X^3 + aXZ^2 + bZ^3$$

This is an equation of the elliptic curve in **projective form**.

Now, one of the motivations to work with the projective form was to unify affine points $E_{\mathbb{A}}/\overline{\mathbb{F}}_p$ and the point at infinity $\mathcal{O}$, which acted as an identity element in the group $E_{\mathbb{A}}(\overline{\mathbb{F}}_p)$. So how do we encode the point at infinity in the projective form?

Well, notice the following observation: all points $(0 : \lambda : 0)$ always lie on the curve $E_{\mathbb{P}}(\overline{\mathbb{F}}_p)$. Moreover, the map from the projective form to the affine form is ill-defined for such points, since we would need to divide by zero. So, we can naturally make the points $(0 : \lambda : 0)$ to be the set of points at infinity. This way, we can define the point at infinity as $\mathcal{O} = (0 : 1 : 0)$.

Finally, let us summarize what we have observed so far.

Definition 9.13. The **homogenous projective form of the elliptic curve** $E_{\mathbb{P}}(\overline{\mathbb{F}}_p)$ is defined as the set of all points $(X : Y : Z) \in \mathbb{P}^2(\overline{\mathbb{F}}_p)$ in the projective space that satisfy the equation

$$E_{\mathbb{P}}(\overline{\mathbb{F}}_p) : Y^2Z = X^3 + aXZ^2 + bZ^3, \quad a, b \in \overline{\mathbb{F}}_p,$$

where the point at infinity is encoded as $\mathcal{O} = (0 : 1 : 0)$.

Example 9.13. Consider the BN254 curve $y^2 = x^3 + 3$ over reals $\mathbb{R}$. Its projective form is given by the equation $Y^2Z = X^3 + 3Z^3$, which gives a surface, depicted below.

⁸Further, we will use notation $E_{\mathbb{A}}$ to represent the elliptic curve equation in the affine form, and $E_{\mathbb{P}}$ to represent the elliptic curve in the projective form.

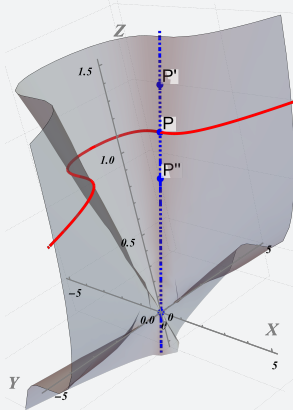


Illustration: BN254 Curve Elliptic Curve in Projective Form over $\mathbb{R}$. In gray is the surface, while red points are the points on the affine curve (lying on the plane $\pi : z = 1$).

Points $P' \approx (0 : 2.165 : 1.25)$ and $P'' \approx (0 : 1.3 : 0.75)$ in projective form both lie on the curve and get mapped to the same point $P \approx (0, 1.732)$ in affine coordinates.

9.4.3 General Projective Coordinates

Hold on, but why did we use the term *homogenous*? The reason why is because we defined equivalence as follows: $(X : Y : Z) \sim (\lambda X : \lambda Y : \lambda Z)$ for some $\lambda \in \overline{\mathbb{K}}$, called **homogenous coordinates**. However, this is not the only way to define equivalence. Consider a more general form of equivalence relation:

$$(X : Y : Z) \sim (X' : Y' : Z') \text{ iff } \exists \lambda \in \overline{\mathbb{K}} : (X, Y, Z) = (\lambda^n X', \lambda^m Y', \lambda Z')$$

In this case, to come back to the affine form, we need to use the map $\phi : (X : Y : Z) \mapsto (X/Z^n, Y/Z^m)$.

Example 9.14. The case $n = 2, m = 3$ is called the **Jacobian Projective Coordinates**. An Elliptic Curve equation might be then rewritten as:

$$Y^2 = X^3 + aXZ^4 + bZ^6$$

The reason why we might want to use such coordinates is that they can be more efficient in some operations, such as point addition. However, we will not delve into this topic much further.

Example 9.15. Consider the BN254 curve $y^2 = x^3 + 3$ over reals $\mathbb{R}$, again. Its *Jacobian projective form* is given by the equation $Y^2 = X^3 + 3Z^6$, which

gives a surface, depicted below.

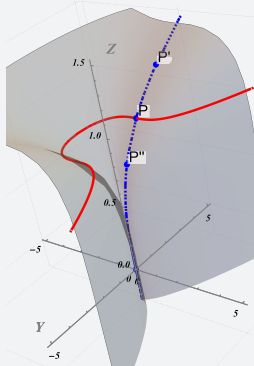


Illustration: BN254 Curve Elliptic Curve in Jacobian Projective Form over $\mathbb{R}$. In gray is the surface, while red points are the points on the affine curve (lying on the plane $\pi : z = 1$).

Notice that now, under the map $(X : Y : Z) \mapsto (X/Z^2, Y/Z^3)$, points in the same equivalence class (in $\mathbb{R}^3$) do not lie on the same line, but rather on the same *curve*. Namely, equivalence class has a form $[(x_0, y_0, z_0)] = \{t^2 x_0, t^3 y_0, t z_0 : t \in \mathbb{R} \setminus \{0\}\}$.

9.4.4 Fast Addition

Let us come back to the affine case and assume that the underlying field is the prime field $\mathbb{F}_p$. Recall that for adding two points $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ to get $R = (x_R, y_R) \leftarrow P \oplus Q$ one used the following formulas (there is no need to understand the derivation fully, just take it as a fact):

$$x_R \leftarrow \left(\frac{y_Q - y_P}{x_Q - x_P} \right)^2 - x_P - x_Q, \quad y_R \leftarrow \left(\frac{y_Q - y_P}{x_Q - x_P} \right) (x_P - x_R) - y_P$$

Denote by **M** the cost of multiplication, by **S** the cost of squaring, and by **I** the cost of inverse operation in $\mathbb{F}_p$. Note that we do not count addition/inverse costs as they are significantly lower than operations listed. Then, the cost of adding two points using above formula is $2\mathbf{M} + 1\mathbf{S} + 1\mathbf{I}$. Indeed, our computation can proceed as follows:

1. Calculate $t_1 \leftarrow (x_Q - x_P)^{-1}$, costing **1I**.
2. Calculate $\lambda \leftarrow (y_Q - y_P)t_1$, costing **1M**.
3. Calculate $t_2 \leftarrow \lambda^2$, costing **1S**.
4. Calculate $x_R \leftarrow t_2 - x_P - x_Q$, costing almost nothing.
5. Calculate $y_R \leftarrow \lambda(x_P - x_R) - y_P$, costing **1M**.

Well, there are just 4 operations in total, so what can go wrong? The problem is that we need to calculate the inverse of $(x_Q - x_P)$, which is a very, very costly

operation. In fact, typically $1\text{I} \gg 20\text{M}$ or even worse, the ratio might reach 80 in certain cases.

Now imagine we want to add 4 points, say $P_1 \oplus P_2 \oplus P_3 \oplus P_4$: this costs $6\text{M} + 3\text{S} + 3\text{I}$. Now we have 3 inverses, which is a lot. Finally, if we are to add much larger number of points (for example, when finding the scalar product), this gets even worse.

Projective coordinates is a way to solve this problem. The idea is to represent points in the projective form $(X : Y : Z)$, so when adding two numbers in projective form, you still get a point in a form $(X : Y : Z)$. Then, after conducting a series of additions, you can convert the point back to the affine form.

But why adding two points, say, $(X_P : Y_P : Z_P)$ and $(X_Q : Y_Q : Z_Q)$, in the projective form is more efficient? We will not derive the formulas, but trust us that they have the following form:

$$\begin{aligned} X_R &= (X_P Z_Q - X_Q Z_P)(Z_P Z_Q (Y_P Z_Q - Y_Q Z_P)^2 - (X_P Z_Q - X_Q Z_P)^2 (X_P Z_Q + X_Q Z_P)); \\ Y_R &= Z_P Z_Q (X_Q Y_P - X_P Y_Q)(X_P Z_Q - X_Q Z_P)^2 - \\ &\quad - (Y_P Z_Q - Y_Q Z_P)((Y_P Z_Q - Y_Q Z_P)^2 Z_P Z_Q - (X_P Z_Q + X_Q Z_P)(X_P Z_Q - X_Q Z_P)^2); \\ Z_R &= Z_P Z_Q (X_P Z_Q - X_Q Z_P)^3. \end{aligned}$$

Do not be afraid, you do not need to understand how this formula is derived. But notice that despite the very scary look, there is no inversions involved! Moreover, this formula can be calculated in only $12\text{M} + 2\text{S}$! So all in all, this is much more effective than $2\text{M} + 1\text{S} + 1\text{I}$.

The only inversion which is unavoidable in the projective form is the inversion of Z since after all additions (and doublings) have been made, we need to use map $(X : Y : Z) \mapsto (X/Z, Y/Z)$ to return back to the affine form. However, this inversion is done only once at the end of the computation, so it is not that costly.

Proposition 9.14. To conclude, typically, when working with elliptic curves, one uses the following strategy:

1. Convert affine points to projective form using the map $(x, y) \mapsto (x, y, 1)$.
2. Perform all operations in the projective form, which do not involve inversions.
3. Convert the result back to the affine form using the map $(X : Y : Z) \mapsto (X/Z, Y/Z)$.

This is illustrated in the Figure below.

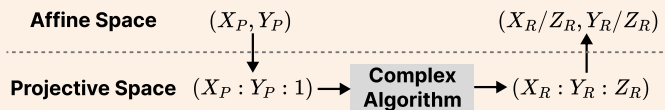


Illustration: General strategy when performing operations over Elliptic Curves.

9.4.5 Scalar Multiplication Basic Implementation

Now, the question is: how do we implement the scalar multiplication $[k]P$ for the given scalar $k \in \mathbb{Z}_r$ and point $P \in E(\mathbb{F}_q)$?

First idea: let us simply add P to itself k times. Well, the complexity would be $O(k)$ in this case, which is even harder than solving the discrete logarithm problem (recall that the discrete logarithm problem has a complexity of $O(\sqrt{k})$). Yikes.

So there should be a better way. In this section we will limit ourselves to the double-and-add method, but the curious reader can look up the NAF (Non-Adjacent Form) method, windowed methods, GLV scalar decomposition and many other methods, which we are not going to cover in this course.

The idea of the double-and-add method is following: we represent the N -bit scalar k in binary form, say $k = (k_{N-1}, k_{N-2}, \dots, k_0)_2$, then we calculate $P, [2]P, [4]P, [8]P, \dots, [2^{N-1}]P$ (which is simply applying the doubling multiple times) and then add the corresponding points (corresponding to positions where $k_i = 1$) to get the result. Formally, we specify the [Algorithm 3](#).

Algorithm 3: Double-and-add method for scalar multiplication

Input : $P \in E(\mathbb{F}_q)$ and $k \in \mathbb{Z}_r$

Output: Result of scalar multiplication $[k]P \in E(\mathbb{F}_q)$

```

1 Decompose  $k$  to the binary form:  $(k_0, k_1, \dots, k_{N-1})$ 
2  $R \leftarrow \mathcal{O}$ 
3  $T \leftarrow P$ 
4 for  $i \in \{0, \dots, N-1\}$  do
5   if  $k_i = 1$  then
6      $R \leftarrow R \oplus T$ 
7   end
8    $T \leftarrow [2]T$ 
9 end
```

Return: Point R

Good news: now we have a complexity of $O(N) = O(\log k)$, which is way much better than a linear one. In fact, many more optimized methods have the same asymptotic complexity (meaning, a logarithmic one), so it turns out that we cannot do much better than that. However, the main advantages of other, more optimized methods is that we can avoid making too many additions (here, in the worst case, we have to make N additions), which is a costly operation (and more expensive than doubling).

Moreover, here we can use projective coordinates to make the addition and doubling operation more efficient! After all, typically the number of operations is even more than 300, so making 300 inversions in affine form is not an option.

9.5 Elliptic Curve Pairing

Pairing is the core object used in threshold signatures, zk-SNARKs constructions, and other cryptographic applications.

Consider the *Decisional Diffie-Hellman problem* which we described in [Section 8](#) (based on g^α, g^β and g^γ , decide whether $\gamma = \alpha\beta$). Turns out that for curves where the so-called *embedding degree*⁹ is small enough, this problem is easy to solve. This might sound like a quite bad thing, but it turns out that although some information about the discrete logarithm is leaked, it is not enough to break the security of the system (basically, solve the *Computational Diffie-Hellman problem*). Pairings is the exact object that allows us to solve the Decisional Diffie-Hellman problem.

However, a more interesting use-case which we are going to use in SNARKs is that pairings allows us to check **quadratic conditions** on scalars using their corresponding elliptic curve representation. For example, just given $u = g^\alpha, v = g^\beta$ we can check whether $\alpha\beta + 5 = 0$ (which is impossible to check without having a pairing).

So what is pairing?

Definition 9.15. Pairing is a bilinear, non-degenerate, efficiently computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_1, \mathbb{G}_2$ are two groups (typically, elliptic curve groups) and $\mathbb{G}_T$ is a target group (typically, a set of scalars). Let us decipher the definition:

- **Bilinearity** means essentially the following:

$$e([a]P, [b]Q) = e([ab]P, Q) = e(P, [ab]Q) = e(P, Q)^{ab}.$$

- **Non-degeneracy** means that $e(G_1, G_2) \neq 1$ (where G_1, G_2 are generators of $\mathbb{G}_1, \mathbb{G}_2$, respectively). This property basically says that the pairing is not trivial.
- **Efficient computability** means that the pairing can be computed in a reasonable time.

The definition is illustrated in [Figure 9.3](#).

⁹We will mention what that is later, but still this term is quite hard to define.

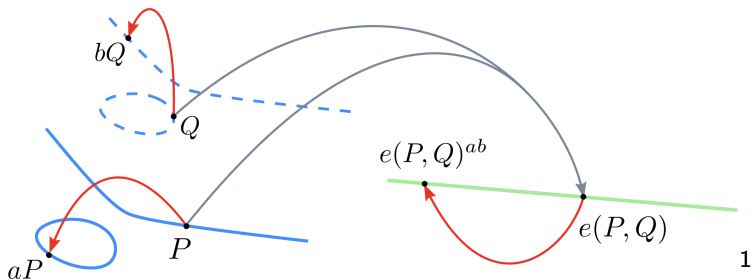


Figure 9.3: Pairing illustration. It does not matter what we do first: (a) compute $[a]P$ and $[b]Q$ and then compute $e([a]P, [b]Q)$ or (b) first calculate $e(P, Q)$ and then transform it to $e(P, Q)^{ab}$. Figure taken from “Pairings in R1CS” talk by Youssef El Housni

Example 9.16. Suppose $\mathbb{G}_1 = \mathbb{G}_2 = \mathbb{G}_T = \mathbb{Z}_r$ are scalars. Then, the map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, defined as:

$$e(x, y) = 2^{xy}$$

is pairing. Indeed, it is bilinear. For example, $e(ax, by) = 2^{abxy} = (2^{xy})^{ab} = e(x, y)^{ab}$ or $e(ax, by) = 2^{abxy} = 2^{(x)(aby)} = e(x, aby)$. Moreover, it is non-degenerate, since $e(1, 1) = 2 \neq 1$. And finally, it is obviously efficiently computable.

However, this is a quite trivial example since working over integers is typically not secure. For example, the discrete logarithm over $\mathbb{Z}_r$ can be solved in subexponential time. For that reason, we want to build pairings over elliptic curves.

Example 9.17. Pairing for BN254. For BN254 (with equation $y^2 = x^3 + 3$), the pairing function $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is defined over the following groups:

- $\mathbb{G}_1$ – points on the regular curve $E(\mathbb{F}_p)$.
- $\mathbb{G}_2$ – r -torsion points on the twisted curve $E'(\mathbb{F}_{p^2})$ over the field extension $\mathbb{F}_{p^2}$ (with equation $y^2 = x^3 + \frac{3}{\xi}$ for $\xi = 9 + u \in \mathbb{F}_{p^2}$).
- $\mathbb{G}_T$ – r th roots of unity $\Omega_r \subset \mathbb{F}_{p^{12}}^\times$.

Well, this one is quite intense and even understanding the input and output parameters is quite hard. So let us decipher some components:

- **r -torsion subgroup on the curve** $E(\mathbb{F}_{p^m})$ is simply a set of points, which multiplied by r give the point at infinity (that is, $[r]P = \mathcal{O}$). Formally, $E(\mathbb{F}_{p^m})[r] = \{P \in E(\mathbb{F}_{p^m}) : [r]P = \mathcal{O}\}$. Of course, for the curve $E(\mathbb{F}_p)$, the r -torsion subgroup is simply the whole curve, but that is generally not the case for the twisted curve over the extension field.

- **r th roots of unity** is a set of elements $\Omega_r = \{z \in \mathbb{F}_{p^{12}}^\times : z^r = 1\}$. This is a group under multiplication, and it has exactly r elements.

Remark. One might ask a reasonable question: where does this 12 come from? The answer is following: the so-called **embedding degree** of BN254 curve is $k = 12$. This number is the key to understanding why we are working over such large extensions when calculating the pairing. The formal description is quite hard, but the intuition is following: the embedding degree is the smallest number k such that all the r th roots of unity lie inside the extended field $\mathbb{F}_{p^k}$. If k was smaller, the output of pairing would contain less than r points and some points would be missing, which would make the pairing more trivial. For that reason, we need to have $\Omega_r \subset \mathbb{F}_{p^k}$.

Definition 9.16. The following conditions are equivalent **definitions** of an embedding degree k of an elliptic curve $E(\overline{\mathbb{F}}_p)$:

- k is the smallest positive integer such that $r \mid (p^k - 1)$.
- k is the smallest positive integer such that $\mathbb{F}_{p^k}$ contains all of the r -th roots of unity in $\overline{\mathbb{F}}_p$, that is $\Omega_r \subset \mathbb{F}_{p^k}$.
- k is the smallest positive integer such that $E(\overline{\mathbb{F}}_p)[r] \subset E(\mathbb{F}_{p^k})$

Pretty obvious observation: lower embedding degree is faster to work with, since it allows us to work over smaller fields. But usually, this embedding degree is quite large and we need to craft elliptic curves specifically to have a small embedding degree. For example, a pretty famous curve `secp256k1` has an embedding degree

$k = 0x2aaaaaaaaaaaaaaaaaaaaaaaaaaaaa74727a26728c1ab49ff8651778090ae0$, which is 254-bit long. For that reason, it is natural to define the term *pairing-friendly elliptic curve*.

Definition 9.17. An elliptic curve is called **pairing-friendly** if it has a relatively small embedding degree k (typically, $k \leq 16$).

Remark. One might ask why usually, when dealing with pairings, we do not get to work with field extensions that much (most likely, if you were to write `groth16` from scratch using some mathematical libraries, you will not need to work with $\mathbb{F}_{p^{12}}$ arithmetic specifically). The reason is that typically, libraries implement the following abstraction: given a set of points $\{(P_i, Q_i)\}_{i=1}^n \subset \mathbb{G}_1^n \times \mathbb{G}_2^n$, the function checks whether

$$\prod_{i=1}^n e(P_i, Q_i) = 1$$

Note that in this case, we do not need to work with $\mathbb{F}_{p^{12}}$ arithmetic, but rather checking the equality in the target group $\mathbb{G}_T$.

Interesting fact: this condition is specified in the ecpairing precompile standard used in Ethereum.

9.5.1 Case Study: BLS Signature

One of the most elegant applications of pairings is the BLS Signature scheme. Compared to ECDSA or other signature schemes, BLS can be formulated in three lines.

Suppose we have pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ (with generators G_1, G_2 , respectively), and a hash function H , mapping message space $\mathcal{M}$ to $\mathbb{G}_1$.

Definition 9.18. BLS Signature Scheme consists of the following algorithms:

- $\text{Gen}(1^\lambda)$: Key generation. $\text{sk} \leftarrow \mathbb{Z}_q$, $\text{pk} \leftarrow [\text{sk}]G_2 \in \mathbb{G}_2$.
- $\text{Sign}(\text{sk}, m)$. Signature is $\sigma \leftarrow [\text{sk}]H(m) \in \mathbb{G}_1$.
- $\text{Verify}(\text{pk}, m, \sigma)$. Check whether $e(H(m), \text{pk}) = e(\sigma, G_2)$.

Let us check the correctness:

$$e(\sigma, G_2) = e([\text{sk}]H(m), G_2) = e(H(m), [\text{sk}]G_2) = e(H(m), \text{pk})$$

As we see, the verification equation holds.

Remark. $\mathbb{G}_1$ and $\mathbb{G}_2$ might be switched: public keys might live instead in $\mathbb{G}_1$ while signatures in $\mathbb{G}_2$.

This scheme is also quite famous for its aggregation properties, which we are not going to consider today.

9.5.2 Case Study: Verifying Quadratic Equations

Example 9.18. Suppose Alice wants to convince Bob that he knows such α, β such that $\alpha + \beta = 2$, but she does not want to reveal α, β . She can do the following trick:

1. Alice computes $P \leftarrow [\alpha]G, Q \leftarrow [\beta]G$ — points on the curve.
2. Alice sends (P, Q) to Bob.
3. Bob verifies whether $P \oplus Q = [2]G$.

It is easy to verify the correctness of the scheme: suppose Alice is honest and she sends the correct values of α, β , satisfying $\alpha + \beta = 2$. Then, $P \oplus Q = [\alpha]G \oplus [\beta]G = [\alpha + \beta]G = [2]G$. Moreover, Bob cannot learn α, β since the computational discrete logarithm problem is hard.

Example 9.19. Well, now suppose I make the problem just a bit more complicated: Alice wants to convince that she knows α, β such that $\alpha\beta = 2$. And it turns out that elliptic curve points on their own are not enough to verify this. However, using pairings, we can do the following trick: assume

we have a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_1$ is generated by G_1 and $\mathbb{G}_2$ is generated by G_2 . Then, Alice can do the following:

1. Alice computes $P \leftarrow [\alpha]G_1 \in \mathbb{G}_1, Q \leftarrow [\beta]G_2 \in \mathbb{G}_2$ – points on two curves.
2. Alice sends $(P, Q) \in \mathbb{G}_1 \times \mathbb{G}_2$ to Bob.
3. Bob checks whether: $e(P, Q) = e(G_1, G_2)^2$.

Remark. The last verification can be also rewritten as $e(P, Q)e(G_1, G_2)^{-2} = 1$, which is more frequently used in practice.

Example 9.20. Finally, let us prove something more interesting. For instance, based on (x_1, x_2) , whether

$$x_1^2 + x_1x_2 = x_2$$

Suppose pairing $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ is symmetric. Alice can then calculate $P \leftarrow [x_1]G, Q \leftarrow [x_2]G \in \mathbb{G}$. Then, the condition can be verified by checking whether

$$e(P, P \oplus Q)e(G, \ominus Q) = 1$$

Let us see the correctness of this equation:

$$\begin{aligned} e(P, P \oplus Q)e(G, \ominus Q) &= e([x_1]G, [x_1 + x_2]G)e(G, [x_2]G)^{-1} \\ &= e(G, G)^{x_1(x_1+x_2)}e(G, G)^{-x_2} \\ &= e(G, G)^{x_1^2+x_1x_2-x_2} \end{aligned}$$

Now, if this is 1, then $x_1^2 + x_1x_2 = x_2$, which was exactly what we wanted to prove.

9.6 Exercises

Exercise 1. Suppose that elliptic curve is defined as $E/\mathbb{F}_7 : y^2 = x^3 + b$. Suppose $(2, 3)$ lies on the curve. What is the value of b ?

Exercise 2. Sum of which of the following pairs of points on the elliptic curve $E/\mathbb{F}_{11}$ is equal to the point at infinity $\mathcal{O}$ for any valid curve equation?

- | | |
|-------------------------------|---|
| a) $P = (2, 3), Q = (2, 8)$. | d) $P = \mathcal{O}, Q = (2, 3)$. |
| b) $P = (9, 2), Q = (2, 8)$. | e) $P = [10]G, Q = G$ where G is a generator. |
| c) $P = (9, 9), Q = (5, 7)$. | |

Exercise 3. Consider an elliptic curve E over $\mathbb{F}_{167^2}$. Denote by r the order of the group of points on E (that is, $r = |E|$). Which of the following **can** be the value of r ?

- | | | |
|-------------------|--------------------------|------------|
| a) $167^2 - 5$ | c) $167^2 + 5 \cdot 167$ | e) 160^2 |
| b) $167^2 - 1000$ | d) 170^2 | |

Exercise 4. Suppose that for some elliptic curve E the order is $|E| = qr$ where both q and r are prime numbers. Among listed, what is the most optimal complexity of algorithm to solve the discrete logarithm problem on E ?

- a) $O(qr)$
- b) $O(\sqrt{qr})$
- c) $O(\sqrt{\max\{q, r\}})$
- d) $O(\sqrt{\min\{q, r\}})$
- e) $O(\max\{q, r\})$

Exercise 5. What is **not** a valid equivalence relation $\sim$ over a set $\mathcal{X}$?

- a) $a \sim b$ iff $a + b < 0$, $\mathcal{X} = \mathbb{Q}$.
- b) $a \sim b$ iff $a \equiv b \pmod{5}$, $\mathcal{X} = \mathbb{Z}$.
- c) $a \sim b$ iff the length of a = the length of b , $\mathcal{X} = \mathbb{R}^2$.
- d) $(a_1, a_2, a_3) \sim (b_1, b_2, b_3)$ iff $a_3 = b_3$, $\mathcal{X} = \mathbb{R}^3$.

Exercise 6. Suppose that over $\mathbb{R}$ we define the following equivalence relation: $a \sim b$ iff $a - b \in \mathbb{Z}$ ($a, b \in \mathbb{R}$). What is the equivalence class of 1.4 (that is, $[1.4]_{\sim}$)?

- a) A set of all real numbers.
- b) A set of all integers.
- c) A set of reals $x \in \mathbb{R}$ with the fractional part of x equal to 0.4.
- d) A set of reals $x \in \mathbb{R}$ with the integer part of x equal to 1.
- e) A set of reals $x \in \mathbb{R}$ with the fractional part of x equal to 0.6.

Exercise 7. Which of the following pairs of points in homogeneous projective space $\mathbb{P}^2(\mathbb{R})$ are **not** equivalent?

- a) $(1 : 2 : 3)$ and $(2 : 4 : 6)$.
- b) $(2 : 3 : 1)$ and $(6 : 9 : 3)$.
- c) $(5 : 5 : 5)$ and $(2 : 2 : 2)$.
- d) $(4 : 3 : 2)$ and $(16 : 8 : 4)$.

Exercise 8. The main reason for using projective coordinates in elliptic curve cryptography is:

- a) To reduce the number of point additions in algorithms involving elliptic curves.
- b) To make the curve more secure against attacks.
- c) To make the curve more efficient in terms of memory usage.
- d) To reduce the number of field multiplications when performing scalar multiplication.
- e) To avoid making too many field inversions in complicated algorithms involving elliptic curves.

Exercise 9. Suppose $k = 19$ is a scalar and we are calculating $[k]P$ using the double-and-add algorithm. How many elliptic curve point addition operations will be performed?

- a) 0.
- b) 1.
- c) 2.
- d) 3.
- e) 4.

Exercise 10. What is the minimal number of inversions needed to calculate

the value of expression (over $\mathbb{F}_p$)

$$\frac{a-b}{(a+b)^4} + \frac{c}{a+b} + \frac{d}{a^2+c^2},$$

for the given scalars $a, b, c, d \in \mathbb{F}_p$?

- a) 1. b) 2. c) 3. d) 4. e) 5.

Exercise 11. Given pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ with G_1 – generator of $\mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ – generator of $\mathbb{G}_2$, which of the following is **not** equal to $e([3]G_1, [5]G_2)$?

- a) $e([5]G_1, [3]G_2)$. d) $e([3]G_1, G_2)e(G_1, [12]G_2)$.
b) $e([4]G_1, [4]G_2)$. e) $e(G_1, G_2)^{15}$.
c) $e([15]G_1, G_2)$.

Exercise 12. Unit Circle Proof. Suppose Alice wants to convince Bob that she knows a point on the unit circle $x^2 + y^2 = 1$. Suppose we are given a symmetric pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ for $\mathbb{G}_1 = \mathbb{G}_2 = \langle G \rangle$ and Alice computes $P \leftarrow [x]G, Q \leftarrow [y]G$. She then proceeds to sending (P, Q) to Bob. Which of the following checks should Bob perform to verify that Alice indeed knows a point on the unit circle?

- a) Check if $e(P, Q)e(Q, P) = 1$.
b) Check if $e([2]P, [2]Q) = e(G, G)$.
c) Check if $e([2]P, Q)e(Q, [2]P) = 1$.
d) Check if $e(P, P) + e(Q, Q) = 1$.
e) Check if $e(P, P)e(Q, Q) = e(G, G)$.

10 Commitment Schemes

Definition 10.1. A cryptographic commitment scheme allows one party to commit to a chosen statement (such as a value, vector, or polynomial) without revealing the statement itself. The commitment can be revealed in full or in part at a later time, ensuring the integrity and secrecy of the original statement until the moment of disclosure.

Before delving into the details, here is the intuition of cryptographic commitments.

Imagine putting a letter with some message into a box and locking it with your key. You then give that box to your friend, who cannot open it without the key. In this scenario, you have made a commitment to the message inside the box. You cannot change the content of the letter, as it is in your friend's possession. At the same time, your friend cannot access the letter since they do not have the key to unlock the box.

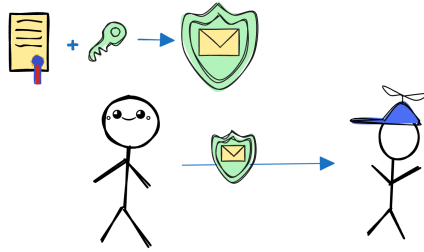


Figure 10.1: Commitment scheme

Definition 10.2 (Commitment Scheme). Commitment Scheme $\Pi_{\text{commitment}}$ is a tuple of three algorithms: $\Pi_{\text{commitment}} = (\text{Setup}, \text{Commit}, \text{Verify})$.

1. $\text{Setup}(1^\lambda)$: returns public parameter pp for both comitter and verifier;
2. $\text{Commit}(pp, m)$: returns a commitment c to the message m using public parameters pp and, optionally, a secret opening hint r ;
3. $\text{Open}(pp, c, m, r)$: verifies the opening of the commitment c to the message m with an opening hint r .

Definition 10.3 (Commitment Scheme). Properties of commitment schemes:

1. *Hiding*: verifier should not learn any information about the message given only the commitment c . To put it formally, we define a game:
 - (a) Adversary chooses two messages m_1, m_2 and sends to the challenger.

- (b) Challenger chooses a random bit b , commits to both messages:
 $c_1 \leftarrow \text{Commit}(pp, m_1), c_2 \leftarrow \text{Commit}(pp, m_2)$, and sends c_b to the adversary.
- (c) Adversary guesses a bit $\hat{b}$.

We define the hiding advantage of a PPT adversary $\mathcal{A}$ as

$$\text{HideAdv}[\mathcal{A}, \Pi_{\text{commitment}}] := \left| \Pr[b = \hat{b}] - \frac{1}{2} \right|$$

We say that the commitment scheme $\Pi_{\text{commitment}}$ is *hiding* if for any adversary, the aforementioned advantage is negligible.

2. *Binding*: prover could not find another message m_1 and open the commitment c without revealing the committed message m . To put it formally, we define a game:

- (a) Adversary chooses five values: commitment c and two distinct pairs (m_0, r_0) and (m_1, r_1) .
- (b) Adversary computes $b_j \leftarrow \text{Open}(pp, c, m_j, r_j)$.

Define the advantage in the binding game as:

$$\text{BindAdv}[\mathcal{A}, \Pi_{\text{commitment}}] = \Pr[b_0 = b_1 \neq 0 \wedge m_0 \neq m_1]$$

We say that the commitment scheme is binding if for any adversary, such advantage is negligible.

10.1 Hash-based commitments

As the name implies, we are using a cryptographic hash function H in such scheme.

1. Prover selects a message m from a message space M which he wants to commit to: $m \leftarrow \mathbb{M}$
2. Prover samples random value r (usually called blinding factor) from a challenge space $\mathcal{C} \subset \mathbb{Z}$: $r \leftarrow_{\$} \mathcal{C}$
3. Both values will be concatenated and hashed with the hash function H to produce the commitment: $c = H(m \parallel r)$

Commitment should be shared with a verifier. During the opening stage, prover reveals (m, r) to the *Verifier*. To check the commitment, verifier computes: $c_1 = H(m \parallel r)$.

If $c_1 = c$, prover has revealed the correct pair (m, r) .

It should be noted that a cryptographic hash function aims to provide collision resistance, meaning that the probability two different messages will result in one output is negligible. Because the *Verifier* knows the hash function digest c before the *Prover* reveals m and r , the *Prover* would need to find a collision $H(m' \parallel r') = H(m \parallel r)$ to be able to convince the *Verifier* that m' value was

committed.

However, due to the collision resistance, finding such m' and r' is computationally infeasible. Which means the *Prover* won't be able to convince the *Verifier* that the commitment was done to another value providing a *binding* property.

A cryptographically secure hash function is a one-way function, which means that finding the hash preimage is almost as hard as bruteforcing all possible input values. Given large challenge space, the probability of the *Verifier* of finding (m, r) such that $H(m, r) = c$ is negligible, which ensures *hiding* property of the commitment scheme.

10.2 Pedersen commitments

Pedersen commitments allow us to represent arbitrarily large vectors with a single elliptic curve point, while optionally hiding any information about the vector. Pedersen commitment uses a public group $\mathbb{G}$ of order q and two random public generators G and U : $U = [u]G$. Secret parameter u should be unknown to anyone, otherwise the *Binding* property of the commitment scheme will be violated. EC point U is chosen randomly using “Nothing-up-my-sleeve” to assure no one knows the discrete logarithm of a selected point.

Remark. Transparent random points generation

User can pick the publicly chosen random number (like a hash of project name, first numbers of π , etc), and hash that result to obtain another value. If that results in an x value that lies on the elliptic curve, use that as the random point and hash the (x, y) pair again (to obtain the next one, it needed). Otherwise, if the x -value does not land on the curve, increment x until it does. Because the committer is not generating the points, they don't know their discrete log.

Pedersen commitment scheme algorithm:

1. Prover and Verifier agrees on G and U points in a elliptic curve point group $\mathbb{G}$, q is the order of the group.
2. Prover selects a value m to commit and a blinder factor r : $m \leftarrow \mathbb{Z}_q, r \leftarrow \mathbb{Z}_q$
3. Prover generates a commitment and sends it to the Verifier: $c \leftarrow [m]G + [r]U$

During the opening stage, prover reveals (m, r) to the verifier. To check the commitment, verifier computes: $c_1 = [m]G + [r]U$.

If $c_1 = c$, prover has revealed the correct pair (m, r) .

Remark. In case the discrete logarithm of U is leaked, the *binding* property can be violated by the *Prover*:

$$c = [m]G + [r]U = [m]G + [r \cdot u]G = [m + r \cdot u]G$$

For example, $(m + u, r - 1)$ will have the same commitment value:

$$[m + u + (r - 1) \cdot u]G = [m + u - u + r \cdot u]G = [m + r \cdot u]G$$

Commitment aggregation

Pedersen commitment have some advantages compared to hash-based commitments. Additively homomorphic property allows to accumulate multiple commitments into one. Consider two pairs: $(m_1, r_1), (m_2, r_2)$.

$$\begin{aligned}c_1 &= [m_1]G + [r_1]U, \\c_2 &= [m_2]G + [r_2]U, \\c_a = c_1 + c_2 &= [m_1 + m_2]G + [r_1 + r_2]U\end{aligned}$$

This works for any number of commitments, so we can encode as many points as we like in a single one. For example, if a set of balances is committed, the sum of any subset can be proven without revealing the exact value of each balance. This is achieved by disclosing the sum of the balances and the corresponding sum of the blinding factors.

10.3 Vector commitments

Vector commitment schemes allows to commit to a vector of values rather than a value and a blinding term.

Pedersen Vector Commitments

Suppose we have a set of random elliptic curve points $(G_1, \dots, G_n)$ of cyclic group $\mathbb{G}$ (that we do not know the discrete logarithm of), a vector $(m_1, m_2 \dots m_n)$ and a random value r . We can do the following:

$$c = [m_1]G_1 + [m_2]G_2 \dots + [m_n]G_n + [r]Q$$

Since the *Prover* does not know the discrete logarithm of the generators, they don't know the discrete logarithm of $[C]$. Hence, this scheme is binding: they can only reveal $(v_1, \dots, v_n)$ to produce $[C]$ later, they cannot produce another vector.

Prover can later open the commitment by revealing the vector $(m_1, m_2 \dots m_n)$ and a blinding term r .

Merkle Tree based Vector Commitments

A naive approach for a vector commitment would be hash the whole vector. More sophisticated scheme uses divide-and-conquer approach by building a binary tree out of vector elements.

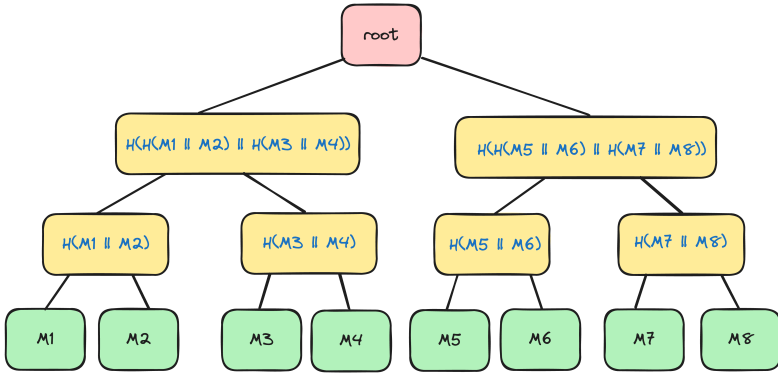


Figure 10.2: Merkle Tree structure

A Merkle Tree is a data structure to efficiently and securely verify the commitments to a vector of data. It is a binary tree where each leaf node represents a hash of a data block, and each non-leaf node is a hash of its child nodes' concatenated hashes. The top node, called the root hash or Merkle root, uniquely represents the entire data set. By comparing this root with a known valid root, one can quickly verify the authenticity and integrity of the data without needing to examine the entire dataset.

To prove the inclusion of element into the tree, a corresponding Merkle Branch is used. On the example below, M_1 inclusion is proved, and $(M_2, H(M_3 \parallel M_4), H(H(M_5 \parallel M_6) \parallel H(M_7 \parallel M_8)))$ is an inclusion branch vector.

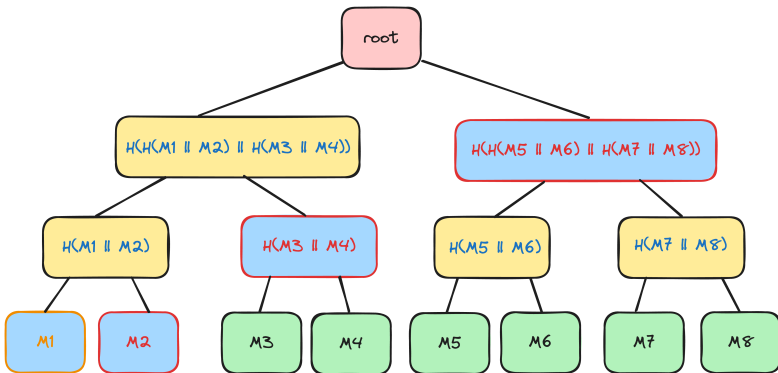


Figure 10.3: Merkle Tree inclusion proof branch

One of Merkle tree key advantages is that it allows for the selective disclosure of specific elements within the data set without revealing the rest.

10.4 Polynomial commitment

Polynomial commitment can be used to prove that the committed polynomial satisfies certain properties $P(x_1, x_2, \dots, x_n) = y$, without revealing what the polynomial is. The commitment is generally succinct, which means that it is much smaller than the polynomial it represents.

The KZG polynomial commitment scheme

The KZG (Kate-Zaverucha-Goldberg) is a polynomial commitment scheme:

1. *One-time “Powers-of-tau” trusted setup stage.* During trusted setup a set of elliptic curve points is generated. Let G be a generator point of some pairing-friendly elliptic curve group $\mathbb{G}$, s some random value in the order of the G point and d be the maximum degree of the polynomials we want to commit to. Public parameters of a trusted setup are calculated as:

$$[\tau^0]G, [\tau^1]G, \dots, [\tau^d]G$$

Parameter τ should be deleted after the ceremony. If it is revealed, the *binding* property of the commitment scheme can be broken. This parameter is usually called the *toxic waste*.

2. *Commit to polynomial.* Given the polynomial $p(x) = \sum_{i=0}^d p_i x^i$, compute the commitment $c = [p(\tau)]G$ using the trusted setup. Although the committer cannot compute $p(\tau)$ directly since the value of τ is unknown, he can compute it using values $([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G)$:

$$[p(\tau)]G = [\sum_{i=0}^d p_i \tau^i]G = \sum_{i=0}^d p_i [\tau^i]G$$

3. *Prove an evaluation.* To prove that at some point x_0 polynomial equals y_0 ($p(x_0) = y_0$), compute polynomial

$$q(x) = \frac{p(x) - y_0}{x - x_0}.$$

Polynomial $q(x)$ is called “quotient polynomial” and only exists if and only if $p(x_0) = y_0$:

- (a) If $p(x_0) = y_0$, we define $r(x) := p(x) - y_0$;
- (b) $r(x)$ has x_0 as a root, as $r(x_0) = 0$ by the definition. That is why there exists $q(x)$, such that $r(x) = q(x) \cdot (x - x_0)$;
- (c) Hence, the expression $q(x) = \frac{p(x) - y_0}{x - x_0}$ is a polynomial.

The existence of this quotient polynomial serves as a proof of the evaluation. *Prover* calculates proof $\pi = [q(\tau)]G$ and sends it to the *Verifier*.

4. *Verify the proof.* Given a commitment $c = [p(\tau)]G$, an evaluation $p(x_0) = y_0$ and a proof $[q(\tau)]G$, we need to ensure that $q(\tau) \cdot (\tau - x_0) = p(\tau) - y_0$. This can be done using trusted setup without knowledge of τ using bilinear mapping:

$$e([q(\tau)]G_1, [\tau]G_2 - [x']G_2) = e([p(\tau)]G_1 - [y]G_1, G_2)$$

Polynomial commitment schemes such as KZG are used in zero knowledge proof system to encode circuit constraints as a polynomial, so that verifier could check random points to ensure that the constraints are met.

10.5 Exercises

Exercise 1. Dmytro and Denis were watching a horse race. Confident in his ability to predict the outcome, Dmytro decided to commit to his prediction. However, in his haste, he forgot to use a blinding factor. Now, Dmytro is concerned that Denis might discover his prediction before the race ends, which would defeat the purpose of his commitment.

We define a dummy hash function $H : \mathbb{Z} \rightarrow \mathbb{F}_{41}$ as $H(x) = 13x + 17$. Dmytro used a *hash-based commitment* and H as a hash function. The set of race horse numbers is $\{3, 5, 8, 15\}$. Help Denis to find out the horse number Dmytro have made a commitment to, if the commitment equals $C = 39$.

- a) 3. b) 5. c) 8. d) 15.

Exercise 2. Denis made a setup (points G and U) for a Pedersen commitment scheme and committed values $(m, r) = (3, 7)$ to Dmytro by sending him $C = [3]G + [7]U$. Dmytro did not verify the setup. Turns out that Denis knows that $U = [6]G$. Denis is planning to send a different message from the one he originally committed to: $m_2 = 15$. Which values of (m_2, r_2) should he send to Dmytro at the opening stage?

- a) (15, 5) b) (15, 7) c) (15, 4) d) (3, 5)

Exercise 3. We define a dummy hash function $H : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{F}_{41}$ as $H(x, y) = 3x + 7y$. You have a Merkle tree built with depth 4 using the hash function H with a root that equals 37. Which inclusion proof is valid for element 3? Position defines how leaves should be hashed:

- if ℓ , then $h_i \leftarrow H(h_{i-1}, \text{branch}[i])$,
- if r , then $h_i \leftarrow H(\text{branch}[i], h_{i-1})$.

- (A) Branch: $[4, 16, 13]$, Position: $[\ell, r, \ell]$
 (B) Branch: $[1, 40, 3]$, Position: $[\ell, \ell, \ell]$
 (C) Branch: $[5, 12, 13]$, Position: $[r, r, \ell]$
 (D) Branch: $[4, 17, 13]$, Position: $[\ell, r, \ell]$

Exercise 4. Given a polynomial $p(x) = x^3 - 10x^2 + 31x - 30$, Oleksandr wants to prove that $p(2) = 0$. To do that, according to the KZG commitment scheme, he constructs the quotient polynomial $q(x)$ and wants to show that $q(\tau) \cdot (\tau - 2) = p(\tau)$. Assuming Oleksandr has conducted these steps correctly, what value of $q(x)$ has Oleksandr calculated?

- (A) $q(x) = 2x^2 + 4x - 6$ (C) $q(x) = x^2 - 8x + 15$
 (B) $q(x) = x^3 - 10x^2 + 30x - 28$ (D) $q(x) = x^2 + 5x + 18$

Part III

Foundational Zero-Knowledge Protocols

Finally, we start with the main content of the book — *zero-knowledge protocols*. This section introduces the concept of zero-knowledge proofs and describes the most fundamental protocols in the field: Σ -protocols, Groth16, PlonK, and Bulletproofs. More specifically, we will cover topics specified in Table 3.

Section	Topic	Key Concepts
11	Introduction to Zero-Knowledge	Proof of Knowledge, Soundness, Relation and Language, P/NP Complexities, Fiat-Shamir Heuristic
12	Σ -Protocols	Shnorr Signatures, Okamoto Representation Protocol, Generalization
13	R1CS	Arithmetical Circuits, Why Rank-1, Matrix Form
14	QAP	Quadratic Arithmetic Program, Polynomial as Universal Encoders
15	Pairing-based SNARKs	Pinnocchio and Groth16 Protocols; why such complicated expressions?
16	Circom	Programming R1CS in Circom, the language of zk-SNARKs
17	PlonK	FFT, Blinding, PlonKish Arithmetization
18	BulletProofs	Bulletproofs construction and performance

Table 3: Topics covered in Part III

11 Introduction to Zero-Knowledge Proofs

11.1 Motivation

Finally, we came to the most interesting part of the course: zero-knowledge proofs. Before we start with SNARKs, STARKs, Bulletproofs, and other zero-knowledge proof systems, let us first define what the zero-knowledge is. But even before that, we need to introduce some formalities. For example, what are “proof”, “witness”, and “statement” — terms that are so widely used in zero-knowledge proofs.

Let us describe the typical setup. We have two parties: **prover** $\mathcal{P}$ and a **verifier** $\mathcal{V}$. The prover wants to convince the verifier that some statement is true. Typically, the statement is not obvious (well, that is the reason for building proofs after all!) and therefore there might be some “helper” data, called **witness**, that helps the prover to prove the statement. The reasonable question is whether the prover can simply send witness to verifier and call it a day. Of course since you are here, reading this lecture, it is obvious that the answer is no. More specifically, by introducing zk-SNARKs, STARKs, and other proving systems, we will try to mitigate the following issues:

- **Zero-knowledge:** The prover wants to convince the verifier that the statement is true without revealing the witness.
- **Argument of knowledge:** Moreover, typically we want to make sure that the verifier, besides the statement correctness, ensures that the prover **knows** such a witness related to the statement.
- **Succinctness:** The proof should be short, ideally logarithmic in the size of the statement. This is crucial for practical applications, especially in the blockchain space where we cannot allow to publish long proofs on-chain. Moreover, verification should be efficient as well.

Example 11.1. Suppose, given a hash function^a $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, the prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that he knows the preimage $x \in \{0, 1\}^*$ such that $H(x) = y$ for some given public value $y \in \{0, 1\}^\lambda$. The properties listed above are interpreted as follows:

- **Zero-knowledge:** The prover $\mathcal{P}$ does not want to reveal *anything* about the pre-image x to the verifier $\mathcal{V}$.
- **Argument of knowledge:** Given a string $y \in \{0, 1\}^\lambda$ it is not sufficient for a prover to merely state that y has a pre-image. The prover $\mathcal{P}$ must demonstrate to a verifier $\mathcal{V}$ that he **knows** such a pre-image $x \in \{0, 1\}^*$.
- **Succinctness:** If the hash function takes n operations to compute^b, the proof should be **much** shorter than n operations. State-of-art solutions can provide proofs that are $O((\log n)^c)$ (polylogarithmic) in size! Moreover, verification time of such proof is also typically

polylogarithmic (or even $O(1)$ in some cases).

^aThe notation $\{0, 1\}^*$ means binary strings of arbitrary length

^bNote that “number of operations” is very vague term. One way to measure the “size” of some computational problem is specifying the number of gates in the arithmetical circuit $\mathcal{C}$, representing the computation of this problem (denoted as $|\mathcal{C}|$, respectively).

11.2 Relations and Languages

Recall that **relation** $\mathcal{R}$ is just a subset of $\mathcal{X} \times \mathcal{Y}$ for two arbitrary sets $\mathcal{X}$ and $\mathcal{Y}$. Now, we are going to introduce the notion of a *language of true statements* based on $\mathcal{R}$.

Definition 11.1 (Language of true statements). Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ be a relation. We say that a statement $x \in \mathcal{X}$ is a **true** statement if $(x, y) \in \mathcal{R}$ for some $y \in \mathcal{Y}$, otherwise the statement is called **false**. We define by $\mathcal{L}_{\mathcal{R}}$ (the language over relation $\mathcal{R}$) the set of all true statements, that is:

$$\mathcal{L}_{\mathcal{R}} = \{x \in \mathcal{X} : \exists y \in \mathcal{Y} \text{ such that } (x, y) \in \mathcal{R}\}.$$

Now, what is the purpose of introducing relations and languages? The idea is that relation is a natural way to formalize the notion of a statement and witness. Namely, we denote the elements of $\mathcal{X}$ as statements and the elements of $\mathcal{Y}$ as witnesses.

Further, we denote by w the witness for the statement $x \in \mathcal{L}_{\mathcal{R}}$. Oftentimes, one might also encounter notation ϕ to denote the statement, but we will stick to x for simplicity.

Example 11.2. Suppose we want to prove the following claim: number $n \in \mathbb{N}$ is the product of two large prime numbers $(p, q) \in \mathbb{N} \times \mathbb{N}$. Here, the relation is the following:

$$\mathcal{R} = \left\{ \begin{array}{l} x = (n) \in \mathbb{N}, \\ w = (p, q) \in \mathbb{N}^2 \end{array} \middle| n = p \cdot q \text{ where } p, q \text{ are primes} \right\}$$

In this particular case, the language of true statements is defined as

$$\mathcal{L}_{\mathcal{R}} = \{n \in \mathbb{N} : \exists w = (p, q) \text{ are primes such that } n = pq\}$$

Therefore, our initial claim we want to prove is $n \in \mathcal{L}_{\mathcal{R}}$. The witness for this statement is the pair (p, q) , where p and q are prime numbers such that $n = p \cdot q$ and typically (but not always) we want to prove this without revealing our witness: p and q . For example, one valid witness for $n = 15$ is $(3, 5)$, while $n = 16$ does not have any valid witness, so $16 \notin \mathcal{L}_{\mathcal{R}}$.

Example 11.3. Another example of claim we want to prove is the following: number $x \in \mathbb{Z}_N^{\times a}$ is a **quadratic residue** modulo N , meaning there exists some $w \in \mathbb{Z}_N^{\times}$ such that $x \equiv w^2 \pmod{N}$ (naturally, w is called a *square root* of x). The relation in this case is:

$$\mathcal{R} = \{(x, w) \in (\mathbb{Z}_N^{\times})^2 : x \equiv w^2 \pmod{N}\}$$

In this case, $\mathcal{L}_{\mathcal{R}} = \{x \in \mathbb{Z}_N^{\times} : \exists w \in \mathbb{Z}_N^{\times} \text{ such that } x \equiv w^2 \pmod{N}\}$. Here, our square root of x modulo N , that is w , is the witness for the statement $x \in \mathcal{L}_{\mathcal{R}}$. For example, for $N = 7$ we have $4 \in \mathcal{L}_{\mathcal{R}}$ since 5 is a valid witness: $5^2 \equiv 4 \pmod{7}$, while $3 \notin \mathcal{L}_{\mathcal{R}}$ since there is no valid witness for 3.

^aBy $\mathbb{Z}_N^{\times}$ we denote the multiplicative group of integers modulo N . In other words, this is a set of integers $\{x \in \mathbb{Z}_N : \gcd(x, N) = 1\}$.

However, we want to limit ourselves to languages that has reasonably efficient verifier (since otherwise the problem is not really practical and therefore of little interest to us). For that reason, we define the notion of a *NP Language* and from now on, we will be working with such languages.

Definition 11.2 (NP Language). A language $\mathcal{L}_{\mathcal{R}}$ belongs to the NP class if there exists a polynomial-time verifier $\mathcal{V}$ such that two properties hold:

- **Completeness:** If $x \in \mathcal{L}_{\mathcal{R}}$, then there is a witness w such that $\mathcal{V}(x, w) = 1$ with $|w| = \text{poly}(|x|)$. Essentially, it states that true claims have *short^a* proofs.
- **Soundness:** If $x \notin \mathcal{L}_{\mathcal{R}}$, then for any w it holds that $\mathcal{V}(x, w) = 0$. Essentially, it states that false claims have no proofs.

^a“Short” is a pretty relative term which would further differ based on the context. Here, we assume that the proof is “short” if it can be computed in polynomial time. However, in practice, we will want to make the proofs even shorter: see SNARKs and STARKs.

However, this construction on its own is not helpful to us. In particular, without having any randomness and no interaction, building practical proving systems is very hard. Therefore, we need some more ingredients to make proving NP statements easier.

11.3 Interactive Probabilistic Proofs

As mentioned above, we will bring two more ingredients to the table: **randomness** and **interaction**:

- **Interaction:** rather than simply passively receiving the proof, the verifier $\mathcal{V}$ can interact with the prover $\mathcal{P}$ by sending challenges and receiving responses.
- **Randomness:** verifier can send random coins (challenges) to the prover,

which the prover can use to generate responses.

Remark. For those who have already worked with SNARKs, the above introduction might seem very confusing. After all, what we are aiming for is to build **non-interactive** zero-knowledge proofs. However, as it turns out, there are a plenty of ways to make *some* interactive proofs non-interactive. We will discuss this in more detail later.

Now, one of the drastic changes is the following: if $x \notin \mathcal{L}_{\mathcal{R}}$, then the verifier $\mathcal{V}$ should reject the claim with overwhelming¹⁰ probability (in contrast to strict probability of 1). Let us consider the concrete example.

11.3.1 Example: Quadratic Residue Test

Again, suppose for relation $\mathcal{R} = \{(x, w) \in (\mathbb{Z}_N^\times)^2 : x \equiv w^2 \pmod{N}\}$ and corresponding language $\mathcal{L}_{\mathcal{R}} = \{x \in \mathbb{Z}_N^\times : \exists w \in \mathbb{Z}_N^\times \text{ such that } x \equiv w^2 \pmod{N}\}$ the prover $\mathcal{P}$ wants to convince the verifier that the given x is in language $\mathcal{L}_{\mathcal{R}}$. Again, sending w is not an option, as we want to avoid revealing the witness. So how can we proceed? The idea is that the prover $\mathcal{P}$ should prove that he *could* prove it if he felt like it.

So here how it goes. The prover $\mathcal{P}$ can first sample a random $r \leftarrow \mathbb{Z}_N^\times$, calculate $a \leftarrow r^2 \pmod{N}$ and say to the verifier $\mathcal{V}$:

- Hey, I could give you the square roots of both a and $ax \pmod{N}$ and that would convince you that the statement is true! But in this case, you would know w ¹¹.
- So instead of providing both values simultaneously, you will choose which one you want to see: either r or $r \times w \pmod{N}$. This way, after a couple of such rounds, you will not learn w but you will be convinced that I know it.

That being said, formally the interaction between prover $\mathcal{P}$ and verifier $\mathcal{V}$ can be described as follows:

1. $\mathcal{P}$ samples $r \leftarrow \mathbb{Z}_N^\times$ and sends $a \leftarrow r^2 \pmod{N}$ to $\mathcal{V}$.
2. $\mathcal{V}$ sends a random bit $b \leftarrow \{0, 1\}$ to $\mathcal{P}$.
3. If $b = 0$, the prover sends $z \leftarrow r$, otherwise, if $b = 1$, he sends $z \leftarrow rw \pmod{N}$.
4. The verifier checks whether $z^2 \equiv a \times x^b \pmod{N}$.
5. Repeat the process for $\lambda \in \mathbb{N}$ rounds.

¹⁰Some technicality: as you know from the Section 8, the value $\varepsilon = \text{negl}(\lambda)$ is called negligible since it is, in practice, very close to 0. In turn, the value $1 - \varepsilon$ is called *overwhelming* since it is, intuitively, close to 1.

¹¹If verifier gets both r and $rw \pmod{N}$, he can divide the latter by former and get w

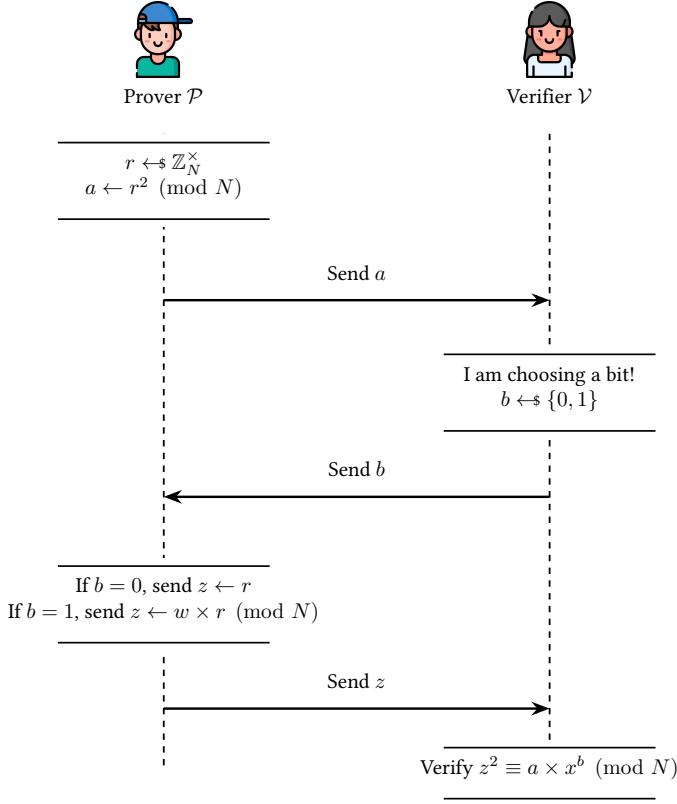


Figure 11.1: The interactive protocol between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for the quadratic residue test.

Now, let us show that the provided protocol is indeed **complete** and **sound**.

Completeness. Suppose the verifier chose $b = 0$ and thus the prover has sent $z = r$. The check would be $r^2 \equiv a \times x^0 \pmod{N}$ which is equivalent to $r^2 \equiv a \pmod{N}$. This obviously holds.

If, in turn, the verifier chose $b = 1$ and the prover sent rw , the check would be $(rw)^2 \equiv ax^{1-0} \pmod{N}$ which is equivalent to $r^2w^2 \equiv ax \pmod{N}$. Since $a = r^2 \pmod{N}$ and $x = w^2 \pmod{N}$, this check also obviously holds.

Soundness. Here, we need to prove that for any dishonest prover who does not know w , the verifier will reject the claim with overwhelming probability. One can easily show the following:

Proposition 11.3. If $x \notin \mathcal{L}_{\mathcal{R}}$, then for any prover $\mathcal{P}$, the verifier $\mathcal{V}$ will reject the claim with probability at least $1/2$.

By making λ rounds, the probability of rejection is $(\frac{1}{2})^\lambda = \text{negl}(\lambda)$ and therefore the verifier can be convinced that $x \in \mathcal{L}_{\mathcal{R}}$ with overwhelming probability of $1 - 2^{-\lambda}$.

To denote the interaction between algorithms $\mathcal{P}$ and $\mathcal{V}$ on the statement $\mathfrak{x}$, we use notation $\langle \mathcal{P}, \mathcal{V} \rangle(\mathfrak{x})$. Finally, now we are ready to define the notion of an **interactive proof system**.

Definition 11.4. A pair of algorithms $(\mathcal{P}, \mathcal{V})$ is called an **interactive proof** for a language $\mathcal{L}_{\mathcal{R}}$ if $\mathcal{V}$ is a polynomial-time verifier and the following two properties hold:

- **Completeness:** For any $\mathfrak{x} \in \mathcal{L}_{\mathcal{R}}$, $\Pr[\langle \mathcal{P}, \mathcal{V} \rangle(\mathfrak{x}) = \text{accept}] = 1$.
- **Soundness:** For any $\mathfrak{x} \notin \mathcal{L}_{\mathcal{R}}$ and for any prover $\mathcal{P}^*$, we have

$$\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(\mathfrak{x}) = \text{accept}] \leq \text{negl}(\lambda)$$

Definition 11.5. Besides classes **P** and **NP**, we now have one more class: **the class of interactive proofs (IP)**:

$$\text{IP} = \{\mathcal{L} : \text{there is an interactive proof } (\mathcal{P}, \mathcal{V}) \text{ for } \mathcal{L}\}.$$

11.4 Zero-Knowledge

Turns out that defining the zero-knowledge to even such a simplistic interactive proof system is not that easy. Informally, we give the following definition.

Definition 11.6. An interactive proof system $(\mathcal{P}, \mathcal{V})$ is called **zero-knowledge** if for any polynomial-time verifier $\mathcal{V}^*$ and any $\mathfrak{x} \in \mathcal{L}_{\mathcal{R}}$, the interaction $\langle \mathcal{P}, \mathcal{V}^* \rangle(\mathfrak{x})$ gives nothing new about the witness $\mathfrak{w}$.

Definition 11.7. The pair of algorithms $(\mathcal{P}, \mathcal{V})$ is called a **zero-knowledge interactive protocol** if it is *complete*, *sound*, and *zero-knowledge*.

Basically, the specified interaction is a **proof**! The prover $\mathcal{P}$ can convince the verifier $\mathcal{V}$ that the statement is true without revealing the witness – that is what we need (quite of).

Remark. The above definition is very informal and, for the most part, complete for the purposes of this course. If you do not want to dive into the formalities, you can skip the next part of this section. However, if you are curious about some technicalities, feel free to continue reading.

11.4.1 The Verifier's View

Suppose that the interaction between $\mathcal{V}$ and $\mathcal{P}$ has ended with the successful verification. What has $\mathcal{V}$ learned? Well, first things first, he has learned that the statement is true, that is $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$. However, he has also learned something more: he has learned the transcript of the interaction, that is the sequence of messages between $\mathcal{P}$ and $\mathcal{V}$.

Definition 11.8. Interaction between $\mathcal{P}$ and $\mathcal{V}$ consists of prover's messages (e.g., commitments and responses) $(m_1, m_2, \dots, m_\ell)$, verifier's queries $(q_1, q_2, \dots, q_\ell)$, and $\mathcal{V}$'s random coins $(r_1, r_2, \dots, r_\ell)$. The view of the verifier $\mathcal{V}$, denoted as $\text{view}_{\mathcal{V}}(\mathcal{P}, \mathcal{V})[\mathbb{x}]$, is a random variable $(m_1, r_1, q_1, m_2, r_2, q_2, \dots, m_\ell, r_\ell, q_\ell)$ that is determined by the random coins of $\mathcal{V}$ and the messages of $\mathcal{P}$ after the interaction with the statement $\mathbb{x}$. See Figure below.

Example 11.4. Suppose that for the aforementioned protocol with $N = 3 \times 2^{30} + 1$, the conversation between the prover $\mathcal{P}$, who wants to convince that $1286091780 \in \mathcal{L}_R$, and $\mathcal{V}$ is the following:

1. During the first round, $\mathcal{P}$ sends 672192003 to $\mathcal{V}$.
2. $\mathcal{V}$ sends $b = 0$ to $\mathcal{P}$.
3. $\mathcal{P}$ sends 2606437826 to $\mathcal{V}$.
4. $\mathcal{V}$ verifies that indeed $2606437826^2 \equiv 672192003 \pmod{N}$.
5. During the second round, $\mathcal{P}$ sends 2619047580 to $\mathcal{V}$.
6. $\mathcal{V}$ chooses $b = 1$ and sends to $\mathcal{P}$.
7. $\mathcal{P}$ sends 1768388249 to $\mathcal{V}$.
8. $\mathcal{V}$ verifies that $1768388249^2 \equiv 2619047580 \times 1286091780 \pmod{N}$.
9. Conversation ends.

The view of the verifier $\mathcal{V}$ is the following:

$$\text{view}_{\mathcal{V}}(\mathcal{V}, \mathcal{P})[1286091780] = (672192003, 0, 2606437826, 2619047580, 1, 1768388249)$$

Essentially, the view that you currently has witnessed is the same as one that $\mathcal{V}$ has seen. After this interaction, you have not learned anything about the witness w that prover $\mathcal{P}$ knows and which we, as of now, has not revealed to you.

In fact, you can verify by yourself, that the witness was $w = 3042517305$ and two randomnesses were $r_1 = 2606437826$ and $r_2 = 3023142760$.

One final note that is essential for the further discussion: variable $\text{view}_{\mathcal{V}}(\mathcal{P}, \mathcal{V})[\mathbb{x}]$ is a random variable. For example, for our particular case, both bits could be 0 or both bits could be 1.

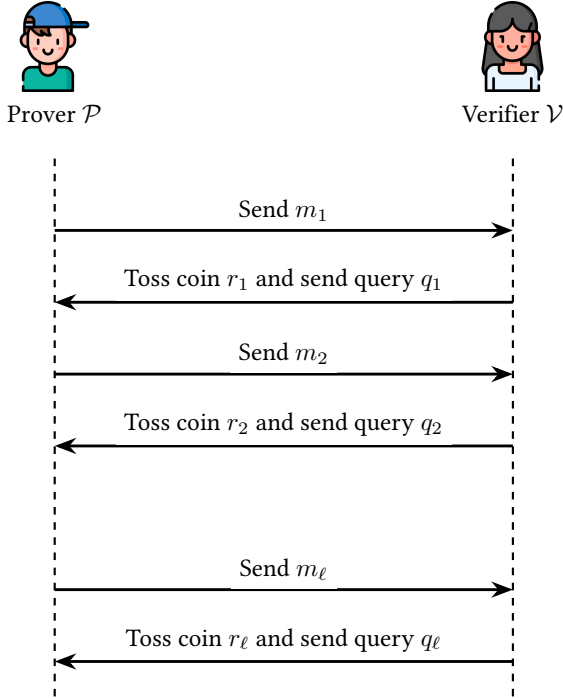


Figure 11.2: The interactive protocol between prover $\mathcal{P}$ and verifier $\mathcal{V}$. Prover's messages consist of messages $\{m_k\}_{k=1}^\ell$, verifier's messages consist of queries $\{q_k\}_{k=1}^\ell$, and additionally verifier samples random coins $\{r_k\}_{k=1}^\ell$.

11.4.2 The Simulation Paradigm

The key idea is the following: $\text{view}_{\mathcal{V}}(\mathcal{V}, \mathcal{P})[\mathbb{X}]$ gives nothing new to the verifier $\mathcal{V}$ about the witness w . But it gives nothing new, if he could have *simulated* this view on his own, without even running the interaction. In other words, the “simulated” and “real” views should be *computationally-indistinguishable*. But let us define the computational indistinguishability first.

Definition 11.9 (Computational Indistinguishability). Given two random distributions D_0 and D_1 , define the following challenger-adversary game:

1. The challenger samples $x_0 \leftarrow D_0, x_1 \leftarrow D_1$ and a bit $b \leftarrow \{0, 1\}$.
2. The challenger sends (x_0, x_1, b) to the adversary.
3. The adversary $\mathcal{A}$ outputs a bit $\hat{b}$.

We define the advantage of the adversary $\mathcal{A}$ in distinguishing D_0 and D_1 as

$$\text{Indadv}[\mathcal{A}, D_0, D_1] = \left| \Pr[b = \hat{b}] - \frac{1}{2} \right|$$

Distributions D_0 and D_1 are called **computationally indistinguishable**, denoted as $D_0 \approx D_1$, if for any polynomial-time adversary $\mathcal{A}$ and polynomial number of rounds in the game, the advantage $\text{Indadv}[\mathcal{A}, D_0, D_1]$ is negligible.

Finally, we are ready to define the **zero-knowledge**.

Definition 11.10 (Zero-Knowledge). An interactive protocol $(\mathcal{P}, \mathcal{V})$ is **zero-knowledge** for a language $\mathcal{L}_{\mathcal{R}}$ if for every poly-time verifier $\mathcal{V}^*$ there exists a poly-time simulator Sim such that for any valid statement $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$:

$$\text{view}_{\mathcal{V}^*}(\mathcal{P}, \mathcal{V}^*)[\mathbb{x}] \approx \text{Sim}(\mathbb{x}, 1^\lambda)$$

However, the condition that verifier might be arbitrary is rather strong. Therefore, we introduce the notion of **honest-verifier zero-knowledge**.

Definition 11.11. Honest-Verifier Zero-Knowledge (HVZK) An interactive protocol $(\mathcal{P}, \mathcal{V})$ is **honest-verifier zero-knowledge** for a language $\mathcal{L}_{\mathcal{R}}$ if there exists a probabilistic poly-time simulator Sim such that for any valid statement $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$ ^a:

$$\text{view}_{\mathcal{V}}(\mathcal{P}, \mathcal{V})[\mathbb{x}] \approx \text{Sim}(\mathbb{x}, 1^\lambda)$$

^aBelow, we assume that the verifier $\mathcal{V}$ is honest: he is following the protocol.

11.5 Proof of Knowledge

Now, the main issue with the above definition is that *we have proven the statement correctness, but we have not proven that the prover **knows** the witness*. These are completely two different things. Let us demonstrate why.

Example 11.5. Suppose that the prover $\mathcal{P}$ wants to convince the verifier that he knows the discrete logarithm of a given point $P \in E(\mathbb{F}_p)$ on a cyclic elliptic curve $E(\mathbb{F}_p)$ of order r . This corresponds to the relation and the corresponding language:

$$\begin{aligned} \mathcal{R} &= \{(P, \alpha) \in E(\mathbb{F}_p) \times \mathbb{Z}_r : P = [\alpha]G\}, \\ \mathcal{L}_{\mathcal{R}} &= \{P \in E(\mathbb{F}_p) : \exists \alpha \in \mathbb{Z}_r \text{ such that } P = [\alpha]G\} \end{aligned}$$

But here is the catch: actually, $\mathcal{L}_{\mathcal{R}} = E(\mathbb{F}_p)$ since any point P has a witness α such that $P = [\alpha]G$ (recall that the curve is cyclic)! So proving that $P \in \mathcal{L}_{\mathcal{R}}$ is completely useless! Rather, we want to prove that the prover knows α , not the fact that the point has a discrete logarithm.

That is why instead of **proof**, we need a **proof of knowledge**. This leads

to even another weird paradigm used for the rigorous definition: the **extractor**. Basically, the knowledge of witness means that we can *extract* the witness while interacting with the prover. Yet, the *extractor* can do more than the verifier: he can call the prover however he wants and he can also rewind the prover (for example, run some pieces multiple times). This sometimes is referred to as “extractor $\mathcal{E}$ uses $\mathcal{P}$ as an oracle”. Now, let us move to the formal definition.

Definition 11.12 (Proof of Knowledge). The interactive protocol $(\mathcal{P}, \mathcal{V})$ is a **proof of knowledge** for $\mathcal{L}_{\mathcal{R}}$ if exists a poly-time extractor algorithm $\mathcal{E}$ such that for any valid statement $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$, in expected poly-time $\mathcal{E}^{\mathcal{P}}(\mathbb{x})$ outputs $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$.

Lemma 11.13. The protocol from Section 11.3.1 is a proof of knowledge for the language $\mathcal{L}_{\mathcal{R}}$.

Proof. Let us define the extractor $\mathcal{E}$ for the statement x as follows:

1. Run the prover to receive $a \equiv r^2 \pmod{N}$ (r is chosen randomly from $\mathbb{Z}_N^*$).
2. Set verifier’s message to $b = 0$ to get $z_1 \leftarrow r$.
3. **Rewind** and set verifier’s message to $b = 1$ to get $z_2 \leftarrow rw \pmod{N}$.
4. Output $z_2/z_1 \pmod{N}$

The extractor $\mathcal{E}$ will always output w if $x \in \mathcal{L}_{\mathcal{R}}$. □

Remark. Note that extractor is given much more than the verifier: he can call the prover multiple times and he can also rewind the prover. This is the main difference between the verifier and the extractor.

11.6 Fiat-Shamir Heuristic

11.6.1 Random Oracle

In cryptography, one frequently encounters the term *cryptographic oracle*. In this section, we are not going to dive into the technical details of what that is, yet it is useful to have a general understanding of what it is.

Definition 11.14 (Cryptographic Oracle). Informally, *cryptographic oracle* is simply a function $\mathcal{O}$ that gives in $O(1)$ an answer to some typically very hard problem.

Example 11.6. Consider the Computational Diffie-Hellman (CDH) problem on the cyclic elliptic curve $E(\mathbb{F}_p)$ of prime order r with a generator G . Recall that such problem consists in computing $[\alpha\beta]G$ given $[\alpha]G$ and $[\beta]G$ where $\alpha, \beta \in \mathbb{Z}_r$.

Typically, it is believed that the Diffie-Hellman problem is hard (meaning, for any adversary strategy the probability of solving the problem is negligible). However, we *could* assume that such problem can be solved in $O(1)$ by a cryptographic oracle $\mathcal{O}_{\text{CDH}} : ([\alpha]G, [\beta]G) \mapsto [\alpha\beta]G$. This way, we can rigorously prove the security of some cryptographic protocols *even* if the Diffie-Hellman problem is suddenly solved.

One of the most popular cryptographic oracles is the **random oracle** $\mathcal{O}_R$. Let us define how the random oracle works.

Suppose someone is inputting x to the random oracle $\mathcal{O}_R$. The oracle $\mathcal{O}_R$ does the following:

1. If x has been queried before, the oracle returns the same value as it returned before.
2. If x has not been queried before, the oracle returns a randomly uniformly sampled value from the output space.

Remark. Of course, the sudden appearance of the random oracle is not a magic trick. In practice, the random oracle is typically implemented as a hash function. Of course, formally, the hash function is not a random oracle, yet it is a very good approximation and it is reasonable to assume that the hash function behaves like a random oracle.

11.6.2 Fiat-Shamir Transformation

Now, the main issue with the interactive proofs is that they are... Well, *interactive*. Ideally, we simply want to accumulate a proof π , publish it (say, in blockchain) so that anyone (essentially, being the verifier) could check its validity. So we need some tools to make *some* interactive protocols non-interactive. This is, of course, not always possible, but there are some ways to achieve this.

While different protocols use different ways to achieve this, one of the most popular methods (which, in particular, is used in STARKs) is the **Fiat-Shamir heuristic**. The idea is the following: instead of verifier sending the challenges, we can replace them with the random oracle applied to all the previous messages.

Here how it goes. Suppose we have an interactive protocol $(\mathcal{P}, \mathcal{V})$ for the statement $\mathbb{x}$. As previously defined, the interaction between $\mathcal{P}$ and $\mathcal{V}$ consists of prover's messages $(m_1, m_2, \dots, m_\ell)$, verifier's queries $(q_1, q_2, \dots, q_\ell)$, and verifier's random coins $(r_1, r_2, \dots, r_\ell)$. In case all the queries are public random coins, such interactive protocol is called **public-coin protocol** (or, more formally, **Arthur-Merlin protocol**). However, as it turns out, when all the verifier's queries are simply uniformly sampled random values, it is an overkill to use the interactive protocol. Instead, suppose at some point the verifier got messages $m_1, m_2, \dots, m_{\ell'}$ ($\ell' \leq \ell$) from the prover. Then, instead of verifier sampling some random value $r_{\ell'}$, we can simply use the random oracle $\mathcal{O}_R$ as follows: $r_{\ell'} \leftarrow \mathcal{O}_R(\mathbb{x}, m_1, m_2, \dots, m_{\ell'})$. Practically, instead of random oracle $\mathcal{O}_R$ we use the hash function H , and use: $r_{\ell'} \leftarrow H(\mathbb{x} \parallel m_1 \parallel m_2 \parallel \dots \parallel m_{\ell'})$.

Remark. Sometimes, to simulate the “interaction” with the verifier, one uses the “Fiat-Shamir Channel”. Its main purpose is to simulate the verifier’s queries and random coins. For example, one might implement it as a class/struct with the following methods:

1. `send_message( $m$ )`: “sends” the message m to the verifier. Under the hood, the proof stream accumulates the current state s and appends m to it.
2. `sample()`: returns the challenge r from the random oracle $\mathcal{O}_R$, applied to the current state s .
3. `get_proof()`: returns the proof π , being the history of interaction, that the prover can publish.

One can check the [winterfell](#) Rust library or a [simpler non-production implementation](#) of the Fiat-Shamir Channel in Golang for more details.

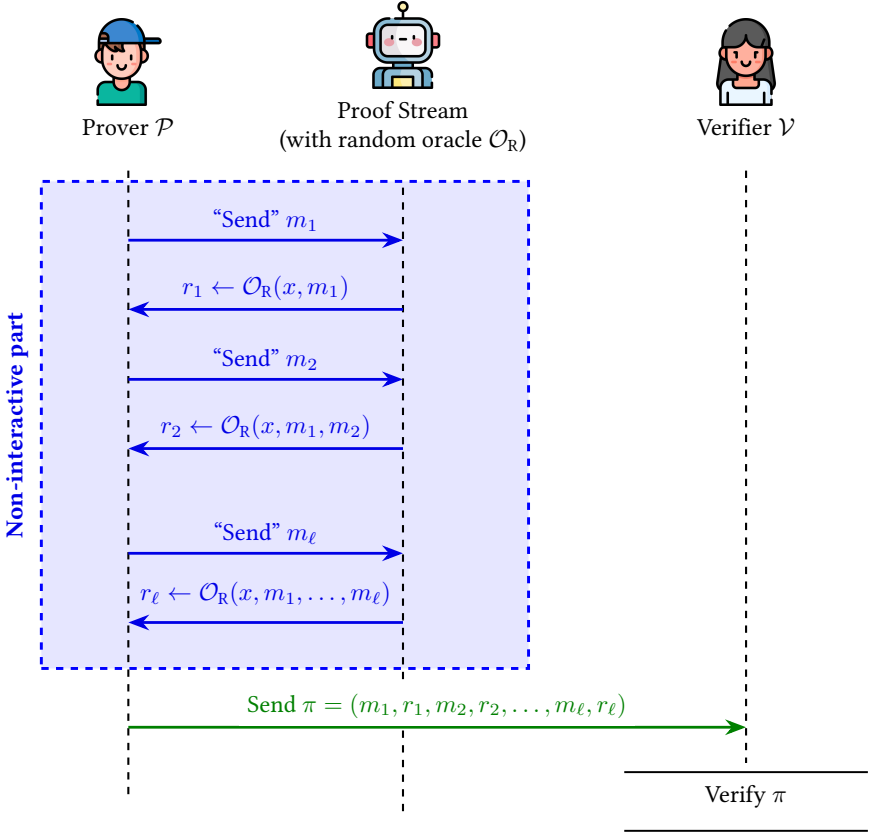


Figure 11.3: The non-interactive protocol between prover $\mathcal{P}$ and verifier $\mathcal{V}$ using Fiat-Shamir Transformation. In **blue** we marked a non-interactive part of the protocol, being the “communication” between a prover and a proof stream. In **green** we marked the final proof π that is sent to the verifier.

The process is illustrated in **Figure 11.3**. The Fiat-Shamir looks as follows:

1. First, the prover $\mathcal{P}$ “sends” the first message m_1 to the verifier $\mathcal{V}$. Here, “sending” is not an actual sending, but rather its simulation.
2. If we had an interactive protocol, the verifier $\mathcal{V}$ would send the random challenge r_1 to the prover $\mathcal{P}$. Instead, we use the random oracle $\mathcal{O}_R$ to get $r_1 \leftarrow \mathcal{O}_R(x, m_1)$.
3. Then, using this challenge, prover does his part in the protocol, and sends the next message m_2 .
4. Again, if we had an interactive protocol, the verifier would send the next challenge r_2 to the prover. Instead, we use the random oracle $\mathcal{O}_R$ to get $r_2 \leftarrow \mathcal{O}_R(x, m_1, m_2)$, which gets “sent” to the prover.

5. The process continues until the protocol is finished.

Note that the whole process can be done by a prover with no interaction with the “verifier”. In this case, one of the ways to represent the proof π is to publish the transcript of the interaction (that is, all the messages sent by the prover and challenges computed using the random oracle). This is exactly what is done in STARKs.

The reason why such transformation works is that the random oracle $\mathcal{O}_R$ is a *random* function. Therefore, the challenges $r_1, r_2, \dots$ are *random* values, and the prover cannot predict them (for example, by fabricating messages to have some specific output). That being said, the following theorem holds (which, of course, we are not going to prove since the proof is complicated).

Theorem 11.15. Suppose that $(\mathcal{P}, \mathcal{V})$ is a public-coin interactive argument of knowledge for some language $\mathcal{L}_{\mathcal{R}}$ with a negligible soundness error. Then, the Fiat-Shamir transformation of $(\mathcal{P}^{\mathcal{O}_R}, \mathcal{V}^{\mathcal{O}_R})$ is a non-interactive argument for $\mathcal{L}_{\mathcal{R}}$ with negligible soundness error in the random oracle model $\mathcal{O}_R$.

Acknowledgements

The material of this section was greatly inspired by the “[Zero Knowledge Proofs MOOC, Spring 2023](#)” series of lectures, in particular by the first lecture by Shafi Goldwasser, and “[Zero Knowledge Proofs Lecture](#)” by Boaz Barak.

11.7 Exercises

Exercise 1. When dealing with RSA protocol, one frequently encounters the following relation where e is a prime number and $n \in \mathbb{N}$:

$$\mathcal{R} = \{(w, x) \in \mathbb{Z}_n^\times \times \mathbb{Z}_n^\times : w^e = x\}$$

Which of the following is the language $\mathcal{L}_{\mathcal{R}}$ that corresponds to the relation $\mathcal{R}$?

- (A) Integers from $\mathbb{Z}_n^\times$ which have a modular root of e -th degree.
- (B) Integers from $\mathbb{Z}_n^\times$ which are divisible by e .
- (C) Integers x from $\mathbb{Z}_n^\times$ with properly defined expression x^e .
- (D) Integers from $\mathbb{Z}_n^\times$ which are prime.
- (E) Integers from $\mathbb{Z}_n^\times$ for which e is a primitive root.

Exercise 2. Suppose that for some interactive protocol $(\mathcal{P}, \mathcal{V})$ during one round, the probability that the verifier $\mathcal{V}$ accepts a false statement is $1/8$. How many rounds of interaction are needed to guarantee 120 bits of security? Assume here that n bits of security means that the probability of accepting a false statement is at most 2^{-n} .

- (A) 30. (B) 40. (C) 60. (D) 90. (E) 120.

Exercise 3. Recall that for relation $\mathcal{R} = \{(w, x) \in \mathbb{Z}_N^\times \times \mathbb{Z}_N^\times : x = w^2\}$ we defined the following interactive protocol $(\mathcal{P}, \mathcal{V})$ to prove that $x \in \mathcal{L}_{\mathcal{R}}$:

- $\mathcal{P}$ samples $r \leftarrow \mathbb{Z}_N^\times$ and sends $a = r^2$ to $\mathcal{V}$.
- $\mathcal{V}$ sends a random bit $b \in \{0, 1\}$ to $\mathcal{P}$.
- $\mathcal{P}$ sends $z = r \cdot w^b$ to $\mathcal{V}$.
- $\mathcal{V}$ accepts if $z^2 = a \cdot x^b$, otherwise it rejects.

Suppose we use the protocol $(\mathcal{P}, \mathcal{V}^*)$ where the “broken” verifier $\mathcal{V}^*$ always outputs $b = 1$. Which of the following statements is true?

- (A) Both the soundness and completeness of the protocol are preserved.
 (B) The soundness of the protocol is preserved, but the completeness is broken.
 (C) The completeness of the protocol is preserved, but the soundness is broken.
 (D) Both the soundness and completeness of the protocol are broken.

Exercise 4. What is the difference between the cryptographic proof and the proof of knowledge?

- (A) Cryptographic proof is a proof of knowledge that is secure against malicious verifiers.
 (B) Cryptographic proof is a proof of knowledge that is secure against malicious provers.
 (C) Cryptographic proof merely states the correctness of a statement, while the proof of knowledge also guarantees that the prover knows the witness.
 (D) While cryptographic proof states that witness exists for the given statement, the proof of knowledge makes sure to make this witness unknown to the verifier.
 (E) Proof of knowledge does not require verifier to know the statement, while cryptographic proof does.

Exercise 5. What is the purpose of introducing the extractor?

- (A) To introduce the algorithm that simulates the malicious verifier trying to extract the witness from the prover.
 (B) To define what it means that the prover knows the witness.
 (C) To give the verifier the ability to extract the witness from the prover during the interactive protocol.
 (D) To define the security of the interactive protocol that uses a more powerful verifier that can extract additional information from the prover.
 (E) To give prover more power to extract randomness generated by the verifier.

Exercise 6. What it means that the interactive protocol $(\mathcal{P}, \mathcal{V})$ is a zero-knowledge?

- (A) The verifier $\mathcal{V}$ cannot know whether the given statement is true or false.
 (B) The verifier $\mathcal{V}$ cannot know whether the prover $\mathcal{P}$ knows the witness.

- (C) View of the prover $\mathcal{P}$ in the protocol is indistinguishable from the view of the verifier $\mathcal{V}$.
- (D) Any view of any verifier $\mathcal{V}$ can be simulated using some polynomial-time algorithm, outputting computationally indistinguishable distribution from the given view.
- (E) The prover $\mathcal{P}$ can convince the verifier $\mathcal{V}$ that the statement is true without knowing the witness.

Hint: View of the participant in the protocol consists of all data he has access to during the protocol execution. For example, verifier $\mathcal{V}$'s view consists of the messages he sends and receives, as well as the random coins he generates.

12 Sigma Protocols

12.1 Schnorr's Identification Protocol

To better illustrate the Fiat-Shamir Transformation in practice, let us consider one of the most basic Sigma protocols: non-interactive Schnorr Identification Protocol. It is a simple and elegant protocol that allows one party to prove to another party that it knows a discrete logarithm of a given element. It is also quite straightforward to generalize it to a signature scheme.

Let us formalize it using theory from [Section 11.2](#). Suppose $\mathbb{G}$ is a cyclic group of order q with a generator g . Then, the relation and language being considered:

$$\mathcal{R} = \{(u, \alpha) \in \mathbb{G} \times \mathbb{Z}_q : u = g^\alpha\}, \quad \mathcal{L}_{\mathcal{R}} = \{u \in \mathbb{G} : \exists \alpha \in \mathbb{Z}_q : u = g^\alpha\}$$

Now, suppose prover $\mathcal{P}$ has a valid statement and a witness $(u, \alpha) \in \mathcal{R}$ and he wants to convince the verifier $\mathcal{V}$ that he knows the witness α to the public statement u (that is, we are building the proof of knowledge). Well, the easiest way how to proceed is simply giving α to $\mathcal{V}$, but this is obviously not what we want. Instead, the Schnorr protocol allows $\mathcal{P}$ to prove the knowledge of α without revealing it.

First, let us start with the interactive version of the protocol.

Definition 12.1. The Schnorr interactive identification protocol $\Pi_{\text{Sch}} = (\text{Gen}, \mathcal{P}, \mathcal{V})$ with a generation function Gen and prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- $\text{Gen}(1^\lambda)$: As with most public-key cryptosystems, we take $\alpha \leftarrow \$ \mathbb{Z}_q$ and $u \leftarrow g^\alpha$. We output the *verification key* as $\text{vk} := u$, and the *secret key* as $\text{sk} := \alpha$.
- The protocol between $(\mathcal{P}, \mathcal{V})$ is run as follows:
 - $\mathcal{P}$ computes $r \leftarrow \mathbb{Z}_q^\times$, $a \leftarrow g^r$ and sends a to $\mathcal{V}$.
 - $\mathcal{V}$ sends a random challenge $e \leftarrow \$ \mathbb{Z}_q$ to $\mathcal{P}$.
 - $\mathcal{P}$ computes $\sigma \leftarrow r + \alpha e \in \mathbb{Z}_q$ and sends σ to $\mathcal{V}$.
 - $\mathcal{V}$ accepts if $g^\sigma = a \cdot u^e$, otherwise it rejects.

This protocol is illustrated in [Figure 12.1](#).

Definition 12.2. An interaction between $\mathcal{P}$ and $\mathcal{V}$ produces the so-called **conversation** $(a, e, \sigma) \in \mathbb{G} \times \mathbb{Z}_q \times \mathbb{Z}_q$. We call such a conversation an **accepting conversation** if $\mathcal{V}$ accepts the proof.

Example 12.1. In case of a Schnorr protocol, the accepting conversation is such that $g^\sigma = a \cdot u^e$.

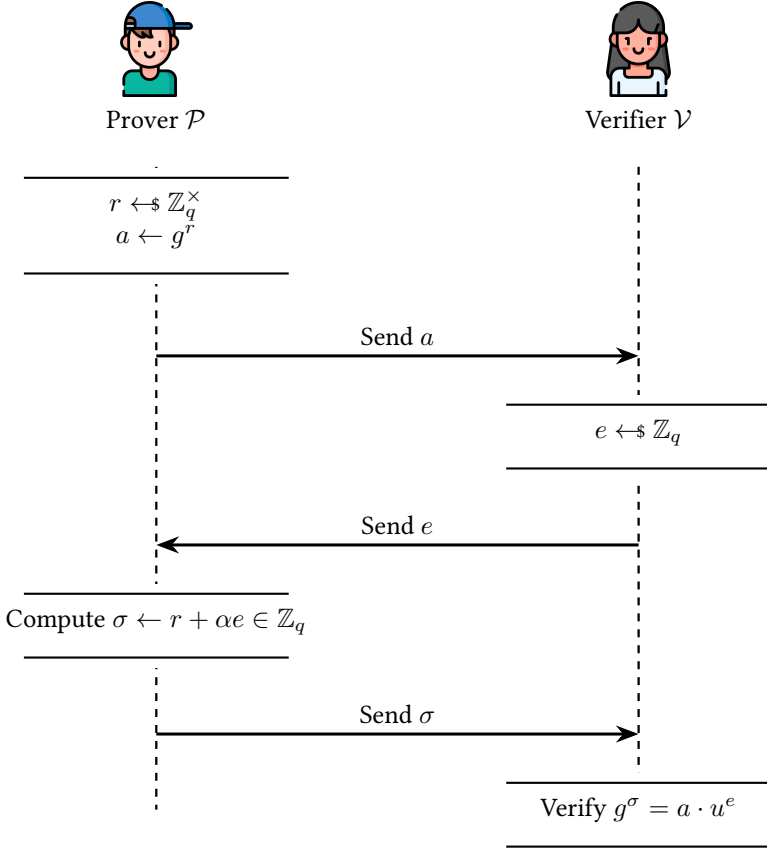


Figure 12.1: The interactive Schnorr protocol between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for proof of knowledge of discrete logarithm relation.

Now, one can prove the following theorem.

Theorem 12.3. The Schnorr protocol Π_{Sch} is complete, sound, and (honest verifier) zero-knowledge proof of knowledge.

Proof. We are not going to prove the zero-knowledge and soundness properly, but completeness and proof of knowledge are quite straightforward to show.

- **Completeness.** Just observe that $g^\sigma = g^{r+\alpha e} = g^r (g^\alpha)^e = a \cdot u^e$.
- **Proof of Knowledge.** To prove that the protocol is a proof of knowledge, we need to construct an extractor $\mathcal{E}^{\mathcal{P}}$. We construct it as follows:
 1. Extractor runs the prover and gets a , e , and σ as a response.
 2. Extractor rewinds back to the verifier's challenge step, generates a new challenge $e' \leftarrow \mathbb{Z}_q$ and gets new prover's response σ' (for the

same prover's randomness r).

3. Extractor outputs the witness $\alpha \leftarrow (\sigma - \sigma')(e - e')^{-1}$.

The reason why this works is following: notice that $g^\sigma = a \cdot u^e$, $g^{\sigma'} = a \cdot u^{e'}$. Therefore, by dividing former by latter, we obtain $g^{\sigma - \sigma'} = u^{e - e'} = g^{\alpha(e - e')}$. It immediately follows that $\alpha = (\sigma - \sigma')(e - e')^{-1}$.

Remark. Before considering how to make such protocol non-interactive correctly, suppose that we instead do the following: after interaction with the verifier, the prover publishes the conversation as a proof of knowledge. Would that be a valid non-interactive proof? In other words, can we convince the independent observer of the interaction that the prover knows the witness? The answer is no (and it is generally so for any interactive protocol). The reason why is that the prover can first sample randomly $e, \sigma \leftarrow \mathbb{Z}_q$, compute $a \leftarrow g^\sigma / u^e$ and simply publish (a, e, σ) as a proof. This is a valid conversation since $g^\sigma = a \cdot u^e = (g^\sigma / u^e) \cdot u^e$ and thus the observer would be convinced that the prover knows the witness. However, the prover might not know the witness at all!

Therefore, either (1) the prover needs to get a challenge e **before** he commits to the value σ , or (2) challenge must be randomized. Otherwise, he can precompute σ and publish it as a proof (or simply make a deal with the verifier to fool the observer).

Now, notice that the provided protocol is a public-coin protocol. Therefore, we can apply the Fiat-Shamir transformation to make it non-interactive. Suppose we have a random oracle $\mathcal{O}_R : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q$:

1. The prover $\mathcal{P}$ computes $r \leftarrow \mathbb{Z}_q^\times$, $a \leftarrow g^r$ and sends a to the Fiat-Shamir Channel.
2. The Fiat-Shamir channel responds with the challenge $e \leftarrow \mathcal{O}_R(u, a)$.
3. The prover $\mathcal{P}$ computes $\sigma \leftarrow r + \alpha e$ and sends σ to the Fiat-Shamir Channel.
4. The Fiat-Shamir channel outputs the proof $\pi = (a, e, \sigma)$, which the verifier can check via previously mentioned equation $g^\sigma = a \cdot u^e$.

Now, notice that e might not be included in the proof since the verifier can compute it by himself. Therefore, the final proof π can be reduced to $(a, \sigma) \in \mathbb{G} \times \mathbb{Z}_q$ and its computation does not need any interaction with the verifier. Moreover, it is still complete, sound, and proof of knowledge due to the Fiat-Shamir transformation. It is also (not easy to prove) zero-knowledge.

12.2 Schnorr's Signature Scheme

Now, turning the Schnorr's Identification Protocol into a signature scheme is quite straightforward. The only modification to the non-interactive proof described in the previous section is that we include the message $m \in \mathcal{M}$ instead of our statement $u \in \mathbb{G}$ in the computation of the challenge e . Additionally,

suppose we use the hash function H as a random oracle from the previous section. Now, let us give a formal definition.

Definition 12.4. The **Schnorr Signature Scheme** Σ_{Sch} is a tuple of algorithms (Gen, Sign, Verify), where:

- Gen(1^λ): We take $\alpha \leftarrow \mathbb{Z}_q$ and $u \leftarrow g^\alpha$. The *public key* is $\text{pk} := u$, while the *secret key* is $\text{sk} := \alpha$.
- Sign(m, sk): The signer computes $r \leftarrow \mathbb{Z}_q^\times, a \leftarrow g^r, e \leftarrow H(m, a), \sigma \leftarrow r + \alpha e$ and outputs the signature (a, σ) .
- Verify($(a, \sigma), m, \text{pk}$): Verifier checks $g^\sigma = a \cdot u^e$ for $e \leftarrow H(m, a)$.

Remark. Typically, one also uses a so-called “*key-prefixed*” variant of the scheme, where the challenge e is computed as $e \leftarrow H(\text{pk}, m, a)$ for a random oracle $H : \mathbb{G} \times \mathcal{M} \times \mathbb{G} \rightarrow \mathbb{Z}_q$. It was argued that such variant has a better multi-user security bound than the classical one.

12.3 Sigma Protocols

Now, the Schnorr Protocol is just one of the many examples of a so-called **Sigma Protocol**. Sigma protocols are a class of interactive proof systems that are used to prove the knowledge of a witness to a statement. They are quite general and can be used to prove the knowledge of a witness to any effective relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$, where $\mathcal{X}$ is the set of public statements and $\mathcal{W}$ is the set of witnesses. Let us define them formally.

Definition 12.5. Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be an effective relation. A **Sigma protocol** for $\mathcal{R}$ is an interactive protocol $(\mathcal{P}, \mathcal{V})$ that satisfies the following properties:

- In the beginning, $\mathcal{P}$ computes a **commitment** a and sends it to $\mathcal{V}$.
- $\mathcal{V}$ chooses a random **challenge** $c \in \mathcal{C}$ from the challenge space $\mathcal{C}$ and sends it to $\mathcal{P}$.
- Upon receiving c , $\mathcal{P}$ computes the response z and sends it to $\mathcal{V}$.
- $\mathcal{V}$ outputs either accept or reject based on the conversation transcript (a, c, z) .

Remark. The name “Sigma” protocol comes from the fact that the “shape” of the message flow vaguely resembles the Greek letter Σ : see [Figure 12.2](#).

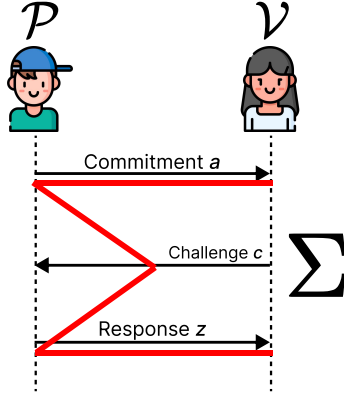


Figure 12.2: Sigma Protocol Illustration: the flow of messages between prover $\mathcal{P}$ and verifier $\mathcal{V}$ closely resembles the Greek letter Σ , which is marked in **red** in the Figure.

Example 12.2. In particular, for the Schnorr Protocol, the Sigma protocol is defined over the relation $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ where:

$$\mathcal{X} = \mathbb{G}, \mathcal{W} = \mathbb{Z}_q, \mathcal{R} = \{(u, \alpha) \in \mathbb{G} \times \mathbb{Z}_q : u = g^\alpha\}$$

Here, the challenge space $\mathcal{C}$ is a subset of $\mathbb{Z}_q$ (or, typically, the whole set).

Similarly to interactive protocols, Sigma protocols also have a property called *soundness*. However, there is an additional property called *special soundness* that simplifies the general notion of soundness.

Definition 12.6 (Special Soundness). Let $(\mathcal{P}, \mathcal{V})$ be a Σ -protocol for $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$. We say that $(\mathcal{P}, \mathcal{V})$ is **special sound** if there exists a witness extractor $\mathcal{E}$ such that, given statement $x \in \mathcal{X}$ and two accepting conversations (a, c, z) and (a, c', z') (where $c \neq c'$)^a, the extractor can always efficiently compute the witness w such that $(x, w) \in \mathcal{R}$.

^aNotice that initial commitments in both conversations are the same!

Example 12.3. In case of the Schnorr Protocol, the special soundness property is satisfied by the extractor $\mathcal{E}$ that we have constructed in the proof of knowledge. In other words, we can extract the discrete logarithm $\alpha = \text{DLog}_{\mathbb{G}}(u)$ given two accepting conversations (a, e, σ) and (a', e', σ') .

Now, let us consider some more examples of Sigma protocols.

12.4 More Sigma Protocol Examples

12.4.1 Okamoto's Protocol for Representations

Again, let $\mathbb{G}$ be a cyclic group of prime order q with a generator $g \in \mathbb{G}$ and let $h \in \mathbb{G}$ an arbitrary group element (for example, it might be yet another group generator). While considering Pedersen Commitments, you already encountered form $g^\alpha h^\beta$. Now, let us generalize this concept a bit.

Definition 12.7. For $u \in \mathbb{G}$, a **representation** relative to g and h is a pair $(\alpha, \beta) \in \mathbb{Z}_q \times \mathbb{Z}_q$ such that $u = g^\alpha h^\beta$.

Remark. Notice that for the given u there are exactly q representations relative to g and h . Indeed, $\forall \beta \in \mathbb{Z}_q \exists! \alpha \in \mathbb{Z}_q : g^\alpha = u h^{-\beta}$.

Now, the *Okamoto's Protocol* is a Sigma protocol that allows one party to prove the knowledge of a representation of a given $u \in \mathbb{G}$ relative to g and h . In other words, we are working with the relation

$$\mathcal{R} = \{(u, (\alpha, \beta)) \in \mathbb{G} \times \mathbb{Z}_q^2 : u = g^\alpha h^\beta\}$$

Now, let us describe the protocol.

Definition 12.8 (Okamoto's Identification Protocol). **Okamoto's Protocol** consists of two algorithms: $(\mathcal{P}, \mathcal{V})$, where the prover is assumed to know $(u, (\alpha, \beta)) \in \mathcal{R}$ defined above. The protocol is defined as follows:

1. $\mathcal{P}$ computes $\alpha_r \leftarrow \mathbb{Z}_q$, $\beta_r \leftarrow \mathbb{Z}_q$, $u_r \leftarrow g^{\alpha_r} h^{\beta_r}$ and sends commitment u_r to $\mathcal{V}$.
2. $\mathcal{V}$ samples the challenge $c \leftarrow \mathbb{Z}_q$ and sends c to $\mathcal{P}$.
3. $\mathcal{P}$ computes $\alpha_z \leftarrow \alpha_r + \alpha c$, $\beta_z \leftarrow \beta_r + \beta c$ and sends $\mathbf{z} = (\alpha_z, \beta_z)$ to $\mathcal{V}$.
4. $\mathcal{V}$ checks whether $g^{\alpha_z} h^{\beta_z} = u_r u^c$ and accepts or rejects the proof accordingly.

Theorem 12.9. Okamoto's Protocol is a Σ -protocol for the relation $\mathcal{R}$ which is Honest-Verifier Zero-Knowledge.

Part of the proof. Again, let us show *correctness* and *special soundness* without honest-verifier zero-knowledge properties.

Completeness. Suppose indeed that $(u, (\alpha, \beta)) \in \mathcal{R}$. Then, the verification condition can be written as follows:

$$g^{\alpha_z} h^{\beta_z} = g^{\alpha_r + \alpha c} h^{\beta_r + \beta c} = g^{\alpha_r} g^{\alpha c} h^{\beta_r} h^{\beta c} = \underbrace{(g^{\alpha_r} h^{\beta_r})}_{=u_r} \cdot \underbrace{(g^{\alpha c} h^{\beta c})}_{=u^c} = u_r u^c$$

Special Soundness. Suppose we are given two accepting conversations: $(u_r, c, (\alpha_z, \beta_z))$ and $(u_r, c', (\alpha'_z, \beta'_z))$ and we want to construct an extractor $\mathcal{E}$ which would give us a witness (α, β) . In this case, we have the following holding:

$$g^{\alpha_z} h^{\beta_z} = u_r u^c, \quad g^{\alpha'_z} h^{\beta'_z} = u_r u^{c'}$$

We can divide the former by the latter to obtain:

$$g^{\alpha_z - \alpha'_z} h^{\beta_z - \beta'_z} = u^{c - c'} = g^{\alpha(c - c')} h^{\beta(c - c')},$$

from which the extractor $\mathcal{E}$ can efficiently compute witness as follows: $\alpha \leftarrow (\alpha_z - \alpha'_z) / (c - c')$ and $\beta \leftarrow (\beta_z - \beta'_z) / (c - c')$.

12.4.2 Chaum-Pedersen protocol for DH-triplets

As with previous examples, suppose we are given the cyclic group $\mathbb{G}$ or prime order q and generator $g \in \mathbb{G}$. Recall that *the Diffie-Hellman Triple* (or, *DH-triple*) is a triple $(g^\alpha, g^\beta, g^\gamma)$ with $\gamma = \alpha\beta$. Now, this definition is not really convenient for us, so we will reformulate the DH-triple using the proposition below.

Proposition 12.10 (Alternative DH-triple Definition). (u, v, w) is a DH-triplet iff $\exists \beta \in \mathbb{Z}_q : v = g^\beta, w = u^\beta$.

Now, this makes it easier to define the relation $\mathcal{R}$ for the Chaum-Pedersen protocol:

$$\mathcal{R} = \{((u, v, w), \beta) \in \mathbb{G}^3 \times \mathbb{Z}_q : v = g^\beta \wedge w = u^\beta\}$$

In other words, here we have a witness $\beta \in \mathbb{Z}_q$, while the statement is a triplet $(u, v, w) \in \mathbb{G}^3$. Again, we want to convert this into a Sigma protocol. We do it as follows.

Definition 12.11 (Chaum-Pedersen Protocol). **Chaum-Pedersen Protocol** consists of two algorithms: $(\mathcal{P}, \mathcal{V})$, where the prover is assumed to know $(\beta, (u, v, w)) \in \mathcal{R}$ defined above. The protocol is defined as follows:

1. $\mathcal{P}$ computes $\beta_r \leftarrow \$ \mathbb{Z}_q$, $v_r \leftarrow \$ g^{\beta_r}$, $w_r \leftarrow u^{\beta_r}$ and sends commitment (u_r, w_r) to $\mathcal{V}$.
2. $\mathcal{V}$ samples the challenge $c \leftarrow \$ \mathbb{Z}_q$ and sends c to $\mathcal{P}$.
3. $\mathcal{P}$ computes $\beta_z \leftarrow \beta_r + \beta c$ and sends β_z to $\mathcal{V}$.
4. $\mathcal{V}$ checks whether two conditions hold: $g^{\beta_z} = v_r v^c$ and $u^{\beta_z} = w_r w^c$, and accepts or rejects the proof accordingly.

Theorem 12.12. Chaum-Pedersen Protocol is a Σ -protocol for the relation $\mathcal{R}$ which is Honest-Verifier Zero-Knowledge.

Part of the proof. As always, let us show *correctness* and *special soundness* without honest-verifier zero-knowledge properties.

Correctness. Again, consider the expression g^{β_z} more closely:

$$g^{\beta_z} = g^{\beta_r + \beta_c} = g^{\beta_r} g^{\beta_c} = \underbrace{g^{\beta_r}}_{=v_r} \underbrace{(g^{\beta})^c}_{=v} = v_r v^c$$

The similar reasoning can be applied to the second verification condition: indeed, here we have $u^{\beta_z} = u^{\beta_r} (u^{\beta})^c = w_r w^c$

Special Soundness. Suppose we are given two accepting conversations: $((u_r, w_r), c, \beta_z)$ and $((u_r, w_r), c', \beta'_z)$ and we want to construct an extractor $\mathcal{E}$ which would give us a witness β . Notice that the following equations hold:

$$\begin{aligned} g^{\beta_z} &= v_r v^c, & g^{\beta'_z} &= v_r v^{c'}, \\ u^{\beta_z} &= w_r w^c, & u^{\beta'_z} &= w_r w^{c'}. \end{aligned}$$

Divide left equations by the right ones to obtain:

$$g^{\beta_z - \beta'_z} = v^{c - c'}, \quad u^{\beta_z - \beta'_z} = w^{c - c'}.$$

Consider the first equation. Since $v = g^{\beta}$ we derive $(\beta_z - \beta'_z) = \beta(c - c')$, from which $\mathcal{E}$ outputs $\beta = \frac{\beta_z - \beta'_z}{c - c'}$. The same value can be extracted from the second equation.

12.5 Generalizing Sigma Protocols

Now, the most interesting part! Probably, you have noticed, that all protocols above (Schnorr, Okamoto, Chaum-Pedersen) have a similar structure. So is there any way to generalize them? The answer is yes and moreover, this done in a very elegant way.

Let $(\mathbb{H}, \oplus)$ and $(\mathbb{T}, \otimes)$ be two finite abelian groups and suppose we have some concrete homomorphism $\psi : \mathbb{H} \rightarrow \mathbb{T}$. Moreover, we require that given $t \in \mathbb{T}$, finding the pre-image of t (meaning, finding some $h \in \mathbb{H}$ such that $\psi(h) = t$) is computationally hard. Suppose $\mathcal{F}$ is a set of all homomorphisms from $\mathbb{H}$ to $\mathbb{T}$ (sometimes denoted as $\text{Hom}(\mathbb{H}, \mathbb{T})$). Now, define the following relation:

$$\mathcal{R} = \{((t, \psi), h) \in (\mathbb{T} \times \mathcal{F}) \times \mathbb{H} : \psi(h) = t\}$$

And now the prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that he knows the witness h to the statement (t, ψ) .

Proposition 12.13. Now, why does this generalize the previous protocols? Well, let us consider all previous examples:

- **Schnorr Protocol:** Here we have $\mathbb{H} = \mathbb{Z}_q$, $\mathbb{T} = \mathbb{G}$, and $\psi : \mathbb{Z}_q \rightarrow \mathbb{G}$ is defined as $\psi(\alpha) = g^\alpha$. Moreover, here ψ is an isomorphism!

- **Okamoto Protocol:** Here we have $\mathbb{H} = \mathbb{Z}_q^2$, $\mathbb{T} = \mathbb{G}$, and $\psi : \mathbb{Z}_q^2 \rightarrow \mathbb{G}$ is defined as $\psi(\alpha, \beta) = g^\alpha h^\beta$. It is also quite easy to see that ψ is a homomorphism:

$$\begin{aligned}\psi((\alpha, \beta) + (\alpha', \beta')) &= \psi(\alpha + \alpha', \beta + \beta') = g^{\alpha + \alpha'} h^{\beta + \beta'} = \\ &= g^\alpha h^\beta g^{\alpha'} h^{\beta'} = \psi(\alpha, \beta) \psi(\alpha', \beta')\end{aligned}$$

- **Chaum-Pedersen Protocol:** Here we have $\mathbb{H} = \mathbb{Z}_q$, $\mathbb{T} = \mathbb{G}^2$, and $\psi : \mathbb{Z}_q \rightarrow \mathbb{G}^2$ is defined as $\psi(\beta) = (g^\beta, u^\beta)$. Again, it is easy to see that ψ is a homomorphism.

Now, we formulate the general Sigma protocol for the relation $\mathcal{R}$ over homomorphism.

Definition 12.14 (Sigma Protocol for the pre-image of a homomorphism). The protocol consists of two algorithms: $(\mathcal{P}, \mathcal{V})$, where the prover is assumed to know the witness $h \in \mathbb{H}$ defined above. The protocol is defined as follows:

1. $\mathcal{P}$ computes $h_r \leftarrow \mathbb{H}$, $t_r \leftarrow \psi(h_r) \in \mathbb{T}$ and sends t_r to the verifier $\mathcal{V}$.
2. $\mathcal{V}$ samples the challenge $c \leftarrow \mathcal{C} \subset \mathbb{Z}$ from the challenge space and sends c to $\mathcal{P}$.
3. $\mathcal{P}$ computes $h_z \leftarrow h_r \oplus h \cdot c$ and sends h_z to $\mathcal{V}$.
4. $\mathcal{V}$ checks whether $\psi(h_z) = t_r \otimes t^c$, and accepts or rejects the proof accordingly.

12.6 Combining Sigma Protocols

Now, suppose we have the Sigma interactive protocol $(\mathcal{P}_0, \mathcal{V}_0)$ for one relation $\mathcal{R}_0 \subseteq \mathcal{W}_0 \times \mathcal{X}_0$ and another Sigma interactive protocol $(\mathcal{P}_1, \mathcal{V}_1)$ for another relation $\mathcal{R}_1 \subseteq \mathcal{W}_1 \times \mathcal{X}_1$. Now, we want to combine these two protocols into a single one. Namely, we want our prover to be able to convince the verifier that:

1. He knows the witnesses w_0, w_1 to both statements x_0, x_1 .
2. He knows the witness $w \in \mathcal{W}_0 \cup \mathcal{W}_1$ to either statement x_0 or x_1 .

Among two, the second one is a bit more interesting since it allows us to prove the knowledge of a witness to either of the statements. This is called the *OR-composition* of Sigma protocols.

12.6.1 The AND Sigma Protocol

Now, let the prover $\mathcal{P}$ prove the witness knowledge of the following relation:

$$\mathcal{R}_{\text{AND}} = \{((x_0, x_1), (w_0, w_1)) \in (\mathcal{X}_0 \times \mathcal{X}_1) \times (\mathcal{W}_0 \times \mathcal{W}_1) : (w_0, x_0) \in \mathcal{R}_0 \wedge (w_1, x_1) \in \mathcal{R}_1\}$$

We define the following protocol.

Definition 12.15 (The AND Sigma Protocol). Define a pair of algorithms $(\mathcal{P}, \mathcal{V})$ which are run as follows:

1. The prover $\mathcal{P}$ runs $\mathcal{P}_0(w_0, x_0)$ to get commitment a_0 and runs $\mathcal{P}_1(w_1, x_1)$ to get a_1 and sends the pair $\mathbf{a} = (a_0, a_1)$ to $\mathcal{V}$.
2. The verifier computes the challenge $c \leftarrow \$ \mathcal{C}$ and sends it to $\mathcal{P}$.
3. The prover feeds provers $\mathcal{P}_0(w_0, x_0)$ and $\mathcal{P}_1(w_1, x_1)$ with the challenge to get responses z_0 and z_1 , respectively. He then sends $\mathbf{z} = (z_0, z_1)$ to $\mathcal{V}$.
4. The verifier checks whether both $\mathcal{V}_0(a_0, c, z_0)$ and $\mathcal{V}_1(a_1, c, z_1)$ pass.

However, such protocol is not very interesting since what we did essentially is just running two protocols separately: one for $(\mathcal{P}_0, \mathcal{V}_0)$, and the other for $(\mathcal{P}_1, \mathcal{V}_1)$. The only difference is that we use the single challenge for both protocols.

12.6.2 The OR Sigma Protocol

The less trivial example is the following: define the relation

$$\mathcal{R}_{\text{OR}} = \{((x_0, x_1), (w, b)) \in (\mathcal{X}_0 \times \mathcal{X}_1) \times ((\mathcal{W}_0 \cup \mathcal{W}_1) \times \{0, 1\}) : (x, w_b) \in \mathcal{R}_b\}$$

Here, the statement is x_0 and x_1 , but the witness is the witness w to either x_0 or x_1 , and the bit $b \in \{0, 1\}$, marking to which of the statement w belongs to. That being said, w might be from either set $\mathcal{W}_0$ or $\mathcal{W}_1$: that is why we say that $w \in \mathcal{W}_0 \cup \mathcal{W}_1$.

To make the interactive protocol work, we add one more assumption about both relations $\mathcal{R}_0$ and $\mathcal{R}_1$. Suppose that the challenge space $\mathcal{C} \subseteq \{0, 1\}^\ell$. This assumption is not very strong as typically $\mathcal{C}$ is some subspace of integers and thus decomposing some $c \in \mathcal{C}$ into the fixed-length bit representation is a trivial task. Now, we describe the algorithm.

Definition 12.16 (The OR Sigma Protocol). Define a pair of algorithms $(\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}_{\text{OR}}$ with $b^* := 1 - b$ as follows:

1. The prover chooses a random challenge $c_{b^*} \leftarrow \$ \mathcal{C}$ and generates random commitment and response (a_{b^*}, z_{b^*}) that form a valid accepting conversation $(a_{b^*}, c_{b^*}, z_{b^*})$ (essentially, the prover runs the simulator $(a_{b^*}, z_{b^*}) \leftarrow \$ \text{Sim}_{b^*}(x_{b^*}, c_{b^*})$). Then, $\mathcal{P}$ also runs $\mathcal{P}_b(x_b, w)$ to get a valid commitment a_b and sends (a_0, a_1) to $\mathcal{V}$.
2. The verifier sends a random challenge $c \leftarrow \$ \mathcal{C} \subseteq \{0, 1\}^\ell$.
3. The prover XORs both challenges: $c_b \leftarrow c \oplus c_{b^*}$. Then it feeds the challenge c_b to the prover $\mathcal{P}_b(x_b, w)$ to get the responses z_b ($b \in \{0, 1\}$) and sends (c_0, z_0, z_1) to $\mathcal{V}$.
4. Verifier computes $c_1 \leftarrow c \oplus c_0$ and checks that both verifications $\mathcal{V}_0(a_0, c_0, z_0)$ and $\mathcal{V}_1(a_1, c_1, z_1)$ pass.

12.7 Okamoto Protocol Sage Implementation

Now, let us implement the Okamoto Protocol from [Section 12.4.1](#) in SageMath! We propose to make the proof over the secp256k1 curve, which is widely used in Bitcoin. We define it as follows (we shorten the curve parameters for brevity):

```
p = 2**256-2**32-2**9-2**8-2**7-2**6-2**4-1
Fp = GF(p)
a = 0
b = 7
E = EllipticCurve(Fp, [a, b])
G = E((55066...29240, 32670...82424))

# Group order
q = 115792...494337
Fq = FiniteField(q)
```

Now, recall that the Okamoto protocol is used to prove the knowledge of a representation $u = g^\alpha h^\beta$ of a given $u \in \mathbb{G}$ relative to g and h . This means that we also need to define the point $h \in \mathbb{G}$. Just for simplicity, let us take some random point on the curve:

```
random_salt = 14159...44592
H = random_salt * G
```

12.7.1 Prover Implementation

Let us start from the prover's side. Let us first generate a random representation for him:

```
alpha = Fq.random_element()
beta = Fq.random_element()
u = alpha * G + beta * H # This is the value we want to
    ↪ prove knowledge of
```

Now, the prover needs to generate the random commitment. This is done by choosing two random scalars $\alpha_r, \beta_r \leftarrow \mathbb{Z}_q$ and computing $u_r = g^{\alpha_r} h^{\beta_r}$:

```
# Next, we compute the commitment
alpha_r = Fq.random_element()
beta_r = Fq.random_element()
u_r = alpha_r * G + beta_r * H
```

Next, we need to compute the challenge. The challenge is computed using the Fiat-Shamir heuristic. We hash the commitment u_r and the public parameter u to get the challenge c : $c \leftarrow H(u, u_r)$. The only caveat, though, is that we need to implement the hash function $H : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q$. For simplicity, let us take the hash function that concatenates the x -coordinates of the points and hashes the

result: $H(u, u_r) = \text{SHA256}(u_x \parallel (u_r)_x)$:

```
def compute_hash(u: E, u_r: E) -> Fq:
    """
    Given two points u and u_r, compute the hash
    """

    result = str(u[0])+str(u_r[0]) # Concatenate x values
    hexdigest = result.encode('utf-8') # Compute the hash
    return Fq('0x'+sha256(hexdigest).hexdigest()) #
    ↪ Return the result as an element of Fq
```

So now we can compute the challenge c :

```
# Next, we compute the challenge
c = compute_hash(u, u_r)
```

Finally, the prover computes the response (as if he received the challenge c randomly from the verifier). It consists of two values: $\alpha_z \leftarrow \alpha_r + \alpha c$ and $\beta_z \leftarrow \beta_r + \beta c$. Finally, the proof is simply the tuple $\pi = (u_r, \alpha_z, \beta_z)$. Let us write this down:

```
# Finally, we compute the response
alpha_z = alpha_r + alpha * c
beta_z = beta_r + beta * c

# The proof is the tuple (u_r, alpha_z, beta_z)
proof = (u_r, alpha_z, beta_z)
```

The prover is pretty much ready. The prover can publish these three values and anyone can verify their correctness. An example proof can be seen below:

proof.json

```
{
  "u": {
    "x": "11050591345070626365101441740
          82670483149869135024966242
          0811897099927372480071",
    "y": "1157074364076179484338
          713344214935372053569769586171
          54462107638165337575096741"
  },
  "u_r": {
    "x": "8574306559776637845628844
          48547110595224508175437242645427
          65740484898155470860",
    "y": "523740737975951915508
          947372920060058211234306708764
          52163938960831568703982553"
  },
  "alpha_z": "431003649207811109284441
              06669886824502097947434355785
              088419275337575495885477",
  "beta_z": "1922706121205495962866045
             80605512169180433142550168819
             52138699213252756999901"
}
```

12.7.2 Verifier Implementation

Finally, the verifier code is extremely simple. All the verifier has to do is to restore the challenge $c \leftarrow H(u, u_r)$ from u and u_r values, specified in proof π , and then check whether $g^{\alpha_z} h^{\beta_z} = u_r u^c$. Also assume that the verifier can get these parameters by calling the method `read_proof`.

```
# Read the proof from the file
(u, u_r, alpha_z, beta_z) = read_proof()

c = compute_hash(u, u_r)
check = (alpha_z * G + beta_z * H) == u_r + c * u

if check:
    print('Proof verified successfully!')
else:
    print('Proof verification failed!')
```

The proof above passes perfectly.

Acknowledgements

This lecture was greatly inspired by “[A Graduate Course in Applied Cryptography](#)” by Dan Boneh and Victor Shoup, in particular the Section 19: “Identification and Signatures from Σ protocols”.

12.8 Exercises

Exercises 1-5. In search of correct Schnorr's Identification Protocol...

You are given the protocol and five ways to implement it. Most of them lack the crucial properties. For each attempt, you need to determine whether the protocol is correct and, if not, specify which of the properties are violated.

Recall, that given the cyclic group $\mathbb{G}$ of order q , the prover wants to convince the verifier that he knows the discrete logarithm α of $h \in \mathbb{G}$ with respect to the generator $g \in \mathbb{G}$ (so that $g^\alpha = h$).

Here are five attempts to construct the protocol:

Attempt 1. Prover sends witness α to the verifier. Verifier checks whether $h = g^\alpha$.

Attempt 2. Prover chooses random $r \leftarrow \mathbb{Z}_q$ and sends $a \leftarrow \alpha + r$ to the verifier. Verifier checks whether $h = g^a$.

Attempt 3. Prover chooses random $r \leftarrow \mathbb{Z}_q$, calculates $a \leftarrow \alpha + r$ and sends both (a, r) to the verifier. Verifier checks whether $g^r h = g^a$.

Attempt 4. Prover chooses random $r \leftarrow \mathbb{Z}_q$, calculates $a \leftarrow g^r$, $z \leftarrow \alpha + r$ and sends (a, z) to the verifier. Verifier checks whether $a \cdot h = g^z$.

Attempt 5. Prover chooses random $r \leftarrow \mathbb{Z}_q$, calculates $a \leftarrow g^r$, and sends a to the verifier. Verifier chooses $e \leftarrow \mathbb{Z}_q$ and sends to the prover. Prover calculates $z \leftarrow \alpha e + r$ and sends to the prover. Verifier checks whether $a \cdot h^e = g^z$.

Below, mark whether the properties of *completeness*, *soundness*, and *zero-knowledge* hold for each attempt.

Attempt #	1	2	3	4	5
Completeness holds?	✓/✗	✓/✗	✓/✗	✓/✗	✓/✗
Soundness holds?	✓/✗	✓/✗	✓/✗	✓/✗	✓/✗
Zero-Knowledge holds?	✓/✗	✓/✗	✓/✗	✓/✗	✓/✗

Exercises 6-10. Non-Interactive Chaum-Pedersen Protocol.

This section explores how to make the previously considered Chaum-Pedersen protocol non-interactive. Fill in the gaps in the following text with the correct statements.

Recall that the Chaum-Pedersen protocol allows the prover $\mathcal{P}$ to convince the skeptical verifier $\mathcal{V}$ that the given triplet $(u, v, w) \in \mathbb{G}^3$ is a Diffie-Hellman (DH) triplet in the cyclic group $\mathbb{G}$ of prime order q with a generator $g \in \mathbb{G}$, meaning that $u = g^\alpha, v = g^\beta, w = g^{\alpha\beta}$ for some $\alpha, \beta \in \mathbb{Z}_q$. However, instead of making (α, β) as a witness, observe that β is sufficient. Indeed, if $u = g^\alpha, v = g^\beta$, then $w = \boxed{6}$. Thus, the relation

is:

$$\mathcal{R} = \left\{ ((u, v, w), \beta) \in \mathbb{G}^3 \times \mathbb{Z}_q : \boxed{7} \right\}$$

Now, we apply the *Fiat-Shamir Transformation*. Recall that prover, instead of getting the random Fiat challenge $c \leftarrow \mathcal{C} \subset \mathbb{Z}_q$ from the verifier interactively, calculates it as the hash function from the public statement (u, v, w) and the prover's commitment. For that reason, define the non-interactive proof system $\Phi = (\text{Gen}, \text{Verify})$ as follows:

- Gen: On input $(u, v, w) \in \mathbb{G}^3$,
 1. Sample $\beta_r \leftarrow \mathbb{Z}_q$ and compute the commitment $\boxed{8}$.
 2. Use the hash function $\boxed{9}$ to get the challenge $c \leftarrow \boxed{10}$.
 3. Compute response $\beta_z \leftarrow \beta_r + \beta c$ and output commitment (v_r, w_r) and β_z as a proof π .
- Verify: Upon receiving statement (u, v, w) and a proof $\pi = (v_r, w_r, \beta_z)$, the verifier:
 1. Recomputes the challenge c using the hash function.
 2. Accepts if and only if $g^{\beta_z} = v_r v^c$ and $u^{\beta_z} = w_r w^c$.

Exercise 6.

- a) v^β b) u^β c) vu d) v^u

Exercise 7.

- a) $v = g^\beta$ and $w = vu$ c) $v = g^\beta$ and $w = u^\beta$
 b) $v = g^\beta$ and $w = v^\beta$ d) $u = g^\beta$ and $w = u^\beta$

Exercise 8.

- a) $(v_r, w_r) = (g^{\beta_r}, g^{\beta_r \beta})$ c) $(v_r, w_r) = (g^{\beta_r}, u^{\beta_r})$
 b) $(v_r, w_r) = (g^{\beta_r}, w^{\beta_r})$ d) $(v_r, w_r) = (g^\beta, g^{\beta_r})$

Exercise 9.

- a) $H : \mathbb{G}^3 \times \mathbb{G}^2 \rightarrow \mathcal{C}$ c) $H : \mathbb{G}^3 \rightarrow \mathcal{C}$
 b) $H : \mathbb{G}^3 \times (\mathbb{G} \times \mathbb{Z}_q) \rightarrow \mathcal{C}$ d) $H : \mathbb{G}^3 \times \mathbb{Z}_q \rightarrow \mathcal{C}$

Exercise 10.

- a) $H((u, v, w), (v_r, w_r))$ c) $H(u, v, w)$
 b) $H((u, v, w), (v_r, \beta_r))$ d) $H((u, v, w), \beta_r)$

13 Introduction to SNARKs. Arithmetic Circuits. R1CS

13.1 What is zk-SNARK?

13.1.1 Informal Overview

Finally, we've reached the most interesting part of the course, where we will consider various zk-SNARK constructions we are using on the daily basis. Again, recall that we have the presence of two parties:

- **Prover** $\mathcal{P}$ — the party who knows the data that resolves the given problem.
- **Verifier** $\mathcal{V}$ — the party that wants to verify the given proof.

Here, the prover wants to convince the verifier that they know the data that resolves the problem (typically, some complex computation) without revealing the data (witness) itself. In the previous lecture, we defined the first practical primitive: zk-NARK — a *zero-knowledge non-interactive argument of knowledge*, and gave the first widely used example: non-interactive Schnorr protocol (which is a special case of a Σ -protocol with the Fiat-Shamir transformation applied). Now, we add one more component which completely changes the game and significantly extends the number of applications: **succinctness**.

Definition 13.1. zk-SNARK — Zero-Knowledge **Succinct** Non-interactive Argument of Knowledge.

Again, since this is a central question considered, we need to recall what do terms like “argument of knowledge”, “succinct”, “non-interactive”, and “zero-knowledge” mean in this context:

- **Argument of Knowledge** — a proof that the prover knows the data (witness) that resolves a certain problem, and this knowledge can be “extracted”.
- **Succinctness** — the proof size and verification time is relatively small relative to the computation size and sometimes even does not depend on the size of the data or statement. This will be explained with examples later.
- **Non-interactiveness** — to produce the proof, the prover does not need any interaction with the verifier.
- **Zero-Knowledge** — the verifier learns nothing about the data used to produce the proof, despite knowing that this data resolves the given problem and that the prover possesses it.

In essence, zk-SNARKs allow one party to prove to another that they know a value without revealing any information about the value itself, and do so with a proof that is both very small and quick to verify. This makes zk-SNARKs a powerful tool for maintaining privacy and efficiency in various cryptographic applications.

This is pretty wide defined and maybe not so obvious if you do not have any background. Let us take a look at the example.

Example 13.1. Imagine you are the part of a treasure hunt, and you've found a hidden treasure chest. You want to prove to the treasure hunt organizer that you know where the chest is hidden without revealing its location. Here's how zk-SNARKs can be used in this context:

The problem: you have found a hidden treasure chest (the secret data), and you want to prove to the organizer (the verifier) that you know its location without actually revealing where it is.

How zk-SNARKs Help:

- **Argument of Knowledge:** You create a proof that demonstrates you know the exact location of the treasure chest. This proof convinces the organizer that you have this knowledge.
- **Succinctness:** The proof you provide is very small and concise. It doesn't matter how large the treasure map is or how many steps it took you to find the chest, the proof remains compact and easy to check.
- **Non-interactiveness:** You don't need to have a back-and-forth conversation with the organizer to create this proof. You prepare it once. The organizer can verify it without needing to ask you any questions.
- **Zero-Knowledge:** The proof doesn't reveal any information about the actual location of the treasure chest. The organizer knows you found it, but they don't learn anything about where it is hidden.

Here you can think of zk-SNARK as a golden coin from the chest where the pirates' sign is engraved, so the organizer can be sure you've found the treasure.

But the problems that we want to solve are in a slightly different format. We can't bring a coin to the verifier. Our goal is to prove that we've executed a specific program on a set of data that resolves a specific challenge or gives us a particular result.

13.1.2 Formal Definition

In this section, we will provide a more formal definition of zk-SNARKs. In case you do not want to dive into the technical details, you can skip this part and move to the next sections where we will consider the arithmetic circuits and the Quadratic Arithmetic Programs.

Previously, we considered NARKs that did not require any setup procedure. However, zk-SNARKs are more complex and require a setup phase. This setup phase is used to generate the proving and verification keys (which we call prover

parameters pp and verifier parameters vp , respectively), which are then used to create and verify proofs. That being said, let us introduce the **preprocessing NARK**.

Definition 13.2. A **preprocessing non-interactive argument of knowledge (preprocessing NARK)** $\Pi_{\text{preNARK}} = (\text{Setup}, \text{Prove}, \text{Verify})$ consists of three algorithms:

- $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow (\text{pp}, \text{vp})$ – the setup algorithm that takes the security parameter λ and relation $\mathcal{R}$, and outputs the public parameters: proving and verification keys.
- $\text{Prove}(\text{pp}, \mathbb{x}, \mathbb{w}) \rightarrow \pi$ – the proving algorithm that takes the prover parameters pp , statement $\mathbb{x}$, and witness $\mathbb{w}$, and outputs a proof π .
- $\text{Verify}(\text{vp}, \mathbb{x}, \pi) \rightarrow \{\text{accept}, \text{reject}\}$ – the verification algorithm that takes the verification key, statement $\mathbb{x}$, and proof π , and outputs a bit indicating whether the proof is valid.

Recall, that from NARK (and now preprocessing NARK, respectively) over relation $\mathcal{R}$ we require the following properties:

- **Completeness** – if the prover is honest and the statement is true, the verifier will always accept the proof:

$$\forall (\mathbb{x}, \mathbb{w}) \in \mathcal{R} : \Pr[\text{Verify}(\text{vp}, \mathbb{x}, \text{Prove}(\text{pp}, \mathbb{x}, \mathbb{w})) = \text{accept}] = 1$$

- **Knowledge Soundness** – the prover cannot (statistically) generate a false proof π that convinces the verifier.
- **Zero-knowledge** – the verifier “learns nothing” about the witness $\mathbb{w}$ from $(\mathcal{R}, \text{pp}, \text{vp}, \mathbb{x}, \pi)$.

While we have formally defined all the terms here, including statistical soundness, we have not defined what **knowledge soundness** is. We give a brief informal definition below.

Definition 13.3 (Knowledge Soundness). Π_{preNARK} is (adaptively) **knowledge sound** for a relation $\mathcal{R}$ if for every PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$, split into two algorithms, such that:

$$\Pr \left[\text{Verify}(\text{vp}, \mathbb{x}, \pi) = \text{accept} \mid \begin{array}{l} (\text{pp}, \text{vp}) \leftarrow \text{Setup}(\cdot) \\ \mathbb{x} \leftarrow \mathcal{A}_0(\cdot) \\ \pi \leftarrow \mathcal{A}_1(\text{pp}, \mathbb{x}) \end{array} \right] > \alpha,$$

where $\alpha = \alpha(\lambda) \neq \text{negl}(\lambda)$ is a non-negligible probability, there exists a PPT extractor $\mathcal{E}^{\mathcal{A}}$ such that

$$\Pr [\mathbb{x}, \mathbb{w}) \in \mathcal{R} \mid \mathbb{x} \leftarrow \mathcal{A}_0(\cdot), \mathbb{w} \leftarrow \mathcal{E}^{\mathcal{A}}(\mathbb{x})] > \alpha - \epsilon,$$

where $\epsilon = \epsilon(\lambda)$ is a negligible function.

Remark. Informally, the aforementioned definition means that if the prover can generate a false proof with a non-negligible probability, then there exists an extractor that can extract the witness with a probability that is almost as high (and thus is also non-negligible).

Finally, to make zk-NARKs more universal and applicable to a wider range of problems, we introduce the **zk-SNARK** by adding the **succinctness** property.

Definition 13.4. A **zk-SNARK** (Succinct NARK) is a preprocessing NARK, where the proof's length $|\pi|$ and verification time T_V are short: the verification time is sublinear in the size of the computation C (denoted by $|C|$), while the proof size is sublinear in the witness size $|\mathbf{w}|$:

$$|\pi| = \text{sublinear}(|\mathbf{w}|), \quad T_V = O_\lambda(|\mathbf{x}|, \text{sublinear}(|C|)).$$

Remark. Sublinearity means that the function $f : \mathbb{N} \rightarrow \mathbb{R}$ grows slower than linearly. For example, functions $f(n) = \log n$, $f(n) = \sqrt{n}$, or $f(n) = n/\log n$ are all sublinear, while $f(n) = 3n + 2$ is linear. Generally, if $\lim_{n \rightarrow \infty} f(n)/(c \cdot n) = 0$ for any $c \in \mathbb{R} \setminus \{0\}$, then $f(n)$ is sublinear.

Example 13.2. Consider the protocol where the proof size is $|\pi| = O(\sqrt{|\mathbf{w}|})$ and $T_V = O(\sqrt[3]{|C|})$. Such protocol is a zk-SNARK, as the proof size is sublinear in the witness size and the verification time is sublinear in the size of the computation.

Although having a proof size and verification time lower than linear is nice, that is still not sufficient to make zk-SNARKs practical in the wild. For that reason, typically, in practice, we require a stricter definition of the succinctness property, where the proof size and verification time are constant or logarithmic in the size of the computation. This is the case for most zk-SNARKs used in practice.

Definition 13.5. A **zk-SNARK** is **strongly succinct** if the proof size and verification time are constant or logarithmic in the size of the computation:

$$|\pi| = O_\lambda(\log |C|), \quad T_V = O_\lambda(|\mathbf{x}|, \log |C|).$$

Example 13.3. Consider three major proving systems used in practice with $N = |C|$ being the complexity of a computation:

- **Groth16** (see [Section 15](#)) with $|\pi| = O_\lambda(1)$, $T_V = O_\lambda(1)$ is definitely a strongly succinct zk-SNARK since both the proof size and verification time are constant.
- **STARKs** with $|\pi| = O_\lambda(\text{polylog}(N))$ and $T_V = O_\lambda(\text{polylog}(N))$ are also strongly succinct zk-SNARKs since both the proof size and verification time are polylogarithmic in the size of the computation.

tion time are logarithmic in the size of the computation.

- **Bulletproofs** (see Section 18) with $|\pi| = O_\lambda(\log N)$ and $T_V = O_\lambda(N)$ is not a strongly succinct zk-SNARK since the verification time is linear in the size of the computation.

13.2 Arithmetic Circuits

13.2.1 What is Arithmetic Circuit?

The cryptographic tools we have learned in the previous lectures operate with numbers or certain primitives above them (like finite field extensions or elliptic curves), so the first question is: how do we convert a program into a mathematical language? Additionally, we need to do this in a way that can be further (a) made succinct, (b) allows us to prove something about it, and (c) be as universal as possible (to be able to prove quite general statements unlike Σ -protocols considered in the previous lecture).

The **Arithmetic Circuits** can help us with these problems. Similar to **Boolean Circuits**, they consist of **gates** and **wires**: gates represent operations acting all elements, connected by wires (see figure below for details). Yet, instead of operations AND, OR, NOT and such, in arithmetic circuits only multiplication/addition/subtraction operations are allowed. Additionally, arithmetic circuits manipulate over elements from some finite field $\mathbb{F}$ (see right figure below).

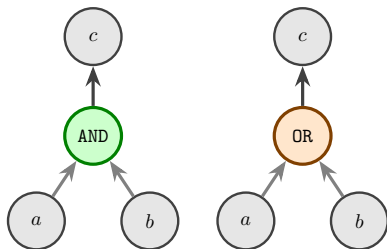


Figure 13.1: Boolean AND and OR Gates

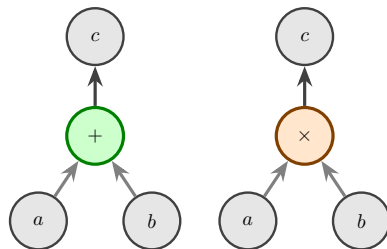


Figure 13.2: Addition and Multiplication Gates

Let us come back to boolean circuits for a moment and consider the AND gate. The *AND Gate Truth Table 4* shows us the results we receive if particular values are supplied to the gate. The main point here is that with this table, we can verify the validity of logical statements. Boolean circuits receive an input vector of $\{0, 1\}$ and resolve to true (1) or false (0); basically, they determine if the input values satisfy the statement.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Table 4: AND Gate Truth Table

However, more notably, we can combine these gates to create more complex circuits that can resolve more complex problems. For example, we might construct a circuit depicted in Figure 13.3, calculating $(a \text{ AND } b) \text{ OR } c$.

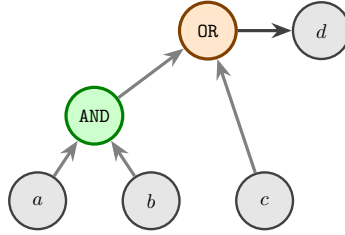


Figure 13.3: Example of a circuit evaluating $d = (a \text{ AND } b) \text{ OR } c$.

Although we can already represent very complex computations using boolean circuits¹², they are not the most convenient way to represent arithmetic operations.

That being said, we can do the same with **arithmetic circuits** to verify computations over some finite field $\mathbb{F}$ without excessive verbosity due to a binary arithmetic, where we had to perceive all intermediate values as binary $\{0, 1\}$.

13.2.2 More advanced examples

Let us take a look at some examples of programs and how can we translate them to the arithmetic circuits.

Example 1: Multiplication. Consider a very simple program, where we are to simply multiply two field elements $a, b \in \mathbb{F}$:

```
def multiply(a: F, b: F) -> F:
    return a * b
```

Since we are doing all the arithmetic in a finite field $\mathbb{F}$, we denote it by F in the code. This can be represented as a circuit with only one (multiplication) gate:

$$r = a \times b$$

The witness vector (essentially, our solution vector) is $\mathbf{z} = (r, a, b)$, for example: $(6, 2, 3)$. We assume that the a and b are input values.

We can think of the “=” in the gate as an assertion, meaning that if $a \times b$ does not equal r , the assertion fails, and the input values do not resolve the circuit.

Good, but this one is quite trivial. Let’s consider a more complex example.

Example 2: Multivariate Polynomial. Now, suppose we want to implement the evaluation of the polynomial $Q(x_1, x_2) = x_1^3 + x_2^2 \in \mathbb{F}[X_1, X_2]$ using arithmetic circuits. The corresponding program is as follows:

¹²such as SHA-256 hash function computation; one might take a look at the example implementation here: <http://stevengoldfeder.com/projects/circuits/sha2circuit.html>.

```
def evaluate(x1: F, x2: F) -> F:
    return x1**3 + x2**2
```

Looks easy, right? But the circuit is now much less trivial. Consider Figure 13.5. Notice that to calculate x_1^3 we cannot use the single gate: we need to multiply x_1 by itself two times. For that reason, we need three multiplication and one addition gate to represent $Q(x_1, x_2)$ calculation.

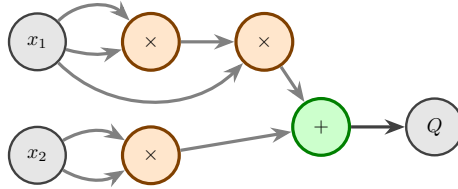


Figure 13.4: Example of a circuit evaluating $x_1^3 + x_2^2$.

Example 3. if statements. Well, it is quite clear how to represent any polynomial-like expressions. But how can we translate if statements? Consider the program below:

```
def if_statement_example(a: bool, b: F, c: F) -> F:
    return b * c if a else b + c
```

We can express this logic in mathematical terms as follows: “If a is true, compute $b \times c$; otherwise, compute $b + c$.” However, only numerical expressions are allowed, so how can we proceed? Assuming that `true` is represented by 1 and `false` by 0, we can transform this logic as follows:

$$r = a \times (b \times c) + (1 - a) \times (b + c)$$

Now, what is the witness vector in this case? One might assume that $\mathbf{z} = (r, a, b, c)$ would suffice. Then, examples of valid witnesses include $(6, 1, 2, 3)$, $(5, 0, 2, 3)$.

But, we need to verify all the intermediate steps! This can be achieved by transforming the above equation using the simplest terms (the gates), ensuring the correctness of each step in the program.

Below, we show to visualize the arithmetic circuit for the if statement example. Corresponding equations for the circuit are:

$$\begin{aligned} r_1 &= b \times c & r_2 &= b + c \\ r_3 &= 1 - a & r_4 &= a \times r_1 \\ r_5 &= r_3 \times r_2 & r &= r_4 + r_5 \end{aligned}$$

With the witness vector: $\mathbf{z} = (r, r_1, r_2, r_3, r_4, r_5, a, b, c)$. One example of a valid witness is $(6, 6, 5, 0, 6, 0, 1, 2, 3)$.

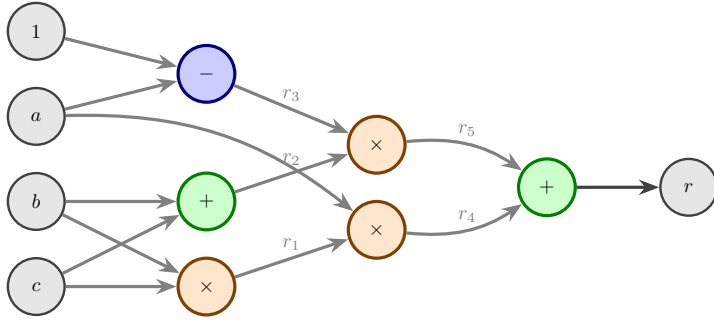


Figure 13.5: Example of a circuit evaluating the if statement logic.

13.2.3 Circuit Satisfiability Problem

Now, let us generalize what we have constructed so far. First, we begin with the arithmetic circuit.

Definition 13.6. Arithmetic circuit $\mathcal{C} : \mathbb{F}^n \rightarrow \mathbb{F}$ with n inputs over a finite field $\mathbb{F}$ is a directed acyclic graph where internal nodes are labeled via $+$, $-$, and $\times$, and inputs are labeled $1, x_1, x_2, \dots, x_n$. By $|\mathcal{C}|$ we denote the number of gates in the circuit.

Example 13.4. For example, previously considered multivariate polynomial $\mathcal{C}(x_1, x_2) = x_1^3 + x_2^2$ can be represented as an arithmetic circuit with three multiplication and one addition gates, as shown in Figure 13.5. It is defined over inputs $\mathbf{x} = (x_1, x_2)$ with $n = 2$ and $|\mathcal{C}| = 4$.

Now, suppose that the circuit is defined over n inputs. We can always split this input into two parts: the first ℓ inputs are the *public inputs*, being our statement $\mathbf{x} \in \mathbb{F}^\ell$, and the remaining $n - \ell$ inputs are the *private inputs*, being our secret witness $\mathbf{w} \in \mathbb{F}^{n-\ell}$. The public inputs are known to everyone, while the private inputs are known only to the prover. The goal of the prover is to show that the circuit is satisfiable, i.e., that for the given $\mathbf{x}$, he *knows* a witness $\mathbf{w}$ that resolves the circuit. Resolving in this context means that the output of the circuit is zero.

Definition 13.7. The **Circuit Satisfiability Problem** is defined as follows: given an arithmetic circuit $\mathcal{C}$ and a public input $\mathbf{x} \in \mathbb{F}^\ell$, determine if there exists a private input $\mathbf{w} \in \mathbb{F}^{n-\ell}$ such that $\mathcal{C}(\mathbf{x}, \mathbf{w}) = 0$. More formally, the problem is determined by relation $\mathcal{R}_{\mathcal{C}}$ and corresponding language $\mathcal{L}_{\mathcal{C}}$ as follows:

$$\begin{aligned}\mathcal{R}_{\mathcal{C}} &= \{(\mathbf{x}, \mathbf{w}) \in \mathbb{F}^\ell \times \mathbb{F}^{n-\ell} : \mathcal{C}(\mathbf{x}, \mathbf{w}) = 0\} \\ \mathcal{L}_{\mathcal{C}} &= \{\mathbf{x} \in \mathbb{F}^\ell : \exists \mathbf{w} \in \mathbb{F}^{n-\ell}, \mathcal{C}(\mathbf{x}, \mathbf{w}) = 0\}\end{aligned}$$

Let us consider some concrete example of the Circuit Satisfiability Problem.

Example 13.5. Suppose our problem (as a prover) is to prove the verifier that we know the point on the circle of “radius $\sqrt{\rho}$ ”, but over the finite field $\mathbb{F}$. More formally, suppose we want to claim that for the given ρ , we have $x_1, x_2 \in \mathbb{F}$ such that:

$$x_1^2 + x_2^2 = \rho$$

For that reason, define the circuit $\mathcal{C}(\rho, x_1, x_2) := x_1^2 + x_2^2 - \rho$. It is constructed as shown in the Figure below.

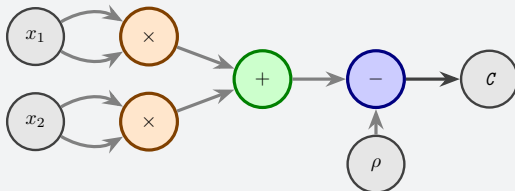


Illustration: Arithmetic circuit for the equation $x_1^2 + x_2^2 - \rho$.

Now, our statement vector is $\mathbf{x} = \rho \in \mathbb{F}$ (so $\ell = 1$) and the witness vector is $\mathbf{w} = (x_1, x_2) \in \mathbb{F}^2$ (so $n - \ell = 2$). The prover wants to prove that he knows the witness $\mathbf{w}$ such that $\mathcal{C}(\mathbf{x}, \mathbf{w}) = 0$. For example, for $\rho = 5$, the prover might have the witness $\mathbf{w} = (2, 1)$ that he wants to show to the verifier.

Now, as with any other previously considered proving systems, suppose we are not concerned about the zero-knowledge property and simply want to prove the evaluation integrity of the circuit. Can the prover simply send the witness $\mathbf{w}$ to the verifier? Prover can send the witness, but this will not be a SNARK (and, surely, not a zk-SNARK either).

Proposition 13.8 (Trivial SNARK is not a SNARK). The protocol in which $\mathcal{P}$ sends the witness $\mathbf{w}$ to $\mathcal{V}$ is not a SNARK for the Circuit Satisfiability Problem. Indeed, in this case, the proof size is $|\pi| = |\mathbf{w}|$ and the verification time is $T_{\mathcal{V}} = O(|\mathcal{C}|)$ (since $\mathcal{C}$ must be evaluated fully). We do not have succinctness (not even mentioning the strong succinctness) in this case.

Proposition above motivates us to look for more advanced techniques to prove the satisfiability of the arithmetic circuits. In the next section, we introduce the Rank-1 Constraint System, which is a more flexible and general way to describe the arithmetic circuits, allowing to further encode the constraints in a more succinct way.

13.3 Rank-1 Constraint System

Almost any program written in high-level programming language can be translated (compiled) into arithmetic circuits, that are really powerfull tool. But

for the ZK proof we need slightly different format of it — **Rank-1 Constraint System**, where the simplest term is **constraint**. This offers a more flexible and general way to describe these parts.

13.3.1 Constraint Definition

Let us define the R1CS constraint.

Definition 13.9. Each **constraint** in the Rank-1 Constraint System must be in the form:

$$\langle \ell, \mathbf{z} \rangle \times \langle \mathbf{r}, \mathbf{z} \rangle = \langle \mathbf{o}, \mathbf{z} \rangle$$

Where $\mathbf{z}$ is a vector containing all the *input*, *output*, and *intermediate* variables involved in the computation. The vectors ℓ , $\mathbf{r}$, and $\mathbf{o}$ are vectors of coefficients corresponding to these variables, and they define the relationship between the linear combinations of $\mathbf{z}$ on the left-hand side and the right-hand side of the equation.

Example 13.6. Consider the most basic circuit with one multiplication gate:

$$r = x_1 \times x_2$$

Since we have 3 variables, the constraint is written as:

$$(a_1 z_1 + a_2 z_2 + a_3 z_3) \cdot (b_1 z_1 + b_2 z_2 + b_3 z_3) = c_1 z_1 + c_2 z_2 + c_3 z_3$$

Coefficients and witness vectors are: $\mathbf{z} = (r, x_1, x_2)$, $\ell = (0, 1, 0)$, $\mathbf{r} = (0, 0, 1)$, $\mathbf{o} = (1, 0, 0)$. Therefore, our expression above reduces to:

$$(0z_1 + 1z_2 + 0z_3)(0z_1 + 0z_2 + 1z_3) = (1z_1 + 0z_2 + 0z_3)$$

$$z_2 \times z_3 = z_1$$

$$x_1 \times x_2 = r$$

An interesting question is how to form coefficient vectors. The lazy answer is straightforward: by placing 1 in the witness vector at position corresponding to the gate input variable.

Example 13.7. Now, let us consider a more complex example. Remember that we want to verify each computational step.

```
def r(x1: bool, x2: F, x3: F) -> F:
    return x2 * x3 if x1 else x2 + x3
```

We know that it can be expressed as:

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3)$$

However, one important consideration was overlooked. If x_1 is neither 0 nor 1, it implies that something else is being computed instead of the desired program. Since we need to add a restriction for x_1 : $x_1 \times (1 - x_1) = 0$, this effectively checks that x_1 is binary.

The next constraints can be build:

$$x_1 \times x_1 = x_1 \quad (\text{binary check}) \quad (1)$$

$$x_2 \times x_3 = \text{mult} \quad (2)$$

$$x_1 \times \text{mult} = \text{selectMult} \quad (3)$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \quad (4)$$

For every constraint we need the coefficients vectors a_i , b_i , c_i , but all of them have the same witness vector $\mathbf{z}$.

$$\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$$

The coefficients vectors:

$$\begin{array}{lll} \ell_1 = (0, 0, 1, 0, 0, 0, 0) & r_1 = (0, 0, 1, 0, 0, 0, 0) & o_1 = (0, 0, 1, 0, 0, 0, 0) \\ \ell_2 = (0, 0, 0, 1, 0, 0, 0) & r_2 = (0, 0, 0, 0, 1, 0, 0) & o_2 = (0, 0, 0, 0, 0, 1, 0) \\ \ell_3 = (0, 0, 1, 0, 0, 0, 0) & r_3 = (0, 0, 0, 0, 0, 1, 0) & o_3 = (0, 0, 0, 0, 0, 0, 1) \\ \ell_4 = (1, 0, -1, 0, 0, 0, 0) & r_4 = (0, 0, 0, 1, 1, 0, 0) & o_4 = (0, 1, 0, 0, 0, 0, -1) \end{array}$$

Now, let us use some specific values to compute an example. Using the arithmetic in a large finite field $\mathbb{F}_p$, consider the following values:

$$x_1 = 1, \quad x_2 = 3, \quad x_3 = 4$$

Verifying the constraints:

1. $x_1 \times x_1 = x_1 \quad (1 \times 1 = 1)$
2. $x_2 \times x_3 = \text{mult} \quad (3 \times 4 = 12)$
3. $x_1 \times \text{mult} = \text{selectMult} \quad (1 \times 12 = 12)$
4. $(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \quad (0 \times 7 = 12 - 12)$

Each constraint enforces that the product of the linear combinations defined by ℓ and r must equal the linear combination defined by o . Collectively, these constraints describe the computation by ensuring that every step, from inputs through intermediates to outputs, satisfies the defined relationships, thus encoding the entire computational process in the form of a system of rank-1 quadratic equations.

13.3.2 Why Rank-1?

The last unresolved question is where the “rank-1” comes from. Using the outer product we can express the constraint in another form.

Lemma 13.10. Suppose we have a constraint $\langle \ell, \mathbf{z} \rangle \times \langle \mathbf{r}, \mathbf{z} \rangle = \langle \mathbf{o}, \mathbf{z} \rangle$ with coefficient vectors $\ell, \mathbf{r}, \mathbf{o}$ and witness vector $\mathbf{z}$ (all from $\mathbb{F}^n$). Then it can be expressed in the form:

$$\mathbf{z}^\top T \mathbf{z} + \mathbf{o}^\top \mathbf{z} = 0$$

Where T is the outer product of vectors $\ell, \mathbf{r}$ (denoted as $\ell \otimes \mathbf{r}$), consequently a **rank-1** matrix.

Lemma proof. Consider the constraint $\langle \ell, \mathbf{z} \rangle \times \langle \mathbf{r}, \mathbf{z} \rangle = \langle \mathbf{o}, \mathbf{z} \rangle$, where $\ell, \mathbf{r}, \mathbf{o}, \mathbf{z} \in \mathbb{F}^n$. Let us expand the inner products:

$$\sum_{i=1}^n \ell_i z_i \cdot \sum_{i=1}^n r_i z_i = \sum_{i=1}^n o_i z_i$$

Combine the products into a double sum on the left side:

$$\sum_{i=1}^n \sum_{j=1}^n \ell_i r_j z_i z_j = \mathbf{z}^\top (\ell \otimes \mathbf{r}) \mathbf{z} = \mathbf{z}^\top T \mathbf{z}$$

Thus, the constraint can be written as:

$$\mathbf{z}^\top T \mathbf{z} + \mathbf{o}^\top \mathbf{z} = 0$$

So, the rank-1 means the rank of the coefficients matrix T in one of the constraint formats.

14 Quadratic Arithmetic Program. Probabilistically Checkable Proofs

14.1 R1CS in Matrix Form

While the Rank-1 Constraint System provides a powerful way to represent computations, it is not succinct at all, since the number of constraints depends linearly on the complexity of the problem being solved. In practical scenarios, this can require tens or even hundreds of thousands of constraints, sometimes even millions. The Quadratic Arithmetic Program (QAP) can address this issue.

Remark. Understanding polynomials and their properties is crucial for this section. If you are not confident in this area, it is better to revisit the corresponding chapter and refresh your knowledge. See [Section 5](#).

To define a constraint in the R1CS we need four vectors: three coefficient vectors (ℓ , r , and o) and the witness one — z . And that's just for one constraint. As you can imagine, many of the values in these vectors are zeros. In circuits with thousands of inputs, outputs, and auxiliary variables, where there are also thousands of constraints, you could end up with a millions of zeroes.

Remark. A matrix in which most of the elements are zero in numerical analysis is usually called **sparse matrix**.

So, we need to change the way how we manage coefficients and make the representation of such matrices and vectors succinct (as required by the definition of SNARK).

Theorem 14.1 (R1CS in the Matrix Form). Consider a Rank-1 Constraint System (R1CS) defined by m constraints. Each constraint is associated with coefficient vectors ℓ_i , r_i , and o_i , where $i \in [m]$. Suppose a witness vector z consists of n elements. Then, this system can also be represented using the corresponding $m \times n$ matrices L, R, O such that all constraints can be reduced to the single equation:

$$Lz \odot Rz = Oz$$

In this representation:

- Each i -th row of the matrices corresponds to the coefficients of a specific constraint.
- Each column of these matrices corresponds to the coefficients associated with a particular element of the witness vector z .

Proof. Matrices defined this way can be expressed as

$$L = \begin{bmatrix} \ell_1^\top \\ \ell_2^\top \\ \vdots \\ \ell_m^\top \end{bmatrix}, \quad R = \begin{bmatrix} r_1^\top \\ r_2^\top \\ \vdots \\ r_m^\top \end{bmatrix}, \quad O = \begin{bmatrix} o_1^\top \\ o_2^\top \\ \vdots \\ o_m^\top \end{bmatrix}$$

Consider an expression Lz :

$$Lz = \begin{bmatrix} \ell_1^\top \\ \ell_2^\top \\ \vdots \\ \ell_m^\top \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} \ell_1^\top z \\ \ell_2^\top z \\ \vdots \\ \ell_m^\top z \end{bmatrix}$$

The last equality is a bit tricky to observe, so let us explain how we ended up with such expression. Notice that since $L \in \mathbb{F}^{m \times n}$ and $z \in \mathbb{F}^n$, the product Lz is a vector from $\mathbb{F}^m$. Now, for j^{th} element of such vector, based on the matrix product definition, we have $(Lz)_j = \sum_{\ell=1}^n a_{j,\ell} z_\ell$ which is exactly an inner product between ℓ_j and z ! Therefore, we have:

$$Lz = \begin{bmatrix} \langle \ell_1, z \rangle \\ \langle \ell_2, z \rangle \\ \vdots \\ \langle \ell_m, z \rangle \end{bmatrix}, \quad Rz = \begin{bmatrix} \langle r_1, z \rangle \\ \langle r_2, z \rangle \\ \vdots \\ \langle r_m, z \rangle \end{bmatrix}, \quad Oz = \begin{bmatrix} \langle o_1, z \rangle \\ \langle o_2, z \rangle \\ \vdots \\ \langle o_m, z \rangle \end{bmatrix}$$

Therefore, $Lz \odot Rz = Oz$ is equivalent to the system of m constraints:

$$\langle \ell_j, z \rangle \times \langle r_j, z \rangle = \langle o_j, z \rangle, \quad j \in [m].$$

Example 14.1. The vectors ℓ_i from the previous examples have the form:

$$\begin{aligned} \ell_1 &= (0, 0, 1, 0, 0, 0, 0) \\ \ell_2 &= (0, 0, 0, 1, 0, 0, 0) \\ \ell_3 &= (0, 0, 1, 0, 0, 0, 0) \\ \ell_4 &= (1, 0, -1, 0, 0, 0, 0) \end{aligned}$$

This corresponds to $n = 7, m = 4$

$$L = \begin{bmatrix} L_{1,1} & L_{1,2} & L_{1,3} & L_{1,4} & L_{1,5} & L_{1,6} & L_{1,7} \\ L_{2,1} & L_{2,2} & L_{2,3} & L_{2,4} & L_{2,5} & L_{2,6} & L_{2,7} \\ L_{3,1} & L_{3,2} & L_{3,3} & L_{3,4} & L_{3,5} & L_{3,6} & L_{3,7} \\ L_{4,1} & L_{4,2} & L_{4,3} & L_{4,4} & L_{4,5} & L_{4,6} & L_{4,7} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

14.2 Polynomial Interpolation

Now is the time to define how we are going to build polynomials! Notice that the columns of these matrices (say, column $(\ell_{1,i}, \ell_{2,i}, \ell_{3,i}, \ell_{4,i})$ in matrix L from example above) represent the mappings from constraint number i to the corresponding coefficient of the j element in the witness vector!

Example 14.2. Consider the witness from the previous examples:

$$\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$$

For element x_1 we are interested in the third columns of the L , R and O matrices, as it's placed on the third position in the witness vector, so $j = 3$. For matrix L :

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Thus, for constraint number 4 ($i = 4$) the coefficient of x_1 is -1 :

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Good, so now we know that for the j th element in the witness vector there are m (the number of constraints) corresponding values in matrices L , R , and O . Now, we want to encode this statement in a form of a polynomial. As we know from the previous chapters, such a mapping in math can be built using the Lagrange polynomial interpolation.

Remark. As a remainder, the Lagrange interpolation polynomial $f(X) \in \mathbb{F}^{\leq n}[X]$ for a given set of points $\{(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)\} \subseteq \mathbb{F} \times \mathbb{F}$ such that $f(x_i) = y_i$ can be built with the following formula:

$$f(X) = \sum_{i=0}^n y_i e_i(X), \quad e_i(X) = \prod_{j=0, j \neq i}^n \frac{X - x_j}{x_i - x_j}.$$

For a given column $j \in [n]$ in a matrix L the set of points that define the variable polynomial $\ell_j(X)$ can be defined as $\{(i, L_{i,j})\}_{i \in [m]}$. In other words, we want to interpolate n polynomials $\ell_j(X) \in \mathbb{F}[X]$ such that:

$$\ell_j(i) = L_{i,j}, \quad i \in [m], \quad j \in [n].$$

The same is true for matrices R and O , resulting in $3n$ polynomials, n for each of the coefficients matrices:

$$\ell_1(X), \ell_2(X), \dots, \ell_n(x), r_1(X), r_2(X), \dots, r_n(X), o_1(X), o_2(X), \dots, o_n(X)$$

Example 14.3. Considering the witness vector $\mathbf{z}$ and matrix L from the previous example, for the variable x_1 , the next set of points can be derived:

$$\{(1, 1), (2, 0), (3, 1), (4, -1)\}$$

We can see that it is used in the 1st, 3rd, and 4th constraints as the values of the coefficients are not zero.

The Lagrange interpolation polynomial for this set of points can be built as follows (for the demonstration purposes, assume we are working over $\mathbb{R}$):

$$\begin{aligned} e_1(X) &= \frac{(X-2)(X-3)(X-4)}{(1-2)(1-3)(1-4)} = -\frac{(X-2)(X-3)(X-4)}{6}, \\ e_2(X) &= \frac{(X-1)(X-3)(X-4)}{(2-1)(2-3)(2-4)} = \frac{(X-1)(X-3)(X-4)}{2}, \\ e_3(X) &= \frac{(X-1)(X-2)(X-4)}{(3-1)(3-2)(3-4)} = -\frac{(X-1)(X-2)(X-4)}{2}, \\ e_4(X) &= \frac{(X-1)(X-2)(X-3)}{(4-1)(4-2)(4-3)} = \frac{(X-1)(X-2)(X-3)}{6}. \end{aligned}$$

Thus, the polynomial is given by:

$$\begin{aligned} \ell_1(X) &= 1 \cdot e_1(X) + 0 \cdot e_2(X) + 1 \cdot e_3(X) + (-1) \cdot e_4(X) \\ &= -\frac{(X-2)(X-3)(X-4)}{6} - \frac{(X-1)(X-2)(X-4)}{2} \\ &\quad - \frac{(X-1)(X-2)(X-3)}{6} = -\frac{5}{6}X^3 + 6X^2 - \frac{79}{6}X + 9. \end{aligned}$$

Therefore, the final Lagrange interpolation polynomial is:

$$\ell_1(X) = -\frac{5}{6}X^3 + 6X^2 - \frac{79}{6}X + 9$$

As shown in Illustration below, the curve intersects all the given points. In this figure, the x -axis represents the constraint number, and the y -axis represents the coefficients of the x_1 witness element.

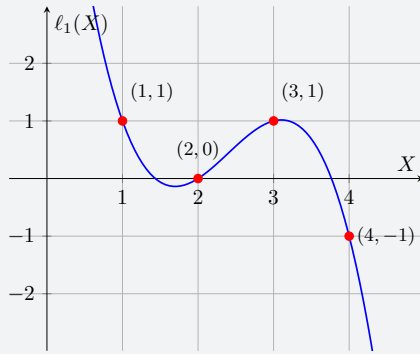


Illustration: The Lagrange interpolation polynomial for points $\{(1, 1), (2, 0), (3, 1), (4, -1)\}$ visualized over $\mathbb{R}$.

Remark. One might a reasonable question: why do we choose x -coordinates to be the indices of the corresponding constraints? Actually, just for convenience purposes. We could have assigned any *unique* index from $\mathbb{F}$ to each constraint (say, t_i for each $i \in [m]$) and interpolate through these points:

$$\ell_j(t_i) = L_{i,j}, \quad i \in [m], \quad j \in [n]$$

As we will see in the subsequent lectures, we can define the x -coordinates in much more clever way to reduce the workload needed for interpolation. But for now, we will stick to this simple version.

14.3 Putting All Together!

Now, using coefficients encoded with polynomials, a constraint number $h \in [m]$, from a constraint system with a witness vector $\mathbf{z}$ can be built in the next way:

$$\begin{aligned} (z_1 \ell_1(h) + z_2 \ell_2(h) + \cdots + z_n \ell_n(h)) \times (z_1 r_1(h) + z_2 r_2(h) + \cdots + z_n r_n(h)) &= \\ &= (z_1 o_1(h) + z_2 o_2(h) + \cdots + z_n o_n(h)) \end{aligned}$$

Or, written more concisely:

$$\sum_{i \in [n]} z_i \ell_i(h) \cdot \sum_{i \in [n]} z_i r_i(h) = \sum_{i \in [n]} z_i o_i(h).$$

Remark. Hold on, but why does it hold? Let us substitute any $X = j$ into this equation:

$$\sum_{i \in [n]} z_i \ell_i(j) \cdot \sum_{i \in [n]} z_i r_i(j) = \sum_{i \in [n]} z_i o_i(j) \quad \forall j \in [m]$$

Recall that we interpolated polynomials to have $\ell_i(j) = L_{j,i}$. Therefore, the

equation above can be reduced to:

$$\sum_{i \in [n]} z_i L_{j,i} \cdot \sum_{i \in [n]} z_i R_{j,i} = \sum_{i \in [n]} z_i O_{j,i} \quad \forall j \in [m]$$

But hold on again! Notice that $\sum_{i \in [n]} z_i L_{j,i} = \langle \mathbf{z}, \mathbf{l}_j \rangle$ and therefore we have:

$$\langle \mathbf{z}, \mathbf{l}_j \rangle \times \langle \mathbf{z}, \mathbf{r}_j \rangle = \langle \mathbf{z}, \mathbf{o}_j \rangle \quad \forall j \in [m],$$

so we ended up with the initial m constraint equations!

Now let us define polynomials $\ell(X)$, $r(X)$, $o(X)$ for easier notation:

$$\ell(X) = \sum_{i \in [n]} z_i \ell_i(X), \quad r(X) = \sum_{i \in [n]} z_i r_i(X), \quad o(X) = \sum_{i \in [n]} z_i o_i(X)$$

Therefore, our set of constraints can be rewritten simply as the single check $\ell(h) \cdot r(h) = o(h)$ for each $h \in [m]$ – much less scary-looking than what we have written before. OK, but what does it give us?

Notice that if $\ell(X) \cdot r(X) = o(X)$ for all $h \in [m]$ then polynomial, defined as $m(X) := \ell(X) \cdot r(X) - o(X)$, has zeros at all elements from the set $\Omega = [m]$. Define the so-called **vanishing polynomial** on Ω as:

$$z_\Omega(X) := \prod_{h \in \Omega} (X - h) = \prod_{i \in [m]} (X - i)$$

Now, if $m(X)$ vanishes on all points from Ω , it means that z_Ω must divide m , so $z_\Omega \mid m$. But that means that m can be divided by z_Ω without remainder! In other words, there exists some polynomial h such that $m = z_\Omega h$. We further drop index Ω for simplicity.

All in all, let us give the definition of a **Quadratic Arithmetic Program**.

Definition 14.2 (Quadratic Arithmetic Program). Suppose that m R1CS constraints with a witness of size n are written in a form

$$Lz \odot Rz = Oz, \quad L, R, O \in \mathbb{F}^{m \times n}$$

Then, the **Quadratic Arithmetic Program** consists of $3n$ polynomials $\ell_1, \dots, \ell_n, r_1, \dots, r_n, o_1, \dots, o_n$ such that:

$$\ell_j(i) = a_{i,j}, \quad r_j(i) = b_{i,j}, \quad o_j(i) = c_{i,j}, \quad \forall i \in [m] \quad \forall j \in [n]$$

Then, $\mathbf{z} \in \mathbb{F}^n$ is a valid assignment for the given QAP and **target polynomial** $z(X) = \prod_{i \in [m]} (X - i)$ if and only if there exists such a polynomial $h(X)$ such that

$$\sum_{i \in [n]} z_i \ell_i(X) \cdot \sum_{i \in [n]} z_i r_i(X) - \sum_{i \in [n]} z_i o_i(X) = z(X) h(X)$$

This was our final step in representing a high-level programming language to some math primitive. We have managed to encode our computation to a single polynomial!

14.4 Probabilistically Checkable Proofs

Before going further we should get acquainted with one more concept from the computational complexity theory, that have an important application in zk-SNARK and provides the theoretical backbone.

L Probabilistically Checkable Proof (PCP) is a type of proof system where the verifier can efficiently check the correctness of a proof by examining only a small, random portion of it, rather than verifying it entirely.

Definition 14.3. L language $\mathcal{L} \subseteq \Sigma^*$ (for some given alphabet Σ) is in the class $\text{PCP}(r, q)$ (**probabilistically checkable proofs**), where r is the *randomness complexity* and q is the *query complexity*, if for a given pair of algorithms $(\mathcal{P}, \mathcal{V})$:

- **Syntax:** $\mathcal{P}$ calculates a proof (bit string) $\pi \in \Sigma^*$ in polynomial time $\text{poly}(|\mathbb{x}|)$ of the common input $\mathbb{x}$. The prover $\mathcal{P}$ and verifier $\mathcal{V}$ interact, where the verifier has an oracle access to π (meaning, he queries it at any position).
- **Complexity:** $\mathcal{V}$ uses at most r random bits to decide which part of the proof to query and the verifier queries at most q bits of the proof.

Such pair of algorithms $(\mathcal{P}, \mathcal{V})$ should satisfy the following properties (with a security parameter $\lambda \in \mathbb{N}$):

- **Completeness:** If $\mathbb{x} \in \mathcal{L}$, then $\Pr[\mathcal{V}^\pi(\mathbb{x}) = 1] = 1$.
- **Soundness:** If $\mathbb{x} \notin \mathcal{L}$, then for any possible (malicious) proof π^* ,

$$\Pr[\mathcal{V}^{\pi^*}(\mathbb{x}) = 1] = \text{negl}(\lambda).$$

This allows a verification of huge statements with high confidence while using limited computational resources. See [Figure 14.2](#).

Theorem 14.4. PCP theorem (PCP characterization theorem)

Any decision problem in NP has a PCP verifier that uses logarithmic randomness $O(\log n)$ and a constant number of queries $O(1)$, independent of n .

$$\text{NP} = \text{PCP}(O(\log n), O(1))$$

However, despite the fact that PCP is a very powerful tool, it is not used directly in zk-SNARKs. We need to extend it a bit to make it more suitable for our purposes.

Three main extensions of PCPs that are frequently used in SNARKs are:

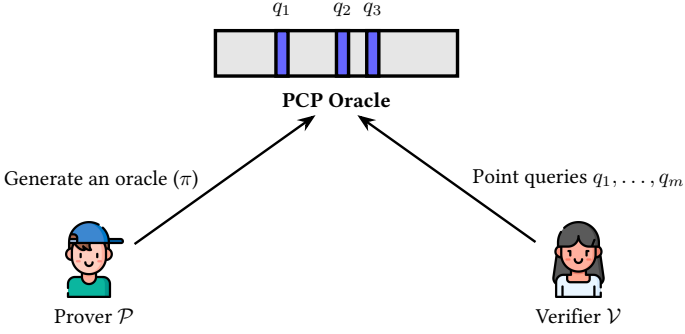


Figure 14.1: Illustration of a Probabilistically Checkable Proof (PCP) system. The prover $\mathcal{P}$ generates a PCP oracle π that is queried by the verifier $\mathcal{V}$ at specific points $q_1, \dots, q_m$.

- **IPCP (Interactive PCP):** The prover commits to the PCP oracle and then, based on the interaction between the prover and verifier, the verifier queries the oracle and decides whether to accept the proof.
- **IOP (Interactive Oracle Proof):** The prover and verifier interact and on each round, the prover commits to a new oracle. The verifier queries the oracle and decides whether to accept the proof.
- **LPCP (Linear PCP):** The prover commits to a linear function and the verifier queries the function at specific points.

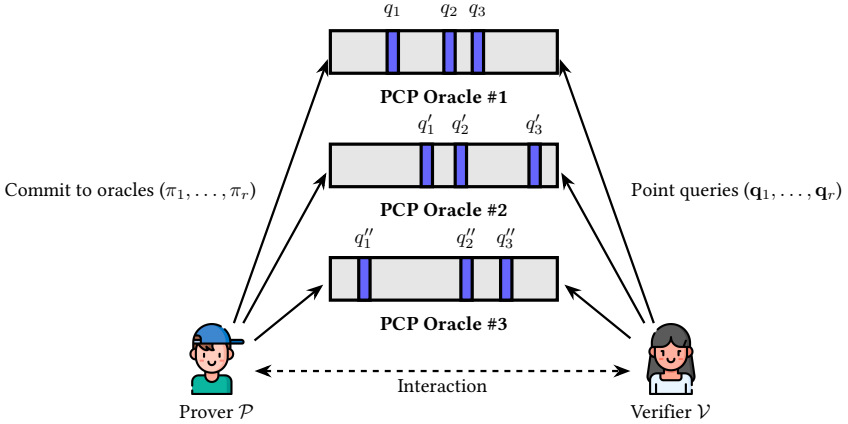


Figure 14.2: Illustration of an Interactive Oracle Proof (IOP). On each round i ($1 \leq i \leq r$), $\mathcal{V}$ sends a message m_i , and $\mathcal{P}$ commits to a new oracle π_i , which $\mathcal{V}$ can query at $\mathbf{q}_i = (q_{i,1}, \dots, q_{i,m})$.

While IOPs will be later used for PLONK and zk-STARKs, we will focus on Linear PCPs in the context of Groth16 zk-SNARK. Let us define it below.

Definition 14.5 (Linear PCP). **L Linear PCP** is a PCP where the prover commits to a linear function $\pi = (\pi_1, \dots, \pi_k)$ and the verifier queries the function at specific points $q_1, \dots, q_r$. Then, the prover responds with the values of the function at these points:

$$\langle \pi_1, q_1 \rangle, \langle \pi_2, q_2 \rangle, \dots, \langle \pi_r, q_r \rangle.$$

Example 14.4 (QAP as a Linear PCP). Recall that key QAP equation is:

$$\sum_{i \in [n]} z_i \ell_i(X) \cdot \sum_{i \in [n]} z_i r_i(X) - \sum_{i \in [n]} z_i o_i(X) = z(X)h(X).$$

Now, the notation might be confusing, but firstly, we denote vectors of polynomials:

$$\begin{aligned} \ell(X) &= (\ell_1(X), \dots, \ell_n(X)), \\ r(X) &= (r_1(X), \dots, r_n(X)), \\ o(X) &= (o_1(X), \dots, o_n(X)). \end{aligned}$$

Now, consider the following **linear PCP for QAP**:

1. $\mathcal{P}$ commits to an extended witness $\mathbf{z}$ and coefficients $\mathbf{h} = (h_1, \dots, h_n)$ of $h(X)$.
2. $\mathcal{V}$ samples $\gamma \leftarrow \mathbb{F}$ and sends query $\gamma = (\gamma, \gamma^2, \dots, \gamma^n)$ to $\mathcal{P}$.
3. $\mathcal{P}$ reveals the following values:

$$\begin{aligned} \pi_1 &\leftarrow \langle \mathbf{z}, \ell(\gamma) \rangle, & \pi_2 &\leftarrow \langle \mathbf{z}, r(\gamma) \rangle, \\ \pi_3 &\leftarrow \langle \mathbf{z}, o(\gamma) \rangle, & \pi_4 &\leftarrow z(\gamma) \cdot \langle \mathbf{h}, \gamma \rangle. \end{aligned}$$

4. $\mathcal{V}$ checks whether $\pi_1 \pi_2 - \pi_3 = \pi_4$.

Of course, the above example cannot be used as it is: at the very least, we have not specified how the prover commits to the extended witness $\mathbf{z}$ and coefficients $\mathbf{h}$ and then ensures consistency of $\pi_1, \dots, \pi_4$ with these commitments. For that reason, we need some more tools to make it work which we learned in the previous lectures.

14.5 QAP as a Linear PCP

When constructing a Quadratic Arithmetic Program (QAP) for a circuit $\mathcal{C}$, we represented the whole circuit's computation using the following relation: $\ell(X)r(X) - o(X) = z(X)h(X)$, where by $\ell(X)$, $r(X)$, $o(X)$ we denote the polynomials that represent the left, right and output wires of the circuit, respectively. $z(X)$ is the target polynomial, while $h(X) := m(X)/z(X)$ for master

polynomial $m(X) = \ell(X)r(X) - o(X)$ is the quotient polynomial.

We effectively managed to transform all the circuit's constraints, and computations in the short form. It still allows one to verify that each computational step is preserved by verifying the polynomial evaluation in specific (random) points, instead of recomputing everything. However, it is not quite clear why such a check is safe and how it can be used in a PCP. In other words, why checking that $\ell(s)r(s) - o(s) = z(s)h(s)$ for randomly selected s is enough to verify the circuit $\mathcal{C}$?

Informal Soundness Justification. Why is it safe to use such a check? As it was said early, we perform all the computations in some finite field $\mathbb{F}$. The polynomials $\ell(X)$, $r(X)$ and $o(X)$ are interpolated polynomials using $|\mathcal{C}|$ (number of gates) points, so $\deg(\ell), \deg(r), \deg(o) \leq |\mathcal{C}|$. Thus, using properties of polynomials' degrees, we can estimate the degree of polynomial $m(X) = \ell(X)r(X) - o(X)$: $\deg(m) \leq \max\{\deg(\ell) + \deg(r), \deg(o)\} \leq 2|\mathcal{C}|$. For that reason, when checking the equality $\ell(s)r(s) - o(s) = z(s)h(s)$ for randomly selected s , we can be sure that the left-hand side and right-hand side polynomials are the same with probability at least $1 - 2|\mathcal{C}|/|\mathbb{F}|$.

15 Pairing-based SNARKs. Pinocchio. Groth16

15.1 Building Pairing-based SNARK

15.1.1 Attempt #1: Encrypted Verification

Now, assume we have the cyclic group $\mathbb{G}$ of prime order q with a generator g . Typically, this is the group of points on an elliptic curve. Assume for simplicity that $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ is a symmetric pairing, where $(\mathbb{G}_T, \times)$ is a target group.

Now, suppose during the setup phase, we have a trusted party that generated a random value $\tau \leftarrow \mathbb{F}$ and public parameters $g^\tau, g^{\tau^2}, \dots, g^{\tau^d}$ for $d = 2|\mathcal{C}|$ — maximum degree of used polynomials (later we will use notation $\{g^{\tau^i}\}_{i \in [d]}$ for brevity). Then, party **deleted** τ . This way, we can now find the KZG commitment for each polynomial. Indeed, for example,

$$\text{com}(\ell) \triangleq g^{\ell(\tau)} = g^{\sum_{i=0}^d \ell_i \tau^i} = \prod_{i=0}^d (g^{\tau^i})^{\ell_i},$$

and the same goes for $g^{r(\tau)}, g^{o(\tau)}, g^{h(\tau)}, g^{z(\tau)}$. Now, with these points, how can we verify that the polynomial $m(X) = \ell(X)r(X) - o(X)$ is correct? Well, first notice that the check is equivalent to

$$\ell(\tau)r(\tau) = z(\tau)h(\tau) + o(\tau).$$

Notice that we transferred $o(\tau)$ to the right side of the equation to further avoid finding the inverse. Now, we can check this equality using encrypted values as follows:

$$e(\text{com}(\ell), \text{com}(r)) = e(\text{com}(z), \text{com}(h)) \cdot e(\text{com}(o), g),$$

Remark. One might ask: why is the above equation correct? Well, let us see:

$$\begin{aligned} e(\text{com}(\ell), \text{com}(r)) &= e(\text{com}(z), \text{com}(h)) \cdot e(\text{com}(o), g) \\ \Leftrightarrow e(g^{\ell(\tau)}, g^{r(\tau)}) &= e(g^{z(\tau)}, g^{h(\tau)}) \cdot e(g^{o(\tau)}, g) \\ \Leftrightarrow e(g, g)^{\ell(\tau)r(\tau)} &= e(g, g)^{z(\tau)h(\tau)} e(g, g)^{o(\tau)} \\ \Leftrightarrow e(g, g)^{\ell(\tau)r(\tau)} &= e(g, g)^{z(\tau)h(\tau) + o(\tau)} \\ \Leftrightarrow \ell(\tau)r(\tau) &= z(\tau)h(\tau) + o(\tau) \\ \Leftrightarrow \ell(X)r(X) &\equiv z(X)h(X) + o(X) \end{aligned}$$

So, sounds like we are done. Let us summarize what we have done so far:

Attempt #1: Non-sound SNARK Protocol

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks a random value $\tau \leftarrow \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i \in [d]}$.
- ✓ **Deletes** τ (toxic waste).
- ✓ **Outputs** prover parameters $\text{pp} \leftarrow \{g^{\tau^i}\}_{i \in [d]}$ and verifier parameters $\text{vp} \leftarrow \text{com}(z)$.

Prove($\text{pp}, \mathbb{x}, \mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit with the statement $\mathbb{x}$ and witness $\mathbb{w}$, obtains the intermediate constraint values encoded in solution witness $\mathbf{z}$, and calculates the polynomials $\ell(X)$, $r(X)$, $o(X)$ through Lagrange Interpolation.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Calculates the KZG commitments as follows:

$$\pi_L \leftarrow \text{com}(\ell), \pi_R \leftarrow \text{com}(r), \pi_O \leftarrow \text{com}(o), \pi_H \leftarrow \text{com}(h),$$

using powers of τ from the prover parameters pp .

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H)$ as a proof.

Verify($\text{vp}, \mathbb{x}, \boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi_R, \pi_O, \pi_H)$, using $\text{com}(z)$ from the verifier parameters vp , the verifier $\mathcal{V}$ checks:

$$e(\pi_L, \pi_R) = e(\text{com}(z), \pi_H) \cdot e(\pi_O, g).$$

This sounds like an end to the story. However, there is a problem with this approach: there is no guarantee that commitments $\pi_L, \pi_R, \pi_O, \pi_H$ were indeed obtained through exponentiating the base g by the corresponding values $\ell(\tau)$, $r(\tau)$, $o(\tau)$, and $h(\tau)$. In other words, how can the verifier know that prover indeed knows, say, polynomial $\ell(X)$ such that $\pi_L = g^{\ell(\tau)}$?

15.1.2 Attempt #2: Including Proof of Exponent

In this section, we introduce the **Proof of Exponent assumption** (PoE) which makes KZG knowledge sound. Let us define it below.

Definition 15.1 (Proof of Exponent for KZG Commitment). A **Proof of Exponent** (PoE) is a protocol that allows the prover $\mathcal{P}$ to convince the verifier $\mathcal{V}$ that he obtained a value $\text{com}(f)$ through exponentiating a base g by $f(\tau)$. The protocol works as follows:

1. **Setup:** Proper parameters pp now contain not only $\{g^{\tau^i}\}_{i \in [d]}$, but also $\{g^{\alpha\tau^i}\}_{i \in [d]}$ for randomly selected $\tau, \alpha \leftarrow \mathbb{F}$ and further **deleted** values.
2. **Commit:** $\mathcal{P}$ commits to two values: $\text{com}(f) = g^{f(\tau)}$ and $\text{com}'(f) = g^{\alpha f(\tau)}$. The latter can be computed using pp as follows:

$$\text{com}'(f) = \prod_{i=0}^d (g^{\alpha\tau^i})^{f_i}$$

3. **Verify:** $\mathcal{V}$ additionally checks $e(\text{com}(f), g^\alpha) = e(\text{com}'(f), g)$.

The informal reason why it makes KZG commitment sound is following: suppose we have an adversary prover $\mathcal{P}^*$ that published commitment c without knowing underlying polynomial $f(X)$. Now, there is no way for him to cheat the verifier. Indeed, what $\mathcal{P}^*$ needs to do is calculating c^α , but he does not know α since, similarly to τ , it was deleted as a part of the toxic waste. Besides, he cannot obtain α for the same reason he cannot obtain τ .

For that reason, we modify the SNARK protocol to include not only commitments to polynomials but also PoE for each of them. Let us see how it looks like.

Attempt #2: SNARK with PoE included

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks a random value $\tau, \alpha \leftarrow \mathbb{F}$.
- ✓ Calculates the public parameters $\{g^{\tau^i}\}_{i \in [d]}, \{g^{\alpha\tau^i}\}_{i \in [d]}$.
- ✓ **Deletes** τ, α (toxic waste).
- ✓ **Outputs** proper parameters $\text{pp} \leftarrow \{\{g^{\tau^i}\}_{i \in [d]}, \{g^{\alpha\tau^i}\}_{i \in [d]}\}$, and verification parameters $\text{vp} \leftarrow \{g^{z(\tau)}, g^\alpha\}$.

Prove(pp, $\mathbb{x}$, $\mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit with the statement $\mathbb{x}$ and witness $\mathbb{w}$, obtains the intermediate constraint values, and calculates the polynomials $\ell(X), r(X), o(X)$ through Lagrange Interpolation.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Calculates the sound KZG commitments as follows:

$$\begin{aligned}\pi_L &\leftarrow g^{\ell(\tau)}, & \pi'_L &\leftarrow g^{\alpha\ell(\tau)} \\ \pi_R &\leftarrow g^{r(\tau)}, & \pi'_R &\leftarrow g^{\alpha r(\tau)} \\ \pi_O &\leftarrow g^{o(\tau)}, & \pi'_O &\leftarrow g^{\alpha o(\tau)} \\ \pi_H &\leftarrow g^{h(\tau)}, & \pi'_H &\leftarrow g^{\alpha h(\tau)}.\end{aligned}$$

using powers $\{g^{\tau^i}\}_{i \in [d]}$ and $\{g^{\alpha\tau^i}\}_{i \in [d]}$ from the proper parameters pp.

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H)$ as a proof.

Verify(vp, $\mathbb{x}$, $\boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H)$, the verifier $\mathcal{V}$ checks:

$$\begin{aligned}e(\pi_L, \pi_R) &= e(g^{z(\tau)}, \pi_H) \cdot e(\pi_O, g), \\ e(\pi_L, g^\alpha) &= e(\pi'_L, g), & e(\pi_R, g^\alpha) &= e(\pi'_R, g), \\ e(\pi_O, g^\alpha) &= e(\pi'_O, g), & e(\pi_H, g^\alpha) &= e(\pi'_H, g).\end{aligned}$$

The provided protocol is secure under the PoE assumption. However, it is still not fully sound. Currently, there is no guarantee that when evaluating π_L, π_R, π_O we used the same extended witness $\mathbf{z}$. In other words, the prover can still cheat by using different extended witnesses for each polynomial (faking the proof system is still hard in this situation, but we want to make sure to eliminate all possible weaknesses). Let us see how to fix this!

15.1.3 Attempt #3: Making SNARK Sound

Besides fixing the issue with consistent use of witness $\mathbf{z}$, we additionally include one more optimization we have not included so far.

Optimization. Left/Right/Output Polynomial Preprocessing. Recall that:

$$\ell(X) = \sum_{i=0}^n z_i \ell_i(X), \quad r(X) = \sum_{i=0}^n z_i r_i(X), \quad o(X) = \sum_{i=0}^n z_i o_i(X).$$

while $m(X) = \ell(X)r(X) - o(X)$ is only known to the prover $\mathcal{P}$ since it contains the extended witness $(\mathbf{z})$ coefficients. However, the set of polynomials

$$\{\ell_i(X)\}_{i \in [n]}, \{r_i(X)\}_{i \in [n]}, \{o_i(X)\}_{i \in [n]}$$

are known in advance. Meaning, we can precompute the values of $\{g^{\ell_i(\tau)}\}_{i \in [n]}$, $\{g^{\alpha \ell_i(\tau)}\}_{i \in [n]}$, $\{g^{r_i(\tau)}\}_{i \in [n]}$, $\{g^{\alpha r_i(\tau)}\}_{i \in [n]}$, $\{g^{o_i(\tau)}\}_{i \in [n]}$, $\{g^{\alpha o_i(\tau)}\}_{i \in [n]}$ and use them in the prover parameters pp.

How? Suppose the prover $\mathcal{P}$ knows the extended witness $\mathbf{z}$. Consider the polynomial $\ell(X) = \sum_{i=0}^n z_i \ell_i(X)$. $\mathcal{P}$ can compute the KZG commitment π_L and its PoE π'_L as follows:

$$\begin{aligned}\pi_L &\triangleq g^{\ell(\tau)} = g^{\sum_{i=0}^n z_i \ell_i(\tau)} = \prod_{i=0}^n (g^{\ell_i(\tau)})^{z_i}, \\ \pi'_L &\triangleq g^{\alpha \ell(\tau)} = g^{\alpha \sum_{i=0}^n z_i \ell_i(\tau)} = \prod_{i=0}^n (g^{\alpha \ell_i(\tau)})^{z_i}.\end{aligned}$$

Fix. Witness consistency check.

Introducing new term with β . To prove that the same $\mathbf{z}$ is used in all commitments, we need some “checksum” term that will somehow combine all polynomials $\ell(X)$, $r(X)$, and $o(X)$ with the witness $\mathbf{z}$. Moreover, we will need to compare this term with proofs π_L , π_R , and π_O . The best candidate for this is the following group element for some other random $\beta \leftarrow \mathbb{F}$:

$$\begin{aligned}\pi_\beta &= g^{\ell(\tau)+r(\tau)+o(\tau)} = \prod_{i=1}^n (g^{\ell_i(\tau)+r_i(\tau)+o_i(\tau)})^{z_i}, \\ \pi'_\beta &= \prod_{i=1}^n (g^{\beta(\ell_i(\tau)+r_i(\tau)+o_i(\tau))})^{z_i}\end{aligned}$$

This way, we get a term that includes all three polynomials $\ell(X)$, $r(X)$, and $o(X)$, and all coefficients of the extended witness $\mathbf{z}$. Moreover, verifier $\mathcal{V}$ can compare π_β with three other commitments π_L , π_R , π_O to ensure that all of them are consistent. This is done through the following check:

$$e(\pi_L \pi_R \pi_O, g^\beta) = e(\pi'_\beta, g).$$

Again, this approach still has a weakness (yeah-yeah, I am also tired of this). Indeed, this check is complete (meaning, if $\mathbf{z}$ is used consistently across π_L , π_R , π_O , then the check will pass), but it is still not sound with an overwhelming probability. Indeed, suppose $\mathbf{z}$ is used inconsistently, meaning, we have extended witnesses $\mathbf{z}_L$, $\mathbf{z}_R$, $\mathbf{z}_O$, and $\mathbf{z}_\beta$, each for corresponding polynomials. If the witness is consistent, the following condition must hold:

$$(z_{\ell,i} \ell_i(\tau) + z_{r,i} r_i(\tau) + z_{o,i} o_i(\tau)) \beta = z_{\beta,i} \beta (\ell_i(\tau) + r_i(\tau) + o_i(\tau)) \quad \forall i \in [n]$$

Assume otherwise. Consider a simple situation where it happens that $\ell_i \equiv r_i$, meaning $\ell_i(\tau)$ and $r_i(\tau)$ are the same (call them q). Then,

$$(z_{\ell,i} + z_{r,i})q + z_{o,i}o_i(\tau) = z_{\beta,i}(2q + o_i(\tau)) \quad \forall i \in [n]$$

For arbitrarily chosen $z_{r,i}$ and $z_{o,i}$, the adversary prover $\mathcal{P}^*$ can set $z_{\beta,i} := z_{o,i}$ and $z_{\ell,i} = 2w_{o,i} - z_{r,i}$. It is easy to verify that the above equation would hold, meaning that $\mathbf{z}$ is not the same across all polynomials.

One might ask: well, situation when $\ell_i \equiv r_i$ is very rare! Indeed, but there also might be situations where $\ell_i \equiv 5r_i$, $r_i \equiv 100o_i$, or $\ell_i \equiv 235o_i$ — all of them would lead to the same issue. So what is the solution?

Introducing separate $\beta_L, \beta_R, \beta_O$. Our proposal is to make β different for each polynomial $\ell(X)$, $r(X)$, $o(X)$, making it much harder for the adversary to find inconsistent witnesses. That being said, during the setup phase, we choose arbitrary $\beta_L, \beta_R, \beta_O \leftarrow \mathbb{F}$ and publish $\{g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O o_i(\tau)}\}_{i \in [n]}$ as a part of the prover parameters pp . Then, the prover $\mathcal{P}$ calculates the following additional commitment:

$$\pi_\beta \leftarrow \prod_{i=1}^n (g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O o_i(\tau)})^{z_i}$$

and then publishes π_β as a part of the proof. The verifier $\mathcal{V}$ checks the following condition:

$$e(\pi_L, g^{\beta_L}) \cdot e(\pi_R, g^{\beta_R}) \cdot e(\pi_O, g^{\beta_O}) = e(\pi_\beta, g).$$

Even that is not the end of the story! We also need to make sure that g^{β_L} , g^{β_R} , and g^{β_O} are incompatible with $g^{\sum_{i=0}^n (\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)) z_i}$ (for reasons we drop here due to already long explanation). For that reason, we multiply $\beta_L, \beta_R, \beta_O$ by some random factor $\gamma \leftarrow \mathbb{F}$. This way, the proving part remains the same, but the check becomes:

$$e(\pi_L, g^{\gamma \beta_L}) \cdot e(\pi_R, g^{\gamma \beta_R}) \cdot e(\pi_O, g^{\gamma \beta_O}) = e(\pi_\beta, g^\gamma).$$

Oof, that was a long fix! Let us come back to the new version of the SNARK protocol (not yet zero-knowledge)!

Attempt #3: Sound SNARK Protocol

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \leftarrow \mathbb{F}$.

✓ **Outputs** prover parameters pp and verification parameters vp :

$$\begin{aligned}\text{pp} &\leftarrow \left\{ \{g^{\tau^i}\}_{i \in [d]}, \{g^{\ell_i(\tau)}, g^{\alpha \ell_i(\tau)}, g^{r_i(\tau)}, g^{\alpha r_i(\tau)}, \right. \\ &\quad \left. g^{\alpha_i(\tau)}, g^{\alpha \alpha_i(\tau)}, g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O \alpha_i(\tau)} \}_{i \in [n]} \right\} \\ \text{vp} &\leftarrow \{g^{z(\tau)}, g^\alpha, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L \gamma}, g^{\beta_R \gamma}, g^{\beta_O \gamma}, g^\gamma\}\end{aligned}$$

✓ **Deletes** aforementioned random scalars (toxic waste).

Prove($\text{pp}, \mathbb{x}, \mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{z}$ and $\ell(X), r(X), o(X)$.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Calculates the sound KZG commitments as follows:

$$\begin{aligned}\pi_L &\leftarrow g^{\ell(\tau)}, & \pi'_L &\leftarrow g^{\alpha \ell(\tau)}, \\ \pi_R &\leftarrow g^{r(\tau)}, & \pi'_R &\leftarrow g^{\alpha r(\tau)}, \\ \pi_O &\leftarrow g^{o(\tau)}, & \pi'_O &\leftarrow g^{\alpha o(\tau)}, \\ \pi_H &\leftarrow g^{h(\tau)}, & \pi'_H &\leftarrow g^{\alpha h(\tau)}.\end{aligned}$$

- ✓ **Calculates the additional commitment π_β as follows:**

$$\pi_\beta \leftarrow g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}$$

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify($\text{vp}, \mathbb{x}, \boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$ checks:

$$\begin{aligned}e(\pi_L, \pi_R) &= e(g^{z(\tau)}, \pi_H) \cdot e(\pi_O, g), \\ e(\pi_L, g^\alpha) &= e(\pi'_L, g), & e(\pi_R, g^\alpha) &= e(\pi'_R, g), \\ e(\pi_O, g^\alpha) &= e(\pi'_O, g), & e(\pi_H, g^\alpha) &= e(\pi'_H, g), \\ e(\pi_L, g^{\gamma \beta_L}) &\cdot e(\pi_R, g^{\gamma \beta_R}) \cdot e(\pi_O, g^{\gamma \beta_O}) &= e(\pi_\beta, g^\gamma).\end{aligned}$$

15.1.4 Attempt #4: Splitting the Extended Witness

Now, recall that the actual circuit $\mathcal{C}$ is defined for some statement $\mathbb{x}$ and witness $\mathbf{z}$. According to the Circuit Satisfiability Problem, the prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that he knows the witness $\mathbf{z}$ such that the circuit $\mathcal{C}(\mathbb{x}, \mathbb{w}) = 0$. Up until now, we have been using the extended witness $\tilde{\mathbf{z}}$ to represent the trace of

computation. However, we can split the witness into two parts: the first part $\mathbf{z}_{\text{mid}}$ – intermediate witness that contains the private witness $\mathbf{z}$ and intermediate variables values, and the second part $\mathbf{z}_{\text{io}}$ – input/output witness that contains public statement information (e.g., $\mathbb{x}$). Suppose for vector $\tilde{\mathbf{z}} = (\tilde{z}_1, \dots, \tilde{z}_n)$ we pick out the set of indices $\mathcal{I}_{\text{mid}} \subset [n]$ to represent the intermediate witness and $\mathcal{I}_{\text{io}} \subset [n]$ to represent the input/output witness (of course, $\mathcal{I}_{\text{mid}} \cap \mathcal{I}_{\text{io}} = \emptyset$ and $\mathcal{I}_{\text{mid}} \cup \mathcal{I}_{\text{io}} = [n]$).

Now, how do we split the proofs $\pi_L, \pi_R, \dots$ into two parts? Well, consider for instance π_L :

$$\pi_L = g^{\sum_{i=0}^n z_i \ell_i(\tau)}.$$

We split this expression as follows:

$$\pi_L = \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(\tau)}}_{\text{new } \pi_L} \times \underbrace{g^{\sum_{i \in \mathcal{I}_{\text{io}}} z_i \ell_i(\tau)}}_{\pi_{\ell, \text{io}}}.$$

This way, the prover $\mathcal{P}$ first calculates the new commitment π_L using only the intermediate witness $\mathbf{z}_{\text{mid}}$ and then the verifier can compute the “public” portion of the proof $\pi_{\ell, \text{io}}$ using the input/output witness $\mathbf{z}_{\text{io}}$ (which is typically quite easy to do since $|\mathcal{I}_{\text{io}}|$ is typically much smaller than $|\mathcal{I}_{\text{mid}}|$). The same goes for other parts of the proof π .

Now, let us formulate the updated SNARK protocol with the extended witness split.

Attempt #4: Sound SNARK for Public/Private Inputs

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \leftarrow \$ \mathbb{F}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp :

$$\begin{aligned} \text{pp} &\leftarrow \left\{ \{g^{\tau^i}\}_{i \in [d]}, \{g^{\ell_i(\tau)}, g^{\alpha \ell_i(\tau)}, g^{r_i(\tau)}, g^{\alpha r_i(\tau)}, \right. \\ &\quad \left. g^{o_i(\tau)}, g^{\alpha o_i(\tau)}, g^{\beta_L \ell_i(\tau) + \beta_R r_i(\tau) + \beta_O o_i(\tau)}\}_{i \in \mathcal{I}_{\text{mid}}} \right\} \\ \text{vp} &\leftarrow \left\{ g^{z(\tau)}, g^\alpha, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L \gamma}, g^{\beta_R \gamma}, g^{\beta_O \gamma}, g^\gamma, \right. \\ &\quad \left. \{g^{\ell_i(\tau)}, g^{r_i(\tau)}, g^{o_i(\tau)}\}_{i \in \mathcal{I}_{\text{io}}} \right\} \end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove($\text{pp}, \mathbb{x}, \mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{z}$ and $\ell(X), r(X), o(X)$.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Splits $\ell(X) = \ell_{\text{mid}}(X) + \ell_{\text{io}}(X)$ – intermediate and input/output parts. That being said, $\ell_{\text{mid}}(X) = \sum_{i \in \mathcal{I}_{\text{mid}}} \ell_i(X)$. Repeat for $r(X)$ and $o(X)$.
- ✓ Calculates the following values:

$$\begin{aligned}\pi_L &\leftarrow g^{\ell_{\text{mid}}(\tau)}, & \pi'_L &\leftarrow g^{\alpha \ell_{\text{mid}}(\tau)}, \\ \pi_R &\leftarrow g^{r_{\text{mid}}(\tau)}, & \pi'_R &\leftarrow g^{\alpha r_{\text{mid}}(\tau)}, \\ \pi_O &\leftarrow g^{o_{\text{mid}}(\tau)}, & \pi'_O &\leftarrow g^{\alpha o_{\text{mid}}(\tau)}, \\ \pi_H &\leftarrow g^{h(\tau)}, & \pi'_H &\leftarrow g^{\alpha h(\tau)}, \\ \pi_\beta &\leftarrow g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}.\end{aligned}$$

- ✓ Publishes $\pi = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify(vp, $\mathbf{x}$, π)

Upon receiving $\pi = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g^{\ell_i(\tau)})^{z_i}$, $\pi_R^* \leftarrow \pi_R \prod_{i \in \mathcal{I}_{\text{io}}} (g^{r_i(\tau)})^{z_i}$, $\pi_O^* \leftarrow \pi_O \prod_{i \in \mathcal{I}_{\text{io}}} (g^{o_i(\tau)})^{z_i}$
- ✓ Checks whether all of the following conditions hold:

$$\begin{aligned}e(\pi_L^*, \pi_R^*) &= e(g^{z(\tau)}, \pi_H) \cdot e(\pi_O^*, g), \\ e(\pi_L, g^\alpha) &= e(\pi'_L, g), & e(\pi_R, g^\alpha) &= e(\pi'_R, g), \\ e(\pi_O, g^\alpha) &= e(\pi'_O, g), & e(\pi_H, g^\alpha) &= e(\pi'_H, g), \\ e(\pi_L, g^{\gamma \beta_L}) \cdot e(\pi_R, g^{\gamma \beta_R}) \cdot e(\pi_O, g^{\gamma \beta_O}) &= e(\pi_\beta, g^\gamma).\end{aligned}$$

15.1.5 Attempt #5: Making SNARK Zero-Knowledge

Finally, we came to the point where we need to make the protocol zero-knowledge. As it turns out, it is not that hard to do (compared to what we have done so far)!

Remark. Currently, you might have a reasonable question: the proof contains quantities such as $g^{\ell(\tau)}$, $g^{r(\tau)}$, $g^{o(\tau)}$ and variations of them. As long as discrete logarithm holds, there is no PPT adversary that would be able to extract coefficients of $\ell(X)$, $r(X)$, $o(X)$ from published KZG commitments (and respective PoE shifts and witness consistency proof). So what could be the issue?

This reasoning is correct. In other words, the adversary will not learn coefficients of polynomials and therefore will not be able to get the witness fully. However, the zero-knowledge property ensures that the adversary cannot draw conclusions based on proof π . However, in our current version, we do have SNARK, but not zk-SNARK. For instance, currently anyone can check whether $\ell \equiv r$ (by checking $\pi_L = \pi_R$) or that $\ell \equiv 11r$ (by checking $\pi_L = \pi_R^{11}$).

The main idea to make our protocol zero-knowledge is to “shift” our commitments by some random factor δ . Of course, this “shift” δ must be different for each commitment and must be chosen by the prover $\mathcal{P}$. This way, we propose to modify the commitments as follows:

$$\begin{aligned}\pi_L &= g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(\tau) + \delta_L z(\tau)} = (g^{z(\tau)})^{\delta_L} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{\ell_i(\tau)})^{z_i}, \\ \pi'_L &= (g^{\alpha z(\tau)})^{\delta_L} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{\alpha \ell_i(\tau)})^{z_i}, \\ \pi_R &= g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i r_i(\tau) + \delta_R z(\tau)} = (g^{z(\tau)})^{\delta_R} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{r_i(\tau)})^{z_i}, \\ \pi'_R &= (g^{\alpha z(\tau)})^{\delta_R} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{\alpha r_i(\tau)})^{z_i}, \\ \pi_O &= g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i o_i(\tau) + \delta_O z(\tau)} = (g^{z(\tau)})^{\delta_O} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{o_i(\tau)})^{z_i}, \\ \pi'_O &= (g^{\alpha z(\tau)})^{\delta_O} \prod_{i \in \mathcal{I}_{\text{mid}}} (g^{\alpha o_i(\tau)})^{z_i}.\end{aligned}$$

for randomly selected $\delta_L, \delta_R, \delta_O \leftarrow \F_q . Good, we concealed all the information about the witness w in the commitment. However, we need to make sure that the verifier $\mathcal{V}$ can still verify the proof, but without modifying the verification mechanism itself. Notice that PoE verifications are still valid, but we need to modify something to make polynomial equality test hold. This can be done by perturbing the polynomial $h(X)$ by some (currently) unknown value Δ_H . Let us derive it from the polynomial equality test:

$$(\ell(X) + \delta_L z(X))(r(X) + \delta_R z(X)) = (h(X) + \Delta_H)z(X) + (o(X) + \delta_O z(X)),$$

which, by expanding, gives us the following equation:

$$\begin{aligned}\cancel{\ell(X)r(X)} + \delta_R \ell(X)z(X) + \delta_L z(X)r(X) + \delta_L \delta_R z(X)^2 &= \\ = \cancel{h(X)z(X)} + \cancel{o(X)} + \Delta_H z(X) + \delta_O z(X),\end{aligned}$$

where we can cancel out $\ell(X)r(X)$ and $h(X)z(X) + o(X)$ terms since they are equal based on initial construction. This way, we get the following expression for Δ_H :

$$\Delta_H = \delta_O + \delta_R \ell(X) + \delta_L r(X) + \delta_L \delta_R z(X)$$

Therefore, our fourth proof system becomes:

$$\pi_H = g^{h(\tau) + \delta_O + \delta_R \ell(\tau) + \delta_L r(\tau) + \delta_L \delta_R z(\tau)}, \quad \pi'_H = g^{\alpha(h(\tau) + \delta_O + \delta_R \ell(\tau) + \delta_L r(\tau) + \delta_L \delta_R z(\tau))}.$$

Finally, we need to make sure that our witness consistency proof π_β is still valid. Since previously we had $\pi_\beta = g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}$, we need to modify it to include the new δ values. Namely, we change $\ell(\tau)$ to $\ell(\tau) + \delta_L z(\tau)$, $r(\tau)$ to $r(\tau) + \delta_R z(\tau)$, and $o(\tau)$ to $o(\tau) + \delta_O z(\tau)$. This way, our new π_β becomes:

This can be done by changing it into:

$$\begin{aligned} \pi_\beta &= g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau) + (\delta_L \beta_L + \delta_R \beta_R + \delta_O \beta_O) z(\tau)} \\ &= \left(g^{\beta_L z(\tau)} \right)^{\delta_L} \left(g^{\beta_R z(\tau)} \right)^{\delta_R} \left(g^{\beta_O z(\tau)} \right)^{\delta_O} g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}. \end{aligned}$$

Oof. Finally, we also have a zero-knowledge property for our SNARK. Let us summarize what has changed for the prover $\mathcal{P}$.

Proposition 15.2 (Including ZK in general-purpose SNARK). Now, the prover $\mathcal{P}$ samples random scalars $\delta_L, \delta_R, \delta_O \leftarrow_{\$} F_q$ and calculates the following commitments:

- **Updated Commitments for ℓ, r, o :** Now, the prover $\mathcal{P}$ needs to calculate the commitments π_L, π_R, π_O with the additional δ 's values to ensure zero-knowledge property:

$$\begin{aligned} \pi_L &\leftarrow g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(\tau) + \delta_L z(\tau)}, & \pi'_L &\leftarrow g^{\alpha \sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(\tau) + \delta_L z(\tau)}, \\ \pi_R &\leftarrow g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i r_i(\tau) + \delta_R z(\tau)}, & \pi'_R &\leftarrow g^{\alpha \sum_{i \in \mathcal{I}_{\text{mid}}} z_i r_i(\tau) + \delta_R z(\tau)}, \\ \pi_O &\leftarrow g^{\sum_{i \in \mathcal{I}_{\text{mid}}} z_i o_i(\tau) + \delta_O z(\tau)}, & \pi'_O &\leftarrow g^{\alpha \sum_{i \in \mathcal{I}_{\text{mid}}} z_i o_i(\tau) + \delta_O z(\tau)}. \end{aligned}$$

- **Updated Commitment for h :** The prover $\mathcal{P}$ needs to calculate the commitment π_H with the additional δ 's values to ensure polynomial equality test is satisfied:

$$\begin{aligned} \pi_H &\leftarrow g^{h(\tau) + \delta_O + \delta_R \ell(\tau) + \delta_L r(\tau) + \delta_L \delta_R z(\tau)}, \\ \pi'_H &\leftarrow g^{\alpha(h(\tau) + \delta_O + \delta_R \ell(\tau) + \delta_L r(\tau) + \delta_L \delta_R z(\tau))}. \end{aligned}$$

- **Updated Witness Consistency Proof:** The prover $\mathcal{P}$ needs to calculate the commitment π_β with the additional δ 's values to ensure witness consistency:

$$\pi_\beta \leftarrow \left(g^{\beta_L z(\tau)} \right)^{\delta_L} \left(g^{\beta_R z(\tau)} \right)^{\delta_R} \left(g^{\beta_O z(\tau)} \right)^{\delta_O} g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}.$$

Let us now look at the final version of our basic SNARK protocol.

Attempt #5: Turning SNARK into zk-SNARK

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha, \beta_L, \beta_R, \beta_O, \gamma \leftarrow \mathbb{F}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp :

$$\begin{aligned} \text{pp} &\leftarrow \left\{ \{g^{\tau^i}\}_{i \in [d]}, \{g^{z(\tau)}, g^{\ell_i(\tau)}, g^{\alpha \ell_i(\tau)}, g^{r_i(\tau)}, g^{\alpha r_i(\tau)}, \right. \\ &\quad \left. g^{o_i(\tau)}, g^{\alpha o_i(\tau)}, g^{\beta_L \ell_i(\tau)}, g^{\beta_R r_i(\tau)}, g^{\beta_O o_i(\tau)} \}_{i \in \mathcal{I}_{\text{mid}}} \right\} \\ \text{vp} &\leftarrow \{g^{z(\tau)}, g^\alpha, g^{\beta_L}, g^{\beta_R}, g^{\beta_O}, g^{\beta_L \gamma}, g^{\beta_R \gamma}, g^{\beta_O \gamma}, g^\gamma, \\ &\quad \{g^{\ell_i(\tau)}, g^{r_i(\tau)}, g^{o_i(\tau)}\}_{i \in \mathcal{I}_{\text{io}}}\} \end{aligned}$$

- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove($\text{pp}, \mathbb{x}, \mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{z}$ and $\ell(X), r(X), o(X)$.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Splits $\ell(X) = \ell_{\text{mid}}(X) + \ell_{\text{io}}(X)$ – intermediate and input/output parts. That being said, $\ell_{\text{mid}}(X) = \sum_{i \in \mathcal{I}_{\text{mid}}} \ell_i(X)$. Repeat for $r(X)$ and $o(X)$.
- ✓ **Samples** $\delta_L, \delta_R, \delta_O \leftarrow \mathbb{F}$ and calculates the following values:

$$\begin{aligned} \pi_L &\leftarrow g^{\ell_{\text{mid}}(\tau)} \left(g^{z(\tau)} \right)^{\delta_L}, & \pi'_L &\leftarrow g^{\alpha \ell_{\text{mid}}(\tau)} \left(g^{\alpha z(\tau)} \right)^{\delta_L}, \\ \pi_R &\leftarrow g^{r_{\text{mid}}(\tau)} \left(g^{z(\tau)} \right)^{\delta_R}, & \pi'_R &\leftarrow g^{\alpha r_{\text{mid}}(\tau)} \left(g^{z(\tau)} \right)^{\delta_R}, \\ \pi_O &\leftarrow g^{o_{\text{mid}}(\tau)} \left(g^{z(\tau)} \right)^{\delta_O}, & \pi'_O &\leftarrow g^{\alpha o_{\text{mid}}(\tau)} \left(g^{z(\tau)} \right)^{\delta_O}, \\ \pi_H &\leftarrow g^{h(\tau)} (g^{\delta_O}) (g^{r(\tau)})^{\delta_L} (g^{\ell(\tau)})^{\delta_R} (g^{z(\tau)})^{\delta_L \delta_R}, \\ \pi'_H &\leftarrow g^{\alpha h(\tau)} (g^{\delta_O}) (g^{r(\tau)})^{\delta_L} (g^{\ell(\tau)})^{\delta_R} (g^{z(\tau)})^{\delta_L \delta_R}, \\ \pi_\beta &\leftarrow \left(g^{\beta_L z(\tau)} \right)^{\delta_L} \left(g^{\beta_R z(\tau)} \right)^{\delta_R} \left(g^{\beta_O z(\tau)} \right)^{\delta_O} g^{\beta_L \ell(\tau) + \beta_R r(\tau) + \beta_O o(\tau)}. \end{aligned}$$

- ✓ Publishes $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$ as a proof.

Verify($\text{vp}, \mathbb{x}, \boldsymbol{\pi}$)

Upon receiving $\boldsymbol{\pi} = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi'_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g^{\ell_i(\tau)})^{z_i}$, $\pi_R^* \leftarrow \pi_R \prod_{i \in \mathcal{I}_{\text{io}}} (g^{r_i(\tau)})^{z_i}$,
 $\pi_O^* \leftarrow \pi_O \prod_{i \in \mathcal{I}_{\text{io}}} (g^{o_i(\tau)})^{z_i}$

✓ Checks whether all of the following conditions hold:

$$\begin{aligned}
e(\pi_L^*, \pi_R^*) &= e(g^{z(\tau)}, \pi_H) \cdot e(\pi_O^*, g), \\
e(\pi_L, g^\alpha) &= e(\pi'_L, g), \quad e(\pi_R, g^\alpha) = e(\pi'_R, g), \\
e(\pi_O, g^\alpha) &= e(\pi'_O, g), \quad e(\pi_H, g^\alpha) = e(\pi'_H, g), \\
e(\pi_L, g^{\gamma\beta_L}) \cdot e(\pi_R, g^{\gamma\beta_R}) \cdot e(\pi_O, g^{\gamma\beta_O}) &= e(\pi_\beta, g^\gamma).
\end{aligned}$$

15.2 Real Protocols

While our built protocol is zk-SNARK, it is still not optimized for practical use. In fact, let us recap what is complexity of our protocol.

Proposition 15.3 (Complexity of the Basic Protocol). Suppose circuit consists of n gates. Then, the complexity of the basic protocol is as follows:

- **Proof Size:** $O(1)$ — we have a constant number of group elements.
- **Setup Time:** $O(n)$ — we need to calculate powers of τ and evaluations at τ .
- **Prover Time:** $O(n \log n)$ — using FFT and wise choice of Ω .
- **Verifier Time:** $O(1)$ — we have a constant number of pairings to evaluate.

However, $O(1)$ is not very descriptive for proof and verifier complexities, so let us provide a more detailed analysis.

- **Proof Size:** $9 \mathbb{G}$ group elements.
- **Verifier Time:** 15 pairings and $O(|\mathcal{I}_{io}|)$ group multiplications.

Now, this is not bad at all! In fact, this is already practical for many applications. However, we can do better by a more clever choice of constants and terms. This is exactly what is done by Bryan Parno and Craig Gentry in their research “Pinocchio: Nearly Practical Verifiable Computation”.

15.3 Pinocchio Protocol

We first begin from the non-zero-knowledge version of the Pinocchio Protocol and then extend it to the zero-knowledge version.

Setup Procedure. The Pinocchio Protocol exploits the idea that we might choose different generators for $\ell(X)$, $r(X)$, and $o(X)$. Namely, the protocol begins from choosing random $\rho_L, \rho_R \leftarrow \mathbb{F}$. Then, we define $\rho_O \triangleq \rho_L \rho_R$, and finally introduce the following generators:

$$g_L \triangleq g^{\rho_L}, \quad g_R \triangleq g^{\rho_R}, \quad g_O \triangleq g^{\rho_O} = g^{\rho_L \rho_R}$$

The next change is that we use different α values for $\ell(X)$, $r(X)$, and $o(X)$.

Namely, we define $\alpha_L, \alpha_R, \alpha_O \leftarrow \$ \mathbb{F}$ and use them in the commitments. This way, in the setup phase, we additionally prepare the following values:

$$\{g_L^{\ell_i(\tau)}, g_L^{\alpha_L \ell_i(\tau)}, g_R^{r_i(\tau)}, g_R^{\alpha_R r_i(\tau)}, g_O^{o_i(\tau)}, g_O^{\alpha_O o_i(\tau)}\}_{i \in \mathcal{I}_{\text{mid}}}$$

Instead of using three different β 's, the Pinocchio Protocol uses a single $\beta \leftarrow \$ \mathbb{F}$:

$$\{g_L^{\beta \ell_i(\tau)}, g_R^{\beta r_i(\tau)}, g_O^{\beta o_i(\tau)}\}_{i \in \mathcal{I}_{\text{mid}}}$$

And finally, since we also have a $\gamma \leftarrow \$ \mathbb{F}$ for the extended witness consistency and some other missing quantities, we also prepare the following values:

$$g_O^{z(\tau)}, g^{\alpha_L}, g^{\alpha_R}, g^{\alpha_O}, g^{\beta \gamma}, g^\gamma$$

Proving. Again, suppose intermediate polynomials for indices $\mathcal{I}_{\text{mid}}$ are $\ell_{\text{mid}}(X) = \sum_{i \in \mathcal{I}_{\text{mid}}} z_i \ell_i(X)$ and similarly for $r_{\text{mid}}(X)$ and $o_{\text{mid}}(X)$. The prover $\mathcal{P}$ calculates the following commitments:

$$\begin{aligned} \pi_L &\leftarrow g_L^{\ell_{\text{mid}}(\tau)}, & \pi'_L &\leftarrow g_L^{\alpha_L \ell_{\text{mid}}(\tau)}, \\ \pi_R &\leftarrow g_R^{r_{\text{mid}}(\tau)}, & \pi'_R &\leftarrow g_R^{\alpha_R r_{\text{mid}}(\tau)}, \\ \pi_O &\leftarrow g_O^{o_{\text{mid}}(\tau)}, & \pi'_O &\leftarrow g_O^{\alpha_O o_{\text{mid}}(\tau)}, \\ \pi_H &\leftarrow g^{h(\tau)}, & \pi_\beta &\leftarrow g_L^{\beta \ell_{\text{mid}}(\tau)} g_R^{\beta r_{\text{mid}}(\tau)} g_O^{\beta o_{\text{mid}}(\tau)}, \end{aligned}$$

and then uses these commitments in the proof $\pi = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi_\beta)$.

Verification. First, the verifier checks the PoE properties:

$$e(\pi_L, g_L^{\alpha_L}) = e(\pi'_L, g_L), \quad e(\pi_R, g_R^{\alpha_R}) = e(\pi'_R, g_R), \quad e(\pi_O, g_O^{\alpha_O}) = e(\pi'_O, g_O).$$

Next, we check the extended witness consistency:

$$e(\pi_L \pi_R \pi_O, g^{\beta \gamma}) = e(\pi_\beta, g^\gamma)$$

The verifier restores the full proof $\pi_L^*, \pi_R^*, \pi_O^*$ by multiplying the public part of the proof with the corresponding private part. This is done as follows:

$$\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g_L^{\ell_i(\tau)})^{z_i}, \quad \pi_R^* \leftarrow \pi_R \prod_{i \in \mathcal{I}_{\text{io}}} (g_R^{r_i(\tau)})^{z_i}, \quad \pi_O^* \leftarrow \pi_O \prod_{i \in \mathcal{I}_{\text{io}}} (g_O^{o_i(\tau)})^{z_i}.$$

And finally, we check the polynomial equality test:

$$e(\pi_L^*, \pi_R^*) = e(g_O^{z(\tau)}, \pi_H) \cdot e(\pi_O^*, g)$$

Zero-Knowledge Extension. The zero-knowledge extension of the Pinocchio Protocol is done exactly in the same way as we did for the basic SNARK protocol.

Namely, we introduce random scalars $\delta_L, \delta_R, \delta_O \leftarrow \mathbb{F}$ and calculate the following commitments:

$$\begin{aligned}\pi_L &\leftarrow g_L^{\ell_{\text{mid}}(\tau)} \left(g_L^{z(\tau)} \right)^{\delta_L}, & \pi'_L &\leftarrow g_L^{\alpha_L L_{\text{mid}}(\tau)} \left(g_L^{\alpha_L z(\tau)} \right)^{\delta_L}, \\ \pi_R &\leftarrow g_R^{r_{\text{mid}}(\tau)} \left(g_R^{z(\tau)} \right)^{\delta_R}, & \pi'_R &\leftarrow g_R^{\alpha_R R_{\text{mid}}(\tau)} \left(g_R^{\alpha_R z(\tau)} \right)^{\delta_R}, \\ \pi_O &\leftarrow g_O^{o_{\text{mid}}(\tau)} \left(g_O^{z(\tau)} \right)^{\delta_O}, & \pi'_O &\leftarrow g_O^{\alpha_O O_{\text{mid}}(\tau)} \left(g_O^{\alpha_O z(\tau)} \right)^{\delta_O}, \\ \pi_H &\leftarrow g^{h(\tau)} (g^{\delta_O}) (g^{\delta_L}) (g^{\delta_R}) (g^{\delta_L \delta_R}), \\ \pi_\beta &\leftarrow \left(g_L^{\beta z(\tau)} \right)^{\delta_L} \left(g_R^{\beta z(\tau)} \right)^{\delta_R} \left(g_O^{\beta z(\tau)} \right)^{\delta_O} g_L^{\beta \ell_{\text{mid}}(\tau)} g_R^{\beta r_{\text{mid}}(\tau)} g_O^{\beta o_{\text{mid}}(\tau)}\end{aligned}$$

Let us see what has changed in the Pinocchio Protocol.

Proposition 15.4. Pinocchio Protocol Proof size and verifier time complexity, compared to the basic SNARK protocol, are as follows:

- **Proof Size:** $8 \mathbb{G}$ group elements.
- **Verifier Time:** 11 pairings and $O(|\mathcal{I}_{\text{io}}|)$ group multiplications (**4 less!**)

Let us see the complete specification of the Pinocchio Protocol.

Pinocchio Protocol

Suppose we are given a circuit $\mathcal{C}$ with a maximum degree d of polynomials used underneath. Thus, all parties additionally know the target polynomial $z(X)$. Additionally, we know that $\mathcal{I}_{\text{io}}$ corresponds to public signals, while $\mathcal{I}_{\text{mid}}$ corresponds to private signals.

Setup(1^λ)

The *trusted party* conducts the following steps:

- ✓ Picks random values $\tau, \alpha_L, \alpha_R, \alpha_O, \beta, \gamma, \rho_L, \rho_R \leftarrow \mathbb{F}$.
- ✓ Calculates $\rho_O \leftarrow \rho_L \rho_R$ and defines generators $g_L \leftarrow g^{\rho_L}, g_R \leftarrow g^{\rho_R}, g_O \leftarrow g^{\rho_O}$.
- ✓ **Outputs** prover parameters pp and verification parameters vp :
$$\begin{aligned}\text{pp} &\leftarrow \left\{ \{g^{\tau^i}\}_{i \in [d]}, \{g_L^{\ell_i(\tau)}, g_L^{\alpha_L \ell_i(\tau)}, g_R^{r_i(\tau)}, g_R^{\alpha_R r_i(\tau)}, g_O^{o_i(\tau)}, g_O^{\alpha_O o_i(\tau)}, \right. \\ &\quad \left. g_L^{\beta \ell_i(\tau)}, g_R^{\beta r_i(\tau)}, g_O^{\beta o_i(\tau)} \}_{i \in \mathcal{I}_{\text{mid}}}, \right. \\ &\quad \left. g_L^{\alpha_L z(\tau)}, g_L^{z(\tau)}, g_R^{z(\tau)}, g_O^{z(\tau)}, g_R^{\alpha_R z(\tau)}, g_O^{\alpha_O z(\tau)}, g_L^{\beta z(\tau)}, g_R^{\beta z(\tau)}, g_O^{\beta z(\tau)} \right\} \\ \text{vp} &\leftarrow \{g_O^{z(\tau)}, g^{\alpha_L}, g^{\alpha_R}, g^{\alpha_O}, g^{\beta \gamma}, g^\gamma, \{g_L^{\ell_i(\tau)}, g_R^{r_i(\tau)}, g_O^{o_i(\tau)}\}_{i \in \mathcal{I}_{\text{io}}}\end{aligned}$$
- ✓ **Deletes** aforementioned random scalars (toxic waste).

Prove($\text{pp}, \mathbb{x}, \mathbb{w}$)

The prover $\mathcal{P}$ conducts the following steps:

- ✓ Runs the circuit to get $\mathbf{z}$ and $\ell(X), r(X), o(X)$.
- ✓ Calculates $h(X) \leftarrow (\ell(X)r(X) - o(X))/z(X)$.
- ✓ Splits $\ell(X) = \ell_{\text{mid}}(X) + \ell_{\text{io}}(X)$ – intermediate and input/output parts. Repeat for $r(X)$ and $o(X)$.
- ✓ Samples $\delta_L, \delta_R, \delta_O \leftarrow \mathbb{F}$ and publishes the following values as a proof π :

$$\begin{aligned}
\pi_L &\leftarrow g_L^{\ell_{\text{mid}}(\tau)} \left(g_L^{z(\tau)} \right)^{\delta_L}, & \pi'_L &\leftarrow g_L^{\alpha_L L_{\text{mid}}(\tau)} \left(g_L^{\alpha_L z(\tau)} \right)^{\delta_L}, \\
\pi_R &\leftarrow g_R^{r_{\text{mid}}(\tau)} \left(g_R^{z(\tau)} \right)^{\delta_R}, & \pi'_R &\leftarrow g_R^{\alpha_R R_{\text{mid}}(\tau)} \left(g_R^{\alpha_R z(\tau)} \right)^{\delta_R}, \\
\pi_O &\leftarrow g_O^{o_{\text{mid}}(\tau)} \left(g_O^{z(\tau)} \right)^{\delta_O}, & \pi'_O &\leftarrow g_O^{\alpha_O O_{\text{mid}}(\tau)} \left(g_O^{\alpha_O z(\tau)} \right)^{\delta_O}, \\
&& \pi_H &\leftarrow g^{h(\tau)} (g^{\delta_O}) (g^{\delta_L}) (g^{\delta_R}) (g^{\delta_L \delta_R}), \\
\pi_\beta &\leftarrow \left(g_L^{\beta z(\tau)} \right)^{\delta_L} \left(g_R^{\beta z(\tau)} \right)^{\delta_R} \left(g_O^{\beta z(\tau)} \right)^{\delta_O} g_L^{\beta \ell_{\text{mid}}(\tau)} g_R^{\beta r_{\text{mid}}(\tau)} g_O^{\beta o_{\text{mid}}(\tau)}
\end{aligned}$$

Verify(vp, $\mathbb{x}$, π)

Upon receiving $\pi = (\pi_L, \pi'_L, \pi_R, \pi'_R, \pi_O, \pi'_O, \pi_H, \pi_\beta)$, the verifier $\mathcal{V}$:

- ✓ Finds $\pi_L^* \leftarrow \pi_L \prod_{i \in \mathcal{I}_{\text{io}}} (g_L^{\ell_i(\tau)})^{z_i}$, $\pi_R^* \leftarrow \pi_R \prod_{i \in \mathcal{I}_{\text{io}}} (g_R^{r_i(\tau)})^{z_i}$, $\pi_O^* \leftarrow \pi_O \prod_{i \in \mathcal{I}_{\text{io}}} (g_O^{o_i(\tau)})^{z_i}$
- ✓ Checks whether all of the following conditions hold:

$$\begin{aligned}
e(\pi_L^*, \pi_R^*) &= e(g_O^{z(\tau)}, \pi_H) \cdot e(\pi_O^*, g), \\
e(\pi_L, g_L^{\alpha_L}) &= e(\pi'_L, g_L), \\
e(\pi_R, g_R^{\alpha_R}) &= e(\pi'_R, g_R), \\
e(\pi_O, g_O^{\alpha_O}) &= e(\pi'_O, g_O), \\
e(\pi_L \pi_R \pi_O, g^{\gamma \beta}) &= e(\pi_\beta, g^\gamma).
\end{aligned}$$

15.4 Groth16 Protocol

Finally, Groth16 allows to reduce the number of pairings **down to 3!** This is done through a technique called **Generic Group Model** (GGM for short). Simply put, GGM allows the adversary to only make oracle requests to compute the group operations. For example, having a set $\{g^{\alpha r_i(\tau)}\}_{i \in [d]}$, adversary can compute only linear combinations of these values. In the particular case of Groth16, instead of considering $\ell_i(X)$, $r_i(X)$, and $o_i(X)$ separately, we construct their linear combinations as $Q_i(X) := \beta \ell_i(X) + \alpha r_i(X) + o_i(X)$, where α and β are toxic parameters.

Let us now concretely describe the Groth16 construction.

Asymmetric Pairing. One change which was made in Groth16 is using the asymmetric pairing function $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ over groups $\mathbb{G}_1$ and $\mathbb{G}_2$ (for more details, see [Section 9.5](#)). Suppose respective group generators are $g_1 \in \mathbb{G}_1$ and

$g_2 \in \mathbb{G}_2$. The primary reason for this is that calculating the pairing in asymmetric setting is more efficient than in symmetric setting. Other than that, Groth16 can be easily formulated using symmetric pairings.

Setup Procedure. In Groth16, we need only five random scalars: $\alpha, \beta, \gamma, \delta$, $\tau \leftarrow \$ \mathbb{F}$. Now, the prover key pk looks as follows:

$$pp \leftarrow \left(g_1^\alpha, g_1^\beta, g_1^\delta, \left\{ g_1^{\tau^i}, \frac{\beta \ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau)}{\gamma}, \frac{\tau^i z(\tau)}{\delta} \right\}_{i \in [n]}, g_2^\beta, g_2^\delta, g_2^\gamma, \{g_2^{\tau^i}\}_{i \in [d]} \right)$$

Proving. Sample random $\delta_L, \delta_R \leftarrow \$ \mathbb{F}$ and compute the following values:

$$\begin{aligned} \pi_L &\leftarrow g_1^{\alpha + \sum_{i=1}^n z_i \ell_i(\tau) + \delta_L \delta}, & \pi_R &\leftarrow g_2^{\beta + \sum_{i=0}^n z_i r_i(\tau) + \delta_R \delta}, \\ \pi_O &\leftarrow g_1^{\frac{Q_{\text{mid}}(\tau) + h(\tau) z(\tau)}{\delta} + \ell \delta_R + r \delta_L - \delta_L \delta_R \delta}, \end{aligned}$$

where we denoted $Q_{\text{mid}} := \sum_{i \in \mathcal{I}_{\text{mid}}} z_i (\beta \ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau))$.

Verification. The verifier first calculates the following value:

$$\pi_{\text{io}} \leftarrow g_1^{\sum_{i \in \mathcal{I}_{\text{io}}} z_i (\beta \ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau)) / \gamma},$$

and then checks the following single condition:

$$e(\pi_L, \pi_R) = e(g_1^\alpha, g_2^\beta) e(\pi_{\text{io}}, g_2^\gamma) e(\pi_O, g_2^\delta)$$

Note that $e(g_1^\alpha, g_2^\beta)$ can be additionally hard-coded in the verifier, thus reducing the number of pairings to 3. Finally, the proof's size is now reduced to 3 group elements: two from $\mathbb{G}_1$, and one from $\mathbb{G}_2$.

Acknowledgements

This section was greatly inspired by “Why and How zk-SNARK works” by Maksym Petkus and “ZK MOOC, Spring 2023” Linear PCP lecture.

16 Circom

In this final lecture, we bridge the gap between the theoretical concepts presented in previous lectures and their practical realization using the **Circom** DSL.

Definition 16.1. **Circom** is a domain-specific language for building arithmetic circuits that can be used to produce zk-SNARK proofs.

Throughout this lecture, we will walk through how concepts like R1CS, witness, trusted setup, and verification keys appear in actual code and practice.

16.1 Journey Begins

In the previous lectures, we covered a variety of theoretical concepts: zk-SNARKs, trusted setup, arithmetic circuits, constraints, witnesses, and the Rank-1 Constraint System (R1CS) representation. Now, let's see how these appear in practice.

16.2 From Theory to Practice: Circom Basics

We learned that a circuit can represent a complex arithmetic computation over a finite field. Circom allows us to write these circuits in a high-level syntax. To begin, consider the arithmetic circuit $r = x \times y$.

It can be represented in Circom syntax as follows:

```
pragma circom 2.1.6;

template Math() {
    signal input x;
    signal input y;

    signal output r <== x * y;
}

component main = Math();
```

Here, we see how easy it is to define a circuit that takes two inputs x, y and outputs their product r . The `template` defines a reusable circuit component, while `signal input` and `signal output` represent inputs and outputs, respectively. Intermediate signals (without input or output) are internal primitives within the circuit.

Public vs Private Signals:

Output signals are always public. You may also define public inputs by specifying them in the main component, for example:

```
component main {public [x]} = Math();
```

This means x is a public input and will appear in the verification context. The order of public signals in the final proof verification step follows the order of their definition inside the template, starting with outputs.

For example, if your circuit looks like this:

```
template Circuit() {
    signal x;

    signal output o2;

    signal input c;
    signal input a;

    signal k1;

    signal input b;

    signal output o1;
}

component main {public [a, b, c]} = Circuit();
```

The order of the public signals that should be passed to the verifier is as follows:

(o2, o1, c, a, b)

16.3 Arguments, Functions, and Vars

Sometimes we need to calculate some values as constants for our circuit. For example, if you want your circuit to be a multi-tool that, based on provided arguments, can work with different cases. For this purpose, we can declare *functions* and *vars* inside the circuit, as shown below:

```
function transformNumber(value) {
    return value ** 2;
}

template Math(padding) {
    signal input x;
    signal input y;

    var elementsNumber = transformNumber(padding);
    signal b <== x * elementsNumber;

    signal output r <== b * y;
}
```

```
component main {public [x]} = Math(12);
```

Here, `var elementsNumber` and the function `transformNumber` are evaluated at compile time. Remember that assignments using `var` and functions do not produce constraints by themselves. Only `<==`, `==>`, or `===` and actual arithmetic on signals produce constraints reflected in R1CS.

Remark. Sometimes, one needs to perform operations like division or non-quadratic multiplication on the signal. For this purpose, you can use the `-->` and `<--` notations to compute the values “out-of-circuit”. For example:

```
template Math() {
    signal input x;
    signal input y;

    signal b <-- x / y;

    signal output r <== b * y;
}

component main = Math();
```

In this case, no constraints are generated with the x input, and it does not even participate in the witness directly.

Notice! This is the main difference between Circom and other languages. When you write a function evaluation $y = f(x)$ in any other language (say, Python or Rust), you are specifying the set of instructions to compute y from x (commonly sequentially). In Circom (or in any other R1CS language) you are merely asserting the **correctness** of the computation and therefore of all intermediate computations. This way, if your task would have been to compute $y = \frac{1}{x}$, you could simply ask y to be the result of the division (that can be computed out of circuit) and then asserting that $x \times y = 1$ with $x \neq 0$. This way, you are not writing the division itself, but the constraints that the division should satisfy.

16.4 Theoretical Recap: Using the Learned Concepts

Next, let us apply the learned concepts to a more complex examples. We start with the `if` statement logic.

Example 16.1. Recall the complex example we analyzed in earlier lectures:

```
def r(x1: bool, x2: F, x3: F) -> F:
    return x2 * x3 if x1 else x2 + x3
```

This can be represented as:

$$r = x_1 \times (x_2 \times x_3) + (1 - x_1) \times (x_2 + x_3).$$

We also had the additional constraint $x_1 \times (1 - x_1) = 0$ to ensure x_1 is binary. The resulting system of constraints was:

$$x_1 \times x_1 = x_1 \quad (1)$$

$$x_2 \times x_3 = \text{mult} \quad (2)$$

$$x_1 \times \text{mult} = \text{selectMult} \quad (3)$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult} \quad (4)$$

It took us quite some time to understand and come up with the constraint system, which can be visualized as follows:

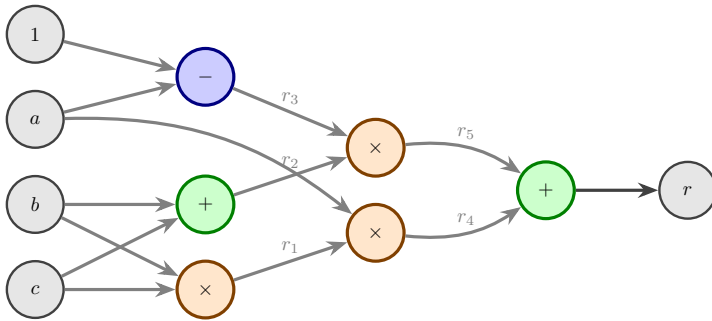


Figure 16.1: Example of a circuit evaluating the if statement logic.

The inputs can be directly transformed into signals like below:

```
template Math() {
    signal output r;

    signal input x1;

    signal input x2;
    signal input x3;
}
```

In our case, we have an additional output signal, so we can "return" it from the circuit. Now, let's compare the mathematical and Circom representations.

Mathematical Constraints:

$$x_1 \times x_1 = x_1$$

$$x_2 \times x_3 = \text{mult}$$

$$x_1 \times \text{mult} = \text{selectMult}$$

$$(1 - x_1) \times (x_2 + x_3) = r - \text{selectMult}$$

$$x1 * x1 === x1;$$

$$\text{signal mult} <== x2 * x3;$$

$$\text{signal selectMult} <== x1 * \text{mult};$$

$$(1 - x1) * (x2 + x3) + \text{selectMult} ==> r;$$

As we can see, the translation from math to Circom is straightforward. We have used *signals* for constraint definitions.

Remark. If you wish to follow along with the explanations in the following chapters:

1. Clone the repository <https://github.com/ZKDL-Camp/hardhat-zkit-template>.
2. Run `npm install` to install dependencies and `npx hardhat zkit make` to compile Circom circuits and generate the necessary artifacts.

16.5 From R1CS to Proof Generation

Now, let us break down everything that is happening during the proof generation process. After compilation, you will find the following files in the `zkit/artifacts/circuits` folder (starting from the project root):

- `.r1cs` file: The Rank-1 Constraint System representation of the circuit.
- `.wasm` and `*.js` files: The code to compute the witness from the given inputs.
- `.zkey` file: Proving keys after the trusted setup.
- `.sym` file: Symbolic reference for signals.

R1CS File. Let us start with the `.r1cs` file. In [Section 16](#), we defined the following coefficient vectors (in simple constraints) for our task:

$$\ell_1 = (0, 0, 1, 0, 0, 0, 0)$$

$$\ell_2 = (0, 0, 0, 1, 0, 0, 0)$$

$$\ell_3 = (0, 0, 1, 0, 0, 0, 0)$$

$$\ell_4 = (1, 0, -1, 0, 0, 0, 0)$$

$$r_1 = (0, 0, 1, 0, 0, 0, 0)$$

$$r_2 = (0, 0, 0, 0, 1, 0, 0)$$

$$r_3 = (0, 0, 0, 0, 0, 1, 0)$$

$$r_4 = (0, 0, 0, 1, 1, 0, 0)$$

$$o_1 = (0, 0, 1, 0, 0, 0, 0)$$

$$o_2 = (0, 0, 0, 0, 0, 1, 0)$$

$$o_3 = (0, 0, 0, 0, 0, 0, 1)$$

$$o_4 = (0, 1, 0, 0, 0, 0, -1)$$

On the other hand, using the test from the `test/Math.witness.test.ts` file and reading the R1CS file, we can see:

test/Math.witness.test.ts

```
expect(constraint1[0]).to.deep.equal([ 0n, 0n, 1n, 0n, 0n, 0n, 0n
  ↪ ]);
expect(constraint1[1]).to.deep.equal([ 0n, 0n, 1n, 0n, 0n, 0n, 0n
  ↪ ]);
expect(constraint1[2]).to.deep.equal([ 0n, 0n, 1n, 0n, 0n, 0n, 0n
  ↪ ]);

expect(constraint2[0]).to.deep.equal([ 0n, 0n, 0n,
  ↪ babyJub.F.negone, 0n, 0n, 0n ]);
expect(constraint2[1]).to.deep.equal([ 0n, 0n, 0n, 0n, 1n, 0n, 0n
  ↪ ]);
expect(constraint2[2]).to.deep.equal([ 0n, 0n, 0n, 0n, 0n,
  ↪ babyJub.F.negone, 0n ]);

expect(constraint3[0]).to.deep.equal([ 0n, 0n, babyJub.F.negone,
  ↪ 0n, 0n, 0n, 0n ]);
expect(constraint3[1]).to.deep.equal([ 0n, 0n, 0n, 0n, 0n, 1n, 0n
  ↪ ]);
expect(constraint3[2]).to.deep.equal([ 0n, 0n, 0n, 0n, 0n, 0n,
  ↪ babyJub.F.negone ]);

expect(constraint4[0]).to.deep.equal([ babyJub.F.negone, 0n, 0n,
  ↪ 0n, 0n, 0n, 0n ]);
expect(constraint4[1]).to.deep.equal([ 0n, 0n, 0n, 1n, 0n, 0n, 0n
  ↪ ]);
expect(constraint4[2]).to.deep.equal([ 0n, babyJub.F.negone, 0n,
  ↪ 0n, 0n, 0n, 0n ]);
```

Mostly, the structure generated by Circom aligns with what we had devised, except for the last constraint. The difference occurs because of Circom's optimization to make proof generation and verification more efficient.

Now, let's take a closer look at how the witness is computed. In [Section 13](#), we had:

$$\mathbf{z} = (1, r, x_1, x_2, x_3, \text{mult}, \text{selectMult})$$

Given the inputs: $x_1 = 1, x_2 = 3, x_3 = 4$, we can quickly do the math and find out that the actual witness should look like this: $\mathbf{z} = (1, 12, 1, 3, 4, 12, 12)$ based on:

$$\begin{aligned}\text{mult} &= 3 \times 4 = 12 \\ \text{selectMult} &= 1 \times 12 = 12 \\ r &= 1 \times (3 \times 4) + (1 - 1) \times (3 + 4) = 12 + 0 = 12\end{aligned}$$

Indeed, it aligns with the test from `test/Math.witness.test.ts`:

test/Math.witness.test.ts

```
expect(witness[0]).to.equal(1n);
expect(witness[1]).to.equal(12n); // r
expect(witness[2]).to.equal(1n); // x1
expect(witness[3]).to.equal(3n); // x2
expect(witness[4]).to.equal(4n); // x3
expect(witness[5]).to.equal(12n); // mult
expect(witness[6]).to.equal(12n); //
  ↪ selectMult
```

The initial 1 in the witness is a constant to facilitate the usage of constants inside the circuit. This corresponds to the fact that $w_0 = 1$ is often used to handle constant terms in R1CS.

The Circom also provides a named representation of all witness elements, which is stored in the `.sym` file and looks as follows:

.sym file for $x_1? x_2 \times x_3 : x_2 + x_3$

```
1,1,0,main.r
2,2,0,main.x1
3,3,0,main.x2
4,4,0,main.x3
5,5,0,main.mult
6,6,0,main.selectMult
```

It not only tells us the names of all signals, but also includes information about the optimized signals.

Recall the previous example where we used division and `<--` to store it in the intermediate signal. The generated `.sym` file would look like this:

.sym file for $b \leftarrow x/y, r \leftarrow by$

```
1,1,0,main.r
2,-1,0,main.x
3,2,0,main.y
4,3,0,main.b
```

As we can see, the `-1` was added to the signal `x` to indicate that it is not used in the witness.

Also, according to the documentation of Circom, there are three levels of optimization.

In the 2.1.9 version of Circom, the default optimization was O2, but it was lowered to O1 in following versions because O2 was too aggressive leading to vulnerable circuits.

Remark. In addition, all linear constraints are optimized on the O2 optimization.

Now, let's examine what the third column in the `.sym` file means.

Consider the following circuit:

```
template BinaryCheck() {
    signal input x1;

    x1 * x1 === x1;
}

template SelectMult() {
    signal input x1;

    signal input x2;
    signal input x3;
```

```

    signal mult <== x2 * x3;

    signal output out <== x1 * mult;
}

template Math() {
    signal output r;

    signal input x1;

    signal input x2;
    signal input x3;

    component binCheck = BinaryCheck();
    binCheck.x1 <== x1;

    component selectMult = SelectMult();
    selectMult.x1 <== x1;
    selectMult.x2 <== x2;
    selectMult.x3 <== x3;

    (1 - x1) * (x2 + x3) + selectMult.out ==> r;
}

```

We split the circuit into three parts, and the .sym file will look like this:

.sym file for $x_1? x_2 \times x_3 : x_2 + x_3$ with templates

```

1,1,2,main.r
2,2,2,main.x1
3,3,2,main.x2
4,4,2,main.x3
5,-1,0,main.binCheck.x1
6,5,1,main.selectMult.out
7,-1,1,main.selectMult.x1
8,-1,1,main.selectMult.x2
9,-1,1,main.selectMult.x3
10,6,1,main.selectMult.mult

```

As we can see, the third column indicates the locality of the signal. Essentially, it tells us which signals are grouped together under the same template.

Another interesting aspect is the order: 0 represents the first component, 1 represents the second component used during computation, and the last component used is the actual 'main'.

Remark. Pay attention to the difference between the modified Math circuit's .sym file and the original one. Even though we added more signals (i.e., constraints), they were actually optimized by Circom back to the original state.

And the last column is the full name of the constraint, including the path to where it is defined in the code.

16.6 Parallel and Custom Keywords

In Circom, there are two special keywords designed to address specific use cases: `custom` and `parallel`.

The `custom` keyword introduces *custom templates* that do not emit R1CS constraints directly. Instead, they delegate logic to `snarkjs` or other libraries at a later stage. Consequently, custom templates **cannot** declare subcomponents or add R1CS constraints within their bodies.

The `custom` keyword is used as follows:

```
pragma circom 2.0.6;
pragma custom_templates;

template custom MyCustomGate() {
    // Custom template's code
    // No R1CS constraints or subcomponents can be declared
    //   ↪ here
    // Logic will be handled by snarkjs as a PLONK custom gates
}
```

Remark. At the moment of writing the document the `snarkjs` does not support any custom gates (as stated in their documentation). Also, they can be used only in turbo-PLONK or UltraPlonk schemes. Nevertheless, you can find an example of how they have been used here: <https://github.com/zkFHE/circomlib-fhe/tree/main>.

Meanwhile, the `parallel` keyword (available from Circom 2.0.8 onward) can be applied at either the template or the component instantiation level to parallelize witness generation for independent computations, thereby accelerating large circuits. Parallelism is *only* applied to the C++ witness generator; it does not affect the constraints themselves.

This keyword will be useful in the structures as below:

```
template parallel ParallelExample(n) {
    signal input in[n];
    signal output out[n];

    // Each iteration is independent, so we can parallelize
    for (var i = 0; i < n; i++) {
        out[i] <== in[i] * 2;
    }
}
```

In summary, you should use the `custom` keyword whenever you want to define a template handled **only** by turbo-PLONK or UltraPlonk schemes. You can also find the exact section in the R1CS binary format where custom gates are stored for later processing by the library in the following link:

https://github.com/iden3/r1csfile/blob/master/doc/r1cs_bin_format.md#custom-gates-list-section-plonk

The `parallel` keyword is helpful when dealing with large or repetitive computations, as it can speed up witness generation. However, in small circuits or wherever computation is inherently sequential (i.e., where the output of one part is the input to another), `parallel` has no effect.

You can find additional examples of the `parallel` keyword usage here: <https://github.com/zkFHE/circomlib-fhe/tree/main>.

With this, we have covered all the important files generated by Circom.

16.7 Generating and Verifying Proofs

Now, it is time to look at proof generation and verification. In this chapter, our main focus will be on the code from `test/Math.circuit.ts`.

To generate a proof, we need to call the `generateProof` method on the circuit object:

```
const proof = await circuit.generateProof(inputs);
```

The actual proof looks like this:

proof.json

```
{
  "proof": {
    "pi_a": [
      "4705801711565477046837119510773988173091957412720
766918367441244292047980064",
      "1400811599548904237959319989696481634963162026439
383059052135976273120564167",
      "1"
    ],
    "pi_b": [
      [
        "1253850816841690029903372652168516381779261463262
0657244409429354131980454661",
        "1091428367996684891779524735521251619761833895668
237487423900550675038442444"
      ],
      [
        "1150463245751857293071931246417067516989932126387
3993433191427524966381618623",
        "1552416371389031307029683708029978103698707118339
7727452907670321368057103914"
      ]
    ],
    "1",
    "0"
  ],
  "pi_c": [
    "26099670053328208608403811624767970928571072694687
5725263543647585988798998",
    "14278428069254250939292704696175748719031859166075
451182707331713513969403299",
    "1"
  ],
  "protocol": "groth16",
  "curve": "bn128"
},
  "publicSignals": {
    "r": "18"
  }
}
```

Also, at the end of the proof, we have the public signals.

Remark. Usually, public signals are represented by an array of elements, but when using the `hardhat-zkit` plugin, they are typed, and we have actual names for them.

The third element of each program does not participate in any computations; it is needed as additional metadata for the library that implements Groth16 verification.

Remark. When submitting the proof, we have to swap elements inside the arrays of the `b` point, so that the proof can be verified correctly.

These three points π_L, π_R, π_O are used by the verifier to check the equality:

$$e(\pi_L, \pi_R) = e(g_1^\alpha, g_2^\beta) e(\pi_{io}, g_2^\gamma) e(\pi_O, g_2^\delta).$$

Other constants needed for the verifier (for example, points g_1^α or g_2^β) are

defined in the following file:

zkkit/artifacts/circuits/Math.circom/Math.vkey.json. We will not show this file due to its size, but in fact, it is similar in formatting to the proof.json file shown before.

Quick reminder about the structure of points in the proof for BN254 curve:

- Each point is either from the regular curve $\mathbb{G}_1 : y^2 = x^3 + b$ over $\mathbb{F}_p$ or from the quadratic extension curve $\mathbb{G}_2 : y'^2 = x'^3 + b'$ over $\mathbb{F}_{p^2}$. For BN254, the quadratic extension is defined as $\mathbb{F}_{p^2} = \mathbb{F}_p(i)$ with $i^2 + 1$. Curve coefficients are $b = 3 \in \mathbb{F}_p$ and $b' = \frac{3}{9+i} \in \mathbb{F}_{p^2}$.
- Left inputs to the pairing function e are the points on the regular curve $\mathbb{G}_1$. They are specified in the form of two field elements $(x, y) \in \mathbb{G}_1$, where $x, y \in \mathbb{F}_p$ are the coordinates.
- Right inputs to the pairing function e are the points over the quadratic extension curve $\mathbb{G}_2$. They are specified of the form of four prime field elements $(x_1, y_1, x_2, y_2) \in \mathbb{G}_2$, where the coordinates are $x_1 + iy_1, x_2 + iy_2 \in \mathbb{F}_{p^2}$.
- $e(g_1^\alpha, g_2^\beta)$ is the element from the multiplicative group $\mathbb{F}_{p^{12}}^\times$. Therefore, we need 12 prime field elements to represent it.

Remark (On representing $\mathbb{F}_{p^{12}}$ element). One might wonder: why is the element from $\mathbb{F}_{p^{12}}$ is represented as a pair of two arrays, each consisting of three pairs of prime field elements? The primary reason is that the most convenient way to construct $\mathbb{F}_{p^{12}}$ element is to use the so-called **tower of extensions**: we represent an element from $\mathbb{F}_{p^{12}}$ as a pair of two $\mathbb{F}_{p^6}$ elements, while each $\mathbb{F}_{p^6}$ consists of a triplet of $\mathbb{F}_{p^2}$ elements. For more details, see ??

Thus, we have covered all the information about the internal structure of the Circom files needed for proof generation and verification.

Finally, we verify the proof in the code:

```
expect(await math.verifyProof(proof)).to.be.true
```

Remark. Here is a set of links that can be used for a deeper dive into the Circom ecosystem:

1. Circom Documentation: <https://docs.circom.io/>
2. Circom Libraries (like circomlib): <https://github.com/iden3/circomlib>

17 PlonK

17.1 Number Theoretical Transform: Universal Polynomial Accelerator

In the previous sections, when considering Groth16, we have seen the idea that polynomials are powerful encoders of data. To encode a set of N values $a_0, \dots, a_{N-1} \in \mathbb{F}$, we interpolate a polynomial $p(x)$ that at specific points $x_0, \dots, x_{N-1} \in \mathbb{F}$ evaluates to these values. The only condition we impose on these points is that they are distinct (unless, the interpolation would not be properly defined). This way, generally, we have the following interpolation problem:

$$p(x_j) = a_j, \quad j = 0, \dots, N-1$$

Particularly, in Groth16 our choice of points was $x_j = j$ for $j = 0, \dots, N-1$, which, for the large enough finite field $\mathbb{F}$, does not cause any issues. However, the complexity of interpolation in this case is not optimal. Let us see why.

Recall that the interpolation formula (see ?? for details) is given by:

$$p(x) = \sum_{i=0}^{N-1} a_i \ell_i(x), \quad \ell_i(x) = \prod_{j=0, j \neq i}^{N-1} \frac{x - x_j}{x_i - x_j}.$$

The naive evaluation of this formula requires $O(N^2)$ operations: we need to compute each ℓ_i , costing $O(N)$ operations, and then sum them up with, again, $O(N)$ operations.

With the specific choice of points $\{x_j\}_{0 \leq j < N}$, we can do much better: in fact, we can reduce the complexity to $O(N \log N)$ operations or even $O(N)$. This is done by utilizing several techniques, including the **Barycentric Interpolation** and the **N th roots of unity**.

17.1.1 Barycentric Interpolation

The idea of $O(N \log N)$ is to exploit the barycentric formula for polynomial interpolation. Let us derive the formula and see how it helps.

First, introduce the quantity $\gamma(x) = \prod_{j=0}^{N-1} (x - x_j)$. Now note that the Lagrange basis polynomials $\ell_j(x)$ can be rewritten as:

$$\ell_j(x) = \gamma(x) \cdot \frac{w_j}{x - x_j}, \quad w_j = \frac{1}{\prod_{k=0, k \neq j}^{N-1} (x_j - x_k)}, \quad j \in [N]$$

Remark. This step might seem unobvious, so let us be careful here. Let us see why expression $\gamma(x) \cdot \frac{w_j}{x - x_j}$ indeed gives the desired basis polynomial:

$$\gamma(x) \cdot \frac{w_j}{x - x_j} = \left(\prod_{k=0}^{N-1} (x - x_k) \right) \cdot \frac{1}{\left(\prod_{k=0, k \neq j}^{N-1} (x_j - x_k) \right) (x - x_j)}$$

Note that we can cancel out $(x - x_j)$ from both numerator and denominator and get

$$\left(\prod_{k=0, k \neq j}^{N-1} (x - x_j) \right) \cdot \frac{1}{\prod_{k=0, k \neq j}^{N-1} (x_j - x_k)} = \prod_{k=0, k \neq j}^{N-1} \frac{x - x_j}{x_j - x_k} = \ell_j(x),$$

which is exactly what we wanted to show.

Good, so why do we even need such an expression for ℓ_j ? Let us substitute it back into the interpolation formula:

$$p(x) = \sum_{i=0}^{N-1} a_i \ell_i(x) = \sum_{i=0}^{N-1} a_i \gamma(x) \cdot \frac{w_i}{x - x_i} = \gamma(x) \sum_{i=0}^{N-1} \frac{w_i}{x - x_i} a_i$$

In regards to this formula, we give the following definition.

Proposition 17.1. The **barycentric interpolation formula** for the interpolation problem $p(x_j) = a_j, j \in [N]$, given by $p(x) = \gamma(x) \sum_{i \in [N]} \frac{w_i}{x - x_i} a_i$ with $\gamma(x) = \prod_{i \in [N]} (x - x_i)$, requires $O(N)$ operations to compute and $O(N^2)$ operations to pre-compute.

Proof. Coefficients $\{w_j\}_{j \in [N]}$ are independent of x , and so are the values $\{a_j\}_{j \in [N]}$. To compute $\{w_j\}_{j \in [N]}$, one needs $O(N)$ operations for each w_j , and thus $O(N^2)$ operations in total. To compute the polynomial $p(x)$, one needs $O(N)$ operations to compute $\gamma(x)$ and $O(N)$ operations to compute the sum, knowing $\{w_j a_j\}_{j \in [N]}$. $\square$

Of course, in reality, storing N values $\{w_j\}_{j \in [N]}$ requires $O(N)$ memory, which is not optimal. Moreover, these points on their own are useless and typically are not used in any other parts of the protocol. This is where the **N th roots of unity** come into play.

17.1.2 Multiplicative Cyclic Subgroup

Again, assume we have the prime field $\mathbb{F}_p$. Let ω be a **primitive N -th root of unity**, i.e., $\omega^N = 1$ and $\omega^j \neq 1$ for $j < N$. The set $\Omega = \{\omega^j\}_{0 \leq j < N}$ is called the **N -th root of unity subgroup** of $\mathbb{F}_p$ of order N . One might ask the following question: why such primitive root even exists? Consider the following lemma, briefly mentioned in [Section 7.3](#).

Lemma 17.2. For $\mathbb{F}_p$ there exists a primitive N -th root of unity if and only if $N \mid (p - 1)$.

What is so special about the set Ω ? The magic of Ω is that it allows to compute certain polynomial operations (such as interpolation or multiplication) using the

Discrete Fourier Transform (DFT) or, equivalently, the **Number Theoretic Transform** (NTT) algorithm. Consider the first central lemma.

Lemma 17.3. The vanishing polynomial of the set Ω is given by $z_\Omega(X) = X^N - 1$.

Proof Idea. If ω is the N th primitive root, then for any $h \in \Omega$ we have $h^N = 1$ and therefore all elements of Ω are the roots of $X^N - 1$. There are precisely N such roots, so $X^N - 1$ can be decomposed as a product of linear factors $c \cdot \prod_{j=0}^{N-1} (X - \omega^j)$. It is easy to see that $c = 1$ by comparing the leading coefficient.

Now, let us come back to the barycentric formula. We have seen that the interpolation polynomial $p(x)$ can be written as $p(x) = \gamma(x) \sum_{i=0}^{N-1} \frac{w_i}{x - x_i} a_i$. The key idea of the FFT is to set $x_j = \omega^j$. What does it give us? We give the following proposition.

Proposition 17.4. Suppose the interpolation domain is chosen so that $a_i = \omega^i$. Then, following the notation of [Definition 17.1](#), certain expressions simplify to the following:

- $\gamma(x) = x^N - 1$.
- $w_i = \omega^i / N$.

Proof Idea. For the first claim, notice that by definition $\gamma(x) = \prod_{i=0}^{N-1} (x - \omega^i)$, which is exactly the vanishing polynomial of Ω . Thus, $\gamma(x) = z_\Omega(x) = x^N - 1$ from [Definition 17.3](#).

As for the second claim, recall that $w_i = 1 / \prod_{j \neq i}^{N-1} (\omega^i - \omega^j)$. Intuitively, the real analysis shows that $w_i = 1 / \gamma'(x_i)$ and since $\gamma(x) = x^N - 1$, we have exactly $w_i = 1 / N x^{N-1} \Big|_{x=\omega^i} = \omega^i / N$. The same result can be obtained by direct computation. $\square$

That being said, the barycentric formula is now given by:

$$p(x) = \frac{x^N - 1}{N} \sum_{j \in [N]} \frac{\omega^j}{x - \omega^j} a_j$$

This formula is a much more convenient form for the computation of the interpolation polynomial! First, the evaluation of the sum requires $O(N)$ operations and it depends only on the values $\{\omega^j\}_{j \in [N]}$, which are typically pre-computed and used in many other parts of the protocol. Second, the evaluation of the vanishing polynomial $x^N - 1$ requires $O(\log N)$ operations using the fast exponentiation algorithm, compared to naive $O(N)$ operations.

17.1.3 Fast Polynomial Multiplication

Forward NTT. Using the N th roots of unity Ω , we can also compute the polynomial multiplication in $O(N \log N)$ operations. The idea is to use the **Number Theoretic Transform** (NTT) algorithm, which is a generalization of the Fast Fourier Transform (FFT) to the finite fields. But first, let us define what NTT is.

Definition 17.5 (NTT). Suppose the polynomial is given by $p(x) = \sum_{j=0}^{N-1} p_j x^j \in \mathbb{F}[x]$. In its essence, the polynomial is defined as a vector of coefficients $\mathbf{p} = (p_0, \dots, p_{N-1})$. The **Number Theoretic Transform** (NTT) of polynomial $p(x)$ is the vector of evaluations at the N th roots of unity Ω : $\text{NTT}(\mathbf{p}) = (p(\omega^0), p(\omega^1), \dots, p(\omega^{N-1}))$.

Remark. Typically, the j th component of the NTT vector is denoted as $\hat{p}_j = \text{NTT}(\mathbf{p})_j$.

If computed naively, the NTT requires $O(N^2)$ operations, since each evaluation of $p(x)$ requires $O(N)$ operations. However, due to the specifics of the selected domain Ω , the NTT can be computed in $O(N \log N)$ operations. Let us emphasize this in the following lemma.

Lemma 17.6. The **Number Theoretic Transform** (NTT) of a polynomial $p(x)$ can be computed in $O(N \log N)$ operations using the N th roots of unity. This is possible only if the prime field $\mathbb{F}_p$ allows to find the primitive 2^k root of unity for $k \in [m]$ with large enough m . Equivalently, $2^k \mid (p-1)$ for $k \in [m]$.

To show this lemma is true, let us develop the concrete algorithm. Notice that our task consists in computing:

$$p(\omega^i) = \sum_{j \in [N]} p_j (\omega^i)^j = \sum_{j \in [N]} p_j \omega^{ij} \text{ for each } i \in [N]$$

Suppose the considered polynomial is such that $N = 2^r$ (we can always pad the polynomial if that is not the case). Now, let us proceed with the polynomial as follows:

$$\begin{aligned} p(\omega^i) &= \sum_{j=0}^{2^r-1} p_j \omega^{ij} = \sum_{j=0}^{2^{r-1}-1} p_{2j} \omega^{2ij} + \sum_{j=0}^{2^{r-1}-1} p_{2j+1} \omega^{i(2j+1)} = \\ &= \sum_{j=0}^{2^{r-1}-1} p_{2j} (\omega^{2i})^j + \omega^i \sum_{j=0}^{2^{r-1}-1} p_{2j+1} (\omega^{2i})^j \end{aligned}$$

This already looks interesting enough. Notice that we can introduce two new polynomials: $p_E(x) = \sum_{j=0}^{2^{r-1}-1} p_{2j} x^j$ and $p_O(x) = \sum_{j=0}^{2^{r-1}-1} p_{2j+1} x^j$, which

are polynomials, containing even and odd coefficients of p , respectively. In that case,

$$p(\omega^i) = p_E(\omega^{2i}) + \omega^i p_O(\omega^{2i})$$

This is quite an interesting observation which already screams divide-and-conquer! However, currently it might still be unclear how to use it: we still have to evaluate N expressions of form $p_E(\omega^{2i}) + \omega^i p_O(\omega^{2i})$ where both polynomials p_E and p_O contain roughly $N/2$ coefficients, totalling in $O(N^2)$ operations again. To counter this, we claim the following: we need only half the domain of Ω to compute both p_E and p_O . To see why, consider the expression $p(\omega^{i+N/2})$:

$$p(\omega^i) = p_E(\omega^{2(i+N/2)}) + \omega^{i+N/2} p_O(\omega^{2(i+N/2)}) = p_E(\omega^{2i}) + \omega^i \omega^{N/2} p_O(\omega^{2i})$$

In other words, having computed $p_E(\omega^{2i})$ and $p_O(\omega^{2i})$, we know not only $p(\omega^i)$, but also $p(\omega^{i+N/2})$ for free! This way, to compute the NTT for N -degree polynomial, we need to evaluate two $\frac{N}{2}$ -degree polynomials at $\frac{N}{2}$ points. This way, on each step: (a) the evaluation domain shrinks in half, (b) the complexity of computing polynomials shrinks in half, (c) we get two new polynomials. This way, on each step, we reduce the problem complexity in half!

The reason why the prime field should support multiplicative cyclic subgroups of order 2^k for sufficiently many k is that not always if ω is the N th primitive root, then ω^2 is the $\frac{N}{2}$ th primitive root. If that is not the case, all the aforementioned magic breaks.

We summarize everything so far in the **Algorithm 4**.

Algorithm 4: Number Theoretic Transform (NTT)

Input : Polynomial $p(x) = \sum_{j=0}^{N-1} p_j x^j$

Output: Vector of evaluations $\text{NTT}(\mathbf{p}, \omega)$ at $\Omega = \{\omega^j\}_{j \in [N]}$

1 **if** $N = 1$ **then**

 | **Return**: (p_0)

2 **end**

3 $H \leftarrow N/2$ /* Compute the domain half-size */

4 $\mathbf{p}_E \leftarrow (p_0, p_2, \dots, p_{N-2})$ /* Find even-indexed coefficients */

5 $\mathbf{p}_O \leftarrow (p_1, p_3, \dots, p_{N-1})$ /* Find odd-indexed coefficients */

6 $\mathbf{y}_E \leftarrow \text{NTT}(\mathbf{p}_E, \omega^2)$ /* Compute NTT for even polynomial via $\frac{N}{2}$ th primitive root ω^2 */

7 $\mathbf{y}_O \leftarrow \text{NTT}(\mathbf{p}_O, \omega^2)$ /* Compute NTT for odd polynomial via $\frac{N}{2}$ th primitive root ω^2 */

Return: $(y_0, \dots, y_{N-1})$ with $y_j = y_E, j \bmod H + \omega^j y_O, j \bmod H$

NTT Domain. OK, so what next? Suppose we want to multiply two polynomials $p(x), q(x) \in \mathbb{F}[X]$ of degree $N = 2^r$ and we have successfully evaluated their NTTs. Say, we got $\hat{p}$ and $\hat{q}$. What can we do next? Here is another trick.

Proposition 17.7. Suppose $m(x) = p(x)q(x)$ is the product of p and q . Then,

$$\hat{m} = \hat{p} \odot \hat{q}$$

Speaking more formally, $\text{NTT} : (\mathbb{F}^{(\leq N)}[X], \times) \rightarrow (\mathbb{F}^N, \odot)$ is a homomorphism between a set of polynomials of degree up to N and their NTT domain. With certain appropriate technicalities, NTT can be extended to the isomorphism.

Intuition. Although this fact might come out of random, we give an intuitive explanation why this holds. One of NTT interpretations is well-known to you *interpolation*. Indeed, polynomials p and q satisfy the following interpolation problem:

$$p(\omega^j) = \text{NTT}(p)_j = \hat{p}_j, \quad q(\omega^j) = \text{NTT}(q)_j = \hat{q}_j.$$

In turn, $\hat{m}_j$ is nothing but the evaluation of m at ω^j . But, if m is the product of p and q and values of p and q at ω^j are $\hat{p}_j$ and $\hat{q}_j$, this immediately implies that $m(\omega^j)$ is nothing but $\hat{p}_j \hat{q}_j$. Meaning, $\hat{m}_j = \hat{p}_j \hat{q}_j$. This, in turn, implies $\hat{m} = \hat{p} \odot \hat{q}$.

Wow! What this essentially means is that multiplication in NTT domain is very cheap: it requires only $O(N)$ field multiplication operations! Now, the algorithm of multiplication of p and q becomes a little bit more clear at this point:

1. Compute NTTs $\hat{p}$ and $\hat{q}$ of p and q .
2. Compute NTT $\hat{m} = \hat{p} \odot \hat{q}$ of their product $m = pq$.
3. Restore $m(x)$ from NTT $\hat{m}$ — this problem is called Inverse NTT (INTT).

Inverse NTT. So, the only problem left is restoring the polynomial from its NTT form. This problem is equivalent to solving the interpolation problem:

$$m(\omega^j) = \hat{m}_j, \quad j \in [N]$$

Suprisingly, the **Inverse NTT** is given by $p_j = \frac{1}{N} \sum_{j=0}^{N-1} \hat{p}_j \omega^{-ij}$, which can be computed by running the forward NTT, but with generator ω^{-1} . Division by N is done over $\mathbb{F}_p$, which is surely trivial. We summarize the whole discussion in [Figure 17.1](#).

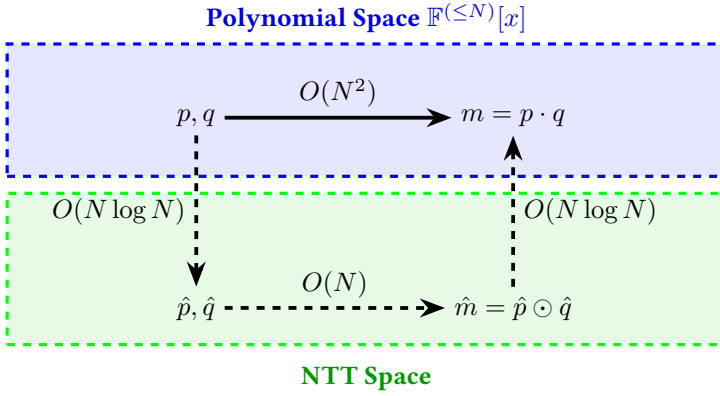


Figure 17.1: Illustration of the NTT Algorithm

17.2 Plonk Arithmetization

Consider we have a certain relation $\mathcal{R}$, which we would like to write down into a processing-prone format over the field $\mathbb{F}$. Plonk arithmetizes this relation into a set of 8 *polynomials*, which are then used to verify the witness knowledge. Let us start with the concrete example.

Example 17.1. To begin with, observe this fairly simple relation $\mathcal{R}_{\text{example}}$: suppose we have a public input $x \in \mathbb{F}$ and public output $y \in \mathbb{F}$, and we want to prove the knowledge of $e \in \mathbb{F}$ such that $e \times x + x - 1 = y$. Formally, we have the following relation:

$$\mathcal{R}_{\text{example}} = \left\{ \begin{array}{ll} \textbf{Public Statement:} & x, y \in \mathbb{F} \\ \textbf{Witness :} & e \in \mathbb{F} \end{array} \mid e \times x + x - 1 = y \right\}$$

Remark. Note that of course, from x and y , it is fairly simple to find e : simply take $\frac{1-x+y}{x}$. However, the Plonk arithmetization is not limited to this simple example, and can be applied to more complex relations, such as hash function pre-image knowledge or any NP statement.

17.2.1 Execution Trace

Standard Plonk is defined as a system with two types of gates: **addition** and **multiplication**. We would explain how to build custom gates later. So, let us consider our program in terms of gates with left, right operands and output.

Example 17.2. We need **three gates** to encode our program:

1. **Gate #1:** left e , right x , output $u = e \times x$

2. **Gate #2:** left u , right x , output $v = u + x$
3. **Gate #3:** left v , right x , output $w = v + (-1)$

You might have glanced the intuitive formation of what is called *execution trace table* – a matrix T with columns L , R and O (it is common to denote those as A, B, C to distinguish from another matrix we will discuss later). Moreover, we will mark columns **A**, **B** and **C** in bold to indicate that they are vectors from $\mathbb{F}^N$, where here and hereafter, unless stated otherwise, N is the number of gates in the program.

Example 17.3. We might visualize the execution trace table T for the example program as follows:

A	B	C
2	3	6
6	3	9
9	✗	8

Notice how the last row has no value in **B** column (marked by ✗) – this is reasoned by the fact it is not a variable, but rather a constant, meaning it doesn't depend on execution. Also note that the number of gates in this particular circuit is $N = 3$.

Remark. As you might notice, in contrast to classic R1CS (which we used for Groth16), the standard Plonk arithmetization as is only allows two input values to be processed at a time. This way, if Groth16 requires only one constraint for verifying $x_1(x_2 + x_3 + x_4) = x_5$, Plonk would need three constraints to verify the same statement. Custom gates partially solve this problem as we will see later, but it is important to keep in mind.

17.2.2 Encode the program

It is essential to distinguish the definition of the program and its specific evaluation for the sake of simplicity and efficiency – once having established encoding for the program, you might apply it for any reasonable inputs. Therefore, let us at first focus on what defines whether execution trace table will be considered valid for our circuit, because having a table by itself does not tell much, since it can be populated with any values.

For that reason, we would define two matrices – $Q \in \mathbb{F}^{N \times 5}$ and $V \in \mathbb{Z}_{\geq 0}^{N \times 3}$ where $N \in \mathbb{N}$, again, is the number of gates in the program:

- Q is the **Gate Matrix**, which encodes the values of the gates and stores all the intermediate values computed.
- V is the **Wiring Matrix**, which encodes the wiring of the gates, i.e., how the output of one gate is carried as input to another.

Definition 17.8. The **gate matrix** $Q \in \mathbb{F}^{N \times 5}$ has one row per each gate with columns $\mathbf{Q}_L, \mathbf{Q}_R, \mathbf{Q}_O, \mathbf{Q}_M, \mathbf{Q}_C$ from $\mathbb{F}^N$. If columns $\mathbf{A}, \mathbf{B}$ and $\mathbf{C} \in \mathbb{F}^N$ of the execution trace table form valid evaluation of the circuit, then the following holds:

$$A_i(Q_L)_i + B_i(Q_R)_i + A_i B_i(Q_M)_i + C_i(Q_O)_i + (Q_C)_i = 0, \forall i \in [N]$$

Using Hadamard product notation, this can be concisely rewritten as:

$$\mathbf{A} \odot \mathbf{Q}_L + \mathbf{B} \odot \mathbf{Q}_R + \mathbf{A} \odot \mathbf{B} \odot \mathbf{Q}_M + \mathbf{C} \odot \mathbf{Q}_O + \mathbf{Q}_C = \mathbf{0}$$

Example 17.4. For our program, we would have a following Q table:

$\mathbf{Q}_L$	$\mathbf{Q}_R$	$\mathbf{Q}_M$	$\mathbf{Q}_O$	$\mathbf{Q}_C$
0	0	1	-1	0
1	1	0	-1	0
1	0	0	-1	-1

You can verify that our claim holds for aforementioned trace matrix:

$$2 \times 0 + 3 \times 0 + 2 \times 3 \times 1 + 6 \times (-1) + 0 = 0$$

$$6 \times 1 + 3 \times 1 + 6 \times 3 \times 0 + 9 \times (-1) + 0 = 0$$

$$9 \times 1 + 0 \times 0 + 9 \times 0 \times 0 + 8 \times (-1) + (-1) = 0$$

Recall that columns of trace matrix T are $\mathbf{A} = \begin{bmatrix} 2 \\ 6 \\ 9 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 3 \\ 3 \\ \text{red } \times \end{bmatrix}$, $\mathbf{C} = \begin{bmatrix} 6 \\ 9 \\ 8 \end{bmatrix}$.

Now, we do have a way of encoding gates separately, yet in order to guarantee how result of one gate is carried in as input of the other (*wirings*), we need another matrix – V .

Definition 17.9. The **wiring matrix** $V \in \mathbb{Z}_{\geq 0}^{N \times 3}$ consists of indices of all inputs and intermediate values, so that if T is a valid trace,

$$\forall (i, j) \forall (k, \ell) : V_{i,j} = V_{k,\ell} \implies T_{i,j} = T_{k,\ell}$$

Put more simply, if two values are equal in V , then the corresponding values (corresponding to these indices) in T must be equal as well.

Example 17.5. For our program, V can be defined as follows:

L	R	O
0	1	2
2	1	3
3	X	4

Here 0 is an index of e , 1 is an index of x , 2 – of intermediate value u , 3 – of v and finally 4 – of output w .

17.2.3 Custom Gates

In order to reach beyond classical operations such as addition and multiplication, one may consider composing a custom gate. The main streamliner of this functionality is a matrix Q , using 5 basic columns of which, you already may build custom logic.

Example 17.6. Our entire program may be encoded as one custom gate.

$$Q = \begin{array}{|c|c|c|c|c|} \hline Q_L & Q_R & Q_M & Q_O & Q_C \\ \hline 0 & 1 & 1 & -1 & -1 \\ \hline \end{array}$$

$$V = \begin{array}{|c|c|c|} \hline L & R & O \\ \hline 0 & 1 & 2 \\ \hline \end{array} \quad T = \begin{array}{|c|c|c|} \hline A & B & C \\ \hline 2 & 3 & 8 \\ \hline \end{array}$$

$$2 \times 0 + 3 \times 1 + 2 \times 3 \times 1 + 8 \times (-1) + (-1) = 0$$

As you can see, custom gates is a good way to reduce the number of constraints needed for the same program.

Remark. Real-world PlonK applications commonly have additional columns in the Q matrix, enabling an even broader set of custom functionality.

17.2.4 Public Inputs

With the current design, we can prove that the computations were done correctly, but we have no restrictions on the values of inputs. For example, when the prover wants to convince the verifier that he knows e for $x = 3$ and $y = 7$, the verifier does not even check whether x is 3 (not to mention whether the result of execution $y = 7$ is correct) in the trace table T . One way of doing this is by incorporating them in three previously defined matrices Q, V, T .

Proposition 17.10. One way to solve this is to use the **equality gates**. Introduce two gadgets:

- **Constant Equality Gate:** Suppose we want to check whether the certain variable equals to the constant value $\alpha \in \mathbb{F}$ at gate with index

i. For *i*th gate, set $(Q_L)_i = -1$, $(Q_C)_i = \alpha$ and other columns to 0. Then, add a row to V with $L = i$, $R = \text{X}$ and $O = \text{X}$. Then, to satisfy the condition, the *i*th left input **must** be equal to α .

- **Nodes Equality Gate:** Suppose we want to check whether the *i*th and *j*th gates have equal outputs in the *k*th gate. Set $(Q_L)_k = 1$, $(Q_R)_k = -1$ with other columns to 0. Add a row to V with $L = i$, $R = j$ and $O = \text{X}$. Then, to satisfy the condition, the *i*th and *j*th outputs **must** be equal.

Example 17.7. Suppose the prover wants to prove that he knows e for the public statement $(x, y) = (3, 8)$. We can encode this as follows:

$Q =$	Q_L	Q_R	Q_M	Q_O	Q_C
	-1	0	0	0	3
	-1	0	0	0	8
	1	1	1	-1	1
	1	-1	0	0	0

$V =$	L	R	O
	0	X	X
	1	X	X
	2	0	3
	1	3	X

$T =$	A	B	C
	3	X	X
	8	X	X
	2	3	8
	8	8	X

As can be seen, besides the original program gate, inscribed in the third row, we have three additional gates:

- The first two gates “allocate” two nodes with indices 0 and 1 to the values 3 and 8 respectively. This is done through the *constant equality gates*.
- The last gate checks whether the result of the third gate is equal to the index 1, corresponding to the allocated value 8. This is done through the *nodes equality gate*.

The primary problem with this approach, is that now we have lost agnosticism in Q and V of concrete evaluations. In other words, our circuit is now “hardcoded” to the specific values of public inputs. In order to resolve this, we would define a separate one-column matrix named $\Pi \in \mathbb{F}^N$, in which we would encode the public inputs.

Example 17.8. With only Q modified, we now have:

Π
3
8
0
0

$Q =$

Q_L	Q_R	Q_M	Q_O	Q_C
-1	0	0	0	0
-1	0	0	0	0
1	1	1	-1	1
1	-1	0	0	0

Proposition 17.11 (Wrap-up). The matrix T with columns $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C} \in \mathbb{F}^N$ encodes correct execution of the program, if the following two conditions hold:

1. $\forall i \in [N] : A_i(Q_L)_i + B_i(Q_R)_i + A_i B_i(Q_M)_i + C_i(Q_O)_i + (Q_C)_i + \Pi_i = 0$
2. $\forall (i, j) \forall (k, \ell) : V_{i,j} = V_{k,\ell} \implies T_{i,j} = T_{k,\ell}$

17.2.5 Matrices to Polynomials

Gates Satisfiability. Now we can traduce the sets of constraints on matrices to just a few equations on polynomials, as we have already done for Groth16. Let ω be a primitive N -th root of unity¹³ and let $\Omega = \{\omega^j\}_{0 \leq j < N}$. In the [Section 17.1](#), we have already discussed the reasoning for choosing such a set Ω . In particular, denote by $\{L_j\}_{0 \leq j < N}$ the Lagrange basis polynomials¹⁴ for the set Ω , which for the selected domain Ω are given by the following convenient form:

$$L_j(x) = \frac{\gamma_j(x^N - 1)}{x - \omega^j}, \quad \text{where } \gamma_j \text{ is some normalizing constant.}$$

Let $a, b, c, q_L, q_R, q_M, q_O, q_C, \pi \in \mathbb{F}^{(\leq N)}[X]$ be polynomials of degree at most N that interpolate corresponding columns from matrices at the domain Ω . In other words, we have $\forall j \in [N] : a(\omega^j) = A_j$ and the same holds for other polynomials.

Notice that if our trace matrix is correct, then the first condition of [Definition 17.11](#) can be reduced to the following polynomial equation:

$$a(\omega^j)q_L(\omega^j) + b(\omega^j)q_R(\omega^j) + a(\omega^j)b(\omega^j)q_M(\omega^j) + c(\omega^j)q_O(\omega^j) + q_C(\omega^j) + \pi(\omega^j) = 0, \quad \forall j \in [N]$$

Notice that this essentially means that the left polynomial $aq_L + bq_R + abq_M + cq_O + q_C + \pi$ has roots at ω^j for all $j \in [N]$. This is equivalent to stating that the vanishing polynomial of Ω divides the left hand side. Due to the [Definition 17.3](#), this vanishing polynomial is given by $z_\Omega(x) = x^N - 1$. Let us wrap this up in the following proposition.

¹³Suppose such ω exists, then $\omega^N = 1$ and $\omega^j \neq 1$ for $0 \leq j < N$.

¹⁴Recall that by definition the Lagrange basis $\{L_j\}_{0 \leq j < N}$ for set $\mathcal{X} = (x_0, \dots, x_{N-1})$ is given by $L_j(x_i) = \delta_{ij}$.

Proposition 17.12. Now we can reduce down our first condition of **Definition 17.11** to checking valid execution trace into the following claim over polynomials:

$$\exists t \in \mathbb{F}^{(\leq 3N)}[x] : aq_L + bq_R + abq_M + cq_O + q_C + \pi = z_\Omega t,$$

where $z_\Omega(x)$ is the vanishing polynomial $x^N - 1$.

Wiring Satisfiability. The next step is to shrink the second condition imposed by the V matrix. This may be achieved by introducing the concept of permutation.

Remark. Permutation of the set S is commonly denoted as $\sigma : S \rightarrow S$. This function is bijective, meaning that for every $s \in S$ there exists a unique $s' \in S$ such that $\sigma(s) = s'$.

Example 17.9. A permutation is a rearrangement of the set, which is in our case:

$$\mathcal{I} = \{(i, j) : \text{such that } 0 \leq i < N, \text{ and } 0 \leq j < 3\}$$

Naturally, the matrix V induces a permutation σ of this set where $\sigma((i, j))$ equals to the pair of indices of the next occurrence of the value at position (i, j) . So, for our example:

$$V =$$

L	R	O
0	X	X
1	X	X
2	0	3
1	3	X

We have the following permutation:

$$\sigma((0,0)) = (2,1), \sigma((0,1)) = (0,3), \sigma((0,2)) = (0,2)$$

$$\sigma((0,3)) = (0,1), \sigma((2,1)) = (0,0), \sigma((3,1)) = (2,2)$$

For demonstration purposes, we marked in **green** the index of the first and second occurrence of the value 0. For proper σ definition (as it has to be bijective), the application of σ to the last occurrence outputs the first one.

Permutation Check. This is probably the most tedious part of PlonK. We split the following derivation into two parts:

- **Set Equality using Polynomials.** We will show how to check whether two sets of field elements are equal using polynomials.
- **Permutation Check using Polynomials.** We will show how to check whether a given function is a permutation using polynomials in several

forms.

Additionally, to avoid confusion, we denote the polynomial variable by capital letters (X, Y) .

Set equality. Having defined permutation, we can now reduce the second condition of [Definition 17.11](#) of valid execution trace matrix into the following check:

$$\forall (i, j) \in \mathcal{I} : T_{i,j} = T_{\sigma(i,j)}$$

You may have noticed how this can be reformulated as equality of two sets A and B :

$$\begin{aligned} A &:= \{((i, j), T_{i,j}) : (i, j) \in \mathcal{I}\} \\ B &:= \{(\sigma((i, j)), T_{i,j}) : (i, j) \in \mathcal{I}\} \end{aligned}$$

We can reduce this check down to polynomial equations! Here is how: suppose for simplicity we have two sets with two elements $A = \{a_0, a_1\}$ and $B = \{b_0, b_1\}$. Introduce two sets of polynomials $A' = \{a_0 + X, a_1 + X\}$ and $B' = \{b_0 + X, b_1 + X\}$.

When do we have the set equality $A' = B'$? Well, $(a_0 + X)(a_1 + X) = (b_0 + X)(b_1 + X)$ works fine. This is true because of linear polynomial unique factorization property, working as prime factors. Now, we can utilize Schwartz-Zippel lemma to replace the latter formula with $(a_0 + \gamma)(a_1 + \gamma) = (b_0 + \gamma)(b_1 + \gamma)$ for some random $\gamma \leftarrow \$ \mathbb{F}$ with overwhelming probability, being at least $1 - 2/|\mathbb{F}|$. If we wish to generalize this for arbitrary sets $A = \{a_0, \dots, a_{k-1}\}$ and $B = \{b_0, \dots, b_{k-1}\}$, apply the following equivalent check:

$$\prod_{i=0}^{k-1} (a_i + \gamma) = \prod_{i=0}^{k-1} (b_i + \gamma)$$

Let Ω be a domain of the form $\{1, \omega, \dots, \omega^{k-1}\}$ for some k -th root of unity ω . Let f and g be polynomials that we interpolate at Ω as follows:

$$f(\omega^j) = a_j + \gamma, \quad g(\omega^j) = b_j + \gamma, \quad j \in [k]$$

Then, $\prod_{i=0}^{k-1} (a_i + \gamma) = \prod_{i=0}^{k-1} (b_i + \gamma)$ holds if and only if there is a polynomial $Z \in \mathbb{F}[X]$ such that for all $h \in \Omega$ we have $Z(\omega^0) = 1$ and $Z(h)f(h) = g(h)Z(\omega h)$.

Now that we can encode equality of sets of field elements, let's expand this to sets of tuples of field elements. Let $A = \{(a_0, a_1), (a_2, a_3)\}$ and $B = \{(b_0, b_1), (b_2, b_3)\}$. Then, similarly, if

$$A' = \{a_0 + a_1 Y + X, a_2 + a_3 Y + X\}, \quad B' = \{b_0 + b_1 Y + X, b_2 + b_3 Y + X\},$$

then $A = B$ if and only if $A' = B'$. As before, we can leverage Schwartz-Zippel lemma to reduce this down into sampling two random β and $\gamma \leftarrow \$ \mathbb{F}$ and checking equality of:

$$(a_0 + \beta a_1 + \gamma)(a_2 + \beta a_3 + \gamma) = (b_0 + \beta b_1 + \gamma)(b_2 + \beta b_3 + \gamma)$$

Permutation Check. Now, to go back to the second condition of **Definition 17.11** which we are trying to formulate in the polynomial domain, it becomes clear that if we somehow encoded inner indices tuple (i, j) into a one field element, we could use the above fact. Recall that $i \in [N]$ and $j \in \{0, 1, 2\}$. Thus, take the $3N$ -th primitive root of unity η and define the bijective map $((i, j), v) \mapsto (\eta^{3i+j}, T_{i,j})$. Thus, consider the modified sets:

$$\begin{aligned} A &= \{(\eta^{3i+j}, T_{i,j}) : (i, j) \in \mathcal{I}\} \\ B &= \{(\eta^{3k+\ell}, T_{i,j}) : (i, j) \in \mathcal{I}, \sigma((i, j)) = (k, \ell)\} \end{aligned}$$

Sample two random field elements β and $\gamma \leftarrow \mathbb{R}$. Let $\mathcal{D} = \{1, \eta, \eta^2, \dots, \eta^{3N-1}\}$. Then, interpolate two polynomials f and g over the defined set $\mathcal{D}$ as follows:

$$\begin{aligned} f(\eta^{3i+j}) &= T_{i,j} + \eta^{3i+j}\beta + \gamma, \quad (i, j) \in \mathcal{I} \\ g(\eta^{3k+\ell}) &= T_{i,j} + \eta^{3k+\ell}\beta + \gamma, \quad (i, j) \in \mathcal{I}, \quad \sigma((i, j)) = (k, \ell) \end{aligned}$$

Similarly to our previous discussion, there should be a polynomial $Z \in \mathbb{F}[X]$ such that $\forall d \in \mathcal{D}$, we have $Z(\eta^0) = 1$ and $Z(d)f(d) = g(d)Z(\eta d)$. This would imply the set equality $A = B$ with overwhelming probability according to Schwartz-Zippel lemma.

Shorter Form. Now, using the $3N$ -th root of unity is a bit of overkill, so let us try compressing it down to $\Omega = \{\omega^j\}_{0 \leq j < N}$ where ω is the N th root of unity. We will define three polynomials $S_{\sigma_1}, S_{\sigma_2}, S_{\sigma_3} \in \mathbb{F}[X]$, which are interpolated as follows:

$$\begin{aligned} S_{\sigma_1}(\omega^i) &= \eta^{3k+\ell}, \quad (i, 0) \in \mathcal{I}, \quad \sigma((i, 0)) = (k, \ell) \\ S_{\sigma_2}(\omega^i) &= \eta^{3k+\ell}, \quad (i, 1) \in \mathcal{I}, \quad \sigma((i, 1)) = (k, \ell) \\ S_{\sigma_3}(\omega^i) &= \eta^{3k+\ell}, \quad (i, 2) \in \mathcal{I}, \quad \sigma((i, 2)) = (k, \ell) \end{aligned}$$

Let k_1 and k_2 be two field elements such that $\omega^i \neq \omega^j k_1 \neq \omega^\ell k_2$ for all possible triplets i, j, ℓ . Recall that β and γ are random field elements. Let f and g be the polynomials that interpolate, respectively, the following values at Ω :

$$\begin{aligned} f(\omega^i) &= (T_{i,0} + \omega^i\beta + \gamma) (T_{i,1} + \omega^i k_1 \beta + \gamma) (T_{i,2} + \omega^i k_2 \beta + \gamma), \quad i \in [N] \\ g(\omega^i) &= (T_{i,0} + S_{\sigma_1}(\omega^i)\beta + \gamma) (T_{i,0} + S_{\sigma_2}(\omega^i)\beta + \gamma) (T_{i,0} + S_{\sigma_3}(\omega^i)\beta + \gamma). \end{aligned}$$

That being said, there is a polynomial $Z \in \mathbb{F}[X]$ such that $\forall d \in \mathcal{D}$ we have $Z(\omega^0) = 1$ and $Z(d)f(d) = g(d)Z(\omega d)$, implying $A = B$ with overwhelming probability. That being said, we now can encode our program using 8 polynomials mentioned at the very beginning:

$$q_L, q_R, q_M, q_O, q_C, S_{\sigma_1}, S_{\sigma_2}, S_{\sigma_3}$$

These are typically called **common preprocessed input**.

17.2.6 Summary

Having a program for relation $\mathcal{R}$, we saw how it can be represented as a sequence of gates with left, right operands and output. The circuit may be encoded using two matrices Q – for capturing gates, and V – for encoding value carries (*wirings*). Upon execution, we get trace execution matrix T and Π for public inputs.

Definition 17.13. Let $T \in \mathbb{F}^{N \times 3}$ be a trace matrix with columns $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^N$ and let $\Pi \in \mathbb{F}^N$ be a public input vector. They correspond to a valid execution instance with public input given by Π if and only if:

1. $\forall i \in [N] : A_i(Q_L)_i + B_i(Q_R)_i + A_i B_i (Q_M)_i + C_i(Q_O)_i + (Q_C)_i + \Pi_i = 0$
2. $\forall (i, j), \forall (k, \ell) : V_{i,j} = V_{k,\ell} \implies T_{i,j} = T_{k,\ell}$
3. $\forall i > n : \Pi_i = 0$, where n is the number of public inputs.

Then, we encode these conditions in terms of polynomials.

Definition 17.14. Let $z_\Omega(x) = x^N - 1$ be a vanishing polynomial. Let $T \in \mathbb{F}^{N \times 3}$ be a trace matrix with columns $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^N$ and $\Pi \in \mathbb{F}^N$ be a vector of public signals. They correspond to a valid execution instance with public input given by Π if and only if:

1. $\exists t_1 \in \mathbb{F}[x] : aq_L + bq_R + abq_M + cq_O + q_C + \pi = z_\Omega t_1$
2. $\exists t_2, t_3, z \in \mathbb{F}[x] : zf - gz' = z_\Omega t_2$ and $(z - 1)L_1 = z_\Omega t_3$, where $z'(x) = z(\omega x)$.

Remark. We can reduce every needed check down to one equation, if we introduce randomness. Let α be a random field element, then:

$$\begin{aligned} z_\Omega t &= aq_L + bq_R + abq_M + cq_O + q_C + \pi \\ &= \alpha(gz' - fz) \\ &= \alpha^2(z - 1)L_1 \end{aligned}$$

The transition between second and third line is very unobvious and requires a bit of algebraic manipulation. Don't worry if you don't see it immediately.

17.3 Plonk Prover and Verifier

In this part, we will observe how the Plonk turns its arithmetization system into the non-interactive proof. We consider how the prover and verifier specifically interact in the protocol.

As in the previous section, let N be the size of our program (number of gates). Let ω be a primitive N -th root of unity. Let $\Omega = \{\omega^j\}_{0 \leq j < N}$ with the vanishing

polynomial $z_\Omega(x) := x^N - 1$.

Assume we have conducted arithmetization of this program, obtaining eight polynomials. Now we will demonstrate commitment-scheme agnostic non-interactive (Fiat-Shamir) algorithms for both prover and verifier.

Remark. Although the further discussion would involve statements like “the prover picks the random $\alpha, \beta, \dots$ ”, it is important to note that these values are not actually chosen by the prover, but rather computed deterministically from the transcript using Fiat-Shamir heuristic. In case you are not familiar with Fiat-Shamir heuristic, we recommend you to revisit the ??.

17.3.1 Gadgets

Before diving into the protocol, let us introduce the gadgets that will be used.

Commitment Scheme. We use the polynomial commitment scheme to commit to the polynomials. Similarly to Groth16, one might safely think of KZG commitment scheme to commit to the polynomials: for $p(x) \in \mathbb{F}^{(\leq N)}[x]$, the commitment is $\text{com}(p) = g^{p(\tau)}$, which is evaluated using the powers of the generator g . The opening procedure at point $\zeta \in \mathbb{F}$ of the commitment is simply finding the encryption $g^{Q(\tau)}$ of the quotient polynomial $Q(x) := (p(x) - y)/(x - \zeta)$, where y is the value of the polynomial at ζ . Finally, verification simply involves applying pairing. Note that other commitment schemes might be used as well.

Blindings. In the protocol, to achieve the HZVK property (honest verifier zero-knowledge), the prover must somehow conceal the polynomials he works with. Specifically, consider the polynomial $b(x)$ that is computed as the interpolated values from the trace table (namely, $b(\omega^j) = T_{j,1}$). However, the prover, by providing the opening $b(\zeta)$, should not reveal any information on polynomial $b(x)$ itself. To achieve this, the prover extends $b(x) \in \mathbb{F}^{(\leq N)}[x]$ to the higher-degree polynomial $\tilde{b}(x)$ with the condition that $\tilde{b}(\omega^j) = b(\omega^j) = T_{j,1}$. However, for higher degrees, there are multitude of choices for $\tilde{b}(x)$, so we need to standardize this choice.

Assume we extend $b(x)$ to $\tilde{b}(x) \in \mathbb{F}^{(\leq N')}[x]$ of higher degree $N' > N$. For convenience, assume $N' = N + \delta$ with $\delta \in \mathbb{N}$. How can we construct such $\tilde{b}(x)$? We can do this by sampling random values $\gamma_0, \gamma_1, \dots, \gamma_\delta \leftarrow \$ \mathbb{F}$ and letting:

$$\tilde{b}(x) = b(x) + z_\Omega(x) \sum_{j=0}^{\delta} \gamma_j x^j$$

Why does this work? Let us substitute any $\omega^i \in \Omega$:

$$\tilde{b}(\omega^i) = b(\omega^i) + z_\Omega(\omega^i) \sum_{j=0}^{\delta} \gamma_j \omega^{ij} = b(\omega^i).$$

Here, we use the fact that $z_\Omega(\omega^i) = 0$ for all $i \in [N]$ since z_Ω is, by definition, the vanishing polynomial of $\Omega \ni \omega^i$.

17.3.2 Proving

Let us finally proceed with the prover protocol.

Round 1. Add to the transcript commitments of 8 arithmetizational polynomials:

$$(\text{com}(S_{\sigma_1}), \text{com}(S_{\sigma_2}), \text{com}(S_{\sigma_3}), \text{com}(q_L), \text{com}(q_R), \text{com}(q_M), \text{com}(q_O), \text{com}(q_C))$$

Interpolate polynomials $a'(x), b'(x), c'(x)$ over corresponding columns of T matrix at the domain Ω . Sample random $b_1, b_2, b_3, b_4, b_5, b_6 \leftarrow \mathbb{F}$. Let the blinded polynomials be:

$$\begin{aligned} a &:= (b_1x + b_2)z_\Omega(x) + a'(x) \\ b &:= (b_3x + b_4)z_\Omega(x) + b'(x) \\ c &:= (b_5x + b_6)z_\Omega(x) + c'(x) \end{aligned}$$

Add to the transcript commitments $(\text{com}(a), \text{com}(b), \text{com}(c))$ of computed above polynomials.

Round 2. Sample random scalars $\beta, \gamma \leftarrow \mathbb{F}$ from the transcript. Let $z_0 = 1$ and define recursively the following sequence $\{z_k\}_{0 \leq k < N}$:

$$z_{k+1} = \frac{(a_k + \beta\omega^k + \gamma)(b_k + \beta\omega^k k_1 + \gamma)(c_k + \beta\omega^k k_2 + \gamma)}{(a_k + \beta S_{\sigma_1}(\omega^k) + \gamma)(b_k + \beta S_{\sigma_2}(\omega^k) + \gamma)(c_k + \beta S_{\sigma_3}(\omega^k) + \gamma)} \cdot z_k.$$

Interpolate polynomial $z'(x)$ over evaluations $(z_0, \dots, z_{N-1})$ at the domain Ω . Sample random $b_7, b_8, b_9 \leftarrow \mathbb{F}$. Let $z(x) = (b_7x^2 + b_8x + b_9)z_\Omega(x) + z'(x)$. Add $\text{com}(z)$ to the transcript.

Round 3. Sample $\alpha \leftarrow \mathbb{F}$ from the transcript. Let $\pi(X)$ be the interpolation polynomial of the input matrix Π at the domain Ω . Define three new polynomials:

$$\begin{aligned} p_1(x) &= aq_L + bq_R + abq_M + cq_O + q_C + \pi \\ p_2(x) &= (a + \beta X + \gamma)(b + \beta k_1 X + \gamma)(c + \beta k_2 X + \gamma)z - \\ &\quad - (a + \beta S_{\sigma_1} + \gamma)(b + \beta S_{\sigma_2} + \gamma)(c + \beta S_{\sigma_3} + \gamma)z(\omega x) \\ p_3(x) &= (z(x) - 1)L_1(x) \end{aligned}$$

Define the composite polynomial $p = p_1 + \alpha p_2 + \alpha^2 p_3$. Compute t such that $p = tz_\Omega$. Write $t = t_{\text{low}} + x^{N+2}t_{\text{mid}} + x^{2(N+2)}t_{\text{high}}$, with $t_{\text{low}}, t_{\text{mid}}$, and $t_{\text{high}} \in \mathbb{F}^{\leq (N+1)}[x]$ polynomials of degree at most $N + 1$. Sample random $b_{10}, b_{11} \leftarrow \mathbb{F}$ and define:

$$\begin{aligned} t_{\text{low}} &= t'_{\text{low}} + b_{10}x^{N+2} \\ t_{\text{mid}} &= t'_{\text{mid}} - b_{10} + b_{11}x^{N+2} \\ t_{\text{high}} &= t'_{\text{high}} - b_{11} \end{aligned}$$

Add to the transcript commitments $(\text{com}(t_{\text{low}}), \text{com}(t_{\text{mid}}), \text{com}(t_{\text{high}}))$.

Round 4. Sample random $\zeta \leftarrow \mathbb{F}$ from the transcript. Compute:

$$\bar{a} = a(\zeta), \bar{b} = b(\zeta), \bar{c} = c(\zeta), \bar{S}_{\sigma_1} = S_{\sigma_1}(\zeta), \bar{S}_{\sigma_2} = S_{\sigma_2}(\zeta), \bar{S}_{\sigma_3} = S_{\sigma_3}(\zeta), \bar{z}_\omega = z(\zeta\omega)$$

and add them to the transcript.

Round 5. Sample random $v \leftarrow \mathbb{F}$ from the transcript. Let:

$$\begin{aligned}\hat{p}_{\text{nc},1}(x) &= \bar{a}q_L + \bar{b}q_R + \bar{a}\bar{b}q_M + \bar{c}q_O + q_C \\ \hat{p}_{\text{nc},2}(x) &= (\bar{a} + \beta\zeta + \gamma)(\bar{b} + \beta k_1\zeta + \gamma)(\bar{c} + \beta k_2\zeta + \gamma)z - \\ &\quad - (\bar{a} + \beta\bar{S}_{\sigma_1} + \gamma)(\bar{b} + \beta\bar{S}_{\sigma_2} + \gamma)\beta\bar{z}_\omega S_{\sigma_3} \\ \hat{p}_{\text{nc},3}(x) &= L_1(\zeta)z(x)\end{aligned}$$

Define:

$$\begin{aligned}p_{\text{nc}} &= p_{\text{nc},1} + \alpha p_{\text{nc},2} + \alpha^2 p_{\text{nc},3} \\ t_{\text{partial}} &= t_{\text{low}} + \zeta^{N+2}t_{\text{mid}} + \zeta^{2(N+2)}t_{\text{high}}\end{aligned}$$

The subscript *nc* stands for “non-constant,” as it is the part of the linearization of p with nonconstant factors. The subscript *partial* indicates that it is a partial evaluation of t at ζ . Partial means that only some power of x is replaced by the powers of ζ . So in particular $t_{\text{partial}}(\zeta) = t(\zeta)$. Let π_B be the opening (**B**atch) proof at ζ of the polynomial f_B defined as:

$$f_B = t_{\text{partial}} + vp_{\text{nc}} + v^2a + v^3b + v^4c + v^5S_{\sigma_1} + v^6S_{\sigma_2}$$

Let π_S be the (**S**ingle) opening proof at $\zeta\omega$ of the polynomial z . Finally, compute $\bar{p}_{\text{nc}} := p_{\text{nc}}(\zeta)$ and $\bar{t} = t(\zeta)$.

Proof. All in all, our proof π looks as follows:

$$\pi = \left(\text{com}(a), \text{com}(b), \text{com}(c), \text{com}(z), \text{com}(t_{\text{low}}), \text{com}(t_{\text{mid}}), \text{com}(t_{\text{high}}), \right. \\ \left. \bar{a}, \bar{b}, \bar{c}, \bar{S}_{\sigma_1}, \bar{S}_{\sigma_2}, \bar{z}_\omega, \pi_B, \pi_S, \bar{p}_{\text{nc}}, \bar{t} \right)$$

17.3.3 Verification

Transcript Initialization. Similarly to prover, the verifier adds the following commitments to the transcript:

$$(\text{com}(S_{\sigma_1}), \text{com}(S_{\sigma_2}), \text{com}(S_{\sigma_3}), \text{com}(q_L), \text{com}(q_R), \text{com}(q_M), \text{com}(q_O), \text{com}(q_C))$$

Extraction of values and commitments. Firstly, the verifier needs to compute all the challenges. For that, he follows the following steps:

- Add $\text{com}(a), \text{com}(b), \text{com}(c)$ to the transcript.
- Sample two challenges β, γ .
- Add $\text{com}(z)$ to the transcript.

- Sample the challenge α .
- Add $\text{com}(t_{\text{low}})$, $\text{com}(t_{\text{mid}})$, $\text{com}(t_{\text{high}})$ to the transcript.
- Sample the challenge ζ .
- Add $\bar{a}$, $\bar{b}$, $\bar{c}$, $\bar{S}_{\sigma_1}$, $\bar{S}_{\sigma_2}$, $\bar{z}_\omega$ to the transcript.
- Sample the challenge v .

Compute $\pi(\zeta)$. Besides, the verifier needs to compute a few values of all these data. First, he computes the Π matrix with the public inputs and outputs. He needs to compute $\pi(\zeta)$, where $\pi(x)$ is the interpolation of Π over the domain Ω . But he does not need to compute π . He can instead compute $\pi(\zeta)$ as follows:

$$\pi(\zeta) = \sum_{i=0}^n L_i(\zeta) \Pi_i,$$

where n is the number of public inputs.

Compute claimed values $p(\zeta)$ and $t(\zeta)$. Next, the verifier computes:

$$\bar{p}_C = p_1(\zeta) + \alpha z_\omega (\bar{c} + \gamma) (\bar{a} + \beta \bar{S}_{\sigma_1} + \gamma) (\bar{b} + \beta \bar{S}_{\sigma_2} + \gamma) - \alpha^2 L_1(\zeta)$$

This is the constant part of the linearization of p . So, adding it to what the prover claims to be $\bar{p}_{\text{nc}}$, he obtains $p(\zeta) = \bar{p}_C + \bar{p}_{\text{nc}}$.

Compute $\text{com}(t_{\text{partial}})$ and $\text{com}(p_{\text{nc}})$. He computes these of the commitments in the proof as follows:

$$\text{com}(t_{\text{partial}}) = \text{com}(t_{\text{low}}) + \zeta^{N+2} \text{com}(t_{\text{mid}}) + \zeta^{2(N+2)} \text{com}(t_{\text{high}})$$

For the second one, compute those:

$$\text{com}(p_{\text{nc},1}) = \bar{a} \cdot \text{com}(q_L) + \bar{b} \cdot \text{com}(q_R) + \bar{a}\bar{b} \cdot \text{com}(q_M) + \bar{c} \cdot \text{com}(q_O) + \text{com}(q_C)$$

$$\begin{aligned} \text{com}(p_{\text{nc},2}) = & (\bar{a} + \beta \zeta + \gamma)(\bar{b} + \beta k_1 \zeta + \gamma)(\bar{c} + \beta k_2 \zeta + \gamma) \cdot \text{com}(z) - \\ & - (\bar{a} + \beta \bar{S}_{\sigma_1} + \gamma)(\bar{b} + \beta \bar{S}_{\sigma_2} + \gamma) \beta z_\omega \cdot \text{com}(S_{\sigma_3}) \end{aligned}$$

$$\text{com}(p_{\text{nc},3}) = L_1(\zeta) \cdot \text{com}(z)$$

Then $\text{com}(p_{\text{nc}}) = \text{com}(p_{\text{nc},1}) + \text{com}(p_{\text{nc},2}) + \text{com}(p_{\text{nc},3})$.

Compute claimed value $f_B(\zeta)$ and $\text{com}(f_B)$. The verifier computes the following two final quantities:

$$f_B(\zeta) = \bar{t} + v \bar{p}_{\text{nc}} + v^2 \bar{a} + v^3 \bar{b} + v^4 \bar{c} + v^5 \bar{S}_{\sigma_1} + v^6 \bar{S}_{\sigma_2}$$

$$\begin{aligned} \text{com}(f_B) = & \text{com}(t_{\text{partial}}) + v \cdot \text{com}(p_{\text{nc}}) + v^2 \cdot \text{com}(a) + \\ & + v^3 \cdot \text{com}(b) + v^4 \cdot \text{com}(c) + v^5 \cdot \text{com}(S_{\sigma_1}) + v^6 \cdot \text{com}(S_{\sigma_2}) \end{aligned}$$

Proof check. Now the verifier has all the necessary values to proceed with the checks.

- Check that $p(\zeta)$ equals $(\zeta^N - 1)t(\zeta)$.
- Verify the opening of f_B at ζ . Run the check: $\text{Verify}(\text{com}(f_B), \pi_B, \zeta, f_B(\zeta))$.
- Verify the opening of z at $\zeta\omega$. Check the validity of the proof π_S using the commitment $\text{com}(z)$ and the value $\bar{z}_\omega$: $\text{Verify}(\text{com}(z), \pi_S, \zeta\omega, \bar{z}_\omega)$.

If all checks pass, output **accept**. Otherwise, output **reject**.

Acknowledgements

This section was greatly inspired by “All you wanted to know about PlonK” blog by LambdaClass and “Understanding PlonK” article by Vitalik Buterin.

18 Bulletproofs

"Just the place for a Snark!" the Bellman cried,
As he landed his crew with care;
Supporting each man on the top of the tide
By a finger entwined in his hair.

– Lewis Carroll, "The Hunting of the Snark"

18.1 Introduction

Bulletproofs is a zero-knowledge proof protocol with logarithmically sized proofs without a trusted setup. Originally, **bulletproofs** was developed to provide efficient range proofs in application to confidential transactions, but it applies also to arbitrary arithmetic circuit (possibly encoded in R1CS). In the heart of protocol lays **inner-product argument** which we describe in details. Technically, the protocol is built in an interactive fashion (like Σ -protocols from Section 12), but one could make it non-interactive with a Fiat-Shamir transform. One key feature that differs it from Σ -protocols is the number of challenges from a verifier $\mathcal{V}$ – in Σ -protocols there is only one challenge, while **bulletproofs** implies a logarithmic in circuit size number of queries.

Also, **bulletproofs**' **inner-product argument** could be used to build various polynomial commitment schemes – crucial building block of proving systems built with *IOP* framework (*Halo*, *Nova*, etc).

The main advantages of **bulletproofs** are an absence of a trusted setup and security against eavesdropping that relies on the *discrete-logarithm* assumption without any other auxiliary structures like bilinear pairings. Also it has quite fast prover for small circuits making it practically useful for client-side proving. However, the main disadvantage of **bulletproofs** is that it isn't a classic *SNARK* due to linear in circuit size verification time, however still very efficient for small circuits.

18.2 Notation

Let $\mathbb{G}$ - cyclic group of prime order p written additively, $\mathbf{G} = (G_1, \dots, G_n)$, $\mathbf{H} = (H_1, \dots, H_n) \in \mathbb{G}^n$ - vectors of independent generators. We denote by $\langle \mathbf{a}, \mathbf{b} \rangle$ - inner product of vectors $\mathbf{a} = (a_1, \dots, a_n)$, $\mathbf{b} = (b_1, \dots, b_n) \in \mathbb{F}_p^n$ and $\langle \mathbf{a}, \mathbf{G} \rangle = \sum_{i=1}^n [a_i]G_i \in \mathbb{G}$ - inner product of vector $\mathbf{a}$ with vector of generators $\mathbf{G}$. Denote by $\mathbf{k}^n$ vector of k 's first n powers: $\mathbf{k}^n = (1, k, k^2, \dots, k^{n-1})$, for example $\mathbf{0}^n, \mathbf{1}^n$ represents vectors of zeros and ones respectively, while $\mathbf{2}^n = (1, 2, 4, \dots, 2^{n-1})$

18.3 Zero-knowledge multiplication

Let $a, b, c \in \mathbb{F}_p$. Here we build a zero-knowledge protocol for relation $\mathcal{R}_{abc} = \{(\perp; c, a, b) | c = ab\}$. We use well-known Σ -protocol framework for that,

but firstly we make very useful generalization that could allow us to prove much larger class of relations.

Consider the first-degree polynomials $l(x) = a + s_L x$, $r(x) = b + s_R x \in \mathbb{F}_p[x]$. Let $t(x) = l(x)r(x)$ and relation

$$\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) | t(x) = l(x)r(x)\}$$

Firstly, observe that proving $t(x) = l(x)r(x)$ may be reduced to evaluation check at some challenge point $u \in \mathbb{F}_p$: $t(u) = l(u)r(u)$, due to the *Schwartz-Zippel lemma*([Definition 5.14](#)):

$$\Pr[l(u)r(u) = t(u) | l(x)r(x) \neq t(x)] \leq \frac{\max(\deg(l(x)r(x)), \deg(t(x)))}{p} = \frac{2}{p}$$

is typically negligible function from security level which makes this check *sound*.

18.3.1 Naïve polynomial multiplication protocol

Let's describe naïve unoptimized version of **polynomial multiplication** protocol $\Pi'_{mul} = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}_{mul}$. During Setup parties agree on group elements $G, B \in \mathbb{G}$. After that parties involve in the following protocol:

- Prover $\mathcal{P}$ computes:

$$t(x) = l(x)r(x) = (a + s_L x)(b + s_R x) = ab + (as_R + bs_L)x + s_L s_R x^2$$

- Prover $\mathcal{P}$ draws blinding factors $\alpha_0, \alpha_1, \beta_0, \beta_1, \tau_0, \tau_1, \tau_2 \xleftarrow{\$} \mathbb{F}_p$ forming blinding polynomials

$$\alpha(x) = \alpha_0 + \alpha_1 x, \beta(x) = \beta_0 + \beta_1 x, \tau(x) = \tau_0 + \tau_1 x + \tau_2 x^2$$

and sends to $\mathcal{V}$ Pedersen commitments ([Section 10](#)) for each coefficient of $l(x), r(x), t(x)$:

$$\begin{aligned} L_0 &= [a]G + [\alpha_0]B & R_0 &= [b]G + [\beta_0]B \\ L_1 &= [s_L]G + [\alpha_1]B & R_1 &= [s_R]G + [\beta_1]B \\ T_0 &= [ab]G + [\tau_0]B \\ T_1 &= [as_R + bs_L]G + [\tau_1]B \\ T_2 &= [s_L s_R]G + [\tau_2]B \end{aligned} \tag{1}$$

Remark. Each commitment could be also seen as a Pedersen commitment to a reciprocal blinding polynomial coefficient as well.

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ random evaluation point $u \xleftarrow{\$} \mathbb{F}_p$

- Prover $\mathcal{P}$ evaluates $l(x), r(x), t(x)$ and $\alpha(x), \beta(x), \tau(x)$ at u :

$$\begin{aligned} l_u &= a + s_L u & \alpha_u &= \alpha_0 + \alpha_1 u \\ r_u &= b + s_R u & \beta_u &= \beta_0 + \beta_1 u \\ t_u &= l_u r_u & \tau_u &= \tau_0 + \tau_1 u + \tau_2 u^2 \end{aligned} \quad (2)$$

and sends $(l_u, r_u, t_u, \alpha_u, \beta_u, \tau_u)$ to $\mathcal{V}$.

- Verifier $\mathcal{V}$ performs checks:

$$\begin{aligned} [l_u]G + [\alpha_u]B &\stackrel{?}{=} L_0 + [u]L_1 \\ [r_u]G + [\beta_u]B &\stackrel{?}{=} R_0 + [u]R_1 \\ [t_u]G + [\tau_u]B &\stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2 \\ t_u &\stackrel{?}{=} l_u r_u \end{aligned} \quad (3)$$

Theorem 18.1. Naïve **polynomial multiplication** protocol Π'_{mul} has *perfect completeness, 3-special soundness, perfect honest-verifier zero-knowledge*

Proof idea. *Perfect completeness* holds due to:

$$\begin{aligned} [l_u]G + [\alpha_u]B &= [a + s_L u]G + [\alpha_0 + \alpha_1 u]B \\ L_0 + [u]L_1 &= [a]G + [\alpha_0]B + [s_L u]G + [\alpha_1 u]B = \\ &= [a + s_L u]G + [\alpha_0 + \alpha_1 u]B \\ [r_u]G + [\beta_u]B &= [b + s_R u]G + [\beta_0 + \beta_1 u]B \\ R_0 + [u]R_1 &= [b]G + [\beta_0]B + [s_R u]G + [\beta_1 u]B = \\ &= [b + s_R u]G + [\beta_0 + \beta_1 u]B \\ [t_u]G + [\tau_u]B &= [l_u r_u]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B \\ T_0 + [u]T_1 + [u^2]T_2 &= [ab]G + [\tau_0]B + [u(as_R + bs_L)]G + \\ &\quad + [u]\tau_1 B + [u^2]s_L s_R G + [u^2]\tau_2 B \\ &= [ab + (as_R + bs_L)u + s_L s_R u^2]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B \\ &= [l_u r_u]G + [\tau_0 + \tau_1 u + \tau_2 u^2]B \end{aligned}$$

Proving *honest-verifier zero-knowledge* is a bit complicated due to proper building of a simulator and proving indistinguishability of distributions, so we briefly describe the idea behind it: each commitment sent in the first phase by $\mathcal{P}$ is a Pedersen commitment which is hiding by design, every second phase response of $\mathcal{P}$ is an evaluation of some first or second degree polynomial at chosen point so there's not enough information for interpolation and polynomial reconstruction, moreover it could be easily simulated.

To prove *3-special soundness* we need to build a knowledge extractor $\mathcal{E}$ which extracts knowledge of witness polynomials $l(x), r(x), t(x)$ such that $l(x)r(x) = t(x)$ using 3 accepting transcripts:

1. $\mathcal{E}$ runs $\mathcal{P}$ to the end and rewinds back the second phase of $\mathcal{P}$, getting three non-equal challenges $u_1, u_2, u_3 \in \mathbb{F}_p$ and three prover responses $(l_{u_i}, r_{u_i}, t_{u_i})_{i=1}^3$
2. $\mathcal{E}$ solves the following systems of linear equations:

$$\begin{cases} l_{u_1} = a + s_L u_1 \\ l_{u_2} = a + s_L u_2 \end{cases} \quad \begin{cases} r_{u_1} = b + s_R u_1 \\ r_{u_2} = b + s_R u_2 \end{cases} \quad \begin{cases} t_{u_1} = t_0 + t_1 u_1 + t_2 u_1^2 \\ t_{u_2} = t_0 + t_1 u_2 + t_2 u_2^2 \\ t_{u_3} = t_0 + t_1 u_3 + t_2 u_3^2 \end{cases}$$

and gets the coefficients of witness polynomials: $(a, s_L, b, s_R, t_0, t_1, t_2)$

3. To prove that $l(x)r(x) = t(x)$ we apply Schwartz-Zippel lemma which asserts polynomial equality with high probability since for random challenges u_i for honest prover we have $l(u_i)r(u_i) = t(u_i)$ $\square$.

18.3.2 Optimized polynomial multiplication protocol

We could optimize our **polynomial multiplication protocol** furthermore. Note that we could simply apply vector Pedersen commitment for constant and linear terms using one more group element $H \in \mathbb{G}$.

Definition 18.2. The **polynomial multiplication protocol** $\Pi_{mul} = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation

$$\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) \mid t(x) = l(x)r(x)\}$$

with prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- Setup returns triple of group generators with unknown discrete log relations $G, H, B \in \mathbb{G}$
- Parties $\mathcal{P}, \mathcal{V}$ run the following protocol:
 - Prover $\mathcal{P}$ computes:

$$t(x) = l(x)r(x) = (a + s_L x)(b + s_R x) = ab + (as_R + bs_L)x + s_L s_R x^2$$

- Prover $\mathcal{P}$ draws blinding factors $\alpha, \beta, \tau_0, \tau_1, \tau_2 \leftarrow \mathbb{F}_p$ and sends to $\mathcal{V}$ the following commitments for coefficients of $l(x), r(x), t(x)$:

$$A = [\alpha]G + [\beta]H + [\alpha]B$$

$$S = [s_L]G + [s_R]H + [\beta]B$$

$$T_0 = [ab]G + [\tau_0]B$$

$$T_1 = [as_R + bs_L]G + [\tau_1]B$$

$$T_2 = [s_L s_R]G + [\tau_2]B$$

(4)

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow \mathbb{F}_p$

- Prover $\mathcal{P}$ evaluates polynomials at u :

$$\begin{aligned} l_u &= a + s_L u & \alpha_u &= \alpha + \beta u \\ r_u &= b + s_R u & \tau_u &= \tau_0 + \tau_1 u + \tau_2 u^2 \\ t_u &= l_u r_u \end{aligned} \tag{5}$$

and sends $(l_u, r_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

- Verifier $\mathcal{V}$ performs checks:

$$\begin{aligned} A + [u]S &\stackrel{?}{=} [l_u]G + [r_u]H + [\alpha_u]B \\ [t_u]G + [\tau_u]B &\stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2 \\ t_u &\stackrel{?}{=} l_u r_u \end{aligned} \tag{6}$$

Theorem 18.3. The **polynomial multiplication** protocol Π_{mul} has *perfect completeness, special soundness, perfect honest-verifier zero-knowledge*

Proof. We left to a reader proof of the theorem in the sake of brevity because it's very similar to the proof of [Definition 18.1](#) $\square$

18.3.3 Zero-knowledge multiplication protocol

Finally, we could easily build the protocol for the zk-multiplication relation where each witness element presented in statement as a Pedersen commitment:

$$\mathcal{R}_{abc} = \left\{ \begin{array}{l} (G, H, B, A, T_0; a, b, \alpha, \tau_0) \\ A = [a]G + [b]H + [\alpha]B \wedge \\ T_0 = [ab]G + [\tau_0]B \end{array} \right\}$$

Where $G, H, B \in \mathbb{G}$ are generators with unknown discrete log relations, $A \in \mathbb{G}$ is a Pedersen commitment to a, b and $T_0 \in \mathbb{G}$ is a Pedersen commitment to their product ab .

Definition 18.4. The **multiplication protocol** $\Pi_{abc} = (\mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}_{abc}$ with prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- Prover $\mathcal{P}$ draws random $s_L, s_R \leftarrow \mathbb{F}_p$ and defines polynomials:

$$l(x) = a + s_L x, \quad r(x) = b + s_R x, \quad t(x) = l(x)r(x)$$

- Parties run Π_{mul} on inputs $(l(x), r(x), t(x))$ along with provided a-priori setup $G, H, B \in \mathbb{G}$ and commitments A, T_0

The protocol obviously has *perfect completeness, special soundness, perfect honest-verifier zero-knowledge* due to the [Definition 18.3](#).

Remark. Now curious reader may wonder why build so overwhelmingly complicated protocol for simple multiplication and not just use classic *Chaum-Pedersen protocol* for DH-triplets? Indeed, it definitely could establish that for given group elements $[a]G, [b]G, [c]G$ equality $c = ab$ holds, but unfortunately commitments $[a]G, [b]G, [c]G$ do not have a *perfect hiding* property (though preserving *computational binding* property) so an adversary could potentially learn a, b, c values especially if they are small or have non-uniform distribution.

Also, there's a folklore version of very similar protocol for establishing product relationship between Pedersen committed values described in [?, section 12].

18.3.4 Zero-knowledge inner-product protocol

We could extend our Π_{mul} protocol even further to provide zero-knowledge proof for the inner-product of vectors: $\langle \mathbf{a}, \mathbf{b} \rangle = v$. The main trick is to substitute polynomials $l(x), r(x) \in \mathbb{F}_p[x]$ by vector polynomials $\mathbf{l}(x), \mathbf{r}(x) \in \mathbb{F}_p^n[x]$ where constant terms are equal to $\mathbf{a}$ and $\mathbf{b}$ respectively, taking inner-product $\langle \mathbf{l}(x), \mathbf{r}(x) \rangle$ results in polynomial with scalar coefficients where constant term is equal to $\langle \mathbf{a}, \mathbf{b} \rangle$.

Example 18.1. Let $\mathbf{a} = (a_1, a_2)$ and $\mathbf{b} = (b_1, b_2)$ be vectors in $\mathbb{F}_p^2$. Consider vector polynomials with vector coefficients:

$$\mathbf{l}(x) = \mathbf{a} + \mathbf{s}_L x = (a_1, a_2) + (s_{L,1}, s_{L,2})x$$

$$\mathbf{r}(x) = \mathbf{b} + \mathbf{s}_R x = (b_1, b_2) + (s_{R,1}, s_{R,2})x$$

where $\mathbf{s}_L = (s_{L,1}, s_{L,2})$, $\mathbf{s}_R = (s_{R,1}, s_{R,2})$.

Their inner-product is a degree two scalar polynomial:

$$\begin{aligned} t(x) &= \langle \mathbf{l}(x), \mathbf{r}(x) \rangle \\ &= \langle \mathbf{a} + \mathbf{s}_L x, \mathbf{b} + \mathbf{s}_R x \rangle \\ &= \langle \mathbf{a}, \mathbf{b} \rangle + \langle \mathbf{a}, \mathbf{s}_R \rangle x + \langle \mathbf{s}_L, \mathbf{b} \rangle x + \langle \mathbf{s}_L, \mathbf{s}_R \rangle x^2 \end{aligned}$$

So the constant term is the inner-product $\langle \mathbf{a}, \mathbf{b} \rangle$.

Definition 18.5. The **zero-knowledge inner-product protocol** $\Pi_{zkip} = (\mathcal{P}, \mathcal{V})$ for the relation

$$\mathcal{R}_{zkip} = \left\{ (\mathbf{G}, \mathbf{H}, G, B, A, V; \mathbf{a}, \mathbf{b}, \alpha, \gamma) \mid \begin{aligned} A &= \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\alpha]B, \\ V &= [\langle \mathbf{a}, \mathbf{b} \rangle]G + [\gamma]B \end{aligned} \right\}$$

where $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$, $G, B \in \mathbb{G}$ – independent group generators with prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- Prover $\mathcal{P}$ choses blinding vectors $\mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n$ and computes polynomi-

als:

$$\mathbf{l}(x) = \mathbf{a} + \mathbf{s}_L x$$

$$\mathbf{r}(x) = \mathbf{b} + \mathbf{s}_R x$$

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = \langle \mathbf{a}, \mathbf{b} \rangle + (\langle \mathbf{a}, \mathbf{s}_R \rangle + \langle \mathbf{s}_L, \mathbf{b} \rangle)x + \langle \mathbf{s}_L, \mathbf{s}_R \rangle x^2$$

- Prover $\mathcal{P}$ draws blinding factors $\beta, \tau_1, \tau_2 \leftarrow \mathbb{F}_p$ and sends to $\mathcal{V}$ the following commitments for coefficients of $\mathbf{l}(x), \mathbf{r}(x), \mathbf{t}(x)$:

$$\begin{aligned} S &= \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B \\ T_1 &= [\langle \mathbf{a}, \mathbf{s}_R \rangle + \langle \mathbf{s}_L, \mathbf{b} \rangle]G + [\tau_1]B \\ T_2 &= [\langle \mathbf{s}_L \mathbf{s}_R \rangle]G + [\tau_2]B \end{aligned} \tag{7}$$

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow \mathbb{F}_p$
- Prover $\mathcal{P}$ evaluates polynomials at u :

$$\begin{aligned} \mathbf{l}_u &= \mathbf{a} + \mathbf{s}_L u & \alpha_u &= \alpha + \beta u \\ \mathbf{r}_u &= \mathbf{b} + \mathbf{s}_R u & \tau_u &= \tau_0 + \tau_1 u + \tau_2 u^2 \\ t_u &= \langle \mathbf{l}_u \mathbf{r}_u \rangle \end{aligned} \tag{8}$$

and sends $(\mathbf{l}_u, \mathbf{r}_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

- Verifier $\mathcal{V}$ performs checks:

$$\begin{aligned} A + [u]S &\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{H} \rangle + [\alpha_u]B \\ [t_u]G + [\tau_u]B &\stackrel{?}{=} V + [u]T_1 + [u^2]T_2 \\ t_u &\stackrel{?}{=} \langle \mathbf{l}_u \mathbf{r}_u \rangle \end{aligned} \tag{9}$$

The protocol also has *perfect completeness*, *special soundness*, *perfect honest-verifier zero-knowledge* due to the [Definition 18.3](#), however building the extractor needs some vector equations we omit for the sake of brevity. Also note that transcript size is linear in size of vectors $\mathbf{l}_u, \mathbf{r}_u$ which is extremely inefficient when vectors are large. So in the next section we present so called **inner-product argument** which is summoned to reduce conversational complexity to logarithmic in vector length making the last check $t_u \stackrel{?}{=} \langle \mathbf{l}_u \mathbf{r}_u \rangle$ quite efficient.

18.4 Inner-product argument

Here we describe the further generalization of Π_{mul} – efficient protocol for the **inner-product argument** - core component of the **bulletproofs** protocol. After that we will apply it to range proofs and arithmetic circuits. We have already seen that inner-products are the main ingredients for R1CS language because

any R1CS relation could be seen as a batch of inner-products though it's not the most efficient representation and we'll see how to amortize all the constraints into inner-products more efficiently.

The **inner-product argument** allows to prove that two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_p^n$ satisfy the relation:

$$\mathcal{R}_{ip} = \{(\mathbf{G}, \mathbf{H}, P, c; \mathbf{a}, \mathbf{b}) \mid P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c\}$$

We refer to $P \in \mathbb{G}$ as a binding Pedersen vector commitment to $\mathbf{a}, \mathbf{b}$.

One way to prove the relation is to use $\Pi_{zk_{ip}}$, but as we've seen it's not efficient due to linear in n size of the proof. We want to build an argument system for the relation $\mathcal{R}_{ip}$ with logarithmic in n size of the proof.

Firstly, let's combine statements $P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c$ into a single statement by multiplying the second one by a random $r \in \mathbb{F}_p$ and some orthogonal generator $B \in \mathbb{G}$, summing up:

$$\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) \mid P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q\}$$

Where $P' = P + [cr]B, Q = [r]B$. Intuitively, if prover $\mathcal{P}$ can prove $\mathcal{R}'_{ip}$ for all $r \in \mathbb{F}_p$, then it can prove $\mathcal{R}_{ip}$ for any valid witness. We use such transformation to compress each vector in half and arrive to the same form of commitment

$$P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q$$

Definition 18.6. The **inner-product** protocol $\Pi_{ip} = (\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}_{ip} = \{(\mathbf{G}, \mathbf{H}, P, c; \mathbf{a}, \mathbf{b}) \mid P = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle \wedge \langle \mathbf{a}, \mathbf{b} \rangle = c\}$ with prover $\mathcal{P}$, verifier $\mathcal{V}$ is defined as follows:

- Parties $\mathcal{P}, \mathcal{V}$ agree on some group element $B \in \mathbb{G}$ with unknown discrete log
- Verifier $\mathcal{V}$ samples random $r \leftarrow \mathbb{F}_p$ and sends it to prover $\mathcal{P}$
- Parties $\mathcal{P}, \mathcal{V}$ compute $Q \leftarrow [r]B$ and $P' = P + [c]Q$
- Parties run protocol Π'_{ip} for relation $\mathcal{R}'_{ip}$ on input $(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b})$

18.4.1 Inner-product compression

Here we describe **inner-product compression** algorithm – main building block of the interactive **inner-product** protocol. Firstly, assuming that $n = 2^d$ define by $\mathbf{G}_{lo} = (G_1, \dots, G_{n/2}), \mathbf{G}_{hi} = (G_{n/2+1}, \dots, G_n) \in \mathbb{G}^{n/2}$ – lower and higher halves of vector $\mathbf{G}$ and $\mathbf{a}_{lo} = (a_1, \dots, a_{n/2}), \mathbf{a}_{hi} = (a_{n/2+1}, \dots, a_n) \in \mathbb{F}_p^{n/2}$ – lower and higher halves of $\mathbf{a} \in \mathbb{F}_p^n$.

Let $u_k \in \mathbb{F}_p$ - be some scalar, define compressed vectors:

$$\begin{aligned}\mathbf{a}^{(k-1)} &= \mathbf{a}_{\text{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{a}_{\text{hi}} \\ \mathbf{b}^{(k-1)} &= \mathbf{b}_{\text{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{b}_{\text{hi}} \\ \mathbf{G}^{(k-1)} &= \mathbf{G}_{\text{lo}} \cdot u_k^{-1} + u_k \cdot \mathbf{G}_{\text{hi}} \\ \mathbf{H}^{(k-1)} &= \mathbf{H}_{\text{lo}} \cdot u_k + u_k^{-1} \cdot \mathbf{H}_{\text{hi}}\end{aligned}$$

Define $P_k \leftarrow P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q$ - current commitment to vectors $\mathbf{a}, \mathbf{b}$ and define P_{k-1} using compressed vectors to have the same form as P_k , but in new basis $(\mathbf{G}^{(k-1)}, \mathbf{H}^{(k-1)})$:

$$P_{k-1} = \langle \mathbf{a}^{(k-1)}, \mathbf{G}^{(k-1)} \rangle + \langle \mathbf{b}^{(k-1)}, \mathbf{H}^{(k-1)} \rangle + [\langle \mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)} \rangle]Q \quad (10)$$

Or alternatively, expressing P_{k-1} in old basis $(\mathbf{G}^{(k)}, \mathbf{H}^{(k)})$ we get:

$$\begin{aligned}P_{k-1} &= \langle u_k^{-1} \cdot \mathbf{a}^{(k-1)}, \mathbf{G}_{\text{lo}}^{(k)} \rangle + \langle u_k \cdot \mathbf{a}^{(k-1)}, \mathbf{G}_{\text{hi}}^{(k)} \rangle + \langle u_k \cdot \mathbf{b}^{(k-1)}, \mathbf{H}_{\text{lo}}^{(k)} \rangle \\ &\quad + \langle u_k^{-1} \cdot \mathbf{b}^{(k-1)}, \mathbf{H}_{\text{hi}}^{(k)} \rangle + [\langle \mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)} \rangle]Q\end{aligned} \quad (11)$$

Substituting compressed vectors and applying bilinearity property of inner product we get:

$$\begin{aligned}P_{k-1} &= \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle + u_k^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u_k^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + \\ &\quad \langle \mathbf{b}_{\text{lo}}, \mathbf{H}_{\text{lo}} \rangle + \langle \mathbf{b}_{\text{hi}}, \mathbf{H}_{\text{hi}} \rangle + u_k^2 \langle \mathbf{b}_{\text{hi}}, \mathbf{H}_{\text{lo}} \rangle + u_k^{-2} \langle \mathbf{b}_{\text{lo}}, \mathbf{H}_{\text{hi}} \rangle + \\ &\quad [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{hi}} \rangle]Q + [u_k^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle + u_k^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle]Q\end{aligned}$$

Note that $\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle = \langle \mathbf{a}, \mathbf{G} \rangle$ so that the first two columns of P_{k-1} definition precisely contains $P_k = P'$:

$$P_k = \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle + \langle \mathbf{b}_{\text{lo}}, \mathbf{H}_{\text{lo}} \rangle + \langle \mathbf{b}_{\text{hi}}, \mathbf{H}_{\text{hi}} \rangle + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{hi}} \rangle]Q$$

Define cross-terms L_k, R_k of P_{k-1} such that:

$$\begin{aligned}P_{k-1} &= P_k + [u_k^2]L_k + [u_k^{-2}]R_k \\ L_k &= \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + \langle \mathbf{b}_{\text{hi}}, \mathbf{H}_{\text{lo}} \rangle + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle]Q \\ R_k &= \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{b}_{\text{lo}}, \mathbf{H}_{\text{hi}} \rangle + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle]Q\end{aligned}$$

The first equation $P_{k-1} = P_k + [u_k^2]L_k + [u_k^{-2}]R_k$ could be used as a check for asserting correctness of next commitment P_{k-1} given cross-terms L_k, R_k , half-sized vectors $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ from which P_{k-1} was computed (10) using updated basis $\mathbf{G}^{(k-1)}, \mathbf{H}^{(k-1)}$ and current commitment value P_k .

But we wish not send $\mathbf{a}^{(k-1)}, \mathbf{b}^{(k-1)}$ directly as this's inefficient due to still linear sizes, instead we apply recursion to compress this vectors to just one

element. Here we come up with some kind of statement compression algorithm reducing size of all vectors in half per compression step. Repeating compression algorithm k times we end up with sending vectors $\mathbf{a}^{(0)}, \mathbf{b}^{(0)}$ each of length one and P_0 containing all accumulated cross-terms:

$$P_0 = [a_1^{(0)}]G_1^{(0)} + [b_1^{(0)}]H_1^{(0)} + [a_1^{(0)}b_1^{(0)}]Q$$

$$P_0 = P_k + \sum_{i=1}^k ([u_i^2]L_i + [u_i^{-2}]R_i)$$

Recalling that $P_k = P'$, the final compressed statement asserting inner-product value will have the following form:

$$P' + \sum_{i=1}^k ([u_i^2]L_i + [u_i^{-2}]R_i) = [a_1^{(0)}]G_1^{(0)} + [b_1^{(0)}]H_1^{(0)} + [a_1^{(0)}b_1^{(0)}]Q \quad (12)$$

Remark. We take $n = 2^d$ without loss of generality, since one could always pad the vectors with zeroes to make their length a power of two and inner-product compression would shrink the size of the vectors by a factor of two per compression step down to one element.

18.4.2 Proving $\mathcal{R}'_{ip}$

Let's describe the **inner-product** protocol Π'_{ip} for relation $\mathcal{R}'_{ip}$.

Definition 18.7. The **inner-product** protocol $\Pi'_{ip} = (\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}'_{ip} = \{(\mathbf{G}, \mathbf{H}, Q, P'; \mathbf{a}, \mathbf{b}) \mid P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q\}$, where all vectors have length $n = 2^d$ with prover $\mathcal{P}$, verifier $\mathcal{V}$ is defined as follows:

- Prover $\mathcal{P}$ sets

$$(k, \mathbf{a}^{(k)}, \mathbf{b}^{(k)}, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{a}, \mathbf{b}, \mathbf{G}, \mathbf{H}, P')$$

- Verifier $\mathcal{V}$ sets

$$(k, \mathbf{G}^{(k)}, \mathbf{H}^{(k)}, P_k) \leftarrow (d, \mathbf{G}, \mathbf{H}, P')$$

- While $k > 0$ then:

- Prover $\mathcal{P}$ computes

$$L_k = \langle \mathbf{a}_{\text{lo}}^{(k)}, \mathbf{G}_{\text{hi}}^{(k)} \rangle + \langle \mathbf{b}_{\text{hi}}^{(k)}, \mathbf{H}_{\text{lo}}^{(k)} \rangle + [\langle \mathbf{a}_{\text{lo}}^{(k)}, \mathbf{b}_{\text{hi}}^{(k)} \rangle]Q$$

$$R_k = \langle \mathbf{a}_{\text{hi}}^{(k)}, \mathbf{G}_{\text{lo}}^{(k)} \rangle + \langle \mathbf{b}_{\text{lo}}^{(k)}, \mathbf{H}_{\text{hi}}^{(k)} \rangle + [\langle \mathbf{a}_{\text{hi}}^{(k)}, \mathbf{b}_{\text{lo}}^{(k)} \rangle]Q$$

- $\mathcal{P}$ sends (L_k, R_k) to $\mathcal{V}$
- $\mathcal{V}$ draws challenge $u_k \leftarrow_{\$} \mathbb{F}_p$ and sends it to $\mathcal{P}$

- Both $\mathcal{P}$ and $\mathcal{V}$ compute:

$$\begin{aligned}\mathbf{G}^{(k-1)} &= \mathbf{G}_{\text{lo}}^{(k)} \cdot u_k^{-1} + u_k \cdot \mathbf{G}_{\text{hi}}^{(k)} \\ \mathbf{H}^{(k-1)} &= \mathbf{H}_{\text{lo}}^{(k)} \cdot u_k + u_k^{-1} \cdot \mathbf{H}_{\text{hi}}^{(k)}\end{aligned}$$

- $\mathcal{P}$ computes:

$$\begin{aligned}\mathbf{a}^{(k-1)} &= \mathbf{a}_{\text{lo}}^{(k)} \cdot u_k + u_k^{-1} \cdot \mathbf{a}_{\text{hi}}^{(k)} \\ \mathbf{b}^{(k-1)} &= \mathbf{b}_{\text{lo}}^{(k)} \cdot u_k^{-1} + u_k \cdot \mathbf{b}_{\text{hi}}^{(k)}\end{aligned}$$

- Prover $\mathcal{P}$ sends $(a, b) \leftarrow (\mathbf{a}_1^{(0)}, \mathbf{b}_1^{(0)})$ to verifier $\mathcal{V}$
- Verifier performs final check:

$$P' + \sum_{i=1}^d ([u_i^2]L_i + [u_i^{-2}]R_i) = [a]G_1^{(0)} + [b]H_1^{(0)} + [ab]Q$$

outputs **accept** if equality holds and **reject** otherwise.

This protocol is illustrated in [Figure 18.1](#).

As we can see, overall communication complexity of Π_{ip} is $2 \log_2 n$ group elements plus 2 field elements so we come up with logarithmic proof size for our inner-product relation $\mathcal{R}_{ip}$. Now one might ask whether Π_{ip} has any desired properties such as completeness, soundness and zero-knowledge. It appears that the first two holds under some generalizations needed for security proofs but not *zero-knowledge* (indeed, if $n = 1$ then $\mathcal{P}$ sends witness pair a, b directly). We'll compile efficient **inner-product argument** Π_{ip} with zero-knowledge Π_{zkip} to achieve efficient zero-knowledge proofs for range proofs and arithmetic circuits.

Theorem 18.8 (Inner-Product Argument). The argument system Π_{ip} for relation $\mathcal{R}_{ip}$ has *perfect completeness and statistical witness-extended emulation* for either extracting a non-trivial discrete logarithm relation between $\mathbf{G}, \mathbf{H}, Q$ or extracting valid witness $\mathbf{a}, \mathbf{b}$.

Proof idea. *Perfect completeness* of Π_{ip} follows because Π_{ip} converts instance of $\mathcal{R}_{ip}$ to instance of $\mathcal{P}'_{ip}$ and Π'_{ip} is trivially complete by construction due to [Equation \(12\)](#). Notation *statistical witness-extended emulation* generalizes *special soundness* in the way applicable for multi-stage complex argument systems where each step of the protocol could be rewound to extract part of the witness so that more accurate definition of protocol security is achieved despite *special soundness* implies building the whole knowledge extractor which might has non-polynomial running time for multi-stage protocols.

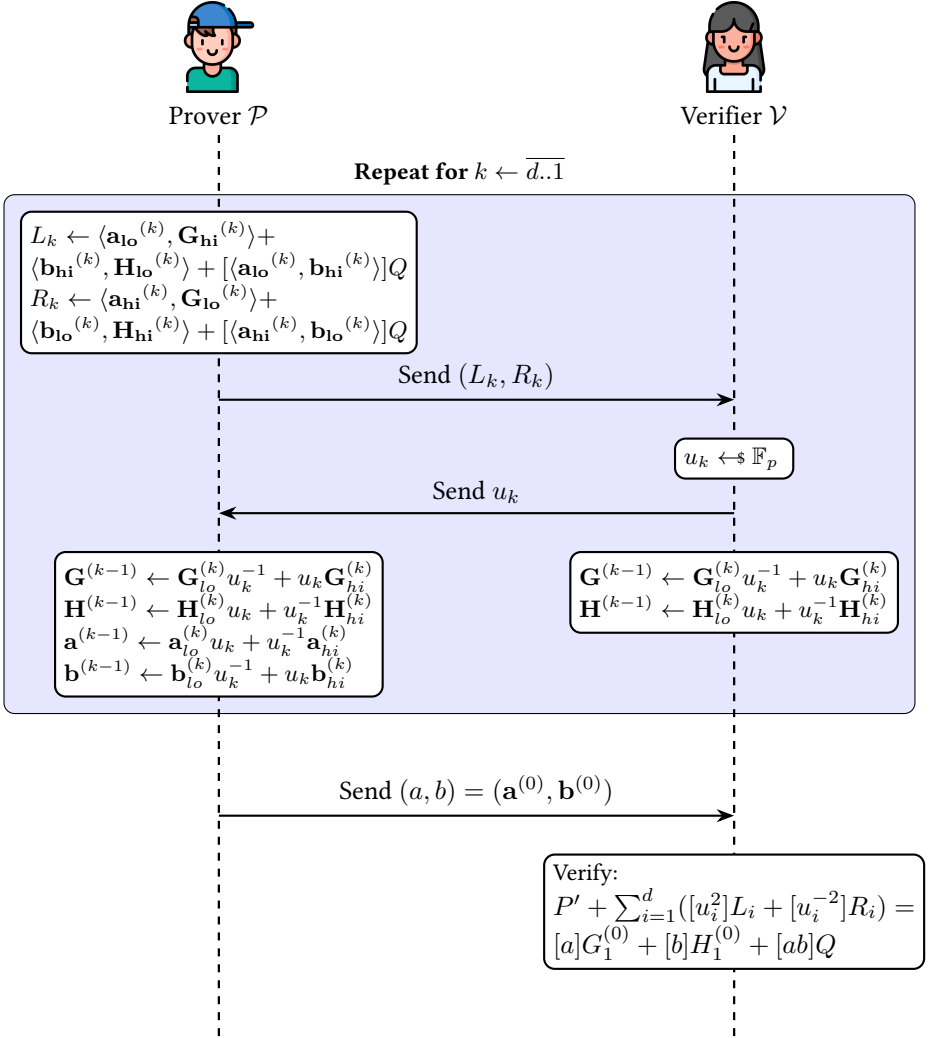


Figure 18.1: Interactive inner-product protocol Π'_{ip} between prover $\mathcal{P}$ and verifier $\mathcal{V}$ for relation $\mathcal{R}'_{ip}$

Here we briefly describe a knowledge extractor $\mathcal{E}'_{ip}$ for a witness $(\mathbf{a}, \mathbf{b})$ or non-trivial discrete logarithm relation for $(\mathbf{G}, \mathbf{H}, Q)$ for Π'_{ip} .

1. $\mathcal{E}'_{ip}$ runs $\mathcal{P}$ to the last stage to obtain $(L, R) \leftarrow (L_1, R_1)$.
2. $\mathcal{E}'_{ip}$ rewinds the Π'_{ip} final stage four times to obtain four challenges x_1, x_2, x_3, x_4 and responses $(\mathbf{a}_1^{(0)}, \mathbf{b}_1^{(0)}), (\mathbf{a}_2^{(0)}, \mathbf{b}_2^{(0)}), (\mathbf{a}_3^{(0)}, \mathbf{b}_3^{(0)}), (\mathbf{a}_4^{(0)}, \mathbf{b}_4^{(0)})$

so that the following equation holds for $i \in \{1, 2, 3, 4\}$ (11):

$$\begin{aligned} [x_i^2]L + P_1 + [x_i^{-2}]R = & \langle x_i^{-1} \cdot \mathbf{a}_i^{(0)}, \mathbf{G}_{\text{lo}}^{(1)} \rangle + \langle x_i \cdot \mathbf{a}_i^{(0)}, \mathbf{G}_{\text{hi}}^{(1)} \rangle \\ & + \langle x_i \cdot \mathbf{b}_i^{(0)}, \mathbf{H}_{\text{lo}}^{(1)} \rangle + \langle x_i^{-1} \cdot \mathbf{b}_i^{(0)}, \mathbf{H}_{\text{hi}}^{(1)} \rangle + [\langle \mathbf{a}_i^{(0)}, \mathbf{b}_i^{(0)} \rangle]Q \end{aligned} \quad (13)$$

3. $\mathcal{E}'_{ip}$ choses $(v_1, v_2, v_3) \in \mathbb{F}_p^3$ as a solution for the system of linear equations:

$$\begin{cases} x_1^2 v_1 + x_2^2 v_2 + x_3^2 v_3 = 0 \\ v_1 + v_2 + v_3 = 1 \\ x_1^{-2} v_1 + x_2^{-2} v_2 + x_3^{-2} v_3 = 0 \end{cases} \quad (14)$$

4. $\mathcal{E}'_{ip}$ takes linear combination of (13) with coefficients v_i for $i \in \{1, 2, 3\}$:

$$\begin{aligned} \sum_{i=1}^3 [v_i x_i^2]L + [v_i]P_1 + [v_i x_i^{-2}]R = & \sum_{i=1}^3 \langle v_i x_i^{-1} \cdot \mathbf{a}_i^{(0)}, \mathbf{G}_{\text{lo}}^{(1)} \rangle + \langle v_i x_i \cdot \mathbf{a}_i^{(0)}, \mathbf{G}_{\text{hi}}^{(1)} \rangle \\ & + \langle v_i x_i \cdot \mathbf{b}_i^{(0)}, \mathbf{H}_{\text{lo}}^{(1)} \rangle + \langle v_i x_i^{-1} \cdot \mathbf{b}_i^{(0)}, \mathbf{H}_{\text{hi}}^{(1)} \rangle + [v_i \langle \mathbf{a}_i^{(0)}, \mathbf{b}_i^{(0)} \rangle]Q \end{aligned}$$

Simplifying equation above we get:

$$\begin{aligned} P_1 = & \left\langle \sum_{i=1}^3 v_i x_i^{-1} \cdot \mathbf{a}_i^{(0)} + v_i x_i \cdot \mathbf{a}_i^{(0)}, \mathbf{G}^{(1)} \right\rangle + \\ & \left\langle \sum_{i=1}^3 v_i x_i \cdot \mathbf{b}_i^{(0)} + v_i x_i^{-1} \cdot \mathbf{b}_i^{(0)}, \mathbf{H}^{(1)} \right\rangle + \left[\sum_{i=1}^3 v_i \langle \mathbf{a}_i^{(0)}, \mathbf{b}_i^{(0)} \rangle \right]Q \end{aligned} \quad (15)$$

On the other hand we have an expression for P_1 from (10):

$$P_1 = \langle \mathbf{a}^{(1)}, \mathbf{G}^{(1)} \rangle + \langle \mathbf{b}^{(1)}, \mathbf{H}^{(1)} \rangle + [\langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle]Q \quad (16)$$

5. Asserting equality (15)=(16) the extractor $\mathcal{E}'_{ip}$ sets:

$$\mathbf{a}^{(1)} = \sum_{i=1}^3 v_i x_i^{-1} \cdot \mathbf{a}_i^{(0)} + v_i x_i \cdot \mathbf{a}_i^{(0)} \quad (17)$$

$$\mathbf{b}^{(1)} = \sum_{i=1}^3 v_i x_i \cdot \mathbf{b}_i^{(0)} + v_i x_i^{-1} \cdot \mathbf{b}_i^{(0)} \quad (18)$$

We need the fourth rewinding to assert equality of inner product:

$$\langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle = \sum_{i=1}^3 v_i \langle \mathbf{a}_i^{(0)}, \mathbf{b}_i^{(0)} \rangle$$

We won't describe it fully since it takes some unwieldy technical details and refer a reader to the original *bulletproofs* paper [?], where the full proof of extraction is described in Theorem 1.

6. The extractor $\mathcal{E}'_{ip}$ recursively extracts $\mathbf{a}^{(k+1)}, \mathbf{b}^{(k+1)}$ from $\mathbf{a}^{(k)}, \mathbf{b}^{(k)}$ using the method described in steps 2-5 until it reaches the final witness $\mathbf{a}^{(d)}, \mathbf{b}^{(d)} = \mathbf{a}, \mathbf{b}$ for which the relation $\mathcal{R}'_{ip}$ holds:

$$P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle]Q$$

Building the extractor $\mathcal{E}_{ip}$ for Π_{ip} is quite simple relatively to what we've done by now:

1. $\mathcal{E}_{ip}$ runs Π_{ip} to the end and applies the extractor $\mathcal{E}'_{ip}$ for P'_{ip} to extract the witness $\mathbf{a}, \mathbf{b}$ such that the following holds for $\mathcal{V}$'s challenge $r_1 \in \mathbb{F}_p$:

$$P + [r_1 c]B = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [r_1 \cdot \langle \mathbf{a}, \mathbf{b} \rangle]B \quad (19)$$

2. $\mathcal{E}_{ip}$ rewinds the prover $\mathcal{P}$ to obtain another challenge from $\mathcal{V}$: $r_2 \in \mathbb{F}_p$ and yield another witness $\mathbf{a}', \mathbf{b}'$ from $\mathcal{E}'_{ip}$:

$$P + [r_2 c]B = \langle \mathbf{a}', \mathbf{G} \rangle + \langle \mathbf{b}', \mathbf{H} \rangle + [r_2 \cdot \langle \mathbf{a}', \mathbf{b}' \rangle]B \quad (20)$$

3. $\mathcal{E}_{ip}$ substitute (19) from (20) to get:

$$[c(r_1 - r_2)]B = \langle \mathbf{a} - \mathbf{a}', \mathbf{G} \rangle + \langle \mathbf{b} - \mathbf{b}', \mathbf{H} \rangle + [r_1 \cdot \langle \mathbf{a}, \mathbf{b} \rangle - r_2 \cdot \langle \mathbf{a}', \mathbf{b}' \rangle]B \quad (21)$$

Unless $a = a'$ and $b = b'$ we get a non-trivial discrete log relation between $\mathbf{G}, \mathbf{H}$ and B , otherwise if equality of witnesses holds we get:

$$[(r_1 - r_2)c]B = [(r_1 - r_2)\langle \mathbf{a}, \mathbf{b} \rangle]B \quad (22)$$

Hence $c = \langle \mathbf{a}, \mathbf{b} \rangle$.

To formally finalize a proof of *witness-extended emulation* we also need to apply so-called *the forking lemma*, we again refer a reader to the original *bulletproofs* paper [?] $\square$.

18.4.3 Zero-knowledge extension of inner-product argument

The main approach to make *inner-product argument zero-knowledge* is to use Π_{zkip} protocol which original bulletproofs [?] does for *range proofs*(Section 18.6) and *arithmetic circuits satisfiability*(Section 18.7). However, there exists an alternative elegant construction based on tweaking *inner-product argument* itself described in [?, Appendix E.2].

The key idea is to bring a blinding factor $\delta \in \mathbb{F}_p$ with a verifier-provided random element $D \in \mathbb{G}$ to the commitment P_k :

$$P_k = \langle \mathbf{a}^{(k)}, \mathbf{G}^{(k)} \rangle + \langle \mathbf{b}^{(k)}, \mathbf{H}^{(k)} \rangle + [\langle \mathbf{a}^{(k)}, \mathbf{b}^{(k)} \rangle]Q + [\delta^{(k)}]D$$

To get L_k, R_k prover draws $\delta_L^{(k)}, \delta_R^{(k)} \xleftarrow{\$} \mathbb{F}_p$ and sets:

$$P_{k-1} = P_k + [u_k^2]L_k + [u_k^{-2}]R_k$$

$$L_k = \langle \mathbf{a}_{lo}^{(k)}, \mathbf{G}_{hi}^{(k)} \rangle + \langle \mathbf{b}_{hi}^{(k)}, \mathbf{H}_{lo}^{(k)} \rangle + [\langle \mathbf{a}_{lo}^{(k)}, \mathbf{b}_{hi}^{(k)} \rangle]Q + [\delta_L^{(k)}]D$$

$$R_k = \langle \mathbf{a}_{hi}^{(k)}, \mathbf{G}_{lo}^{(k)} \rangle + \langle \mathbf{b}_{lo}^{(k)}, \mathbf{H}_{hi}^{(k)} \rangle + [\langle \mathbf{a}_{hi}^{(k)}, \mathbf{b}_{lo}^{(k)} \rangle]Q + [\delta_R^{(k)}]D$$

Then $\mathcal{P}$ updates next-round δ using verifier-provided challenge u_k :

$$\delta^{(k-1)} = \delta^{(k)} + u_k^2 \delta_L^{(k)} + u_k^{-2} \delta_R^{(k)}$$

On the last step prover must prove that he possesses $a = a^{(0)}, b = b^{(0)}, \delta = \delta^{(0)}$ such that for $G = G^{(0)}, H = H^{(0)}$ equality holds:

$$P_0 = P_k + \sum_{i=1}^k ([u_i^2]L_i + [u_i^{-2}]R_i) = [a]G + [b]H + [a \cdot b]Q + [\delta]D$$

$P_0 = P_k + \sum_{i=1}^k ([u_i^2]L_i + [u_i^{-2}]R_i)$ is a Pedersen commitment to $a, b, a \cdot b$ with blinding factor δ and could easily be computed by verifier. One could prove knowledge of an opening (a, b, δ) to that commitment using protocol similar to Π_{zkip} .

1. $\mathcal{P}$ draws blinders $u, v, r, s \leftarrow \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments:

$$\begin{aligned} A &= [r]G + [s]H + [as + br]Q + [u]D \\ T &= [rs]Q + [v]D \end{aligned}$$

2. $\mathcal{V}$ draws challenge $c \leftarrow \mathbb{F}_p$ and sends it to $\mathcal{P}$
3. $\mathcal{P}$ evaluates and sends to $\mathcal{V}$:

$$\begin{aligned} a' &= a + rc \\ b' &= b + sc \\ \delta' &= \delta + uc + vc^2 \end{aligned}$$

4. $\mathcal{V}$ performs check:

$$[a']G + [b']H + [a'b']Q + [\delta']D \stackrel{?}{=} P_0 + [c]A + [c^2]T$$

Protocol is *knowledge-sound* as value δ could easily be extracted from three accepting transcripts. To argue *zero-knowledge* we stress that an adversary couldn't learn anything from transcript and each prover's message (L_k, R_k) could be easily simulated by random element, at the base of recursion simulator simulates zero-knowledge proof of opening (a, b, δ) to commitment P_0 .

18.5 Inner-product based polynomial commitment scheme

Here we describe one of the main theoretical applications of the *inner-product argument* – **polynomial commitment scheme** that relies only on *discrete logarithm* assumption, while studied before KZG commitment scheme needs bilinear pairings.

Definition 18.9. The inner-product non-hiding polynomial commitment scheme $\mathcal{C}_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows. Let $f(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{F}_p[x]$ be a polynomial of degree $n - 1$.

- Setup returns a vector of independent generators $\mathbf{G} = (G_1, \dots, G_n)$.
- Commit returns polynomial commitment $\text{Com}(f) = \langle \mathbf{f}, \mathbf{G} \rangle$ where $\mathbf{f} = (a_0, \dots, a_{n-1})$
- Open given evaluation point $u \in \mathbb{F}_p$ computes $\mathbf{u}^n = (1, u, u^2, \dots, u^{n-1})$, obtains $f(u) = \langle \mathbf{f}, \mathbf{u}^n \rangle$ and runs *inner-product argument* Π_{ip} non-interactively setting

$$\mathbf{a} = \mathbf{f}, \mathbf{b} = \mathbf{u}^n, P = \text{Com}(f), c = f(u)$$

to produce an evaluation proof π_{ip}

- VerifyOpen given evaluation point $u \in \mathbb{F}_p$ and commitment $\text{Com}(f)$ validates proof π_{ip} running the non-interactive verifier $\mathcal{V}$ of *inner-product argument*.

Remark. As the second vector $\mathbf{b} = \mathbf{u}^n$ is known to the verifier, the prover don't have to commit to it using vector $\mathbf{H}$, so the parties might adjust all the steps eliminating vector $\mathbf{H}$ and $\mathbf{b}$ vector compression as well. The full scheme is described [here](#). Note that described scheme is not zero-knowledge as classic *inner-product argument* is not zero-knowledge. But we could make it zero-knowledge using Π_{zkip} protocol or using alternative construction from [Section 18.4.3](#).

18.6 Range proofs

Let $G, B \in \mathbb{G}$ – independent group generators. Let's consider the relation:

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) | V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

This relation is often called the **range proof** relation. It asserts that committed value v lays in the interval $[0, 2^n)$. Range proofs have very significant applications in various privacy *blockchain* protocols since they usually imply proving that transaction inputs or outputs are valid, e.g. have positive value or satisfy other relations between them.

For the first view it seems very inconspicuous why **inner-product argument** is useful for proving the range proof relation, but we'll show it ab initio.

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lceil \log_2 v \rceil} 2^i v_i$ and $\mathbf{a}_L = (v_0, v_1, \dots, v_{n-1})$ be the vector of bits padded with zeroes to length n , so the range validation that v lays in $[0, 2^n)$ implies two checks:

- Each bit v_i must be either 0 or 1
- The following inner-product equality holds: $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$

We already know how to prove the second one inner product equality – simply

by taking evaluation point $u \leftarrow 2$ in *inner-product based polynomial commitment scheme*.

The first relation is a bit more tricky to check algebraically, but still we'll manage to do that, note that binary check for v_i takes form $v_i(v_i - 1) = 0$, or in vector form:

$$\begin{aligned}\mathbf{a}_R &= \mathbf{a}_L - \mathbf{1}^n \Leftrightarrow \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n = \mathbf{0}^n \\ \mathbf{a}_L \circ \mathbf{a}_R &= \mathbf{0}^n\end{aligned}$$

Example 18.2. Let $\mathbf{a}_L = (1, 0, 1, 0)$, $\mathbf{a}_R = (0, -1, 0, -1)$, then $\mathbf{a}_L \circ \mathbf{a}_R = (0, 0, 0, 0)$

This two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle = 0 \text{ and } \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$$

This checks are sound because the prover doesn't know challenge y in advance.

Note that $\langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle = \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle$ so the prover could commit to $\mathbf{a}_L$, $\mathbf{a}_R$ and verifier will adjust commitment for $\mathbf{a}_R$ using modified generators $\mathbf{H} \circ \mathbf{y}^{-n}$.

Here we came up with three inner-product checks:

1. $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$
2. $\langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = 0$
3. $\langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$

Here we could soundly combine all three checks into one using random linear combination with some verifier-provided challenge $z \in \mathbb{F}_p$:

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + z \cdot \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v$$

Remark. Naïve check $\langle \mathbf{a}_L, \mathbf{2}^n \rangle + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle + \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = v$ is not sound as Prover could adjust vectors to be non-zero but still satisfy the check.

Now simplify this expression having only one inner-product check:

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + z \cdot \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v$$

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + \boxed{z \cdot \langle \mathbf{a}_L, \mathbf{y}^n \rangle - z \cdot \langle \mathbf{a}_R, \mathbf{y}^n \rangle - z \cdot \langle \mathbf{1}^n, \mathbf{y}^n \rangle} + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v$$

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + \boxed{\langle \mathbf{a}_L, z \cdot \mathbf{y}^n \rangle + \langle -\mathbf{a}_R, z \cdot \mathbf{y}^n \rangle + \langle -z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle} + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v$$

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + \langle \mathbf{a}_L, z \cdot \mathbf{y}^n \rangle + \boxed{\langle \mathbf{a}_R \circ \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle} + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \boxed{\langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle}$$

$$\boxed{\langle \mathbf{a}_L, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n \rangle} + \langle \mathbf{a}_R \circ \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle + \boxed{\langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle} = z^2 v + \langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle$$

$$\boxed{\langle \mathbf{a}_L, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle} + \langle \mathbf{a}_R \circ \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle = z^2 v + \langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle$$

Now adding to both sides $\langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle$:

$$\begin{aligned}
& \langle \mathbf{a}_L, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle + \langle \mathbf{a}_R \circ \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle + \boxed{\langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle} = \\
& = z^2 v + \langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle + \boxed{\langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle} \\
& \langle \mathbf{a}_L, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle + \boxed{\langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle} = \\
& = z^2 v + \boxed{\langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle + \langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle} \\
& \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \boxed{\delta(y, z)}
\end{aligned}$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = \langle z \cdot \mathbf{1}^n, \mathbf{y}^n \rangle + \langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n, -z \cdot \mathbf{1}^n \rangle = (z - z^2) \langle \mathbf{1}^n, \mathbf{y}^n \rangle - z^3 \langle \mathbf{1}^n, \mathbf{2}^n \rangle$$

Now we have only one inner-product check left:

$$\langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z) \quad (23)$$

We will use a technique presented in Π_{zkp} to provide zero-knowledge and **inner-product argument** to achieve logarithmic size-proof. One key problem is that the verifier must adjust commitments to compensate auxiliary terms.

Firstly, construct the blinding polynomials for $\mathbf{a}_L$ and $\mathbf{a}_R$ with substitution:

$$\mathbf{a}'_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x \quad \mathbf{a}'_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x$$

Compute polynomials $\mathbf{l}(x) = \mathbf{l}_0 + \mathbf{l}_1 x$, $\mathbf{r}(x) = \mathbf{r}_0 + \mathbf{r}_1 x$:

$$\begin{aligned}
\mathbf{l}(x) &= \mathbf{a}'_L - z \cdot \mathbf{1}^n = (\mathbf{a}_L + \mathbf{s}_L x) - z \cdot \mathbf{1}^n = \mathbf{a}_L - z \cdot \mathbf{1}^n + \mathbf{s}_L x \\
\mathbf{r}(x) &= z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}'_R \circ \mathbf{y}^n = z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + (\mathbf{a}_R + \mathbf{s}_R x) \circ \mathbf{y}^n \\
&= z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n + \mathbf{s}_R \circ \mathbf{y}^n x
\end{aligned}$$

So that $\langle \mathbf{l}_0, \mathbf{r}_0 \rangle = z^2 v + \delta(y, z)$ - inner product that we want to prove using a bit modified Π_{zkp} .

Definition 18.10. The **range proof protocol** $\Pi_{rp} = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) \mid V = [v]G + [\gamma]B, v \in [0, 2^n)\}$ with prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- Setup returns vectors of group generators with unknown discrete log relations $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$
- Prover does bit decomposition of v to obtain vectors $\mathbf{a}_L \leftarrow \mathbf{v}, \mathbf{b}_L \leftarrow \mathbf{a}_L - \mathbf{1}^n$ and choses blinding terms $\mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n, \alpha, \beta \in \mathbb{F}_p$ computing

and sending commitments:

$$\begin{aligned} A &= \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{b}_L, \mathbf{H} \rangle + [\alpha]B \\ S &= \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B \end{aligned}$$

- Verifier $\mathcal{V}$ samples challenges $y, z \leftarrow \mathbb{F}_p$ and sends them to $\mathcal{P}$
- Prover $\mathcal{P}$ combines the three inner products into one using provided challenges y, z (23) and reconstructs vector polynomials $\mathbf{l}(x), \mathbf{r}(x), \mathbf{t}(x)$:

$$\begin{aligned} \mathbf{l}(x) &= \mathbf{a}_L - z \cdot \mathbf{1}^n + \mathbf{s}_L x \\ \mathbf{r}(x) &= z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n + \mathbf{s}_R \circ \mathbf{y}^n x \\ t(x) &= \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_0 + t_1 x + t_2 x^2 \end{aligned}$$

Where

$$\begin{aligned} t_0 &= \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z) \\ t_1 &= \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, \mathbf{y}^n \circ \mathbf{s}_R \rangle + \langle \mathbf{y}^n \circ (\mathbf{a}_R + z \cdot \mathbf{1}^n) + z^2 \cdot \mathbf{2}^n, \mathbf{s}_L \rangle \\ t_2 &= \langle \mathbf{s}_L, \mathbf{y}^n \circ \mathbf{s}_R \rangle \end{aligned}$$

- Prover $\mathcal{P}$ draws blinding factors $\tau_1, \tau_2 \leftarrow \mathbb{F}_p$ and sends to $\mathcal{V}$ the following commitments for coefficients of $\mathbf{t}(x)$:

$$\begin{aligned} T_1 &= [t_1]G + [\tau_1]B \\ T_2 &= [t_2]G + [\tau_2]B \end{aligned} \tag{24}$$

Note: prover does not have to send commitment to t_0 as it's the inner-product we want to prove and it could be computed from high-level commitment V .

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow \mathbb{F}_p$
- Prover $\mathcal{P}$ evaluates polynomials at u :

$$\begin{aligned} \mathbf{l}_u &= \mathbf{l}(u) & \alpha_u &= \alpha + \beta u \\ \mathbf{r}_u &= \mathbf{r}(u) & \tau_u &= z^2 \gamma + \tau_1 u + \tau_2 u^2 \\ t_u &= t(u) = z^2 v + \delta(y, z) + t_1 u + t_2 u^2 \end{aligned} \tag{25}$$

and sends $(\mathbf{l}_u, \mathbf{r}_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

- Verifier $\mathcal{V}$ performs checks:

$$\begin{aligned} A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle \\ \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B \\ [t_u]G + [\tau_u]B \stackrel{?}{=} [z^2]V + [\delta(y, z)]G + [u]T_1 + [u^2]T_2 \\ t_u \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle \end{aligned} \tag{26}$$

Remark. The last two steps of Π_{rp} could be substituted with an inner-product argument Π_{ip} to provide logarithmic size-proof with the following steps:

- After $\mathcal{P}$ evaluates polynomials at u he sends $(t_u, \alpha_u, \tau_u$ to $\mathcal{V}$) and computes commitment:

$$P = \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle$$

- $\mathcal{V}$ performs check $[t_u]G + [\tau_u]B \stackrel{?}{=} [z^2]V + [\delta(y, z)]G + [u]T_1 + [u^2]T_2$, halts if it fails and reconstructs commitment P otherwise:

$$P = A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle - [\alpha_u]B$$

- Parties run inner-product argument Π_{ip} on $(\mathbf{G}, \mathbf{y}^{-n} \circ \mathbf{H}, P, t_u; \mathbf{l}_u, \mathbf{r}_u)$

Theorem 18.11. The range proof protocol Π_{rp} has perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge

Proof idea. *Perfect completeness*

Expand LHS of the polynomial correctness check:

$$\begin{aligned} & A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle = \\ & \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{b}_L, \mathbf{H} \rangle + [\alpha]B + u \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + u \cdot \langle \mathbf{s}_R, \mathbf{H} \rangle + [u\beta]B + \\ & \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle = \\ & \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{b}_L, \mathbf{H} \rangle + u \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + u \cdot \langle \mathbf{s}_R, \mathbf{H} \rangle + \\ & \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{1}^n, \mathbf{H} \rangle + \langle z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha + u\beta]B \end{aligned}$$

Expand RHS of the polynomial correctness check:

$$\begin{aligned} & \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B = \\ & \langle \mathbf{a}_L - z \cdot \mathbf{1}^n + \mathbf{s}_L \cdot u, \mathbf{G} \rangle + \langle z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n + \mathbf{s}_R \circ \mathbf{y}^n \cdot u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle \\ & + [\alpha + u\beta]B = \\ & \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + u \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + \langle z \cdot \mathbf{1}^n, \mathbf{H} \rangle + \\ & \langle \mathbf{a}_R, \mathbf{H} \rangle + u \cdot \langle \mathbf{s}_R, \mathbf{H} \rangle + [\alpha + u\beta]B \end{aligned}$$

We could see that *LHS* is equal to *RHS* so the first check pass.

Taking the t_0 correctness check:

$$\begin{aligned} & [t_u]G + [\tau_u]B \stackrel{?}{=} [z^2]V + \delta(y, z) + [u]T_1 + [u^2]T_2 \\ & [t_u]G + \cancel{[z^2\gamma + \tau_1u + \tau_2u^2]B} \stackrel{?}{=} [z^2v]G + \cancel{[z^2\gamma]B} + [\delta(y, z)]G + \\ & \quad [ut_1]G + \cancel{[u\tau_1]B} + [t_2u^2]G + \cancel{[t_2u^2]B} \\ & [t_u]G \stackrel{?}{=} [z^2v + \delta(y, z) + t_1u + t_2u^2]G \end{aligned}$$

Which holds since $t_u = t(u) = z^2v + \delta(y, z) + t_1u + t_2u^2$. The third inner-product also holds as $t_u = \langle \mathbf{l}_u, \mathbf{r}_u \rangle$.

Perfect honest-verifier zero-knowledge follows from the zero-knowledge construction of Π_{zkip} protocol as Verifier learns no information about $\mathbf{a}_L, \mathbf{a}_R$.

Computational extended witness emulation implies building extractor that combines extractors for two subprotocols: $\mathcal{E}_{ip}$ extracts witness $(\mathbf{l}_u, \mathbf{r}_u)$ from Π_{ip} than the extractor $\mathcal{E}_{zkip}$ extracts high-level witness $(\mathbf{a}_L, \mathbf{a}_R)$ from Π_{zkip} $\square$

The proof size of **range-proof** protocol is $2 \log_2 n + 4$ group $\mathbb{G}$ elements and 5 field $\mathbb{F}_p$ elements.

Remark. Range proofs could be efficiently aggregated: e.g. one could prove the relation using slightly modified range proof protocol Π_{rp}

$$\mathcal{R}_{rpm} = \{(G, B, \vec{V}, n; \vec{v}, \vec{\gamma}) | \forall i \in 1..m : V_i = [v_i]G + [\gamma_i]B, v_i \in [0, 2^n)\}$$

Where $\vec{v} = (v_1, v_2, \dots, v_m)$ and $\vec{\gamma} = (\gamma_1, \gamma_2, \dots, \gamma_m)$ – respectively secrets and blinding factors. Detail explanation of aggregation protocol could be found in original bulletproofs paper [?]

Example 18.3. One of the most famous *NP-complete* problems is the **subset-sum problem**: given a set of numbers presented as vector $\mathbf{s}$ and number $v \in \mathbb{N}$, does a some subset sums up to v . It turns out that we could use our **range-proof** protocol for this problem. One could simply replace first inner-product check $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$ with $\langle \mathbf{a}_L, \mathbf{s} \rangle = v$ where $\mathbf{a}_L$ is the secret vector of bits that encode positions of $\mathbf{s}$ that sum up to v .

For example take $\mathbf{s} = (6, 8, 2, 3)$ and $v = 14$. Then setting $\mathbf{a}_L = (1, 1, 0, 0)$ we could use Π_{rp} to prove that there exists a subset of $\mathbf{s}$ that sums up to $v = 14$ without disclosing that subset.

Therefore, **bulletproofs range proof** protocol is capable to prove a knowledge of witness to any *NP*-problem as they all could be reduced to the **subset-sum problem**

18.7 Arithmetic circuits proofs

Bulletproofs presents not only range proofs, but also efficient proofs for arithmetic circuits satisfiability. As we could see before, inner-product relation is quite powerful tool and could be used to prove a knowledge of witness to any *NP*-problem. But here we present a more convenient way to compile arithmetic circuits into inner-product relation.

18.7.1 Arithmetization

Bulletproofs arithmetization slightly differs from the classic R1CS we studied before at Section 13, however it could be transformed vice-versa easily. Also **bulletproofs** arithmetization is more convenient and human-friendly for encoding

most of the arithmetic circuits than the R1CS.

There are two types of variables in *bulletproofs* constraint system: *low-level* and *high-level*. Typically *high-level* variables are provided with Pedersen commitments $\mathbf{V} \in \mathbb{G}^m$ as the private witness inputs $\mathbf{v} \in \mathbb{F}_p^m$ to the circuit, while *low-level* variables $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$ are the intermediate witness values of computation. We will define circuit as a set of multiplication constraints operating with *low-level* variables and set of linear constraints which links *low-level* variables between each other and *high-level* variables as well.

Multiplication constraints are defined with one vector equation:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O$$

Linear constraints are defined via:

$$\mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c}$$

Where $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O$ – vectors of left and right inputs for multiplication gates and output values (all of them are low-level variables). $\mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O \in \mathbb{F}_p^{q \times n}$, $\mathbf{W}_V \in \mathbb{F}_p^{q \times m}$ – public matrices of weights for linear constraints (obviously known to verifier). $\mathbf{c} \in \mathbb{F}_p^q$ – public vector of constants. Typically they encode wiring of the circuit and other linear relations between variables.

Example 18.4. Consider the following elliptic curve membership circuit. Here witness (v_1, v_2) should satisfy elliptic curve equation:

$$y^2 = x^3 + ax + b$$

The arithmetization for this circuit is as follows:

Low-level variables:

$$\mathbf{a}_L = \begin{bmatrix} x \\ x \\ y \end{bmatrix}, \quad \mathbf{a}_R = \begin{bmatrix} x \\ x^2 \\ y \end{bmatrix}, \quad \mathbf{a}_O = \begin{bmatrix} x^2 \\ x^3 \\ y^2 \end{bmatrix}$$

High-level variables:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

Multiplication constraints:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \Rightarrow \begin{bmatrix} x \cdot x = x^2 \\ x \cdot x^2 = x^3 \\ y \cdot y = y^2 \end{bmatrix}$$

Linear constraints:

$$\begin{aligned}
\mathbf{a}_L^{(1)} &= v_1 \\
\mathbf{a}_R^{(1)} &= v_1 \\
\mathbf{a}_L^{(2)} - \mathbf{a}_L^{(1)} &= 0 \\
\mathbf{a}_R^{(2)} - \mathbf{a}_O^{(1)} &= 0 \\
\mathbf{a}_L^{(3)} &= v_2 \\
\mathbf{a}_R^{(3)} &= v_2 \\
\mathbf{a}_O^{(3)} - \mathbf{a}_O^{(2)} - a \cdot \mathbf{a}_L^{(1)} &= b
\end{aligned}$$

$$\mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c}$$

Where:

$$\mathbf{W}_L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ -a & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_R = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_O = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 1 \end{bmatrix},$$

$$\mathbf{W}_V = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ b \end{bmatrix}$$

Classic R1CS arithmetization:

We can also represent the same circuit using the classic R1CS formalism:

$$(A \cdot \mathbf{w}) \circ (B \cdot \mathbf{w}) = (C \cdot \mathbf{w})$$

where $\mathbf{w}$ is extended witness vector $\mathbf{w} = (1, x, y, x^2, x^3, y^2)$

The R1CS constraints for the circuit $y^2 = x^3 + ax + b$ are:

- Constraint 1: $x \times x = x^2$
- Constraint 2: $x^2 \times x = x^3$
- Constraint 3: $y \times y = y^2$
- Constraint 4: $x^3 + ax + b = y^2$

Explicitly, the R1CS matrices are:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & a & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

18.7.2 Proving a circuit satisfiability

In this section we build an argument system for the following arithmetic circuit satisfiability relation:

$$\mathcal{R}_{sat} = \left\{ \begin{array}{l} (G, B, \mathbf{V}, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O, \mathbf{W}_V, \mathbf{c}; \mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v}, \mathbf{r}) \\ \forall i = 1..m : V_i = [v_i]G + [r_i]B \wedge \\ \mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \wedge \\ \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c} \end{array} \right\} \quad (27)$$

Where $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$, $\mathbf{v}, \mathbf{r} \in \mathbb{F}_p^m$, $\mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O \in \mathbb{F}_p^{q \times n}$, $\mathbf{W}_V \in \mathbb{F}_p^{q \times m}$, $\mathbf{c} \in \mathbb{F}_p^q$. Informally this relation states that there exists a valid witness $\mathbf{v}$ that satisfies all constraints of the circuit. For the verifier witness is presented only as commitments vector $\mathbf{V}$.

We could use similar to *range-proofs* technique to compile constraints of the circuit into inner-product relation. For multiplicative constraints take random $y \in \mathbb{F}_p$ and apply zero check:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle = 0$$

Do the same for linear constraints, but for different randomness $z \in \mathbb{F}_p$:

$$\langle \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c}, \mathbf{z}^q \rangle = 0$$

Combine this two checks to one using the same randomness z :

$$\begin{aligned} \langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + z \cdot \langle \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c}, \mathbf{z}^q \rangle &= 0 \\ \langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle z \cdot \mathbf{z}^q, \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c} \rangle &= 0 \end{aligned}$$

This check is sound as typically a prover could not control values of y, z before he commits to $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O$ and $\mathbf{v}$.

Then split the second inner product and factor-out public terms to RHS:

$$\begin{aligned} \langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_L \cdot \mathbf{a}_L \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_R \cdot \mathbf{a}_R \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_O \cdot \mathbf{a}_O \rangle \\ - \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_V \cdot \mathbf{v} \rangle - \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{c} \rangle = 0 \\ \langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_L \cdot \mathbf{a}_L \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_R \cdot \mathbf{a}_R \rangle + \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_O \cdot \mathbf{a}_O \rangle \\ - \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{W}_V \cdot \mathbf{v} \rangle = \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{c} \rangle \end{aligned}$$

Applying conjugation rule(if A – linear operator and A^T – its transpose(conjugate) then $\langle \mathbf{a}, A\mathbf{b} \rangle = \langle A^T \mathbf{a}, \mathbf{b} \rangle$) to the second inner product we get:

$$\begin{aligned} \langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{W}_L^T \cdot (\mathbf{z} \cdot \mathbf{z}^q), \mathbf{a}_L \rangle + \langle \mathbf{W}_R^T \cdot (\mathbf{z} \cdot \mathbf{z}^q), \mathbf{a}_R \rangle + \\ \langle \mathbf{W}_O^T \cdot (\mathbf{z} \cdot \mathbf{z}^q), \mathbf{a}_O \rangle - \langle \mathbf{W}_V^T \cdot (\mathbf{z} \cdot \mathbf{z}^q), \mathbf{v} \rangle = \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{c} \rangle \end{aligned}$$

Denote $w_c = \langle \mathbf{z} \cdot \mathbf{z}^q, \mathbf{c} \rangle$ and flattened linear constraints(still public and easily computed by verifier):

$$\begin{aligned} \mathbf{w}_L &= \mathbf{W}_L^T \cdot (\mathbf{z} \cdot \mathbf{z}^q) \\ \mathbf{w}_R &= \mathbf{W}_R^T \cdot (\mathbf{z} \cdot \mathbf{z}^q) \\ \mathbf{w}_O &= \mathbf{W}_O^T \cdot (\mathbf{z} \cdot \mathbf{z}^q) \\ \mathbf{w}_V &= \mathbf{W}_V^T \cdot (\mathbf{z} \cdot \mathbf{z}^q) \end{aligned}$$

Now we could finally rewrite the equation as:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle + \langle \mathbf{w}_R, \mathbf{a}_R \rangle + \langle \mathbf{w}_O, \mathbf{a}_O \rangle - \langle \mathbf{w}_V, \mathbf{v} \rangle = w_c$$

Then rearrange and combine terms so that $\mathbf{a}_L, \mathbf{a}_O$ be on the left side and $\mathbf{a}_R$ on the right side:

$$\begin{aligned} \langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle - \langle \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle + \langle \mathbf{w}_R, \mathbf{a}_R \rangle + \langle \mathbf{w}_O, \mathbf{a}_O \rangle &= \langle \mathbf{w}_V, \mathbf{v} \rangle + w_c \\ \langle \mathbf{a}_L, \mathbf{y}^n \circ \mathbf{a}_R \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle + \langle \mathbf{w}_R, \mathbf{a}_R \rangle &= \langle \mathbf{w}_V, \mathbf{v} \rangle + w_c \\ \langle \mathbf{a}_L, \mathbf{y}^n \circ \mathbf{a}_R \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle + \langle \mathbf{y}^n \circ \mathbf{a}_R, \mathbf{y}^{-n} \circ \mathbf{w}_R \rangle &= \langle \mathbf{w}_V, \mathbf{v} \rangle + w_c \\ \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle &= \langle \mathbf{w}_V, \mathbf{v} \rangle + w_c \end{aligned}$$

Add $\delta(y, z) = \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{w}_L \rangle$ to both sides:

$$\begin{aligned} w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) &= \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R \rangle + \langle \mathbf{a}_L, \mathbf{w}_L \rangle + \\ &\quad \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{w}_L \rangle \\ w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) &= \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R \rangle + \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{w}_L \rangle + \\ &\quad \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle \\ w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) &= \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle \end{aligned}$$

Now it seems we are stuck as there is two separate inner products so one could not simply linearly blind witness parts, multiply corresponding polynomials and

obtain desired inner product in constant term just as we did in the *range-proof* case. But fortunately we could take polynomials of higher degree and obtain desired sum of inner-products as some coefficient of the product of polynomials:

$$\langle \mathbf{a}x + \mathbf{c}x^2, \mathbf{d} + \mathbf{b}x \rangle = s_1x + s_2x^2 + s_3x^3 = x \cdot \langle \mathbf{a}, \mathbf{d} \rangle + x^2 \cdot (\langle \mathbf{a}, \mathbf{b} \rangle + \langle \mathbf{c}, \mathbf{d} \rangle) + x^3 \cdot \langle \mathbf{c}, \mathbf{b} \rangle \quad (28)$$

So take:

$$\begin{aligned} \mathbf{a} &\leftarrow \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R & \mathbf{b} &\leftarrow \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \\ \mathbf{c} &\leftarrow \mathbf{a}_O & \mathbf{d} &\leftarrow -\mathbf{y}^n + \mathbf{w}_O \end{aligned} \quad (29)$$

So than we could get desired sum of inner products as the second-degree coefficient s_2 :

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) = s_2$$

In order to obtain final polynomials $\mathbf{l}(x), \mathbf{r}(x)$ we must firstly blind $\mathbf{a}_L, \mathbf{a}_R$:

$$\mathbf{a}_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x^2 \quad \mathbf{a}_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x^2 \quad (30)$$

Remark. We multiplied blinders $\mathbf{s}_L, \mathbf{s}_R$ with the second power of challenge x^2 because we want the blinding terms to do not interfere with other parts of inner-product.

$\mathbf{a}_O$ does not need separate blinding as it's located on the left side of the inner-product (28) along with $\mathbf{a}_L$, which is already blinded by $\mathbf{s}_L$.

Now we could compute polynomials $\mathbf{l}(x), \mathbf{r}(x)$ from (28) using assignments from (29) and blinders from (30):

$$\begin{aligned} \mathbf{l}(x) &= (\mathbf{a}_L + \mathbf{s}_L \cdot x^2) \cdot x + \mathbf{y}^{-n} \circ \mathbf{w}_R \cdot x + \mathbf{a}_O \cdot x^2 \\ &= \mathbf{a}_L \cdot x + \mathbf{s}_L \cdot x^3 + \mathbf{y}^{-n} \circ \mathbf{w}_R \cdot x + \mathbf{a}_O \cdot x^2 \\ &= \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x \\ \mathbf{r}(x) &= \mathbf{y}^n \circ (\mathbf{a}_R + \mathbf{s}_R \cdot x^2) \cdot x + \mathbf{w}_L \cdot x - \mathbf{y}^n + \mathbf{w}_O \\ &= \mathbf{y}^n \circ \mathbf{a}_R \cdot x + \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + \mathbf{w}_L \cdot x - \mathbf{y}^n + \mathbf{w}_O \\ &= \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O \end{aligned} \quad (31)$$

Now we could compute the polynomial $t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle$:

$$\begin{aligned}
t(x) &= \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_1x + t_2x^2 + t_3x^3 + t_4x^4 + t_5x^5 + t_6x^6 = \sum_{i=1}^6 t_i x^i \\
t_1 &= \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, -\mathbf{y}^n + \mathbf{w}_O \rangle \\
t_2 &= \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle \\
&= \langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle - \langle \mathbf{a}_O, \mathbf{y}^n \rangle + \langle \mathbf{w}_L, \mathbf{a}_L \rangle + \langle \mathbf{w}_R, \mathbf{a}_R \rangle + \langle \mathbf{w}_O, \mathbf{a}_O \rangle + \delta(y, z) \\
&= w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) \\
t_3 &= \langle \mathbf{s}_L, -\mathbf{y}^n + \mathbf{w}_O \rangle + \langle \mathbf{a}_O, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle \\
t_4 &= \langle \mathbf{s}_L, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle + \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{s}_R \rangle \\
t_5 &= \langle \mathbf{a}_O, \mathbf{y}^n \circ \mathbf{s}_R \rangle \\
t_6 &= \langle \mathbf{s}_L, \mathbf{y}^n \circ \mathbf{s}_R \rangle
\end{aligned} \tag{32}$$

Our proving strategy is the same as in the *range-proof* case:

- Prove that $\mathbf{l}(x), \mathbf{r}(x)$ are correct using binding commitments to their coefficients.
- Prove that t_2 is correct (as it's the inner-product we want to prove) using evaluation at challenge point u .
- Apply inner-product argument to compress the proof of evaluation of $t(x)$ at challenge point u .

Definition 18.12. The **arithmetic circuit satisfiability** protocol $\Pi_{sat} = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}_{sat}$ with prover $\mathcal{P}$ and verifier $\mathcal{V}$ is defined as follows:

- Setup: returns vector of group generators with unknown discrete log relations $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$.
- Prover $\mathcal{P}$ choses blinding factors $\alpha, \beta, \gamma \in \mathbb{F}_p, \mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n$ and sends the following commitments to $\mathcal{V}$:

$$\begin{aligned}
A_I &= \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \\
A_O &= \langle \mathbf{a}_O, \mathbf{G} \rangle + [\gamma]B \\
S &= \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B
\end{aligned}$$

- Verifier samples challenges $y, z \leftarrow \mathbb{F}_p$ and sends them to $\mathcal{P}$.
- Using provided challenges y, z prover forms polynomials $\mathbf{l}(x), \mathbf{r}(x)$:

$$\begin{aligned}
\mathbf{l}(x) &= \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x \\
\mathbf{r}(x) &= \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O
\end{aligned}$$

computes

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_1x + t_2x^2 + t_3x^3 + t_4x^4 + t_5x^5 + t_6x^6$$

chooses random blinding factors $\tau_1, \tau_3, \tau_4, \tau_5, \tau_6 \in \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments to its coefficients:

$$T_1 = [t_1]G + [\tau_1]B$$

$$T_3 = [t_3]G + [\tau_3]B$$

$$T_4 = [t_4]G + [\tau_4]B$$

$$T_5 = [t_5]G + [\tau_5]B$$

$$T_6 = [t_6]G + [\tau_6]B$$

Note: Prover does not send separate commitment to t_2 as the verifier could derive it from $\mathbf{V}$ and the circuit public parameters:

$$t_2 = w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z)$$

$$T_2 = \langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G$$

- Verifier samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow \mathbb{F}_p$.
- Prover evaluates polynomials at u :

$$\mathbf{l}_u = \mathbf{l}(u)$$

$$\mathbf{r}_u = \mathbf{r}(u)$$

$$t_u = \langle \mathbf{l}_u, \mathbf{r}_u \rangle = t(u)$$

$$\tau_u = \tau_1 \cdot u + \langle \mathbf{w}_V, \mathbf{r} \rangle u^2 + \tau_3 \cdot u^3 + \tau_4 \cdot u^4 + \tau_5 \cdot u^5 + \tau_6 \cdot u^6$$

$$\alpha_u = \alpha u + \gamma u^2 + \beta u^3$$

and sends $(\mathbf{l}_u, \mathbf{r}_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

- Verifier performs checks:

$$[u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle +$$

$$u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle$$

$$\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B$$

$$[t_u]G + [\tau_u]B \stackrel{?}{=} [u]T_1 + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G) +$$

$$[u^3]T_3 + [u^4]T_4 + [u^5]T_5 + [u^6]T_6$$

$$t_u \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle$$

Remark. As in the *range proof* protocol case the last two steps of Π_{sat} could be substituted with an inner-product argument Π_{ip} to provide logarithmic size-proof with the following steps:

- After $\mathcal{P}$ evaluates polynomials at u he sends (t_u, α_u, τ_u) to $\mathcal{V}$ and

computes commitment:

$$P = \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle$$

- $\mathcal{V}$ performs check:

$$[t_u]G + [\tau_u]B \stackrel{?}{=} \sum_{k=1, k \neq 2}^6 [u^k]T_k + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G)$$

halts if it fails and reconstructs commitment P otherwise:

$$P = [u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle + u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle - [\alpha_u]B$$

- Parties run inner-product argument Π_{ip} on $(\mathbf{G}, \mathbf{y}^{-n} \circ \mathbf{H}, P, t_u; \mathbf{l}_u, \mathbf{r}_u)$

Theorem 18.13. The arithmetic circuit satisfiability protocol Π_{sat} has perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge

Proof idea. *Perfect completeness*

- **Polynomial correctness check:**

$$\begin{aligned} LHS &= [u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle + \\ &\quad u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle \\ &= u \cdot \langle \mathbf{a}_L, \mathbf{G} \rangle + u \cdot \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha u]B + u^2 \cdot \langle \mathbf{a}_O, \mathbf{G} \rangle + [\gamma u^2]B + \\ &\quad + u^3 \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + u^3 \cdot \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta u^3]B - \langle \mathbf{1}, \mathbf{H} \rangle + \\ &\quad + u \cdot \langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + u \cdot \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle \\ RHS &= \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B \\ &= u^3 \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + u^2 \cdot \langle \mathbf{a}_O, \mathbf{G} \rangle + u \cdot \langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{G} \rangle + \\ &\quad + u^3 \cdot \langle \mathbf{y}^n \circ \mathbf{s}_R, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + u \cdot \langle \mathbf{y}^n \circ \mathbf{a}_R, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + u \cdot \langle \mathbf{w}_L, \mathbf{y}^n \circ \mathbf{H} \rangle \\ &\quad - \langle \mathbf{y}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + \langle \mathbf{w}_O, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha u + \gamma u^2 + \beta u^3]B \\ &= u^3 \cdot \langle \mathbf{s}_L, \mathbf{G} \rangle + u^2 \cdot \langle \mathbf{a}_O, \mathbf{G} \rangle + u \cdot \langle \mathbf{a}_L, \mathbf{G} \rangle + u \cdot \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{G} \rangle + \\ &\quad + u^3 \cdot \langle \mathbf{s}_R, \mathbf{H} \rangle + u \cdot \langle \mathbf{a}_R, \mathbf{H} \rangle + u \cdot \langle \mathbf{w}_L, \mathbf{y}^n \circ \mathbf{H} \rangle \\ &\quad - \langle \mathbf{1}^n, \mathbf{H} \rangle + \langle \mathbf{w}_O, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha u]B + [\gamma u^2]B + [\beta u^3]B \end{aligned}$$

As we see $LHS = RHS$ so the check holds.

- t_2 correctness check:

$$\begin{aligned} [t_u]G + [\tau_u]B &\stackrel{?}{=} [u]T_1 + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G) + \\ &\quad + [u^3]T_3 + [u^4]T_4 + [u^5]T_5 + [u^6]T_6 \end{aligned}$$

Note that

$$\begin{aligned}
\langle \mathbf{w}_V, \mathbf{V} \rangle &= \sum_{i=1}^m [w_{V,i}] V_i = \sum_{i=1}^n ([w_{i,k} \cdot v_i] G + [w_{V,i} \cdot r_i] B) \\
&= [\sum_{i=1}^m w_{V,i} \cdot v_i] G + [\sum_{i=1}^m w_{V,i} \cdot r_i] B \\
&= [\langle \mathbf{w}_V, \mathbf{v} \rangle] G + [\langle \mathbf{w}_V, \mathbf{r} \rangle] B
\end{aligned}$$

so simplifying the LHS we get:

$$\begin{aligned}
&[t_u]G + [\tau_u]B = \\
&[t_1 u + (w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z))u^2 + t_3 u^3 + t_4 u^4 + t_5 u^5 + t_6 u^6]G + \\
&[\tau_1 u + \langle \mathbf{w}_V, \mathbf{r} \rangle u^2 + \tau_3 u^3 + \tau_4 u^4 + \tau_5 u^5 + \tau_6 u^6]B = \\
&[u]T_1 + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G) + \\
&+ [u^3]T_3 + [u^4]T_4 + [u^5]T_5 + [u^6]T_6 = RHS \quad \square
\end{aligned}$$

Perfect honest-verifier zero-knowledge follows from the zero-knowledge construction of Π_{zkip} protocol as Verifier learns no information about $\mathbf{a}_L, \mathbf{a}_R, \mathbf{v}$.

Computational extended witness emulation implies building extractor that combines extractors for two subprotocols: $\mathcal{E}_{ip}$ extracts witness $(\mathbf{l}_u, \mathbf{r}_u)$ from Π_{ip} than the extractor built on top of $\mathcal{E}_{zkip}$ extracts high-level witness $(\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v})$ from Π_{zkip} $\square$

The proof size of **arithmetic circuit satisfiability** protocol is $2 \log_2 n + 8$ group $\mathbb{G}$ elements and 5 field $\mathbb{F}_p$ elements.

Usually the number of multipliers n is not a power of 2 so that the intermediate witness vectors $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O$ must be padded with zeroes to the next power of 2 in order to apply the *inner-product argument* efficiently.

Remark. The Π_{sat} protocol could be slightly modified to provide intermediate random challenges inside the circuit. For example it would allow proving *permutation check*: $\{a, b\} = \{c, d\} \Leftrightarrow (a - x) \cdot (b - x) = (c - x) \cdot (d - x)$ for some random challenge x .

Acknowledgements

This section was heavily inspired by:

- [dalek's crate bulletproofs](#)
- [bulletproofs whitepaper](#)
- [RareSkills explanation](#)

Part IV

Sum-Check-based Protocols and Lookup Checks

This section introduces a new class of zero-knowledge protocols which are based on the Sum-Check protocol. If previously we mostly worked over univariate polynomials, Sum-Check derivatives typically operate over multilinear polynomials in many variables. Turns out that such representation can give a competitive performance to protocols relying on high-degree univariate polynomials.

Besides Sum-Check-based approaches, we also introduce the notion of *lookup checks*, which is a quite recent development, allowing to significantly reduce the arithmetical circuit size for non-native problems for the a zero-knowledge protocol (such as hash function computation or zero-knowledge Machine Learning). While this is a standalone research direction, we still provide it after discussing Sum-Check protocol since historically one of the first lookup protocols (namely, logup) is in fact Sum-Check-based.

Specifically, we list sections of this chapter in [Table 5](#).

Section	Topic	Key Concepts
19	Sum-Check Protocol	Construction of Sum-Check protocol, its performance and soundness
20	GKR Protocol	Sum-Check-based SNARK over arithmetical circuits
22	Lookup Checks	Motivation for lookup checks; plookup and logup constructions
23	UltraGroth	Implementing lookup checks in Groth16

Table 5: Topics covered in Part IV

19 Sum-Check Protocol

19.1 Multivariate Polynomials

In the previous sections, we worked with univariate polynomials $\mathbb{F}[X]$, which are polynomials in a single variable. However, in a set of applications such as *Spartan*, we need to work with multivariate polynomials $\mathbb{F}[X_1, \dots, X_v]$ where v is the number of variables.

Definition 19.1. We define a *monomial* in v variables as a product of the form: $X_1^{\alpha_1} \dots X_v^{\alpha_v}$ where $\alpha_1, \dots, \alpha_v$ are non-negative integers. The *degree of a monomial* is defined as $\alpha_1 + \dots + \alpha_v$. **Multivariate polynomial** from the space $\mathbb{F}[X_1, \dots, X_v]$ is defined as a finite linear combination of such monomials where coefficients are from $\mathbb{F}$. The **degree** of a multivariate polynomial is defined as the maximum degree of its monomials.

Example 19.1. For example, $f(X_1, X_2, X_3) = X_1^3 + 3X_1^2X_2^2 + X_3^2 + X_3$ is a linear combination of monomials $\{X_1^3, X_1^2X_2^2, X_3^2, X_3\}$, thus it is a multivariate polynomial in three variables X_1, X_2 , and X_3 . The maximum degree has a monomial $X_1^2X_2^2$ which is 4, thus $\deg(f) = 4$.

However, in cryptography, we won't work with arbitrary multivariate polynomials, compared to the univariate case. What we need is a *multilinear polynomial*.

Definition 19.2. A **multilinear polynomial** $f(X_1, \dots, X_v)$ is a multivariate polynomial which is linear in each variable, meaning that each variable appears with degree at most 1. In other words, for each variable X_i , function $f(X_1, \dots, X_v)$ is a linear function for fixed values of all other variables $X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_v$:

$$\begin{aligned} f(X_1, \dots, X_v) &= \alpha_i(X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_v) \cdot X_i \\ &\quad + \beta_i(X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_v) \end{aligned}$$

Example 19.2. For example, $f(X_1, X_2, X_3) = X_1X_2 + 3X_1X_3 + X_2X_3$ is a multilinear polynomial in three variables X_1, X_2 , and X_3 . Note that it is linear in X_1 . Indeed, it holds

$$f(X_1, X_2, X_3) = \alpha_1(X_2, X_3)X_1 + \beta_1(X_2, X_3)$$

for $\alpha_1(X_2, X_3) = X_2 + 3X_3$ and $\beta_1(X_2, X_3) = X_2X_3$.

Similarly, $f(X_1, \dots, X_v) = \prod_{i=1}^v X_i$ is also a multilinear polynomial in v variables.

19.2 Multilinear Extension

Compared to univariate polynomials, multilinear polynomials can hold information with a much smaller degree. For example, if our interpolation domain is $\{0, 1, \dots, n-1\}$, then we need up to n^{th} degree univariate polynomial to interpolate the values over this domain. However, for multilinear polynomials, we can encode the same domain with only $\log n$ -variate multilinear polynomial. Now, the details.

Definition 19.3. Suppose we are given the function $f : \{0, 1\}^v \rightarrow \mathbb{F}$, mapping the value from the v -dimensional hypercube to the field $\mathbb{F}$. We can define the **extension** of f as a v -variate polynomial $\tilde{f} \in \mathbb{F}[X_1, \dots, X_v]$ which agrees with f on the points of the hypercube, meaning that for each $\mathbf{b} \in \{0, 1\}^v$ it holds that $f(\mathbf{b}) = \tilde{f}(\mathbf{b})$.

Example 19.3. A more intuitive way to think about the extension is that given 2^v points of the hypercube, we construct the “interpolation” function $\tilde{f}$ which agrees with the given values over the hypercube. For example, if $f(0, 0) = 1$, $f(0, 1) = 2$, $f(1, 0) = 2$, and $f(1, 1) = 3$, then the extension $\tilde{f}$ can be defined as $\tilde{f}(X_1, X_2) = X_1^2 + X_2^2 + 1$.

There are possibly very large number of possible extensions for a given function f . What we are interested in is the multilinear extension, which turns out to be unique.

Theorem 19.4. Any function over the v -dimensional hypercube $f : \{0, 1\}^v \rightarrow \mathbb{F}$ has a unique v -variate multilinear extension $\tilde{f} \in \mathbb{F}[X_1, \dots, X_v]$. It is defined using the *Lagrange interpolation of multilinear polynomials* formula as follows:

$$\tilde{f}(\mathbf{X}) = \sum_{\mathbf{b} \in \{0, 1\}^v} f(\mathbf{b}) \cdot \text{eq}(\mathbf{X}; \mathbf{b}),$$

where the set $\{\text{eq}(\mathbf{X}; \mathbf{b})\}_{\mathbf{b} \in \{0, 1\}^v}$ is referred to as *the set of multilinear Lagrange basis polynomials* over the set $\{0, 1\}^v$. Each $\text{eq}(\mathbf{X}; \mathbf{b})$ is defined as:

$$\text{eq}(\mathbf{X}; \mathbf{b}) \triangleq \prod_{i=1}^v \{X_i b_i + (1 - X_i)(1 - b_i)\}.$$

Proof. First, let us show that the provided $\tilde{f}$ is indeed a multilinear polynomial. Notice that each $\text{eq}(\mathbf{X}; \mathbf{b})$ is a multilinear polynomial, since it is a product of linear polynomials in each variable X_i . Thus, $\tilde{f}$ is a linear combination of multilinear polynomials, hence it is also a multilinear polynomial.

Why is it an extension? Notice that the formula for $\text{eq}(\mathbf{X}; \mathbf{b})$ has the following property: $\text{eq}(\mathbf{X}; \mathbf{b}) = 1$ if $\mathbf{X} = \mathbf{b}$, and 0 otherwise. Indeed, if $X_i \neq b_i$, then the term $X_i b_i + (1 - X_i)(1 - b_i)$ is equal to 0, and thus the whole product is equal to 0. If $X_i = b_i$ for each i , then each term is equal to 1 and thus the product is 1. Therefore, $\tilde{f}(\mathbf{b}')$ for each $\mathbf{b}' \in \{0, 1\}^v$ is equal to:

$$\tilde{f}(\mathbf{b}') = \sum_{\mathbf{b} \in \{0, 1\}^v} f(\mathbf{b}) \cdot \text{eq}(\mathbf{b}'; \mathbf{b}) = f(\mathbf{b}') \cdot \underbrace{\text{eq}(\mathbf{b}'; \mathbf{b}')}_{=1} + \sum_{\mathbf{b} \neq \mathbf{b}'} f(\mathbf{b}) \cdot \underbrace{\text{eq}(\mathbf{b}'; \mathbf{b})}_{=0} = f(\mathbf{b}').$$

Example 19.4. Suppose function $f : \{0, 1\}^2 \rightarrow \mathbb{F}_p$ for $p = 11$ is given by $f(0, 0) = 3$, $f(0, 1) = 4$, $f(1, 0) = 1$, and $f(1, 1) = 2$. We first build the multilinear Lagrange basis polynomials:

$$\begin{aligned} \text{eq}(X_1, X_2; (0, 0)) &= (1 - X_1)(1 - X_2), & \text{eq}(X_1, X_2; (0, 1)) &= (1 - X_1)X_2, \\ \text{eq}(X_1, X_2; (1, 0)) &= X_1(1 - X_2), & \text{eq}(X_1, X_2; (1, 1)) &= X_1X_2. \end{aligned}$$

The multilinear extension $\tilde{f}(X_1, X_2)$ is thus computed as:

$$\tilde{f}(X_1, X_2) = 3(1 - X_1)(1 - X_2) + 4(1 - X_1)X_2 + X_1(1 - X_2) + 2X_1X_2 = 3 - 2X_1 + X_2.$$

Now, the question is, how fast can we compute the multilinear extension $\tilde{f}$ given 2^v values of f ? Consider the following lemma.

Lemma 19.5. Fix some positive integer v and let $n = 2^v$. Given an input $f(\mathbf{b})$ for all $\mathbf{b} \in \{0, 1\}^v$ and a vector $\mathbf{r} = (r_1, \dots, r_v) \in \mathbb{F}^{\log n}$, one can compute $\tilde{f}(\mathbf{r})$ in $\mathcal{O}(n)$ time and space.

19.3 The Sum-Check Protocol

19.3.1 Protocol Description

Suppose we are given the v -variate polynomial (possibly non-multilinear) $f : \{0, 1\}^v \rightarrow \mathbb{F}$ over a finite field $\mathbb{F}$. The main goal of the Sum-Check protocol is to convince the verifier $\mathcal{V}$ that

$$\sum_{b_1 \in \{0, 1\}} \sum_{b_2 \in \{0, 1\}} \cdots \sum_{b_v \in \{0, 1\}} f(b_1, \dots, b_v) = H$$

for the given value $H \in \mathbb{F}$. In other words, we can convince that the sum of all the values of f over the hypercube $\{0, 1\}^v$ (which we further write as $\sum_{\mathbf{b} \in \{0, 1\}^v} f(\mathbf{b})$ for short) is equal to the given value H . Such check might be useless at first glance, but as it turns out many protocols can be reduced to the Sum-Check protocol, similarly how any NP statement can be encoded as a high-degree univariate polynomial check (see the lecture on QAP).

Why can't the verifier just compute the sum? Typically, v is a fairly large number. According to the [Definition 19.5](#), the verifier can compute the

multilinear extension $\tilde{f}$ in $\mathcal{O}(2^v)$ time and space, which is infeasible for large v . The Sum-Check protocol allows the verifier to check the sum in up to $\mathcal{O}(v^2)$ time and space, which is much more adequate for large v .

How does it work? First, the prover $\mathcal{P}$ sends the value $C_1 \in \mathbb{F}$, which he claims to be the value of the sum (that is, H). The protocol proceeds in v rounds. For each round j , define the following univariate polynomial in the variable X_j :

$$f_j(X_j) = \sum_{(b_{j+1}, \dots, b_v) \in \{0,1\}^{v-j}} f(r_1, \dots, r_{j-1}, X_j, b_{j+1}, \dots, b_v),$$

where values $r_1, \dots, r_{j-1} \in \mathbb{F}$ are fixed values of the variables $X_1, \dots, X_{j-1}$ (which are randomnesses selected during previous rounds).

Consider the first round, when $j = 1$. In such case, according to the definition of $f_j(X_j)$, the prover $\mathcal{P}$ computes the univariate polynomial $f_1(X_1) = \sum_{(b_2, \dots, b_v) \in \{0,1\}^{v-1}} f(X_1, b_2, \dots, b_v)$ and sends the *claimed* polynomial $s_1(X_1)$ as the first round message. How can the verifier $\mathcal{V}$ be sure that $s_1(X_1)$ is indeed the univariate polynomial $f_1(X_1)$? Since $f_1(0) + f_1(1) = H$, the verifier can check that $s_1(0) + s_1(1) = C_1$.

However, this is not enough: there are many univariate polynomials that satisfy the condition $s_1(0) + s_1(1) = C_1$. For that reason, we are going to apply the Schwartz-Zippel lemma, which states that as long as $|\mathbb{F}| \gg \deg f_1$, it is safe to check the equality $s_1(r_1) = f_1(r_1)$ at a random point $r_1 \leftarrow \mathbb{F}$. In such case, the soundness of such check is $1 - \deg f_1 / |\mathbb{F}|$.

However, can the verifier evaluate both $s_1(r_1)$ and $f_1(r_1)$ effectively to verify the equality? Good news is that $s_1(r_1)$ can be evaluated efficiently: in fact, in $\mathcal{O}(\deg s_1)$ where typically degree is small (in case of multilinear polynomials, the degree is at most 1). But what about $f_1(r_1)$? Here is the bad news: $f_1(r_1)$ is a sum of 2^{v-1} terms, and thus it cannot be computed efficiently. Another good news is that we do not need to! The idea of the sumcheck protocol is to reduce the computation of $f_1(r_1)$ to the computation of f_2 , then f_3 etc. until the computation is trivial.

For concrete example, consider the second round. Now, the prover computes the polynomial $f_2(X_2) = \sum_{(b_3, \dots, b_v) \in \{0,1\}^{v-2}} f(r_1, X_2, b_3, \dots, b_v)$ and sends the claimed polynomial $s_2(X_2)$. The verifier now checks that $s_2(0) + s_2(1) = s_1(r_1)$ and that $s_2(r_2) = f_2(r_2)$ at random point $r_2 \leftarrow \mathbb{F}$. Since computing $f_2(r_2)$ consists in adding up 2^{v-2} terms, it is still infeasible to compute it directly, but we reduced the problem $4\times$ already.

Recursive definition. For the j -th round, the prover computes $f_j(X_j)$ and sends the claimed polynomial $s_j(X_j)$. The verifier checks whether $s_j(0) + s_j(1) = s_{j-1}(r_{j-1})$. Additionally, the verifier rejects if $\deg s_j$ is too large (for example, if $\deg s_j > \deg_j f$ where $\deg_j f$ is a degree of the polynomial in variable X_j). During the last round, the prover sends the claimed value $s_v(r_v)$ and the verifier checks whether $s_v(r_v) = f(r_1, \dots, r_v)$ ¹⁵. If the check succeeds, then the verifier

¹⁵Here we assume that the verifier has the oracle access to the function f and can compute it at

accepts the claim that

$$\sum_{\mathbf{b} \in \{0,1\}^v} f(\mathbf{b}) = H.$$

One might ask: how secure is the Sum-Check protocol? Consider the following lemma.

Lemma 19.6. Let $f \in \mathbb{F}[X_1, \dots, X_v]$ be a multivariate polynomial of degree at most d in each variable, defined over the finite field $\mathbb{F}$. For any given $H \in \mathbb{F}$, let $\mathcal{L}$ be the language of all polynomials f (given as an oracle) such that

$$H = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_v \in \{0,1\}} f(b_1, \dots, b_v).$$

The sumcheck protocol is an IOP for $\mathcal{L}$ with the completeness error $\delta_C = 0$ and the soundness error $\delta_S \leq vd/|\mathbb{F}|$.

Finally, consider the efficiency of the Sum-Check protocol.

Lemma 19.7. Suppose f is given according to the representation in Lemma 19.6. Then, the following holds:

- Communication consists of $\mathcal{O}(dv)$ field elements.
- The verifier $\mathcal{V}$ runs in $\mathcal{O}(vd) + T$ where T is the cost of the oracle access to f .
- The prover $\mathcal{P}$ runs in $\mathcal{O}(2^v T)$.

19.3.2 Sum-Check Protocol Implementation

Let us implement this algorithm in SageMath! First, define the multivariate polynomial ring and the function $f : \{0,1\}^v \rightarrow \mathbb{F}$. As an example, we will use $v = 10$ while f will be sampled randomly from $\mathbb{F}_p[X_1, \dots, X_v]$. As a prime field, we use $p = 2^{31} - 1$. Here is the code that sets everything up:

```
# Defining the finite field GF(p) where p is a large prime
p = (1<<31) - 1
Fp = GF(p)

# Defining the multivariate polynomial ring over the finite field GF(p)
v = 10 # Degree of the polynomial
variable_names = [f'x{i}' for i in range(v)]
R = PolynomialRing(Fp, names=variable_names)
variables = R.gens()
```

any randomly selected point.

Now, let us define f and find the corresponding value $H = \sum_{b \in \{0,1\}^v} f(b)$. For that, we simply take the random element of the polynomial ring R and manually add up all 2^v values over the boolean hypercube $\{0,1\}^v$.

```
def boolean_hypercube_sum(f: PolynomialRing) -> Fp:
    """
    Computes the sum of the polynomial f over the boolean hypercube {0,1}^v.
    The boolean hypercube is represented by evaluating the polynomial at all
    combinations of 0 and 1 for each variable.
    """

    total_sum = Fp(0)
    for i in range(1<<v):
        # Convert i to a binary representation of length v
        binary_representation = [(i >> j) & 1 for j in range(v)]
        # Evaluate the polynomial at this point
        point_value = f(*binary_representation)
        total_sum += point_value
    return total_sum

f = R.random_element(degree=v) # Random polynomial
H = boolean_hypercube_sum(f)
```

The SageMath generated the following f and H :

$$f(X_1, \dots, X_{10}) = 226921892X_2^3X_3X_4^2X_5X_6X_8^2 + 274566947X_0X_1^2X_2^2X_4X_7^2X_8X_9 \\ + 850030718X_1^2X_4X_6^3X_7^2X_8X_9 + 560601732X_0X_3X_5X_6X_7X_8X_9^4 - 39 \\ H = \sum_{b \in \{0,1\}^{10}} f(b) = 1053620759.$$

Now, we are going to implement the prover algorithm. For that, we need to define the univariate polynomial $f_j(X_j)$ for each round j and compute the claimed polynomial $s_j(X_j)$ based on previously sampled randomness values $r_1, \dots, r_{j-1}$. Since our protocol is a *public-coin* protocol, we can apply the *Fiat-Shamir transform* to make the randomness values $r_1, \dots, r_v$ deterministically computable by the prover (thus making the protocol non-interactive). For that, we will compute the challenge r_j as $\mathcal{H}(s_1, s_2, \dots, s_{j-1})$ where $\mathcal{H} : \{0,1\}^* \rightarrow \mathbb{F}_p$ is a cryptographic hash function (SHA256 in our case). That said, we define $\mathcal{H}$ as follows:

```
def fiat_shamir_sample(input_string: str) -> Fp:
    """
    Samples a random value from the finite field GF(p) using the Fiat-Shamir h
    The input string is hashed to produce a random value.
    """

    fs_hash = hashlib.sha256(input_string.encode('utf-8')).hexdigest()
```

```
return Fp(int(fs_hash, 16) % p)
```

Finally, the verifier logic is even more simple: the verifier checks that (a) randomnesses $\{r_j\}_{j \in \{1, \dots, 10\}}$ are sampled correctly, (b) the claimed polynomials $s_j(X_j)$ satisfy the conditions $s_j(0) + s_j(1) = s_{j-1}(r_{j-1})$ for each $j \in \{1, \dots, 10\}$ (instead of s_0 , we simply use the claimed value H), and (c) the final check $s_v(r_v) = f(r_1, \dots, r_v)$ holds. Additionally, we should check the degrees of each s_j , but we omit this check for simplicity. Here is the implementation of the prover and verifier:

```
class SumCheckProtocol:
    """
    Prover and Verifier for the Sum Check protocol.
    """

    def __init__(
        self,
        polynomial: PolynomialRing,
        claimed_sum: Fp
    ) -> None:
        """
        Initializes the SumCheck protocol with a polynomial and the claimed sum.

        Args:
            polynomial (PolynomialRing): The polynomial to be checked.
            claimed_sum (Fp): The sum claimed by the prover over the boolean hy

        self.f = polynomial
        self.H = claimed_sum

    @staticmethod
    def fiat_shamir_from_polynomials(polynomials: list[PolynomialRing]) -> Fp:
        """
        Samples a random value from the finite field GF(p) using the Fiat-Shamir
        based on the provided polynomials.

        Args:
            polynomials (list[PolynomialRing]): List of polynomials to be used

        Returns:
            Fp: A random value from the finite field GF(p).
        """

        fs_input = ''.join(str(poly) for poly in polynomials)
        return fiat_shamir_sample(fs_input)
```

```

def prove(self) -> dict:
    """
    Prover generates the transcript of the polynomial and the claimed sum.

    Returns:
        dict: A transcript containing the polynomial, claimed sum, random
    """

    # Initialize the transcript representing the
    # interaction between the prover and verifier.
    transcript = {
        'polynomial': self.f,
        'claimed_sum': self.H,
        'random_values': [],
        'polynomials': []
    }

    for j in range(v):
        # The protocol consists of v rounds.
        # Finding polynomial s_j that represents the sum of f over the booleans
        Rj = PolynomialRing(Fp, 'x')
        x = Rj.gen()
        s_j = Rj.zero()

        for i in range(2**(v-j-1)):
            # Convert i to a binary representation of length v-j-1
            binary_representation = [(i >> k) & 1 for k in range(v-j-1)]
            # Evaluate the polynomial at this point
            point_value = self.f(*transcript['random_values'][:j], variables[j])
            univariate_poly = point_value.subs({variables[j]: x})
            s_j += univariate_poly

        # Append the polynomial s_j to the transcript
        transcript['polynomials'].append(s_j)

        # Sample the random value using Fiat-Shamir heuristic
        random_value = SumCheckProtocol.fiat_shamir_from_polynomials(
            polynomials=transcript['polynomials']
        )
        transcript['random_values'].append(random_value)

    return transcript

def verify(self, transcript: dict) -> bool:
    """
    Verifier checks the validity of the transcript.

```


Args:

transcript (dict): The transcript generated by the prover.

Returns:

bool: True if the claimed sum is valid, False otherwise.

"""

```
# Decipher the transcript
H = transcript['claimed_sum']
f = transcript['polynomial']
random_values = transcript['random_values']
polynomials = transcript['polynomials']

# Assert that random_values is formed correctly using Fiat-Shamir heuristic
if len(random_values) != v:
    print("Invalid number of random values in the transcript.")
    return False
for i in range(v):
    if random_values[i] != self.fiat_shamir_from_polynomials(polynomials, i):
        print(f"Random value at index {i} does not match the Fiat-Shamir output.")
        return False

# Assert that each s_r(0) + s_r(1) matches the claimed sum H
for j in range(v):
    # During the first round, simply check that s_1(0) + s_1(1) = H
    if j == 0:
        s_1 = polynomials[0]
        if s_1(0) + s_1(1) != H:
            print(f"First round sum {s_1(0) + s_1(1)} does not match the claimed sum {H}.")
            return False
        continue

    # For subsequent rounds, check that the sum of the polynomials matches
    s, s_previous = polynomials[j], polynomials[j-1]
    if s(0) + s(1) != s_previous(random_values[j-1]):
        print(f"Round {j} sum {s(0) + s(1)} does not match the previous round's output {s_previous(random_values[j-1])}.")
        return False

# Final round checks whether f(r) = s_v(r_v)
last_polynomial = polynomials[-1]
last_random = random_values[-1]

if last_polynomial(last_random) != f(*random_values):
    print(f"Final check failed: {last_polynomial(last_random)} != {f(*random_values)}")
    return False

return True
```

```

protocol = SumCheckProtocol(f, H)
transcript = protocol.prove()
print(f'Prover has the transcript: {transcript}')
print(f'Verifier checks the proof: {protocol.verify(transcript)}')

```

The example transcript is the following:

$$\begin{aligned}
s_1(X) &= 763349684X^2 + 238898491X + 25686292, & r_1 &= 493136960 \\
s_2(X) &= 1430156880X^2 + 361631142, & r_2 &= 2831006 \\
s_3(X) &= 658473518X^3 + 208437747X^2 + 1648222287, & r_3 &= 321757611 \\
s_4(X) &= 111234379X + 1501686780, & r_4 &= 1835658 \\
s_5(X) &= 2135686754X^2 + 1501686780X + 1698421434, & r_5 &= 1970078146 \\
s_6(X) &= 53232575X + 259099570, & r_6 &= 1616339175 \\
s_7(X) &= 669263002X^3 + 514137765X + 1942401931, & r_7 &= 887816643 \\
s_8(X) &= 1524551309X^2 + 1551518026X + 55160592, & r_8 &= 421872749 \\
s_9(X) &= 1766229428X^2 + 826393776X + 1291949229, & r_9 &= 1169032581 \\
s_{10}(X) &= 2030644729X^4 + 1291949229X^3 + 560585988X + 1720313399, \\
r_{10} &= 1461328437
\end{aligned}$$

19.4 Sumcheck Applications

As of now, the Sum-Check protocol seems too abstract and not very useful: why do we even need to sum some random multivariate polynomial over the boolean hypercube? In this section, we provide some (rather theoretic) applications of the Sum-Check protocol, but as we go further, it will become more clear why it is so important.

19.4.1 The #SAT Problem

Let $\mathcal{C} : \{0, 1\}^\ell \rightarrow \{0, 1\}$ be any boolean formula of size $S = \mathcal{O}(\text{poly}(\ell))$. In the #SAT problem, the goal is to compute the number of satisfying (boolean) assignments of $\mathcal{C}$, that is, find the value $H = \sum_{b \in \{0, 1\}^\ell} \mathcal{C}(b)$. Such problem is believed to be very difficult with the fastest known algorithm to still run in the exponential time: that is, no much better than brute-force the formula in time $\mathcal{O}(2^\ell S)$. Even determining whether $H > 0$ is widely believed to be NP-hard. But suppose we have a prover $\mathcal{P}$ who does know H and wants to prove any verifier $\mathcal{V}$ that H was computed correctly in the polynomial time.

Now, similarly to how it is done in R1CS, we *arithmetize* the circuit $\mathcal{C}$ to be computable over the finite field $\mathbb{F}$ (let us call it $\tilde{\mathcal{C}} : \mathbb{F}^\ell \rightarrow \mathbb{F}$). Here how it goes:

- Instead of the gate $x \wedge y$, we use the multiplication $x \cdot y$.
- Instead of the gate $x \vee y$, we use the addition $x + y - x \cdot y$.

- Instead of the gate $\neg x$, we use the subtraction $1 - x$.
- Instead of the gate $x \oplus y$, we use the addition $x + y - 2xy$.

As an example, suppose we have the circuit:

$$C(x_1, x_2, x_3, x_4) = (\neg x_1 \wedge x_2) \wedge (x_3 \vee x_4)$$

The extension $\tilde{C}$ of the circuit C is

$$\tilde{C}(X_1, X_2, X_3, X_4) = (1 - X_1)X_2(X_3 + X_4 - X_3X_4).$$

It is fairly easy to see that $\tilde{C}(\mathbf{b}) = C(\mathbf{b})$ for all $\mathbf{b} \in \{0, 1\}^\ell$ with $\ell = 4$. Then, we can apply the sum-check protocol over $\tilde{C}$ to prove that H was computed correctly. In such case, the prover runs in $\mathcal{O}(S^2 2^\ell)$ time, while the verifier in time $\mathcal{O}(S)$.

19.4.2 Matrix Multiplication Verification (MatMul Check-Sum Protocol)

Now, here is the application which gets much more interesting. Consider two matrices $A, B \in \mathbb{F}^{n \times n}$ and the goal is to verify that the product $C = A \cdot B$ is computed correctly. The naive way is to make verifier compute the product, taking $\mathcal{O}(n^3)$ time, and then compare the result with the claimed value C . However, it is possible to verify the correctness of the product in $\mathcal{O}(n^2)$ time and space using the *Freivalds'* protocol by the simple observation that we can sample a random challenge $\alpha \leftarrow \mathbb{F}^n$ and verify that $A(B\alpha) = C\alpha$ (idea is pretty similar to the Schwartz-Zippel lemma where we check the polynomial equation at a random point). However, is there any better way to do that? Sumcheck protocol can help to keep the same asymptotics, but do not reveal the whole matrices A, B to the verifier.

Protocol. Here is where *multilinear extension* comes into play. Instead of perceiving matrices A, B, C as n^2 field elements, we perceive them as functions $f_A, f_B, f_C : \{0, 1\}^{\log n} \times \{0, 1\}^{\log n} \rightarrow \mathbb{F}$, mapping two indices of the matrix to the corresponding value. This way, for instance:

$$f_A(\mathbf{i}, \mathbf{j}) = A_{\mathbf{i}, \mathbf{j}}, \quad \text{where } \mathbf{i} = (i_1, \dots, i_{\log n}), \mathbf{j} = (j_1, \dots, j_{\log n}).$$

Denote by $\tilde{f}_A, \tilde{f}_B, \tilde{f}_C : \mathbb{F}^{\log n} \times \mathbb{F}^{\log n} \rightarrow \mathbb{F}$ the multilinear extensions of the functions f_A, f_B, f_C . Now, how to reduce the seemingly difficult check $C = AB$ into the Sum-Check protocol? Consider the following lemma.

Lemma 19.8. $\tilde{f}_C(\mathbf{x}, \mathbf{y}) = \sum_{\mathbf{b} \in \{0, 1\}^{\log n}} \tilde{f}_A(\mathbf{x}, \mathbf{b}) \tilde{f}_B(\mathbf{b}, \mathbf{y}).$

Proof. Obviously, both left and right-hand sides are multilinear polynomials in $\mathbf{x}$ and $\mathbf{y}$. Since multilinear extension of C is unique, it suffices to check that the equality holds for all boolean assignments $\mathbf{i}, \mathbf{j} \in \{0, 1\}^{\log n}$. Indeed, we have $\tilde{f}_C(\mathbf{i}, \mathbf{j}) = \sum_{\mathbf{b} \in \{0, 1\}^{\log n}} \tilde{f}_A(\mathbf{i}, \mathbf{b}) \tilde{f}_B(\mathbf{b}, \mathbf{j})$. But notice that this check literally

checks whether $C_{i,j} = \sum_{b=1}^n A_{i,b}B_{b,j}$ which is exactly the definition of matrix multiplication!

Now, the interactive proof is immediate: sample random $\mathbf{r}_1, \mathbf{r}_2 \leftarrow \mathbb{F}^{\log n}$, and apply the sum-check on $g(\mathbf{z}) := \tilde{f}_A(\mathbf{r}_1, \mathbf{z})\tilde{f}_B(\mathbf{z}, \mathbf{r}_2)$ to prove that it equals $\tilde{f}_C(\mathbf{r}_1, \mathbf{r}_2)$.

It can be shown that both the prover's and verifier's time is $\mathcal{O}(n^2)$, while the proof size is $\mathcal{O}(\log n)$ (compared to $\mathcal{O}(n^2)$ for the naive approach).

19.4.3 MatMul Sum-Check Protocol Implementation

Now, let us implement the matrix multiplication verification protocol in SageMath! For simplicity, we assume that the matrix has a size $n = 2^v$ so that MLE of matrices are of degree $2v$. Also, we will work over the small prime field $\mathbb{F}_{11}$ this time to make outputs somewhat more readable. So here we go:

```
# Defining the finite field GF(p) where p is a large prime
p = 11
Fp = GF(p)

# Assume for simplicity that matrices are of size 2^v x 2^v
v = 2 # Matrix of size 4x4
n = 1<<v

# Defining how MLE are computed
variable_names = [f'x{i}' for i in range(2*v)]
R = PolynomialRing(Fp, names=variable_names)
variables = R.gens()
```

Now, for concreteness, we initialize the matrices A, B and $C = AB$.

```
# Generate two random matrices A and B over the finite field GF(p)
A = Matrix(Fp, n, n, [Fp.random_element() for _ in range(n*n)])
B = Matrix(Fp, n, n, [Fp.random_element() for _ in range(n*n)])

# Find the product matrix C=A*B
C = A * B
print(f'Matrix A:\n{A}')
print(f'Matrix B:\n{B}')
print(f'Matrix C:\n{C}')
```

Here, we obtained the following matrices:

$$A = \begin{bmatrix} 1 & 1 & 3 & 10 \\ 5 & 6 & 4 & 5 \\ 1 & 8 & 3 & 4 \\ 6 & 2 & 2 & 7 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 7 & 9 & 5 \\ 5 & 10 & 0 & 10 \\ 5 & 0 & 2 & 3 \\ 2 & 6 & 6 & 6 \end{bmatrix}, \quad C = \begin{bmatrix} 9 & 0 & 9 & 7 \\ 4 & 4 & 6 & 6 \\ 10 & 1 & 6 & 8 \\ 2 & 5 & 1 & 10 \end{bmatrix}$$

260

Now, we need to build the multilinear extensions $\tilde{f}_A, \tilde{f}_B, \tilde{f}_C : \mathbb{F}^{2 \times 2} \rightarrow \mathbb{F}$. To achieve that, we first write down the function to compute the MLE given the set of values $\{(b, f(b))\}_{b \in \{0,1\}^\ell}$ given interpolation formula from Theorem 19.4.

```
def mle_from_hypercube(hypercube: list) -> R:
    """
    Computes the Multivariate Linear Extension (MLE) of a hypercube.
    The hypercube is represented as a list of tuples, where each tuple
    contains the coordinates of a point in the hypercube.

    Args:
        hypercube (list): A list of tuples representing points in the hypercube.
                           Each tuple should have length equal to the dimension.
    """

    # Create a polynomial for each point in the hypercube
    mle = R.zero()
    for point, value in hypercube:
        eq_poly = R(1)
        for i, bit in enumerate(point):
            eq_poly *= bit*variables[i] + (1-bit)*(1-variables[i])

        mle += eq_poly * value

    return mle
```

We can verify the correctness of this function by running the Example in the first section. Indeed:

```
print('Example MLE:', mle_from_hypercube([
    ((0, 0), Fp(3)),
    ((0, 1), Fp(4)),
    ((1, 0), Fp(1)),
    ((1, 1), Fp(2))
])) # Output: -2*x0 + x1 + 3
```

At this point, we can finally derive the MLEs. We simply iterate through all indices (i, j) , bit-decompose them to get bit-vectors $\mathbf{i} = (i_1, i_2)$ and $\mathbf{j} = (j_1, j_2)$, and then interpolate the MLEs: for matrix A , for instance, we have $\{(i_1, i_2, j_1, j_2), A_{i,j}\}_{i,j \in [n]^2}$.

```
def mle_from_matrix(matrix: Matrix) -> R:
    """
    Computes the Multivariate Linear Extension (MLE) of a matrix.
    The MLE is a polynomial that represents the matrix entries as variables.
    """
```

```

assert matrix.nrows() == matrix.ncols(), "Matrix must be square."
assert matrix.nrows() == n, "Matrix size must be 1<<v"

# Range over all indices, bit-decompose them, and build the mle
hypercube = []
for i in range(n):
    for j in range(n):
        # Convert i and j to binary representation of length v
        point = [(i >> k) & 1 for k in range(v)] + [(j >> k) & 1 for k in range(v)]
        hypercube.append((tuple(point), matrix[i, j]))

return mle_from_hypercube(hypercube)

```

Now we find the MLEs of matrices A , B , and C :

```

A_mle = mle_from_matrix(A)
B_mle = mle_from_matrix(B)
C_mle = mle_from_matrix(C)
print(f'MLE of A: \n{A_mle}')
print(f'MLE of B: \n{B_mle}')
print(f'MLE of C: \n{C_mle}')

```

The results are the following:

$$\begin{aligned}
 \tilde{f}_A(X_1, X_2, Y_1, Y_2) &= -X_1X_2Y_1 - 3X_1X_2Y_2 + 4X_1Y_1Y_2 - 2X_2Y_1Y_2 \\
 &\quad + X_1X_2 + X_1Y_1 - 4X_2Y_1 - 3X_1Y_2 - 4Y_1Y_2 \\
 &\quad + 4X_1 + 2Y_2 + 1 \\
 \tilde{f}_B(X_1, X_2, Y_1, Y_2) &= -2X_1X_2Y_1Y_2 - 2X_1X_2Y_1 - 3X_1X_2Y_2 + 3X_1Y_1Y_2 \\
 &\quad + 4X_2Y_1Y_2 + 5X_1X_2 + X_2Y_1 - X_1Y_2 + X_2Y_2 \\
 &\quad + 2Y_1Y_2 + 3X_1 + 3X_2 + 5Y_1 - 4Y_2 + 2 \\
 \tilde{f}_C(X_1, X_2, Y_1, Y_2) &= 2X_1X_2Y_1Y_2 + 3X_1X_2Y_1 + X_1X_2Y_2 + 4X_1Y_1Y_2 \\
 &\quad + 4X_2Y_1Y_2 - 3X_1X_2 - 2X_1Y_1 + 2X_1Y_2 - 4X_2Y_2 \\
 &\quad - 4Y_1Y_2 - 5X_1 + X_2 + 2Y_1 - 2
 \end{aligned}$$

One can easily check that they indeed encode the matrices A , B , and C as expected. For example, it is trivial to check the upper left element in all three cases:

$$\tilde{f}_A(0, 0, 0, 0) = 1, \quad \tilde{f}_B(0, 0, 0, 0) = 2, \quad \tilde{f}_C(0, 0, 0, 0) = 9.$$

Now, we can implement the Sum-Check protocol to verify that $C = AB$ holds. We first sample randomnesses $\mathbf{r}_1, \mathbf{r}_2 \leftarrow \mathbb{F}_{11}^2$:

Now, we are going to apply the sumcheck protocol. First,

```
# sample two random vectors r1 and r2 of size v = log(n)
r1 = [Fp.random_element() for _ in range(v)]
r2 = [Fp.random_element() for _ in range(v)]
```

Then, we define the function $g(\mathbf{z}) = \tilde{f}_A(\mathbf{r}_1, \mathbf{z})\tilde{f}_B(\mathbf{z}, \mathbf{r}_2)$ and apply the Sum-Check protocol to prove that it equals $f_C(\mathbf{r}_1, \mathbf{r}_2)$. For that, we first define the function g :

```
claimed_sum = C_mle*(r1 + r2))
g = A_mle.subs({
    variables[i]: r1[i] for i in range(v)
}).subs({
    variables[v+i]: variables[i] for i in range(v)
}) * B_mle.subs({
    variables[i+v]: r2[i] for i in range(v)
})
```

Turns out that the claimed sum is $H = 6$ while the polynomial is:

$$g(X_1, X_2) = 5X_1^2X_2^2 - 5X_1^2X_2 + 3X_1X_2^2 + 3X_1^2 + 4X_1X_2 - 3X_2^2 + 4X_1 - 3X_2 + 2$$

So the only thing left is to run the already implemented Sum-Check protocol. Here how it goes:

```
# Defining a smaller polynomial ring for the protocol (since g
# is polynomial in v variables instead of 2*v)
Q = PolynomialRing(Fp, names=variable_names)
variables = Q.gens()
g = Q(g)

# Running the sum-check protocol
protocol = SumCheckProtocol(Fp, Q, g, claimed_sum, degree=v)
transcript = protocol.prove()
print_transcript(transcript)
verification_result = protocol.verify(transcript)
print(f'Verification result: {verification_result}')
```

Our interaction consisted of only two polynomials (since $v = 2$):

$$\begin{aligned} s_1(X) &= 6X^2 + 4X + 9, & r_1 &= 4 \\ s_2(X) &= X^2 + 10X, & r_2 &= 2 \end{aligned}$$

The final verification result is True, meaning prover has successfully proved that $C = AB$ holds.

20 GKR Protocol

20.1 Motivation

Previous discussion, while giving certain applications of the Sum-Check protocol, still provides quite “artificial” constructions not directly applicable in the real-world use-cases. For instance, the #SAT problem is not very practical as the prover $\mathcal{P}$ cannot even compute the value H in polynomial time. Ideally, we would like to have a protocol that allows the prover to compute the value H in the polynomial time, while the verifier $\mathcal{V}$ should be able to verify the correctness of the claimed value H in the logarithmic time.

Goldwasser, Kalai, and Rothblum (GKR) described a protocol which solves exactly this issue over the arithmetical circuits, which we solved using QAP $\rightarrow$ NILP reduction in Groth16. Here we take the Sum-Check approach.

Suppose we are given the *layered* arithmetical circuit $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ of size S (number of gates). The *layered* here means that the circuit C can be decomposed into d layers (note that GKR can be generalized to the unstructured arithmetical circuits as well). The GKR protocol allows to achieve the following performance:

- The communication consists of $\mathcal{O}(d \cdot \text{polylog}(S))$ field elements.
- The verifier runs in $\mathcal{O}(n + d \cdot \text{polylog}(S))$ time.
- The prover runs in $\mathcal{O}(\text{poly}(S))$ time.
- The soundness error is just $\mathcal{O}(d \log(S)/|\mathbb{F}|)$.

20.2 Protocol Description

20.2.1 Circuit Representation

Again, assume we are given the circuit $C : \mathbb{F}^n \rightarrow \mathbb{F}^m$ of size S , depth d , and fan-in two (i.e., each gate has at most two inputs, but might have multiple outputs). Let us number the layers of the circuit from 0 to d where 0 denotes the output layer while d is the input layer (notice that the layers are numbered in the reverse order compared to the usual circuit notation). Assume the number of gates in the layer i is S_i and without the loss of generality, we assume that S_i is the power of two: $S_i = 2^{v_i}$. Now, we are going to introduce certain functions to further build up the Sum-Check.

Gates Encodings. Suppose $W_i : \{0, 1\}^{v_i} \rightarrow \mathbb{F}$ is structured so that it outputs the value of the i -th layer gate given the gate label. As usual, we denote the multilinear extension of W_i as $\widehat{W}_i : \mathbb{F}^{v_i} \rightarrow \mathbb{F}$.

Wiring Predicates. We introduce the wiring predicates $\text{in}_{1,i}, \text{in}_{2,i} : \{0, 1\}^{v_i} \rightarrow \{0, 1\}^{v_{i+1}}$ which indicate which pairs of wiring are connected to the i -th layer gate from the layer $i + 1$. It takes the label of the gate in the layer i (say, a) and the function $\text{in}_{1,i}(a)$ outputs the label of the first input connected to the gate a in the layer $i + 1$, while $\text{in}_{2,i}(a)$ outputs the label of the second input connected to the gate a in the layer $i + 1$.

Operations Encodings. Define two functions: $\text{add}, \text{mul} : \{0, 1\}^{v_i+2v_{i+1}} \rightarrow \{0, 1\}$ which take three gate labels (say, (a, b, c)) and return 1 if and only if $(b, c) = (\text{in}_{1,i}(a), \text{in}_{2,i}(a))$ (that is, the gate a in the layer i is connected to the gates b and c in the layer $i + 1$) and a is an addition gate. Similarly, $\text{mul}(a, b, c)$ returns 1 if and only if $(b, c) = (\text{in}_{1,i}(a), \text{in}_{2,i}(a))$ and a is a multiplication gate. We denote the MLEs of these functions as $\widetilde{\text{add}}$ and $\widetilde{\text{mul}} : \mathbb{F}^{v_i+2v_{i+1}} \rightarrow \mathbb{F}$, respectively.

Example. Consider the following circuit with $d = 3$ layers:

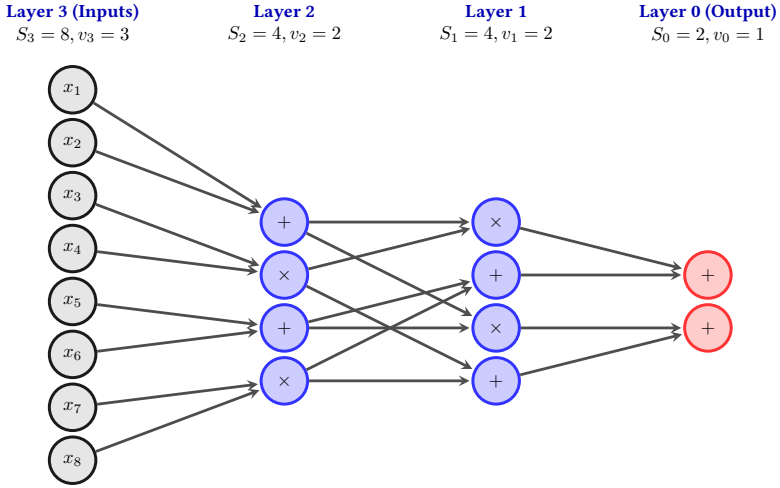


Figure 20.1: Example of a layered arithmetical circuit $C : \mathbb{F}^8 \rightarrow \mathbb{F}^2$ with $d = 3$ layers.

Let us assume that we have inputted the values $\mathbf{x}^{(3)} = (1, 2, 0, 1, 0, 2, 0, 1)$. Then, the value of the layer 2 is $\mathbf{x}^{(2)} = (3, 0, 2, 0)$ and the layer 1 is $\mathbf{x}^{(1)} = (0, 2, 6, 0)$. Finally, the output layer is $\mathbf{x}^{(0)} = (2, 6)$. Let us now see how to encode gates, wiring predicates, and operations encodings on this circuit. Consider $W_1 : \{0, 1\}^2 \rightarrow \mathbb{F}$. We have

$$W_1(0, 0) = 0, \quad W_1(0, 1) = 2, \quad W_1(1, 0) = 6, \quad W_1(1, 1) = 0.$$

Therefore, the multilinear extension of W_1 is $\widetilde{W}_1(X_1, X_2) = 2(1 - X_1)X_2 + 6X_1(1 - X_2)$.

Now, let us define the wiring predicates $\text{in}_{1,2}, \text{in}_{2,2} : \{0, 1\}^2 \rightarrow \{0, 1\}^3$. Consider the 2nd gate ($2 = (1, 0)$) in layer 2. It is connected to the 5th and 6th gates in layer 3 (the input layer). Since $5 = (1, 0, 1)$ and $6 = (1, 1, 0)$ in the binary form, we have

$$\text{in}_{1,2}(1, 0) = (1, 0, 1), \quad \text{in}_{2,2}(1, 0) = (1, 1, 0).$$

Now, consider the operation encodings add_2 and $\text{mul}_2 : \{0, 1\}^8 \rightarrow \{0, 1\}$. The addition gate is non-zero only on the following inputs:

$$((0, 0), (0, 0, 0), (0, 0, 1)), \quad ((1, 0), (1, 0, 0), (1, 0, 1)).$$

Therefore, the multilinear extension of add_2 is

$$\begin{aligned} \widetilde{\text{add}}_2(X_1, X_2, Y_1, Y_2, Y_3, Z_1, Z_2, Z_3) &= (1 - X_1)(1 - X_2)(1 - Y_1)(1 - Y_2) \\ &\quad \cdot (1 - Y_3)(1 - Z_1)(1 - Z_2)Z_3 \\ &\quad + X_1(1 - X_2)Y_1(1 - Y_2)(1 - Y_3)Z_1(1 - Z_2)Z_3 \end{aligned}$$

Similarly, the multiplication encoding mul_2 is non-zero only on the following inputs:

$$((0, 1), (0, 1, 0), (0, 1, 1)), \quad ((1, 1), (1, 1, 0), (1, 1, 1)),$$

The multilinear extension of mul_2 is

$$\begin{aligned} \widetilde{\text{mul}}_2(X_1, X_2, Y_1, Y_2, Y_3, Z_1, Z_2, Z_3) &= (1 - X_1)X_2(1 - Y_1)Y_2(1 - Y_3)(1 - Z_1)Z_2Z_3 \\ &\quad + X_1X_2Y_1Y_2(1 - Y_3)Z_1Z_2Z_3. \end{aligned}$$

Remark. Note that the operations encodings add_i and mul_i (and thus MLEs $\widetilde{\text{add}}_i$ and $\widetilde{\text{mul}}_i$) do not depend on the solution witness $\{\mathbf{x}^{(i)}\}_{i \in [d+1]}$, while the gates encodings W_i do depend.

20.2.2 Protocol Specification

The GKR protocol consists of d rounds, one for each layer of the circuit. In the i -th round, the prover $\mathcal{P}$ claims a value for $\widetilde{W}_i(\mathbf{r}_i)$ at the randomly selected point $\mathbf{r}_i \leftarrow \$ \mathbb{F}^{v_i}$.

At the start of the 1st round, this claim is made about the circuit outputs. Namely, if we have $S_0 = 2^{v_0}$ gates in the output layer, let $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$ denote the function which maps the gate label to the claimed output value, sent by the prover $\mathcal{P}$. The verifier picks a random point $\mathbf{r}_0 \leftarrow \$ \mathbb{F}^{v_0}$ and evaluates $\widetilde{D}(\mathbf{r}_0)$ in time $\mathcal{O}(S_0)$. By the Schwartz-Zippel lemma, if $\widetilde{D}(\mathbf{r}_0) = \widetilde{W}_0(\mathbf{r}_0)$, then with the soundness error of $v_0/|\mathbb{F}|^{v_0}$ the verifier is assured that these two polynomials are indeed equal. However, for obvious reasons, the verifier cannot compute $\widetilde{W}_0(\mathbf{r}_0)$ without the prover's help. So the core idea of GKR is to reduce the claim about the value of $\widetilde{W}_i(\mathbf{r}_i)$ to a claim about the value of $\widetilde{W}_{i+1}(\mathbf{r}_{i+1})$ for some randomly chosen $\mathbf{r}_{i+1} \leftarrow \$ \mathbb{F}^{v_{i+1}}$. How? Consider the following lemma.

Lemma 20.1. The following statement holds:

$$\begin{aligned} \widetilde{W}_i(\mathbf{z}) = & \sum_{\mathbf{b}, \mathbf{c} \in \{0,1\}^{v_{i+1}}} \left(\widetilde{\text{add}}_i(\mathbf{z}, \mathbf{a}, \mathbf{b})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) \right. \\ & \left. + \widetilde{\text{mul}}_i(\mathbf{z}, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}) \right). \end{aligned}$$

Proof. Since both sides are multilinear polynomials, it suffices to check that the equality holds for all boolean assignments $\mathbf{z} \in \{0,1\}^{v_i}$. Fix some concrete $\mathbf{z}_0 \in \{0,1\}^{v_i}$ and without the loss of generality, assume that $\mathbf{z}_0$ is the addition gate in the layer i (the case of multiplication gate is similar). In such case, all the terms $\widetilde{\text{mul}}_i(\mathbf{z}, \mathbf{b}, \mathbf{c})$ will be zero, so we can reduce the sum down to:

$$\widetilde{W}_i(\mathbf{z}_0) = \sum_{\mathbf{b}, \mathbf{c} \in \{0,1\}^{v_{i+1}}} \widetilde{\text{add}}_i(\mathbf{z}_0, \mathbf{a}, \mathbf{b})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c}))$$

Now, according to the definition of the $\widetilde{\text{add}}_i$ predicate, the $\widetilde{\text{add}}_i(\mathbf{z}_0, \mathbf{b}, \mathbf{c})$ is non-zero (equals to 1) only if $(\mathbf{b}, \mathbf{c}) = (\text{in}_{1,i}(\mathbf{z}_0), \text{in}_{2,i}(\mathbf{z}_0))$. Since we know that $\mathbf{z}_0$ is an addition gate, we can conclude that two such gates $\mathbf{b}$ and $\mathbf{c}$ exist. Therefore, we can rewrite the sum as:

$$\widetilde{W}_{i+1}(\text{in}_{1,i}(\mathbf{z}_0)) + \widetilde{W}_{i+1}(\text{in}_{2,i}(\mathbf{z}_0)) = \widetilde{W}_i(\mathbf{z}_0).$$

That said, to check the $\mathcal{P}$'s claim about $\widetilde{W}_i(\mathbf{r}_i)$, the verifier can apply the Sum-Check protocol on the function

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$

However, note that *the verifier $\mathcal{V}$ does not know the polynomial $\widetilde{W}_{i+1}$* . However, note that he does not need to know it until the last round, during which $\mathcal{V}$ has to make the oracle request $\mathcal{O}^{f_i}$ to compute the value of f_i at the randomly selected point $(\mathbf{b}^*, \mathbf{c}^*) \leftarrow \$_{\mathbb{F}^{2v_{i+1}}}$. Evaluating $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ requires evaluating $\widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$, $\widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}^*, \mathbf{c}^*)$, and $\widetilde{W}_{i+1}(\mathbf{b}^*)$ with $\widetilde{W}_{i+1}(\mathbf{c}^*)$. Note that evaluating the first two terms can be done by the verifier $\mathcal{V}$ in $\mathcal{O}(\text{poly}(v_i, v_{i+1}))$. However, evaluating the last two terms requires the prover $\mathcal{P}$ to assist the verifier. So we are going to do the following: the prover $\mathcal{P}$ sends the values $z_b = \widetilde{W}_{i+1}(\mathbf{b}^*)$ and $z_c = \widetilde{W}_{i+1}(\mathbf{c}^*)$ to the verifier $\mathcal{V}$. Now the verifier has to check both these values are correct. However, here is the issue: during the sum-check protocol, the verifier can only use a single random point $\mathbf{r}_{i+1} \in \mathbb{F}^{v_{i+1}}$. So how do we check both conditions using one random point? Here is the trick.

Proposition 20.2. Let $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ be the line such that $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$. Then, the prover $\mathcal{P}$ sends the univariate polynomial $q(X)$ claimed to be equal to $\widetilde{W}_{i+1} \circ \ell$ – the restriction of $\widetilde{W}_{i+1}$ to the line ℓ . $\mathcal{V}$ checks

whether indeed $\ell(0) = z_b$ and $\ell(1) = z_c$, then chooses a random point $\mathbf{r}^* \leftarrow \$ \mathbb{F}^{v_{i+1}}$ and checks whether $\widetilde{W}_{i+1}(\ell(\mathbf{r}^*)) = q(\mathbf{r}^*)$.

This way, the interaction between the prover and the verifier at this round ends with the new claim about the value of $\widetilde{W}_{i+1}(\mathbf{r}_{i+1})$ with $\mathbf{r}_{i+1} := \ell(\mathbf{r}^*)$.

Finally, we need to define the last round of the protocol. Here, the $\mathcal{V}$ simply evaluates $\widetilde{W}_d(\mathbf{r}_d)$ on his own, costing $\mathcal{O}(n)$ time.

Proposition 20.3. We finally summarize the GKR protocol:

- At the start of the first round, $\mathcal{P}$ sends a function $D : \{0, 1\}^{v_0} \rightarrow \mathbb{F}$ claimed to equal W_0 (that is, he essentially sends the values of the output layer gates).
- $\mathcal{V}$ picks a random point $\mathbf{r}_0 \leftarrow \$ \mathbb{F}^{v_0}$ and finds $m_0 \leftarrow \widetilde{D}(\mathbf{r}_0)$.
- For each $i \in [d]$ we do the following:
 - Define the $2v_i$ -variate polynomial:

$$f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i) = \widetilde{\text{add}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})(\widetilde{W}_{i+1}(\mathbf{b}) + \widetilde{W}_{i+1}(\mathbf{c})) \\ + \widetilde{\text{mul}}_i(\mathbf{r}_i, \mathbf{b}, \mathbf{c})\widetilde{W}_{i+1}(\mathbf{b})\widetilde{W}_{i+1}(\mathbf{c}).$$

- $\mathcal{P}$ sends the value m_i claimed to equal $\sum_{\mathbf{b}, \mathbf{c} \in \{0, 1\}^{v_{i+1}}} f_i(\mathbf{b}, \mathbf{c}; \mathbf{r}_i)$.
- $\mathcal{V}$ applies the Sum-Check protocol on the function $f_i(\cdot; \mathbf{r}_i)$ up until the last round, where the $\mathcal{V}$ has to compute the value $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ at random $(\mathbf{b}^*, \mathbf{c}^*) \in \mathbb{F}^{2v_{i+1}}$.
- Both prover and the verifier computes the line $\ell : \mathbb{F} \rightarrow \mathbb{F}^{v_{i+1}}$ such that $\ell(0) = \mathbf{b}^*$ and $\ell(1) = \mathbf{c}^*$. $\mathcal{P}$ sends the univariate polynomial $q(X)$ claimed to equal $\widetilde{W}_{i+1} \circ \ell$.
- $\mathcal{V}$ computes the required value $f_i(\mathbf{b}^*, \mathbf{c}^*; \mathbf{r}_i)$ using $q(0)$ and $q(1)$ instead of $\widetilde{W}_{i+1}(\mathbf{b}^*)$ and $\widetilde{W}_{i+1}(\mathbf{c}^*)$.
- $\mathcal{V}$ chooses a random point $\mathbf{r}^* \leftarrow \$ \mathbb{F}^{v_{i+1}}$ and sets $\mathbf{r}_{i+1} \leftarrow \ell(\mathbf{r}^*)$ and $m_{i+1} \leftarrow q(\mathbf{r}_{i+1})$.
- The check reduces to verifying that $\widetilde{W}_{i+1}(\mathbf{r}_{i+1}) = m_{i+1}$, so proceed to the next step.
- At the last round, $\mathcal{V}$ directly checks whether $m_d = \widetilde{W}_d(\mathbf{r}_d)$.

21 Offline Memory Checking

21.1 Sum-Check-based Grand Product Protocol

This subsection describes a transparent SNARK, which may be used for proving grand product relations of the following form:

$$\mathcal{R}_{GP} = \left\{ (p \in \mathbb{F}, \mathbf{v} \in \mathbb{F}^m) : p = \prod_{i=0}^m v_i \right\} \quad (33)$$

Without loss of generality, assume that m is a power of 2. We can think of $\mathbf{v}$ as a vector of evaluations of a $\log m$ -variate multilinear polynomial $v(x)$ over $\{0, 1\}^{\log m}$ in a natural fashion. We assume the prover first opens p and then commits to $v(x)$; the following protocol then verifies both the polynomial's validity and the corresponding grand-product equality derived from that commitment.

Lemma 21.1. A scalar p and a vector $\mathbf{v}$ satisfies the relation $\mathcal{R}_{GP}$ if and only if there exists a multilinear polynomial f in $\log m + 1$ variables such that $f(1, \dots, 1, 0) = p$ and $\forall x \in \{0, 1\}^{\log m}$ the following hold:

$$\begin{aligned} f(0, x) &= v(x) \\ f(1, x) &= f(x, 0) \cdot f(x, 1) \end{aligned}$$

Such polynomial f has the following construction:

- $f(1, \dots, 1) = 0$
- For all $\ell \in [\log m]$ and $x \in \{0, 1\}^{\log m - \ell}$:

$$f(1^\ell, 0, x) = \prod_{y \in \{0, 1\}^\ell} v(x, y)$$

One can simply visualize the structure of f as a binary tree.

Example 21.1. Let $m = 4$ ($\log m = 2$), $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

$$\begin{aligned} v(x_1, x_2) &= 1 + 2x_1 + x_2 \\ v(x_1, x_2) : \quad &v(0, 0) = 1, \quad v(0, 1) = 2, \quad v(1, 0) = 3, \quad v(1, 1) = 4. \end{aligned}$$

Now, we define f as follows:

$$f(0, 0, 0) = 1, \quad f(0, 0, 1) = 2, \quad f(0, 1, 0) = 3, \quad f(0, 1, 1) = 4,$$

and:

$$\begin{aligned}
 f(1, 0, 0) &= f(0, 0, 0) \times f(0, 0, 1) = 1 \times 2 = 2, \\
 f(1, 0, 1) &= f(0, 1, 0) \times f(0, 1, 1) = 3 \times 4 = 12, \\
 f(1, 1, 0) &= f(1, 0, 0) \times f(1, 0, 1) = 2 \times 12 = 24 = p, \\
 f(1, 1, 1) &= 0.
 \end{aligned}$$

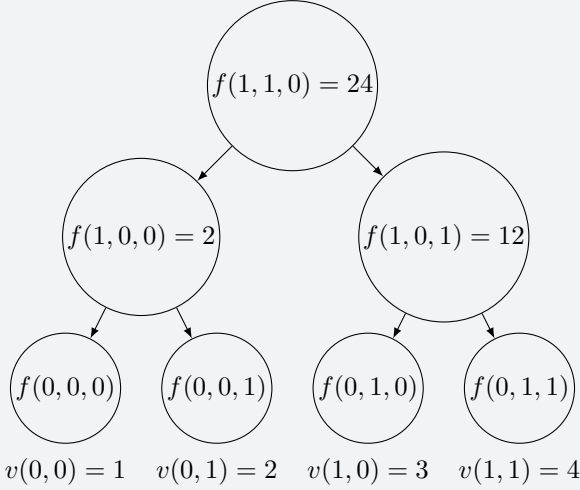


Illustration: Binary tree of the Grand Product constraints for $v(x_1, x_2) = 1 + 2x_1 + x_2$.

Then, to check that $\forall x \in \{0, 1\}^{\log m}$ the equation $f(1, x) = f(x, 0) \cdot f(x, 1)$ holds, we can use a sum-check protocol to prove the evaluation of g that is referred to a MLE of $f(1, x) - f(x, 0) \cdot f(x, 1)$:

$$g(t) = \sum_{x \in \{0, 1\}^{\log m}} \tilde{\text{eq}}(t, x) \cdot (f(1, x) - f(x, 0) \cdot f(x, 1))$$

By the Schwartz–Zippel lemma, except for a soundness error of $\frac{\log m}{|\mathbb{F}|}$ (which should be negligible), $g(\tau) = 0$ for τ uniformly random in $\mathbb{F}^{\log m}$ if and only if $g = 0$, which implies that $f(1, x) - f(x, 0) \cdot f(x, 1) = 0$ for all $x \in \{0, 1\}^{\log m}$.

Similarly, to prove that $v(x) = f(0, x)$ for all $x \in \{0, 1\}^{\log m}$ it suffices to prove that $v(\gamma) = f(0, \gamma)$ for a public coin $\gamma \in \mathbb{F}^{\log m}$.

Thus, to prove the existence of f and hence the grand product relationship, it

suffices to prove, for some verifier selected random $\tau, \gamma \in \mathbb{F}^\ell$, that:

$$0 = \sum_{x \in \{0,1\}^{\log m}} \tilde{\text{eq}}(x, \tau) \cdot (f(1, x) - f(x, 0) \cdot f(x, 1)) \quad (34)$$

$$f(0, \gamma) = v(\gamma) \quad (35)$$

$$f(1, \dots, 1, 0) = p \quad (36)$$

This can be achieved by running the sum-check protocol between $\mathcal{P}$ and $\mathcal{V}$, where $\mathcal{V}$ has oracle access to v and f . Additionally, $\mathcal{V}$ evaluates the f, v at the random point γ to verify that $f(0, x) = v(x)$. So, the sum-check-based protocol for the grand products looks as follows:

Algorithm 5: Sum-Check-based Protocol for Grand Products

1 $\mathcal{P}$: Compute polynomials $v \in \mathbb{F}^{\log m}[x]$, $f \in \mathbb{F}^{\log m+1}[x]$ such that

$$p = \prod_{x \in \{0,1\}^{\log m}} v(x) \quad \text{and} \quad f, v \text{ satisfy (34), (35), (36)}.$$

2 $\mathcal{P}$: $C_f \leftarrow \text{Commit}(f)$; $C_v \leftarrow \text{Commit}(v)$; send C_f, C_v to $\mathcal{V}$.

3 $\mathcal{V}$: Choose random $\tau, \gamma \in \mathbb{F}^{\log m}$ and send them to $\mathcal{P}$.

4 $\mathcal{P}$: Compute

$$g(x) = \tilde{\text{eq}}(x, \tau) (f(1, x) - f(x, 0) f(x, 1)).$$

5 $\mathcal{P} \& \mathcal{V}$: Run SumCheckProtocol(0, g , C_f)

/* we assume this sum-check runs over the commitment to f , not directly on g . */

6 $\mathcal{V}$: $r \leftarrow \text{Query}(C_f, (1, \dots, 1, 0))$.

7 **if** $r \neq p$ **then**

8 | $\mathcal{V}$ **rejects**.

9 **end**

10 $\mathcal{V}$: $a \leftarrow \text{Query}(C_f, (0, \gamma))$, $v(\gamma) \leftarrow \text{Query}(C_v, \gamma)$.

11 **if** $a \neq v(\gamma)$ **then**

12 | $\mathcal{V}$ **rejects**.

13 **end**

Remark. Note, that during the call to the sum-check protocol, we pass the polynomial g in a couple with the commitment to f . Here we assume that each invocation of the Query method to g inside the sum-check instance will be more complex: **instead of evaluating a single function and returning result, we evaluate f at three different points, check the proofs and return results according to the construction of g .**

21.2 Randomized Permutation Check

The goal of the randomized permutation check is to verify, with high probability, whether two sequences of tuples are permutations of each other, without

performing a full sort or pairwise comparison.

Definition 21.2 (Reed-Solomon Fingerprinting). Let $\mathbf{a} \in \mathbb{F}^n$, then for a random $\gamma \in \mathbb{F}$, the Reed-Solomon fingerprinting of $\mathbf{a}$ is defined as:

$$h_\gamma(\mathbf{a}) = \sum_{i \in n} a_i \cdot \gamma^i.$$

$h_\gamma(\mathbf{a})$ uniquely identifies the sequence $\mathbf{a}$ with high probability, i.e., let $\mathbf{b} \in \mathbb{F}^n$ and $\mathbf{a} \neq \mathbf{b}$, then, according to the Schwartz-Zippel lemma:

$$\Pr[h_\gamma(\mathbf{a}) = h_\gamma(\mathbf{b})] \leq \frac{n}{|\mathbb{F}|}.$$

Definition 21.3 (Randomized Permutation Check). Let A and B be two multisets of tuples in $\mathbb{F}^n$. Define

$$\mathcal{H}_{\tau, \gamma}(X) = \prod_{x \in X} (h_\gamma(x) - \tau).$$

Then comparing $\mathcal{H}_{\tau, \gamma}(A)$ and $\mathcal{H}_{\tau, \gamma}(B)$ yields a randomized test for whether A and B are permutations of one another. Concretely:

- (Completeness) If $A = B$ (as multisets), then

$$\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)$$

with probability 1 over uniform $\tau, \gamma \in \mathbb{F}$.

- (Soundness) If $A \neq B$, then

$$\Pr[\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)] \leq \frac{\max(|A|, |B|)}{|\mathbb{F}|}.$$

In other words, by first “hashing” each tuple via the map function h_γ and then taking the τ -shifted product, one obtains a fingerprint that is invariant under permutation but unlikely to collide on distinct sets.

Example 21.2. Consider all operations in $\mathbb{F}_7$, and set $n = 3$, $\tau = 5$, and $\gamma = 3$. Let

$$A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\},$$

First compute the Reed-Solomon fingerprints modulo 7:

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$

$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Now form the shifted products:

$$\mathcal{H}_{\tau,\gamma}(A) = (6 - 5)(2 - 5) = 1 \cdot (-3) \equiv 4,$$

$$\mathcal{H}_{\tau,\gamma}(B) = (2 - 5)(6 - 5) = (-3) \cdot 1 \equiv 4,$$

so the test *accepts* A vs. B (they are indeed permutations).

Now consider a non-permutation B' :

$$B' = \{(1, 2, 3), (2, 1, 3)\}.$$

$$h_\gamma(2, 1, 3) = 2 \cdot 1 + 1 \cdot 3 + 3 \cdot 2 = 2 + 3 + 6 = 11 \equiv 4.$$

$$\mathcal{H}_{\tau,\gamma}(B') = (6 - 5)(4 - 5) = 1 \cdot (-1) \equiv 6 \neq 4,$$

so the test *rejects* A vs. B' . In this way a single choice of (γ, τ) distinguishes permutations from non-permutations.

21.3 Offline Memory Checking

This protocol lets you, after doing a bunch of read/writes from an untrusted memory, verify in one quick algebraic step that every answer really came from the same original contents – without re-running any reads.

For the first time it's not so obvious, why we may need need such a protocol, so let's start with a simple example.

Example 21.3. Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100), (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. Alice writes a new value 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. Alice reads from address 0 and obtains the reply 150.
3. Alice reads from address 1 and (honestly) obtains 200.

At this point, both replies individually look correct. However, Bob can cheat on the very last step by returning, say, 300 instead of 200:

$$(1, 200) \longrightarrow (1, 300).$$

Alice only sees the single read result “300 at address 1,” which could simply be the true value after some write she missed—and she has no immediate way to detect the inconsistency.

In other words, without keeping an auditable record of all reads, Alice cannot later prove that all replies came from a single, immutable memory

state. This gap is exactly what an *offline memory check* fills: after all reads are done, Alice runs one fast verification that guarantees “every read you saw really did come from one fixed initial memory + your writes,” and thus catches any cheating by Bob.

Each memory cell can be described as a tuple (addr, val, counter), where addr is the address of the memory cell, val is the value stored at that address, and counter is the number of times the memory cell has been accessed. You can think of a counter as a timestamp. The memory is a vector of memory cels.

The protocol utilizes four sets of tuples:

- **init** – contains the initial memory state, where all counters are set to 0;
- **write** – contains memory cels that represent write operations, where val is the value stored at the memory address addr after the specified counter;
- **read** – contains memory cels that represent read operations, where val is the value read from the memory address addr at the specified counter;
- **final** – contains the final memory state, where all counters are set to the last value after the last read/write operation.

At the beginning, **init** is populated with the initial memory state, where all counters are set to 0, indicating that the values were first accessed during the initialization phase, while the **read**, **write** are empty. Then, for each read operation, the untrusted memory is queried at addr, returning a pair (val, counter), and writing a tuple (addr, val, counter) to the **read** set. For each write operation a corresponding tuple (addr, newval, counter + 1), where newval is a value being written, is added, with counter incremented by one, to the **write** set.

We assume that before each write operation, the read operation is performed. And the read operation is always followed by a write operation to the same address. Thus, to make a plain read operation, the prover first reads the value from the memory, then writes it back to the memory.

After all reads and writes are done, the **final** set is populated with the final memory state. Now, the verifier can check the consistency of all the operations by verifying the following equation:

$$\text{read} \cup \text{final} = \text{write} \cup \text{init}.$$

Thus, using randomized permutation check, one can rewrite the equation as

follows:

$$\begin{aligned}
& \mathcal{H}_{\tau,\gamma}(\mathbf{read}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{final}) = \mathcal{H}_{\tau,\gamma}(\mathbf{write}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{init}), \\
& \prod_{(a,v,t) \in \mathbf{read}} (h_\gamma(a, v, t) - \tau) \prod_{(a,v,t) \in \mathbf{final}} (h_\gamma(a, v, t) - \tau) = \\
& = \prod_{(a,v,t) \in \mathbf{write}} (h_\gamma(a, v, t) - \tau) \prod_{(a,v,t) \in \mathbf{init}} (h_\gamma(a, v, t) - \tau), \\
& \prod_{(a,v,t) \in \mathbf{read}} ((a\gamma^2 + v\gamma + t) - \tau) \prod_{(a,v,t) \in \mathbf{final}} ((a\gamma^2 + v\gamma + t) - \tau) = \\
& = \prod_{(a,v,t) \in \mathbf{write}} ((a\gamma^2 + v\gamma + t) - \tau) \prod_{(a,v,t) \in \mathbf{init}} ((a\gamma^2 + v\gamma + t) - \tau).
\end{aligned}$$

Example 21.4. Suppose the we track a single *row-memory*, the algorithm performs three reads and two writes, the initial memory is given by the following table:

Address	0	1	2	3
Value	2	5	7	9

Initial memory (counter 0):

$$\mathbf{init} = \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\},$$

while

$$\mathbf{read} = \emptyset \quad \text{and} \quad \mathbf{write} = \emptyset.$$

Trace 1 - consistent execution

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ (5, 0)	(1, 5, 0)	-
2	write((1,6))	-	(1, 6, 1)
3	read(2) $\rightarrow$ (7, 0)	(2, 7, 0)	-
4	write((2,7))	-	(2, 7, 1)

After the four steps

$$\mathbf{read} = \{(1, 5, 0), (2, 7, 0)\},$$

$$\mathbf{write} = \{(1, 6, 1), (2, 7, 1)\}.$$

And we can fill **final** as:

$$\mathbf{final} = \{(0, 2, 0), (1, 6, 1), (2, 7, 1), (3, 9, 0)\}$$

One can clearly see that $\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}$.

$$\begin{aligned}
& \{(1, 5, 0), (2, 7, 0)\} \cup \{(0, 2, 0), (1, 6, 1), (2, 7, 1), (3, 9, 0)\} = \\
& = \{(1, 6, 1), (2, 7, 1)\} \cup \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\}
\end{aligned}$$

Trace 2 - inconsistent execution

Let the second reading return a wrong value – a , instead of 5. So, the new trace would look like:

step	operation	$\Delta\text{read}_{\text{step}}$	$\Delta\text{write}_{\text{step}}$
1	$\text{read}(1) \rightarrow (a, 0)$	$(1, a, 0)$	-
2	$\text{write}((1,6))$	-	$(1, 6, 1)$

After the two steps

$$\text{read} = \{(1, a, 0)\},$$

$$\text{write} = \{(1, 6, 1)\},$$

The verifier checks $\text{read} \cup \text{final} = \text{write} \cup \text{init}$:

$$\begin{aligned} & \{(1, a, 0)\} \cup \{(0, 2, 0), (1, 6, 1), (2, 7, 0), (3, 9, 0)\} \neq \\ & \neq \{(1, 6, 1)\} \cup \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\} \end{aligned}$$

and *rejects*.

Actually, no multiset final that depends on the valid values and addresses can reconcile them.

22 Lookup Checks

22.1 Motivation

Suppose you need to develop the zero-knowledge circuit over some “unfriendly” construction such as AES-128, SHA-256, or pairing implemented on the non-native finite field. One way to resolve this issue is to use the proving scheme that is most “native” to the logic needed to implement such constructions (for example, you might want to switch to STARK instead of Groth16 for implementing hash functions). However, zero-knowledge world came up with one more way to resolve this issue without changing the whole system.

Consider the following problem: you have the witness $\mathbb{w}$ in which you want to check whether elements, say, $\vec{z} := \{z_i\}_{i \in [n]} \subseteq \mathbb{F}$ are contained in the lookup table $\vec{t} := \{t_j\}_{j \in [d]} \subseteq \mathbb{F}$ of size d . We write this check as $\{z_i\}_{i \in [n]} \subseteq \{t_j\}_{j \in [d]}$ or $\vec{z} \subseteq \vec{t}$ for short.

Example 22.1. As a not very practical example, suppose $\vec{t} = \{0, 1\}$. Then the check $\vec{z} \subseteq \vec{t}$ is read as “check whether each of elements in $\vec{z}$ is binary”. Another trivial example might be the following: set $\vec{t} := \{1, 6, 7, 10\}$. Then, for example, $\vec{z} = \{10, 6, 7, 1, 1, 6, 10, 7, 1\}$ should pass the lookup check since $\vec{z}$ consists of elements from $\vec{t}$ only. In turn, $\vec{z}^* = \{1, 6, 10, 5\}$ is not a valid witness corresponding to $\vec{t}$ since $5 \notin \vec{t}$.

Modern protocols have the following property: such check costs only $\mathcal{O}(n + d)$ constraints. Let us consider why this is significant for many applications in the wild.

Range Checks. The most obvious application is the range check. Assume that $t_j = j$ and $d = 2^w$. This way, the check $\{z_i\}_{i \in [n]} \subseteq \{t_j\}_{j \in [d]}$ simply corresponds to checking whether “is each z_i indeed a w -bit integer”? Needless to say, this has significant implications in various ZK applications: privacy-preserving payments, implementation of non-native arithmetic, or even zkML. In the case of zkML, for instance, *Bionetta* uses lookup checks to speed up the computation of the ReLU function, given by $\text{ReLU}(x) = \max\{0, x\}$, by the factor of up to $\times 20$.

Binary Arithmetic. Suppose you are implementing the SHA256, in which you need to frequently XOR binary strings. To simplify the problem, instead of XORing two, say, 256-bit strings, you XOR 8-bit strings: $\mathbf{c} = \mathbf{a} \oplus \mathbf{b}$ for $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \{0, 1\}^8$. The naive approach is to bit decompose each of $\mathbf{a}, \mathbf{b}, \mathbf{c}$ and verify that the check holds, but that requires enormous number of constraints. The more clever lookup-based approach is as follows: compose the table $\vec{t} := \{(t_j^{(1)}, t_j^{(2)}, t_j^{(3)})\}_{j \in [d]} \subseteq \mathbb{F}^3$ which consists of all distinct valid 8-bit XORs: $t_j^{(1)} \oplus t_j^{(2)} = t_j^{(3)}$. Note that in such case $d = 2^{16}$ since there are 2^{16} distinct equalities of such form (which simply enumerate over all possibilities of two 8-bit inputs to the XOR gate). We thus want to show that $\vec{z} := \{(a_i, b_i, c_i)\}_{i \in [n]} \subseteq \vec{t}$ where a_i, b_i, c_i are the part of the witness.

Now what remains is to show how to reduce this check to the “one-dimensional” case: recall that lookup protocols can merely check for inclusion for the ordered set of *scalars* and not *tuples*. It is done fairly easy: sample some random $\alpha \leftarrow \$\mathbb{F}$ and instead of viewing $\vec{z}$ as a tuple of three elements, view it as a list $\vec{z}_\alpha$ of n scalars of form $a_i + \alpha b_i + \alpha^2 c_i$. Similarly, each table elements of $\vec{t}_\alpha$ is viewed as $t_j^{(1)} + \alpha t_j^{(2)} + \alpha^2 t_j^{(3)}$. Then, the inclusion check $\vec{z} \subseteq \vec{t}$ due to Schwartz-Zippel lemma is equivalent to $\vec{z}_\alpha \subseteq \vec{t}_\alpha$ with overwhelming probability.

22.2 Plookup Protocol

One of the first lookup protocols that became practical in the zero-knowledge world is the *plookup protocol*. It is mostly used in Poly-IOPs but can presumably be compiled to other types of protocols as well, as long as *multiset equality* check can be implemented optimally.

Now, if you recall the *PlonK* construction, we can check that the multiset $\{s'_i\}_{i \in [n]}$ is the permutation of the multiset $\{s_i\}_{i \in [n]}$. This was done by comparing the products $\prod_{i \in [n]}(\gamma + s'_i) = \prod_{i \in [n]}(\gamma + s_i)$ at a random point γ . Due to Schwartz-Zippel lemma, equality in such case implies $s_i = s'_{\sigma(i)}$ with overwhelming probability for arbitrary $i \in [n]$ and some permutation $\sigma \in S_n$.

The primary issue with the lookup check is that instead of multisets equality, we need to check whether one multiset is the subset of another, which is a much trickier problem: we need to show that polynomials $Z(X) := \prod_{i \in [n]}(X - z_i)$ and $T(X) := \prod_{j \in [d]}(X - t_j)$ have the same roots, ignoring multiplicities. Let us introduce the following tool which will help to reduce the problem to a simpler one.

Definition 22.1. The **difference set** of $\vec{s} = \{s_i\}_{i \in [n]}$, denoted by $\delta\vec{s}$, is $\delta\vec{s} := \{s_{i+1} - s_i\}_{i \in [n-1]}$. Additionally, by $\partial\vec{s}$ denote the non-zero difference set: the set $\delta\vec{s}$, but without zero elements.

Example 22.2. Suppose $\vec{s} = \{1, 2, 5, 10, 13\}$. Then, we have

$$\delta\vec{s} = \{2 - 1, 5 - 2, 10 - 5, 13 - 10\} = \{1, 3, 5, 3\} = \partial\vec{s}$$

A slightly more motivating example is $\vec{s} := \{1, 1, 4, 4, 8, 8, 8\}$. Then:

$$\delta\vec{s} = \{0, 3, 0, 4, 0, 0\}, \quad \partial\vec{s} = \{3, 4\}.$$

Now let us get to the initial goal of proving $\vec{z} = \{z_i\}_{i \in [n]} \subseteq \{t_j\}_{j \in [d]} =: \vec{t}$. Let us look at a sorted version $\vec{s}$ of the values of $\vec{z}$ and compare the non-zero difference sets $\partial\vec{s}$ and $\partial\vec{t}$. Notice the following: if $\vec{z} \subseteq \vec{t}$, then $\partial\vec{s} = \partial\vec{t}$. So the prover $\mathcal{P}$ might want to simply prove this set equality. Unfortunately, the converse (that is, $\partial\vec{s} = \partial\vec{t} \Rightarrow \vec{z} \subseteq \vec{t}$) is not generally true. Indeed, consider the

following example:

$$\vec{t} = \{1, 4, 8\}, \quad \vec{s} = \{1, 1, 4, 8, 8, 8\}, \quad \vec{s}^* = \{1, 5, 5, 5, 8, 8\}.$$

Note that $\partial\vec{t} = \partial\vec{s} = \partial\vec{s}^* = \{3, 4\}$, but obviously $\vec{s}^* \not\subseteq \vec{t}$. So we need to modify our checks to introduce more robust lookup check condition.

Now, consider the following version: let $\vec{s}$ be the *sorted* multiset of concatenation, which we denote by $(\vec{z}, \vec{t}) = \{z_0, \dots, z_{n-1}, t_0, \dots, t_{d-1}\}$. Now we impose two checks: (1) $\vec{s} = (\vec{z}, \vec{t})$, (2) $\partial\vec{s} = \partial\vec{t}$. Turns out this condition is necessary and sufficient.

Proposition 22.2. $\vec{z} \subseteq \vec{t} \Leftrightarrow \vec{s} = (\vec{z}, \vec{t}) \wedge \partial\vec{s} = \partial\vec{t}$.

Proof. $\Rightarrow$ direction. The first condition holds due to construction. Note that the second condition holds for the reason that $\partial\vec{s}$ contains only differences of form $z_i - t_j$, $z_i - z_j$, or $t_i - t_j$. If $\vec{z} \subseteq \vec{t}$ and given $\vec{s}$ is sorted, $\partial\vec{s}$ contains only elements of form $t_{j+1} - t_j$, which is exactly $\partial\vec{t}$.

$\Leftarrow$ direction. Given first condition, we know that $\vec{z} \subseteq \vec{s}$. Thus, if we show $\vec{s} \subseteq \vec{t}$, we are done. Suppose, on the contrary, that $\vec{s} \not\subseteq \vec{t}$. Without loss of generality, assume thus that there is some $h \in \vec{s}$ such that $t_i < h < t_{i+1}$ for some $i \in [d]$. Then, $h - t_i, t_{i+1} - h \in \partial\vec{s}$ but clearly $h - t_i, t_{i+1} - h \notin \partial\vec{t}$ since $h \notin \vec{t}$. Contradiction. Thus $\vec{z} \subseteq \vec{s} \subseteq \vec{t}$.

Example 22.3. Again, consider the following lookup table and (sorted) witness elements:

$$\vec{t} = \{1, 4, 8\}, \quad \vec{z} = \{1, 1, 4, 8, 8, 8\}, \quad \vec{z}^* = \{1, 5, 5, 5, 8, 8\}.$$

Now consider the sorted multisets of concatenations:

$$\vec{s} = \{1, 1, 1, 4, 4, 8, 8, 8, 8\}, \quad \vec{s}^* = \{1, 1, 4, 5, 5, 5, 8, 8, 8\}$$

Let us check whether both conditions are satisfied. The first condition is satisfied for both $\vec{s}$ and $\vec{s}^*$ due to construction. The second condition, though, is satisfied only for $\vec{s}$. Indeed:

$$\partial\vec{t} = \{3, 4\} = \partial\vec{s} \neq \partial\vec{s}^* = \{1, 3\}$$

Reducing two checks to one. At this point, one can apply the *PlonK*-based protocol to check whether $\vec{s} = (\vec{z}, \vec{t})$ and $\partial\vec{s} = \partial\vec{t}$. However, *lookup* allows to use just a single one. Here how it works.

Definition 22.3. The **randomized difference set** with randomness $\beta \in \mathbb{F}^\times$

of multiset $\vec{s} = \{s_i\}_{i \in [n]}$, further denoted as $\partial_\beta \vec{s}$, is defined as follows:

$$\partial_\beta \vec{s} \triangleq \{s_i + \beta s_{i+1}\}_{i \in [n-1]}.$$

Remark. Note that $\partial_{-1} \vec{s}$ corresponds to the difference set of $\vec{s}$ (albeit with the negative sign).

Now, the verifier chooses a random $\beta \leftarrow \mathbb{F}^\times$ and it suffices to impose a single multiset check between $\partial_\beta \vec{s}$ and $((1 + \beta)\vec{z}, \partial_\beta \vec{t})$. We provide the following proposition (without proof).

Proposition 22.4. $\vec{z} \subseteq \vec{t}$ if and only if $\partial_\beta \vec{s} = ((1 + \beta)\vec{z}, \partial_\beta \vec{t})$ for $\beta \leftarrow \mathbb{F}^\times$.

Intuition. Such proposition came out of nowhere, so let us explain why (intuitively) it works. First, let us see how $\partial_\beta \vec{s}$ for $\vec{z} \subseteq \vec{t}$ looks like. Compared to $\partial \vec{s}$, two same consecutive elements s_i, s_{i+1} with $s_i = s_{i+1}$ don't get zeroed: instead, one gets $(1 + \beta)s_i$. So at least we know where $1 + \beta$ coefficient comes from! Other than same elements, in $\partial_\beta \vec{s}$, one gets coefficients of form $t_i + \beta t_{i+1}$, which is exactly $\partial_\beta \vec{t}$ by definition. This way $\partial_\beta \vec{s}$ consists of:

- Expressions of form $(1 + \beta)z_i$, which together form $(1 + \beta)\vec{z}$.
- Expressions of form $t_i + \beta t_{i+1}$, which together form $\partial_\beta \vec{t}$.

Thus we naturally require $\partial_\beta \vec{s} = ((1 + \beta)\vec{z}, \partial_\beta \vec{t})$. Why this condition is sufficient is less obvious, so we leave it without proof.

Example 22.4. Once again, consider the following lookup table and (sorted) witness elements:

$$\vec{t} = \{1, 4, 8\}, \quad \vec{z} = \{1, 1, 4, 8, 8, 8\}.$$

As previously shown, the sorted concatenation is $\vec{s} = \{1, 1, 1, 4, 4, 8, 8, 8, 8\}$. Now, suppose we have sampled some random $\beta \leftarrow \mathbb{F}^\times$. Then we have:

$$\partial_\beta \vec{s} = \{1 + \beta, 1 + \beta, 1 + 4\beta, (1 + \beta)4, 4 + 8\beta, (1 + \beta)8, (1 + \beta)8, (1 + \beta)8\}$$

Now note that for $(1 + \beta)\vec{z}$ and $\partial_\beta \vec{t}$ we have:

$$(1 + \beta)\vec{z} = \{1 + \beta, 1 + \beta, (1 + \beta)4, (1 + \beta)8, (1 + \beta)8, (1 + \beta)8\}, \quad \partial_\beta \vec{t} = \{1 + 4\beta, 4 + 8\beta\}$$

Clearly, $\partial_\beta \vec{s} = ((1 + \beta)\vec{z}, \partial_\beta \vec{t})$.

22.2.1 pllookup Precise Scheme

Fix integers n, d – witness size and lookup table size. Given witness $\mathbf{z} \in \mathbb{F}^n$ and lookup table $\mathbf{t} \in \mathbb{F}^d$, we want to ensure $\mathbf{z} \subseteq \mathbf{t}$. Let $\Omega = \{\omega^j\}_{j \in [n]} \leq \mathbb{F}^\times$ be a

multiplicative field subgroup. We say $z \subseteq t$ is *sorted by t* when values appear in the same order in z as they do in t .

Now, given $t \in \mathbb{F}^d$, $z \in \mathbb{F}^n$, and $s \in \mathbb{F}^{n+d}$, define bi-variate polynomials Z and T as follows:

$$Z(\beta, \gamma) \triangleq (1 + \beta)^n \prod_{i \in [n]} (\gamma + z_i) \prod_{i \in [d-1]} (\gamma(1 + \beta) + t_i + \beta t_{i+1})$$

$$T(\beta, \gamma) \triangleq \prod_{i \in [n+d-1]} (\gamma(1 + \beta) + s_i + \beta s_{i+1})$$

Why on Earth do we need these two polynomials? Consider the following proposition:

Proposition 22.5. $Z \equiv T$ if and only if $z \subseteq t$ and s is (z, t) sorted by t .

Now, this fact is completely unobvious, and you can see the proof in the [original lookup paper](#). This motivates us to formulate the following protocol. Without loss of generality, assume $d = n + 1$ (if $d \leq n$, pad t with $n - d + 1$ repetitions of the last element).

Preprocessing. Compute the polynomial $t(X)$ that encodes the values of lookup table $t \in \mathbb{F}^{n+1}$ over the domain Ω : that is, $t(\omega^j) = t_j$.

Inputs. Polynomial $z(X)$, encoding vector $z \in \mathbb{F}^n$ over Ω : that is, $z(\omega^j) = z_j$.

Protocol.

1. $\mathcal{P}$ computes $s = (z, t) \in \mathbb{F}^{2n+1}$ sorted by t . Interpolate two polynomials $h_1, h_2 \in \mathbb{F}^{<(n+1)}[X]$ as follows: $h_1(\omega^j) = s_j$ for $j \in [n+1]$ and $h_2(\omega^j) = s_{n+j}$ for $j \in [n+1]$. That is, h_1 interpolates first $n+1$ elements of s , while h_2 the remaining n .
2. $\mathcal{P}$ sends commitments of h_1 and h_2 to verifier $\mathcal{V}$ (namely, so that $\mathcal{V}$ will have an oracle access to both h_1 and h_2).
3. $\mathcal{V}$ picks random $\beta, \gamma \leftarrow \mathbb{F}$ and sends them to $\mathcal{P}$.
4. $\mathcal{P}$ computes a polynomial $F \in \mathbb{F}^{<(n+1)}[X]$ that aggregate the value $Z(\beta, \gamma)/T(\beta, \gamma)$ where Z, T are as described above. Specifically, let:
 - (a) $F(\omega) = 1$.
 - (b) For each $2 \leq j \leq n$, compute:

$$F(\omega^j) = \frac{(1 + \beta)^{i-1} \prod_{j < i} (\gamma + z_j) \prod_{1 \leq j < i} (\gamma(1 + \beta) + t_j + \beta t_{j+1})}{\prod_{1 \leq j < i} (\gamma(1 + \beta) + s_j + \beta s_{j+1}) (\gamma(1 + \beta) + s_{n+j} + \beta s_{n+j+1})}$$

$$(c) F(\omega^{n+1}) = 1.$$

5. $\mathcal{P}$ sends commitment of F to $\mathcal{V}$.
6. $\mathcal{V}$ checks that F is of the form described above, and that $F(\omega^{n+1}) = 1$. More precisely, $\mathcal{V}$ checks the following identities for each $\omega^j \in \Omega$:
 - (a) $L_1(\omega^j)(F(\omega^j) - 1) = 0$.

- (b) $(\omega^j - \omega^{n+1})F(\omega^j)(1+\beta)(\gamma+z(\omega^j))(\gamma(1+\beta)+t(\omega^j)+\beta t(\omega \cdot \omega^j)) = (\omega^j - \omega^{n+1})F(\omega \cdot \omega^j)(\gamma(1+\beta) + h_1(\omega^j) + \beta h_1(\omega \cdot \omega^j))(\gamma(1+\beta) + h_2(\omega^j) + \beta h_2(\omega \cdot \omega^j)).$
- (c) $L_{n+1}(\omega^j)(h_1(\omega^j) - h_2(\omega \cdot \omega^j)) = 0.$
- (d) $L_{n+1}(\omega^j)(F(\omega^j) - 1) = 0.$

Recall that $L_j(X)$ is the Lagrange basis polynomial: that is, $L_j(\omega^i) = \delta_{i,j}$.

Intuition. Four checks that $\mathcal{V}$ imposes are not very obvious (especially the second one). So here is the brief explanation of their meaning:

1. First equation checks whether $F(\omega) = 1$: note that since $L_1(\omega^j)$ is non-zero only for $j = 1$, the equation essentially reduces to $F(\omega) - 1 = 0$.
2. This is the main verification that is needed to check whether $Z(\beta, \gamma) \equiv T(\beta, \gamma)$. However, since both Z and T are high-degree polynomials, we cannot directly check their equivalence. For that reason, we introduce the “accumulator” polynomial F that should satisfy $F(\omega) = F(\omega^{n+1}) = 1$ and recursive condition

$$\frac{F(\omega^{j+1})}{F(\omega^j)} = \frac{(1+\beta)(\gamma+z_j)(\gamma(1+\beta)+t_j+\beta t_{j+1})}{(\gamma(1+\beta)+s_j+\beta s_{j+1})(\gamma(1+\beta)+s_{n+j}+\beta s_{n+j+1})}$$

Note that given $F(\omega) = 1$ and this recursive relation, by taking the product of both sides from 1 to n one obtains $F(\omega^{n+1}) = Z(\beta, \gamma)/T(\beta, \gamma)$ which is one, according to the previously specified lemma.

3. Since $L_{n+1}(\omega^j)$ is non-zero only for $j = n+1$, this check verifies that $h_1(\omega^{n+1}) = h_2(\omega^{n+2})$. This is the consistency check for h_1 and h_2 that checks whether h_1 and h_2 are properly “glued” together to encode s .
4. Finally, the last check verifies whether $F(\omega^{n+1}) = 1$.

Lemma 22.6. Soundness of the above protocol is at least $1 - (5n+4)/|\mathbb{F}|$.

22.3 Logup

While *plookup* is the Poly-IOP-based protocol, *logup* allows to prove the lookup inclusion using SumCheck and rather more algebraic (compared to *plookup*) approach. Let’s see how it works.

22.3.1 Preliminaries

Compared to previously discussed Sum-Check protocol, to stick with the original logup notation, we consider a slightly different hypercube: instead of boolean version $\{0, 1\}^n$, we consider the hypercube $\{-1, 1\}^n = \{\pm 1\}^n$, which we denote by $\mathcal{Q}_n$ for short. Note that the SumCheck protocol is exactly the same as for the boolean case, but we need to use “−1” instead of “0”. Besides, the

multilinear lagrange basis polynomials have a much nicer form:

$$\text{eq}(\mathbf{x}; \mathbf{y}) = \frac{1}{2^n} \prod_{j=1}^n (1 + x_j y_j)$$

22.3.2 Logarithmic Derivative

One of the ideas of the logup protocol is to reduce the polynomial check to the fractional one. To show this transition, we need a couple of formalities before we proceed.

Definition 22.7. Given a polynomial $q(X) := \sum_{j=0}^d q_j X^j \in \mathbb{F}[X]$, its **formal derivative**, in a similar fashion to calculus, is given by $q'(X) \triangleq \sum_{j=1}^d j q_j X^{j-1}$.

Example 22.5. For instance, given $q(X) = 1 + 2X + 3X^2$, the formal derivative is $q'(X) = 2 + 6X$.

It can be shown that the formal derivative obeys all the usual rules taught in calculus. Now, one more definition.

Definition 22.8. For a function $q(X)/r(X)$ from the rational function field $\mathbb{F}(X)$, the formal derivative is defined as:

$$\left(\frac{q(X)}{r(X)} \right)' \triangleq \frac{q'(X)r(X) - q(X)r'(X)}{r(X)^2}$$

For the further derivation, we will be interested in particular cases when derivatives turn out to be zero: that is $q'(X) = 0$ for the polynomial case or $\left(\frac{q(X)}{r(X)} \right)' = 0$ for a rational one. If polynomials are given in the real field, then these conditions immediately imply $q(X) \equiv \text{const}$ for polynomial case and $q(X) = cr(X)$ with $c = \text{const}$ for the rational case. Surprisingly, when considering the field $\mathbb{F}$ of finite characteristic p , these results are not generally true. Fortunately for us, they turn out to be false only for extreme case when degrees of polynomials are more than p , which, as you can imagine, never happens in practice. So we give the following lemma.

Lemma 22.9. Assume $q(X)$ and $r(X)$ are polynomials of degree less than p . Then:

- If $q'(X) = 0$, then q is constant.
- If $(q(X)/r(X))' = 0$, then $q(X)/r(X) = c$ for some $c \in \mathbb{F}$.

Finally, the logarithmic derivative.

Definition 22.10. The **logarithmic derivative** of $q(X) \in \mathbb{F}[X]$ is given by the rational function $q'(X)/q(X)$.

Remark. The name “logarithmic” comes from the fact that in calculus, for a function $f : \mathbb{R} \rightarrow \mathbb{R}$, the derivative of $\log f(x)$ is given by $f'(x)/f(x)$ by the chain rule.

One reason why logarithmic derivatives are attractive for our application is the following: the logarithmic derivative of $q(X)r(X)$ is the sum of logarithmic derivatives of $q(X)$ and $r(X)$. Indeed, note that the logarithmic derivative of $q(X)r(X)$ is nothing but:

$$\frac{(q(X)r(X))'}{q(X)r(X)} = \frac{q'(X)r(X) + q(X)r'(X)}{q(X)r(X)} = \frac{q'(X)}{q(X)} + \frac{r'(X)}{r(X)}$$

In particular, logarithmic derivative of $\prod_{j=1}^n (X + z_j)$ is given by $\sum_{j=1}^n \frac{1}{X + z_j}$. From all previously mentioned facts we can finally formulate some useful facts about lookup checks. Let us start from the simple observation.

Lemma 22.11. Let $\vec{a} = \{a_i\}_{i \in [n]}$ and $\vec{b} = \{b_i\}_{i \in [n]}$ be two sequences of elements from $\mathbb{F}$. To verify with overwhelming probability whether two multisets are equal, it suffices to check

$$\sum_{j=1}^n \frac{1}{\gamma + a_j} = \sum_{j=1}^n \frac{1}{\gamma + b_j}$$

for randomly chosen $\gamma \leftarrow \$ \mathbb{F}$.

Proof. Recall that two multisets are equal with overwhelming probability if for randomly selected $\gamma \leftarrow \$ \mathbb{F}$, we have $\prod_{j=1}^n (a_j + \gamma) = \prod_{j=1}^n (b_j + \gamma)$. Thus it suffices to show that imposing this product equality is equivalent to the statement in the lemma. Notice that if products are indeed the same, so are their logarithmic derivatives (with respect to γ), so we immediately get $\sum_{j=1}^n \frac{1}{\gamma + a_j} = \sum_{j=1}^n \frac{1}{\gamma + b_j}$. To show the opposite direction, assume that logarithmic derivatives of $q_{\vec{a}}(X) = \prod_{j=1}^n (X + a_j)$ and $q_{\vec{b}}(X) = \prod_{j=1}^n (X + b_j)$ are the same, so $\frac{q'_{\vec{a}}(X)}{q_{\vec{a}}(X)} = \frac{q'_{\vec{b}}(X)}{q_{\vec{b}}(X)}$. Then:

$$\left(\frac{q_{\vec{a}}(X)}{q_{\vec{b}}(X)} \right)' = \frac{q'_{\vec{a}}(X)q_{\vec{b}}(X) - q_{\vec{a}}(X)q'_{\vec{b}}(X)}{q_{\vec{b}}(X)^2} = 0$$

Thus $\frac{q_{\vec{a}}(X)}{q_{\vec{b}}(X)} = c$ for constant $c \in \mathbb{F}$. Since the leading coefficients of both $q_{\vec{a}}(X)$ and $q_{\vec{b}}(X)$ are 1, we conclude that $c = 1$, which immediately implies

$$\prod_{j=1}^n (a_j + \gamma) = \prod_{j=1}^n (b_j + \gamma).$$

The consequence of this fact is the following.

Lemma 22.12. Given two sequences of elements $\{t_i\}_{i \in [d]}$ and $\{z_i\}_{i \in [n]}$, the inclusion check $\{z_i\}_{i \in [n]} \subseteq \{t_i\}_{i \in [d]}$ is satisfied if and only if there exist the set of multiplicities $\{\mu_i\}_{i \in [d]}$ where $\mu_i = \#\{j \in [n] : z_j = t_i\}$ such that:

$$\sum_{i \in [n]} \frac{1}{X + z_i} = \sum_{i \in [d]} \frac{\mu_i}{X + t_i}$$

In particular, checking such equality at random point from $\mathbb{F}$ results in the soundness error of up to $(n + d)/|\mathbb{F}|$, which becomes negligible for fairly large $|\mathbb{F}|$.

22.3.3 Applying SumCheck

Now we need to build the SumCheck equation. Instead of considering the inclusion check for scalars, we consider the inclusion check for *multivariate polynomials*. Notice that the lookup table $\vec{t} = \{t_j\}_{j \in [d]}$ for $d = 2^v$ can be viewed as a function $t : \mathcal{Q}_v \rightarrow \mathbb{F}$. However, here is the issue: our witness vector $\vec{z} = \{z_i\}_{i \in [n]}$ typically has a significantly larger size than $\vec{t}$, so we cannot view $\vec{z}$ as a function $\mathcal{Q}_v \rightarrow \mathbb{F}$ since the input domain size is too small (also quite obvious remark – we cannot have two different hypercubes for the sumcheck protocol). For that reason, we consider a list of $m := \lceil n/d \rceil$ functions $z_1, \dots, z_m : \mathcal{Q}_v \rightarrow \mathbb{F}$ that encode all n values of $\{z_i\}_{i \in [n]}$.

This way, we want to check whether $\bigcup_{i \in [m]} \{z_i(\mathbf{x})\}_{\mathbf{x} \in \mathcal{Q}_v} \subseteq \{t(\mathbf{x})\}_{\mathbf{x} \in \mathcal{Q}_v}$. This way, we can rewrite our condition $\sum_{i \in [n]} \frac{1}{X + z_i} = \sum_{i \in [d]} \frac{\mu_i}{X + t_i}$ in the “sumcheck” world as follows:

$$\sum_{\mathbf{x} \in \mathcal{Q}_v} \sum_{i \in [m]} \frac{1}{\gamma + z_i(\mathbf{x})} = \sum_{\mathbf{x} \in \mathcal{Q}_v} \frac{\mu(\mathbf{x})}{\gamma + t(\mathbf{x})}$$

Here, $\mu(\mathbf{x})$ (if it is injective, which is typically the case) gives the number of occurrences of $t(\mathbf{x})$ in $z_1, \dots, z_m$ altogether, i.e. $\mu(\mathbf{x}) = \sum_{i \in [m]} \#\{\mathbf{y} \in \mathcal{Q}_v : z_i(\mathbf{y}) = t(\mathbf{x})\}$.

Now, the idea of logup is to sample random $\gamma \leftarrow \mathbb{F}$ and apply the SumCheck on the function:

$$\zeta(\mathbf{x}) = \sum_{i \in [m]} \frac{1}{\gamma + \tilde{z}_i(\mathbf{x})} - \frac{\tilde{\mu}(\mathbf{x})}{\gamma + \tilde{t}(\mathbf{x})},$$

where by $\tilde{t}_i$ we denoted the multilinear extension, that is:

$$\tilde{t}(\mathbf{x}) = \sum_{\mathbf{y} \in \mathcal{Q}_v} t(\mathbf{y}) \text{eq}(\mathbf{x}; \mathbf{y}) = \frac{1}{2^v} \sum_{\mathbf{y} \in \mathcal{Q}_v} t(\mathbf{y}) \prod_{j=1}^v (1 + x_j y_j)$$

The only issue left is that sumcheck protocol cannot work with rational functions, which is the case here for $\zeta(\mathbf{x})$. For that reason, roughly, the prover $\mathcal{P}$ will divide the sum of m terms into ℓ chunks and provide multilinear helper functions for each such sum. Note that ℓ is chosen depending on the chosen polynomial commitment scheme.

Namely, suppose we split $[m] = \bigcup_{j \in [k]} \mathcal{I}_j$ into $k = \lceil m/\ell \rceil$ subintervals. Suppose

$$\zeta_j(\mathbf{x}) = \sum_{i \in \mathcal{I}_j} \frac{\mu_i(\mathbf{x})}{\phi_i(\mathbf{x})}, \quad j \in [k],$$

is the respective partial sum of consecutive terms in the overall expression $\frac{\mu(\mathbf{x})}{\gamma + \tilde{t}(\mathbf{x})} - \sum_{i \in [m]} \frac{1}{\gamma + \tilde{z}_i(\mathbf{x})}$. That is, we used the notation $\mu_0(\mathbf{x}) = \mu(\mathbf{x})$ and $\mu_i(\mathbf{x}) \equiv -1$ for $i > 0$. Similarly, $\phi_0(\mathbf{x}) = \gamma + \tilde{t}(\mathbf{x})$ and $\phi_i(\mathbf{x}) = \gamma + \tilde{z}_{i-1}(\mathbf{x})$ for $i > 0$.

Then, the prover provides oracles for $\{\zeta_i\}_{i \in [k]}$ subject to $\sum_{\mathbf{x} \in \mathcal{Q}_v} \sum_{i \in [k]} \zeta_i(\mathbf{x}) = 0$ and the domain identities:

$$\zeta_j(\mathbf{x}) \prod_{i \in \mathcal{I}_j} \phi_i(\mathbf{x}) = \sum_{i \in \mathcal{I}_j} \mu_i(\mathbf{x}) \prod_{k \in \mathcal{I}_j \setminus \{i\}} \phi_k(\mathbf{x})$$

These identities are finally checked using SumCheck protocol and then combined into a single one using random scalars $\lambda_0, \dots, \lambda_{k-1} \leftarrow \$ \mathbb{F}$.

22.3.4 Precise Scheme

Suppose the prover $\mathcal{P}$ wants to convince the verifier $\mathcal{V}$ that $\{z_i\}_{i \in [n]} \subseteq \{t_j\}_{j \in [d]}$. Assume $d = 2^v$.

Preprocessing stage. Compute multilinear extension $\tilde{t} : \mathbb{F}^v \rightarrow \mathbb{F}$ that encodes the lookup table $\{t_j\}_{j \in [d]}$ and, if needed, multilinear Lagrange basis polynomials $\{\text{eq}(\mathbf{x}; \mathbf{y})\}_{\mathbf{y} \in \mathcal{Q}_v}$.

Protocol. Interaction between $\mathcal{P}$ and $\mathcal{V}$ proceeds as follows:

1. $\mathcal{P}$ divides witness into $k = \lceil n/d \rceil$ pieces and perceives values $\{z_i\}_{i \in [n]}$ as a set of m functions $z_j : \{\pm 1\}^v \rightarrow \mathbb{F}$. Additionally, $\mathcal{P}$ computes the multilinear extensions $\{\tilde{z}_j(\mathbf{x})\}_{j \in [m]}$.
2. $\mathcal{P}$ computes the multiplicity function multilinear extension $\tilde{\mu}$ and sends the oracle access to it $\mathcal{O}^\mu(\cdot)$ to the verifier $\mathcal{V}$.
3. $\mathcal{V}$ samples the challenge $\gamma \leftarrow \$ \mathbb{F}$ and sends to $\mathcal{P}$.
4. $\mathcal{P}$ computes $k = \lceil m/\ell \rceil$ functions $\{\zeta_j(\mathbf{x})\}_{j \in [k]}$ according to constraints above and sends oracle access $\mathcal{O}^{\zeta_0}(\cdot), \mathcal{O}^{\zeta_1}(\cdot), \dots, \mathcal{O}^{\zeta_{k-1}}(\cdot)$ to the verifier $\mathcal{V}$.
5. $\mathcal{V}$ responds with a random vector $\alpha \leftarrow \$ \mathbb{F}^n$ and random scalars $\lambda_0, \dots, \lambda_{k-1} \leftarrow \$ \mathbb{F}$. Now, both $\mathcal{P}$ and $\mathcal{V}$ engage in the sumcheck protocol for:

$$\sum_{\mathbf{x} \in \mathcal{Q}_v} G(\mathbf{x}, \text{eq}(\mathbf{x}; \alpha), \mu(\mathbf{x}), \phi_0(\mathbf{x}), \dots, \phi_{m-1}(\mathbf{x}), \zeta_0(\mathbf{x}), \dots, \zeta_{k-1}(\mathbf{x})) = 0,$$

where the function G is defined as follows:

$$G(\mathbf{x}, \star) = \sum_{r \in [k]} \zeta_r(\mathbf{x}) + \text{eq}(\mathbf{x}; \alpha) \lambda_r \left(\zeta_r \prod_{i \in \mathcal{I}_r} \phi_i(\mathbf{x}) - \sum_{i \in \mathcal{I}_r} \mu_i(\mathbf{x}) \prod_{j \in \mathcal{I}_r \setminus \{i\}} \phi_j(\mathbf{x}) \right)$$

During sumcheck, $\mathcal{V}$ uses oracles provided by $\mathcal{P}$.

Lemma 22.13. The cost for running the protocol is $\mathcal{O}(n\ell)$ field multiplications, while the communication size is $\mathcal{O}(m/\ell)$ oracles of size $\mathcal{O}(d)$.

23 UltraGroth

23.1 Protocol Motivation

UltraGroth is addressed to insignificantly modify Groth16 protocol to enable sampling random field elements effectively and further pass them as signals to the circuit. The primary implication of such possibility is, of course, lookup checks!

Moreover, such modification saves all the verification benefits of Groth16: in UltraGroth's basic form, it adds only a single $\mathbb{G}_1$ point to the proof, a single pairing operation, and a single hashing operation per randomness.

One of the examples where UltraGroth has already been used is *Bionetta* – an R1CS-based zkML framework used by *Rarimo*. When implementing $\text{ReLU}(x) = \max\{0, x\}$ non-linearity in-circuit, in the vanilla Groth16 one needs to perform the full bit-decomposition of x , thus costing ≈ 254 constraints per call. As a result, the circuit size for the neural network is roughly $\mathcal{O}(n)$ with $n = 254L$ with L being the number of ReLU calls. Using UltraGroth, we reduced the number of constraints to the *sublinear complexity*: $\mathcal{O}(n/\log n)$.

23.1.1 Historical Notes

Before delving into specifics, we briefly mention the history of this protocol, which is quite curious. This blog is based on the original *Lev Soukhanov's post* in 2023. However, as we prepared the conference paper for Bionetta, we discovered that his construction is nothing but a generalization of *MIRAGE protocol*, posted back in 2020! (in subsequent notation, MIRAGE is an UltraGroth protocol with $d = 1$). Even more surprisingly, it seems that independently of Lev Soukhanov's construction, Alex Ozdemir, Evan Laufer, and Dan Boneh introduced *MIRAGE+ protocol* in 2024, which was used for proving RAM computations correctness. This protocol, which is hard to believe, but also coincides with the Lev Soukhanov's construction (although I believe the former generalizes the construction a bit more rigorously). That said, to the best of my knowledge, Lev Soukhanov's construction is the first to generalize MIRAGE to the multiple-round interactive protocol.

All three papers (Bionetta included) proved completeness, soundness, and zero-knowledge of this construction, so the protocol that follows can be safely integrated into production systems. We, of course, drop formalities in this blog and recommend checking *MIRAGE+ protocol* paper for proof specifics in case someone is interested.

Finally, our personal opinion is that this construction is vastly underestimated and it is surprising that it is not yet used in production. Indeed, integrating lookup checks comes with a tiny cost of additional pairing which enables wrapping non-native protocols in Groth16 (e.g., zk-STARKs) *much* more effectively (up to a factor of $\times 20$).

Remark. The subsequent explanations will be quite technical, so if you want easier and more straightforward overview, you can additionally check out *our*

23.2 Lookup Checks

Here we restate the key fact that we explored in Section 22.

Theorem 23.1. Given two sequences of elements $\{t_i\}_{i \in [v]}$ and $\{z_i\}_{i \in [n]}$, the inclusion check $\{z_i\}_{i \in [n]} \subseteq \{t_i\}_{i \in [v]}$ is satisfied with overwhelming probability if and only if there exists the set of multiplicities $\{\mu_i\}_{i \in [v]}$ where $\mu_i = \#\{j \in [n] : z_j = t_i\}$ such that for randomly sampled $\gamma \leftarrow \$ \mathbb{F}$:

$$\sum_{i \in [n]} \frac{1}{\gamma + z_i} = \sum_{i \in [v]} \frac{\mu_i}{\gamma + t_i}$$

More specifically, checking such equality at a random point from $\mathbb{F}$ results in the soundness error of up to $(n + v)/|\mathbb{F}|$, which becomes negligible for large $|\mathbb{F}|$.

Now, the right hand side of the equation, given that we have sampled the randomness $\gamma \leftarrow \mathbb{F}$, can be computed in v constraints, while the left hand side is computed in additional n constraints (recall that inversion costs a single constraint to prove). Again, since Groth16 is a non-interactive protocol in its essence (meaning, compared to PlonK or sumcheck-based protocols, it is not constructed by converting an interactive protocol to non-interactive), there is no yet effective way to sample γ (besides probably in-circuit hashing which is insanely costly). Yet, we do propose a way to sample γ by turning the Groth16 into the "interactive-like" proof system.

The proceeding *UltraGroth* construction is essentially devoted to sampling γ and passing it as a signal to the circuit soundly.

23.2.1 Some other implications

Sampling randomnesses overall enables numerous applications besides lookup checks. For instance, suppose you have matrices $A, B \in \mathbb{F}^{n \times n}$ and you want to write the circuit that computes their product: $C = AB$. The naive way to do that is to compute it by definition. However, such circuit costs $\mathcal{O}(n^3)$ constraints, which is a lot. Turns out that sampling randomnesses can reduce this significantly. Indeed, one can apply the *Freiveld's* protocol: first sample $\gamma \leftarrow \$ \mathbb{F}^n$ and instead of checking $AB = C$, check whether matrices AB and C project γ to the same vector (this looks somewhat similar to Schwartz-Zippel lemma, but in the world of matrices):

$$(AB)\gamma = C\gamma \Leftrightarrow A(B\gamma) = C\gamma$$

Now notice that this can be implemented in approximately $3n^2$ constraints: one first multiplies $u \leftarrow B\gamma$, then multiplies Au and compares it to the product $C\gamma$. Note that matrix-vector product can be implemented in n^2 constraints, which is much more effective than matrix-matrix product with n^3 constraints.

However, currently, in Circom there is no way to pass γ as a signal to the circuit. Indeed, one can write `signal input gamma[n]` and mark it as public, but how can the verifier be ensured that corresponding values in witness were indeed sampled uniformly randomly by the prover? So let's see how to fix it.

23.3 UltraGroth

23.4 Two-round Interactive R1CS

Suppose we split our R1CS into multiple rounds. Consider the case where we want to implement lookup checks. In such case, it would be nice if we could compile the following interactive protocol (IP) into the zk-SNARK for a relation $\mathcal{R}_{\text{QAP}}$:

- *Round 0:* The prover runs the circuit without imposing lookup checks. The prover sends resultant witness w_0 to verifier. Verifier responds with the random challenge $x_1 \leftarrow \mathbb{F}$.
- *Round 1:* The prover computes the witness w_1 that corresponds to the lookup check (that is, that verifies equality $\sum_i \frac{1}{x_1 + z_i} = \sum_i \frac{\mu_i}{x_1 + t_i}$) using the randomness x_1 provided by verifier. The prover sends w_1 and polynomial $h(X)$ according to $\mathcal{R}_{\text{QAP}}$.
- Finally, the verifier checks the same QAP equation $\ell(X)r(X) = o(X) + t_\Omega(X)h(X)$.

We would like to compile such IP into zk-SNARK by applying the Fiat-Shamir heuristic. However, here is the issue: x_1 must be derived by hashing the previous transcript, which is the public statement $\mathfrak{x}_0$ and witness w_0 . However, how can we hash the witness w_0 , given that the whole Groth16 proving system compiles the verification equation to just three points?

Now, suppose we partition the index set of public signals $\mathcal{I}_X$ into two parts (corresponding to each round): $\mathcal{I}_X^{(0)}$ and $\mathcal{I}_X^{(1)}$. Corresponding private signals are indexed by $\mathcal{I}_W^{(0)}$ and $\mathcal{I}_W^{(1)}$ which partition $\mathcal{I}_W$. In the example above, $\mathcal{I}_X^{(0)}$ corresponds to the regular public signals in relation $\mathcal{R}_{\text{QAP}}$ while $\mathcal{I}_X^{(1)}$ indexes the single solution witness element, corresponding to sampled randomness x_1 .

The idea is the following: we split the polynomial $c(X)$ (which was described in Groth16 construction above) into two pieces $c_0(X)$ and $c_1(X)$. The first piece corresponds to the first round indexed by $(\mathcal{I}_X^{(0)}, \mathcal{I}_W^{(0)})$ and is formed as follows: sample $r_0 \leftarrow \mathbb{F}$, add toxic parameter δ_0 , and compute the point $\pi_C^{(0)} \leftarrow g_1^{c_0(\tau)}$ where:

$$c_0(X) = \delta_0^{-1} \sum_{j \in \mathcal{I}_W^{(0)}} z_j f_j(X) + r_0 \delta$$

Notice that the point $\pi_C^{(0)}$ encodes all witness elements in Round 0. Thus, to sample randomness, we can hash prover parameters and this point: $x_1 \leftarrow \mathcal{H}(\text{pp}, \pi_C^{(0)})$. Then, points $\pi_A = g_1^{a(\tau)}$ and $\pi_B = g_1^{b(\tau)}$ are computed in the same way as in the

regular Groth16, while the third piece $\pi_C^{(1)} = g_1^{c_1(\tau)}$ is computed using:

$$c_1(X) = \delta^{-1} \left(\sum_{j \in \mathcal{I}_W^{(1)}} z_j f_j(X) + h(X) t_\Omega(X) \right) + a(X)s + b(X)r - r_0 \delta_0 - r s \delta$$

Essentially, such construction ensures that $\delta_0 c_0(X) + \delta c_1(X)$ gives exactly the polynomial $\delta c(X)$ in the original Groth16. This allows the prover to construct the proof as a tuple $\pi = (\pi_A, \pi_B, \pi_C^{(0)}, \pi_C^{(1)})$ and verifier to check it using:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(g_1^{i(\tau)}, g_2^\gamma) \cdot e(\pi_C^{(0)}, g_2^{\delta_0}) \cdot e(\pi_C^{(1)}, g_2^\delta),$$

where $i(X) = \gamma^{-1} \sum_{i \in \mathcal{I}_X} z_i f_i(X)$ as before. As in Groth16, the term $e(g_1^\alpha, g_2^\beta)$ can be pre-computed, thus the cost of verification is 4 pairing operations.

Remark. The aforementioned protocol, as mentioned earlier, is essentially the MIRAGE protocol, and can be implemented as is. It is complete, sound, and zero-knowledge. In the subsequent section, we will generalize this protocol for more than 1 round.

23.5 Generalizing to multiple rounds

To generalize the aforementioned protocol, we define the $(d + 1)$ -round quadratic arithmetic program (or d QAP, for short) as follows:

$$\mathcal{R}_{d\text{QAP}} = \left\{ \begin{array}{l} \mathbb{x}_i = \{z_j\}_{j \in \mathcal{I}_X^{(i)}} \\ \mathbb{w}_i = \{z_j\}_{j \in \mathcal{I}_W^{(i)}} \\ \text{for } i \in [d + 1] \end{array} \middle| \begin{array}{l} \ell(X) \cdot r(X) = o(X) + t_\Omega(X)h(X) \\ \ell(X) = \sum_{i \in [n]} z_i \ell_i(X), \\ r(X) = \sum_{i \in [n]} z_i r_i(X), \\ o(X) = \sum_{i \in [n]} z_i o_i(X), \\ \text{for some } h(X) \in \mathbb{F}[X] \end{array} \right\},$$

where indices $\{\mathcal{I}_X^{(i)}\}_{i \in [d+1]}$ and $\{\mathcal{I}_W^{(i)}\}_{i \in [d+1]}$ partition $[n]$. In this notation, all $\{\mathbb{w}_i\}_{i \in [d+1]}$ are called *private witnesses*, $\mathbb{x}_0$ is a *public statement*, while $\{\mathbb{x}_i\}_{i \in [d+1] \setminus \{0\}}$ are *challenges* (that are going to be sampled by means of Ultra-Groth).

Since prover and verifier participate in the interactive protocol, the prover cannot compute the whole witness on his own without verifier's randomnesses. For that reason, we define *strategy* as the collection of functions $S = \{S_i\}_{i \in [d]}$, each of which computes the witness for the given round given previous witnesses and challenges and the current challenge, sampled by the verifier. In other words,

$$\mathbb{w}_i = S_i(\mathbb{x}_0, \dots, \mathbb{x}_i, \mathbb{w}_0, \dots, \mathbb{w}_{i-1})$$

Example 23.1. For instance, 0QAP represents the regular Groth16 construction, where strategy S_0 is a regular witness calculator: $\mathbb{w}_0 = S_0(\mathbb{x}_0)$. The protocol which involves lookup checks is a SNARK over 1QAP, where S_0 computes

the R1CS witness without lookup constraints, while S_1 , based on the sampled randomness(es) $\mathbb{x}_1$, imposes lookup check condition: $\mathbb{w}_1 = S_1(\mathbb{x}_0, \mathbb{x}_1, \mathbb{w}_0)$.

Now, we can describe the *UltraGroth* protocol over $\mathcal{R}_{\text{dQAP}}$.

Setup($1^\lambda, \mathcal{R}_{\text{dQAP}}$). As in previous sections, denote by $f_i(X) := \beta \ell_i(X) + \alpha r_i(X) + o_i(X)$ the random linear combination of ℓ_i, r_i, o_i . During the trusted setup, one generates the toxic waste $\alpha, \beta, \gamma, \{\delta_i\}_{i \in [d+1]}, \tau \leftarrow \mathbb{F}$ and computes the following common parameters:

$$\begin{aligned} \text{pp} &= \left(g_1^\alpha, g_1^\beta, \{g_1^{\delta_i}\}_{i \in [d]}, g_2^\beta, g_2^\gamma, \{g_2^{\delta_i}\}_{i \in [d]}, \right. \\ &\quad \left. \left\{ g_1^{\tau^i}, g_2^{\tau^i}, g_1^{\tau^i t(\tau)/\delta_d} \right\}_{i \in [n]}, \left\{ g_1^{f_i(\tau)/\gamma} \right\}_{i \in \mathcal{I}_X}, \left\{ g_1^{f_j(\tau)/\delta_i} \right\}_{(i,j) \in [d] \times \mathcal{I}_W^{(i)}} \right) \\ \text{vp} &= \left(g_1, g_2, g_2^\gamma, \{g_2^{\delta_i}\}_{i \in [d]}, g_T^{\alpha\beta}, \left\{ g_1^{f_i(\tau)/\gamma} \right\}_{i \in \mathcal{I}_X} \right) \end{aligned}$$

Prove(pp, $\mathbb{x}_0, S$). The prover initializes the accumulator $a_0 := \mathcal{H}(\text{pp})$. For each round $i \in [d]$ she conducts the following steps:

- Samples $r_i \leftarrow \mathbb{F}$
- Computes witness $\mathbb{w}_i \leftarrow S_i(\mathbb{x}_0, \dots, \mathbb{x}_i, \mathbb{w}_0, \dots, \mathbb{w}_{i-1})$.
- Computes $\pi_C^{(i)} \leftarrow g_1^{c_i(\tau)}$ with $c_i(X) := \delta_i^{-1} \sum_{j \in \mathcal{I}_W^{(i)}} z_j f_j(X) + r_i \delta_d$.
- Updates accumulator as $a_{i+1} \leftarrow \mathcal{H}(a_i, \pi_C^{(i)})$.
- If $i < d$, for each $j \in \mathcal{I}_X^{(i+1)}$ sets $z_j \leftarrow \mathcal{H}(a_{i+1}, g_1^j)$.

After finishing d rounds, the prover does the following:

- Computes polynomial $h(X)$ from QAP as in Groth16.
- Samples random $r, s \leftarrow \mathbb{F}$ and computes $\pi_A \leftarrow g_1^{a(\tau)}, \pi_B \leftarrow g_2^{b(\tau)}$ and the last proof piece $\pi_C^{(d)} \leftarrow g^{c_d(\tau)}$ where $a(X), b(X)$, and $c(X)$ are defined as follows:

$$\begin{aligned} a(X) &= \alpha + \sum_{i \in [n]} z_i \ell_i(X) + r \delta_d, \\ b(X) &= \beta + \sum_{i \in [n]} z_i r_i(X) + s \delta_d \\ c_d(X) &= \delta_d^{-1} \left(\sum_{i \in \mathcal{I}_W^{(d)}} z_i f_i(X) + h(X) t_\Omega(X) \right) + a(X)s + b(X)r - \sum_{i \in [d]} r_i \delta_i - r s \delta_d \end{aligned}$$

- **Outputs** proof $\pi = (\pi_A, \pi_B, \pi_C^{(0)}, \dots, \pi_C^{(d)}) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1^{d+1}$.

Verify(vp, $\mathbb{x}_0, \pi$). The verifier recomputes all the challenges $\{\mathbb{x}_i\}_{i \in [d+1] \setminus \{0\}}$ and verifies that corresponding public signals are correct. Then, she checks:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(g_1^{i(\tau)}, g_2^\gamma) \cdot \prod_{i \in [d+1]} e(\pi_C^{(i)}, g_2^{\delta_i})$$

23.6 Efficiency Analysis

Finally, we specify the efficiency of UltraGroth.

- The prover’s work is essentially the same as in Groth16: if the total witness size is n , the prover needs to conduct $\mathcal{O}(n \cdot E_1 + n \cdot E_2)$ operations with a couple of additional hashes (formally, $\sum_{i \in [d+1] \setminus \{0\}} |\mathbb{X}_i|$ hashes, which is a negligible amount in practice). Note however, that due to such R1CS splitting, the circuit’s size n is drastically decreased: in many cases it becomes sublinear in the original size.
- Verifier’s work is $(d + 3)E + \mathcal{O}(l \cdot E_1)$.
- Proof size is 1 $\mathbb{G}_2$ point and $(d + 2)$ $\mathbb{G}_1$ points.

23.7 Practical Implementation and Further Steps

Currently, we have implemented the UltraGroth as a part of the Bionetta project for $d = 1$ in the following repository: <https://github.com/rarimo/ultragroth>. Also, we have a custom *SnarkJS* fork that implements smart contract generation and witness verify and export operations. But the current implementation is vastly inflexible and currently primarily works in conjunction with *Bionetta’s witness generator* only. This implementation takes the regular Circom-generated `.wtns` and `.sym` files, parses them, figures out which witness element corresponds to the randomness, and during the proving procedure fills in this spot using the protocol above.

We look forward to completing the implementation that could be easily integrated into other’s projects. One nice way to implement that would be to add a special keyword to the Circom compiler. For instance, it would be nice if we could write `random signal r[N]` that fills in the vector `r` with random values, although such task is surely challenging.

23.8 Acknowledgements

Of course, without Lev Soukhanov this section would not exist. I also thank Artem Sdobnov, Vitalii Volovyk, Yevhenii Sekhin, and Illia Dovgopoly for the UltraGroth implementation for Bionetta.

23.9 Summary

In case you implement complex circuits involving non-native operations (e.g., wrapping the verification of some ZK protocol), it is definitely worth checking out the UltraGroth protocol. With the tiny verification overhead, one gets an immense boost to the proving procedure. In particular, it makes client-side ZK much more practical and realistic without sacrificing verification efficiency.

Part V

Concluding Remarks

Solutions to Exercises

Introduction to Number Theory

Exercise 1. Answer: E. Statement “any prime number p equals to the product of its divisors” is true since the only divisors of p are 1 and p by definition.

Exercise 2. Answer: D. Statement $\text{lcm}(n+1, n-1) = n^2 - 1$ is false. For instance, take $n := 5$. Then, $\text{lcm}(6, 4) = 12 \neq 5^2 - 1 = 24$.

Exercise 3. Answer: C. Answer is 9. Indeed, $9 \cdot 5 = 45 \equiv 1 \pmod{11}$.

Exercise 4. Answer: C. Answer is 15. Indeed, $\text{gcd}(15, 21) = 3 \neq 1$, so $15 \notin \mathbb{Z}_{21}^\times$ by definition.

Exercise 5. Answer: A. Notice that $n^2 + 1 = n(n+1) - (n-1)$. Thus, if $(n+1) \mid (n^2 + 1)$ we should have $(n+1) \mid (n-1)$ which is possible only if $n-1 = 0$. Thus, $n = 1$ is the only such number.

Exercise 6. Answer: C. Let us find roots of f . Notice that:

$$f(x) = 2x^2 - (3p+q)x + (p^2 + pq)$$

By equating $f(x) = 0$ we get two solutions: $r_1 = p$ and $r_2 = \frac{p+q}{2}$. One can easily check that the only possible p and q for r_2 to be prime is $(7, 3)$. Thus, the sum is 10.

Exercise 7. Answer: B. We have the only $\alpha_1 = 100$ which by the definition of τ function gives $\tau(p^{\alpha_1}) = \alpha_1 + 1 = 101$.

Exercise 8. Answer: D. Suppose $n = \prod_{j=1}^r p_j^{\alpha_j}$. Then, $\tau(n) = \prod_{j=1}^r (\alpha_j + 1) = 7$. The only possible way to represent 7 as a product of integers is to have the only non-zero α such that $\alpha + 1 = 7$. Thus, $\alpha = 6$, so $n = p^6$ for arbitrary prime p . Therefore, $\tau(n^2) = \tau(p^{12}) = 13$, following the reasoning from the previous exercise.

Exercise 9. Answer: B. Suppose $m = \prod_{j=1}^r p_j^{\alpha_j}$, so again $\tau(m) = \prod_{j=1}^r (\alpha_j + 1)$. For this expression to be odd, we need every term $\alpha_j + 1$ to be odd. This is possible only if every α_j is even. Therefore, m must be a square of a number. The smallest square greater than n^2 is of course $(n+1)^2$.

Introduction to Abstract Algebra

Exercise 1. Answer: E. Notice that if $xy = 1$ then $y = \frac{1}{x}$. If x is an integer then y is integer only if $x = \pm 1$. Therefore,

$$X \cap \mathbb{Z}^2 = \{(1, 1), (-1, -1)\}, \quad X \cap \mathbb{N}^2 = \{(1, 1)\}.$$

Therefore, first two statements are correct. When it comes to the third statement, it is also true: indeed, the identity element is $(1, 1)$ while the

inverse element for (x_1, x_2) is $(1/x_1, 1/x_2)$ (such operation is closed since $(1/x_1)(1/x_2) = 1/(x_1x_2) = 1$). Other properties are also satisfied.

Exercise 2. Answer: A. Identity element in this group is 1. Indeed, $a \oplus 1 = 1 \oplus a = a + 1 - 1 = a$. Associativity follows immediately from the sum operation associativity. Inverse element for a is $2 - a$ since:

$$a \oplus (2 - a) = a + 2 - a - 1 = 1$$

All operations are closed since the sum and subtraction of two integers is an integer.

Exercise 3. Answer: C. Suppose the identity element $E \in \mathbb{R}^2$ exists. Then, $A \oplus E = E$ by definition. However, the midpoint of AE is E only if $A = E$. Since identity element depends on the chosen group element A , it does not exist.

Exercise 4. Answer: E. Indeed, take $A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$. Suppose inverse B exists. Then, $AB = E_{2 \times 2}$ where $E_{2 \times 2}$ is an identity matrix. Therefore, $\det(AB) = \det A \cdot \det B$. Since $\det A = 4$ we have $\det B = \frac{1}{4}$. However, the determinant of a matrix, composed of integers, cannot be a fraction. Therefore, the inverse does not exist for A .

Exercise 5. Answer: A. Let us check all the properties carefully:

- **Closure:** Take $f, g \in \mathcal{F}$. The function $f \times g$ has a domain Ω and range $\mathbb{G}$ since $f(\omega) \times g(\omega)$ always outputs $\mathbb{G}$ for any $\omega \in \Omega$ due to the closure of $\mathbb{G}$.
- **Identity Element:** Define identity element $i \in \mathcal{F}$ as a function $i(\omega) = e$ for any $\omega \in \Omega$ where e is an identity element in $\mathbb{G}$. Then:

$$(f \star i)(\omega) = f(\omega) \times i(\omega) = f(\omega) \times e = f(\omega) \implies f \star i = f$$

- **Inverse:** Define inverse element for $f \in \mathcal{F}$ as g defined as $g := (f(\omega))^{-1}$ for every $\omega \in \Omega$. Then:

$$(f \star g)(\omega) = f(\omega) \times (f(\omega))^{-1} = e \quad \forall \omega \in \Omega \implies f \star g = i$$

- **Associativity:** Holds due to the associativity of $\times$.
- **Commutativity:** Holds due to the commutativity of $\times$.

Exercise 6. Answer: A. First, $f : x \mapsto x^2$ is obviously closed over $[0, 1]$. Let us check why it is an automorphism. First, check whether it is a homomorphism:

$$f(x \cdot y) = (x \cdot y)^2 = x^2 \cdot y^2 = f(x) \cdot f(y)$$

Now, let us show that f is a bijection. Indeed, f is injective since $f(x) = f(y)$ implies $x^2 = y^2$ which is possible only if $x = y$ when both x and y are positive. f is surjective since for any $y \in [0, 1]$ there is $x = \sqrt{y}$ such that $f(x) = y$. Therefore, f is an automorphism.

Exercise 7. Answer: C. In fact, $\varphi_1 = \det$ while $\varphi_2 = \text{trace}$. It is well-known that $\det(AB) = \det A \det B$ and $\text{trace}(A+B) = \text{trace}(A) + \text{trace}(B)$. Therefore, both functions are homomorphisms.

Exercise 8. Answer: A. Let us check the homomorphism property. Take $a + bi, c + di \in \mathbb{C} \setminus \{0\}$. Then:

$$\psi((a + bi)(c + di)) = \psi((ac - bd) + (ad + bc)i) = \begin{bmatrix} ac - bd & -(ad + bc) \\ ad + bc & ac - bd \end{bmatrix}$$

On the other hand:

$$\psi(a + bi)\psi(c + di) = \begin{bmatrix} a & -b \\ b & a \end{bmatrix} \cdot \begin{bmatrix} c & -d \\ d & c \end{bmatrix} = \begin{bmatrix} ac - bd & -(ad + bc) \\ ad + bc & ac - bd \end{bmatrix}$$

Therefore, $\psi((a + bi)(c + di)) = \psi(a + bi)\psi(c + di)$, so ψ is a homomorphism. Let us show it is a bijection. Suppose $\psi(a + bi) = \psi(c + di)$. Then:

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix} = \begin{bmatrix} c & -d \\ d & c \end{bmatrix} \implies a = c, b = d$$

Thus, ψ is injective. To show the surjectivity notice that for any matrix $\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$ there is $a + bi$ such that $\psi(a + bi) = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}$. Therefore, ψ is a bijection. Therefore, ψ is an isomorphism.

Exercise 9. Let $\mathbb{G}$ be a group of order 9. If $\mathbb{G}$ contains the element of order 9, then it is cyclic and thus abelian, so we are done. Suppose this is not the case and we have the element $x \in \mathbb{G}$ of order other than 9. Due to Lagrange's theorem, the order of g must divide the order of $\mathbb{G}$, so it is either 1, 3 or 9. Since 1 corresponds to the identity element, suppose $\mathbb{H}_X = \langle x \rangle$ has order 3. Let $y \in \mathbb{G} \setminus \mathbb{H}_X$ be some other non-identity element of $\mathbb{G}$ and $\mathbb{H}_Y := \langle y \rangle$. Since $\mathbb{H}_X \cap \mathbb{H}_Y$ is a subgroup of $\mathbb{H}_X$ and $|\mathbb{H}_X| = 3$ by Lagrange's Theorem either $\mathbb{H}_X \cap \mathbb{H}_Y = \{e\}$ or equals $\mathbb{H}_X$. Since $y \notin \mathbb{H}_X$, we conclude that $\mathbb{H}_X \cap \mathbb{H}_Y = \{e\}$. Therefore,

$$|\mathbb{H}_X \mathbb{H}_Y| = \frac{|\mathbb{H}_X| \cdot |\mathbb{H}_Y|}{|\mathbb{H}_X \cap \mathbb{H}_Y|} = 9$$

Therefore, $\mathbb{G} = \mathbb{H}_X \mathbb{H}_Y = \{e, x, x^2, y, y^2, xy, xy^2, x^2y, x^2y^2\}$ since $x^3 = y^3 = e$. Now, consider $yx \in \mathbb{G}$. It must be one of elements listed in $\mathbb{G}$:

- $yx \neq e$, otherwise $y = x^2$ but $\mathbb{H}_X \cap \mathbb{H}_Y = \{e\}$.
- $yx \neq x$, otherwise $y = e$.
- $yx \neq x^2$, otherwise $x = y$.
- $yx \neq y$, otherwise $x = e$.
- $yx \neq y^2$, otherwise $x = y$.
- $yx \neq xy^2$, otherwise $xyx = x$ (multiplied both sides by y), but $(yx)^3 = yx(yxy)x = yx^3 = y = e$, so $y = e$.
- $yx \neq x^2y$, otherwise $xyx = y$ and thus $x = e$ (see explanation above).
- $yx \neq x^2y^2$, otherwise $(yx)^2 = (yx)(yx) = (x^2y^2)(yx) = e$. Yet, $(yx)^3 = e$, so $yx = e$ or $y = x^2$, contradicting $\mathbb{H}_X \cap \mathbb{H}_Y = \{e\}$.

Therefore, $yx = xy$, so $\mathbb{G}$ is an abelian group.

Polynomials

Exercise 1. Answer: A. Note that $\deg(p - q) = 4$ since p and q are of different degrees. Therefore, $\deg((p - q)r) = \deg(p - q) + \deg(r) = 4 + 5 = 9$.

Exercise 2. Answer: D. It suffices to plug each of $\{0, 1, 2, 3, 4\}$ in $4x^2 + 7$ and verify that the result is never 0 over $\mathbb{F}_5$.

Exercise 3. Answer: A. Note that $p(x) = a(x - 1)(x - 2)$. Since $p(0) = 2$, we have $a = 1$. Therefore, $p(x) = x^2 - 3x + 2$ and thus $b = -3$, $c = 2$. The sum $a + b + c$ is then 0.

Exercise 4. Answer: C. Roots of Q are $\pm i$. If $Q(x) \mid P(x)$, we must have $P(\pm i) = 0$. Thus we have $P(i) = i^{2n} + i^n - 2 = (-1)^n + i^n - 2$. Consider four cases:

- $n = 4k$. Then, $P(i) = (-1)^{4k} + i^{4k} - 2 = 0$.
- $n = 4k + 1$. Then, $P(i) = (-1)^{4k+1} + i^{4k+1} - 2 = -3 + i \neq 0$.
- $n = 4k + 2$. Then, $P(i) = (-1)^{4k+2} + i^{4k+2} - 2 = -2 \neq 0$.
- $n = 4k + 3$. Then, $P(i) = (-1)^{4k+3} + i^{4k+3} - 2 = -3 - i \neq 0$.

Therefore, n must be divisible by 4. It is easy to check that in such case $P(-i)$ equals 0 as well.

Exercise 5. Answer: A. Since polynomial has d distinct roots, the probability of choosing a root is *exactly* d/p . The probability that Alice fools Bob after n rounds is then $(d/p)^n$.

Exercise 6. Answer: B. According to the Schwartz-Zippel Lemma, we have $\Pr[f(\mathbf{x}) = 0 \mid \mathbf{x} \xleftarrow{R} \mathbb{S}] \leq \deg f / |\mathbb{S}|$, so we need to find the size $|\mathbb{S}|$. Our claim is that the number of solutions to $x_1 + \cdots + x_n = 1$ over $\mathbb{F}_p$ is exactly p^{n-1} . Indeed, for any p^{n-1} possible values of $(x_2, \dots, x_n)$ there is a unique x_1 such that $x_1 + \cdots + x_n = 1$, which is $1 - \sum_{i=2}^n x_i$. Therefore, $\Pr[f(\mathbf{x}) = 0] \leq \deg f / p^{n-1}$.

Field Extensions

Exercise 1. Answer: D. Since $i^2 = -1$, we have $(3+i)(4+i) = 12 + 7i + i^2 = 11 + 7i$. Reducing this modulo 7 gives 4.

Exercise 2. Answer: D. Note that $2/i = 2i/i^2 = 2i/(-2) = -i = (p-1)i$.

Exercise 3. Answer: A. Such field extension is valid as long as $v^2 + v + 1$ has no roots over $\mathbb{F}_p$. Note that $v^2 + v + 1 = 0$ has no solutions if and only if $v^3 - 1$ has no trivial solutions: indeed, $v^3 - 1 = (v - 1)(v^2 + v + 1)$. From the problem statement we know this is the case when $p \equiv 1 \pmod{3}$. From the provided choice, this is the case for $p = 8431$.

Exercise 4. Answer: B. Field extension $\mathbb{F}_5[i]/(r(i))$ is valid only when $r(x)$ is irreducible over $\mathbb{F}_5$. This is the case when $r(x) = x^2 + 2$.

Exercise 5. Answer: A. The zero of the provided polynomial is:

$$x_0 = \frac{i + 3}{i} = \frac{i^2 + 3i}{i^2} = 1 + \frac{3i}{-2} = 1 - 3i \cdot 3 = 1 - 9i = 1 + i$$

Exercise 6. Answer: A. Notice that $j^4 = (j^2)^2 = \xi^2$. Since $\xi = 1 + i$ by

definition, we have $j^4 = (1 + i)^2 = -1 + 2i = 4 + 2i$.

Exercise 7. Answer: C. Again, using identity $j^2 = \xi$, we arrive at $j^3 + 2i^2\xi = j\xi - 4\xi = j\xi + \xi$. Therefore, both the real and imaginary parts are same and equal to ξ .

Exercise 8. Answer: B. Let us expand the expression: $(a_0 + a_1j)b_1j = a_0b_1j + a_1b_1j^2 = a_1b_1\xi + a_0b_1j$. So the real part is thus $a_1b_1\xi$.

Exercise 9. Answer: B. $z\bar{z} = (c_0 + c_1j)(c_0 - c_1j) = c_0^2 - c_1^2j^2 = c_0^2 - \xi c_1^2$.

Elliptic Curves and Pairing

Exercise 1. Answer: 1. If $(2, 3) \in E/\mathbb{F}_7$, then $3^2 = 2^3 + b$ and thus $b = 9 - 8 = 1$.

Exercise 2. Answer: A. If $P \oplus Q = \mathcal{O}$, then P and Q must have the same x coordinates and opposite y coordinates. Points $P = (2, 3)$ and $Q = (2, 8)$ satisfy this condition.

Exercise 3. Answer: A. According to Hasse's theorem, we have

$$r = p^2 + 1 - t, \quad |t| \leq 2p$$

In our case, $p = 167$, so $r = 167^2 + 1 - t$ for $|t| \leq 334$. The only suitable option among provided is $167^2 - 5$.

Exercise 4. Answer: C. Upper bound for factors of $|E| = qr$ is $\max\{q, r\}$, so the complexity is $\mathcal{O}(\sqrt{\max\{q, r\}})$.

Exercise 5. Answer: A. Note that reflexivity is not satisfied. Indeed, $a + a < 0$ does not hold for $a \in \mathbb{Q}_{>0}$.

Exercise 6. Answer: C. By definition, $[1.4]_{\sim} = \{x \in \mathbb{R} : x - 1.4 \in \mathbb{Z}\}$. That means that every $x \in [1.4]_{\sim}$ is of form $x = 1.4 + n$ where $n \in \mathbb{Z}$. This can be further simplified to $x = 0.4 + n'$ with $n' \in \mathbb{Z}$. Thus, the fractional part must be 0.4.

Exercise 7. Answer: D. For homogeneous projective coordinates we must have $X_2/X_1 = Y_2/Y_1 = Z_2/Z_1$. In our case, the fraction $X_2/X_1 = 16/4 = 4$ while $Y_2/Y_1 = 8/3$, so two points are not equivalent.

Exercise 8. Answer: E. Projective coordinates are effective since addition of points over the projective space does not require inversion. The only inversion needed is during the transforming point back to the affine space.

Exercise 9. Answer: D. The number of additions equals the number of non-zero bits in the binary representation of k . Since $19 = 10011_2$, the number of additions is 3.

Exercise 10. Answer: A. One way to reduce the number of inversions is to calculate the $t \leftarrow (a + b)^{-1}$ first, which gives $(a + b)^{-4} = t^4$ for free. All left to do is to calculate $(a^2 + c^2)^{-1}$, making the total cost 2 inversions. However, we can use only one inversion: notice that

$$\frac{a - b}{(a + b)^4} + \frac{c}{a + b} + \frac{d}{a^2 + c^2} = \frac{(a - b)(a^2 + c^2) + c(a + b)^3(a^2 + c^2) + d(a + b)^4}{(a + b)^4(a^2 + c^2)}$$

Therefore, we need only one inversion: first we calculate $z \leftarrow (a+b)^4(a^2+c^2)$ and then calculate z^{-1} . Then we multiply by the numerator, costing zero inversions. So the optimal solution is 1 inversion. Note that the second approach requires larger number of field multiplications, so it might be not optimal in case inversion is cheap.

Exercise 11. Answer: B. Notice that $e([4]G_1, [4]G_2) = e(G_1, G_2)^{4 \cdot 4} = e(G_1, G_2)^{16} \neq e([3]G_1, [5]G_2) = e(G_1, G_2)^{15}$.

Exercise 12. Answer: E. Note that $e(P, P)e(Q, Q) = e(G, G)^{x^2} e(G, G)^{y^2} = e(G, G)^{x^2+y^2}$. Therefore, checking whether $e(P, P)e(Q, Q) = e(G, G)$ is equivalent to checking whether $x^2 + y^2 = 1$.

Commitment Schemes

Exercise 1. Answer: C. We have $H(x) = C$. By the definition of H , we have $13x + 17 = 39$ over $\mathbb{F}_{41}$. Therefore, $13x = 22$, so $x = 22 \cdot 13^{-1} = 8$.

Exercise 2. Answer: A. If $U = [6]G$ then the Pedersen commitment for element (m, r) can be written as $\text{com}(m, r) = [m]G + [r]U = [m]G + [6r]G = [m + 6r]G$. Therefore, to fool Dmytro, it suffices to take such (m_2, r_2) that $m_2 + 6r_2 = m_1 + 6r_1$. Substituting numbers we get $15 + 6r_2 = 3 + 6 \cdot 7 = 45$. Therefore, $6r_2 = 30$ so finally $r_2 = 5$.

Exercise 3. Answer: D. It suffices to manually check every option. For option (D) we indeed have $H(H(17, H(3, 4)), 13) = 37$.

Exercise 4. Answer: C. The quotient polynomial $q(x)$ must satisfy $q(x) \cdot (x - 2) = p(x)$. It is easy to check that for $q(x) = x^2 - 8x + 15$ we indeed have $(x^2 - 8x + 15)(x - 2) = x^3 - 10x^2 + 31x - 30$.

Security Analysis

Exercise 1. Answer: B. Adversary $\mathcal{A}$ can safely send two different messages m_0 and m_1 to the challenger with $\text{lsb}(m_b) = b$ for $b \in \{0, 1\}$. Then, having received c , $\mathcal{A}$ can output $b' \leftarrow \mathcal{O}_{\text{lsb}}(c)$ where $\mathcal{O}_{\text{lsb}}(c)$ is an oracle that returns the least significant bit of m . Thus, $\Pr[b = \hat{b}] = 1$ and thus $\text{SSAdv}[\mathcal{A}] = 1/2$.

Exercise 2. Answer: D. The probability should be subtracted from the probability of random guess. The random guess corresponds to the probability $1/|\mathcal{M}|$, thus the answer.

Exercise 3. Answer: D. Take $f = g$. Then, $f/g = 1 \neq \text{negl}(\lambda)$.

Introduction to Zero-Knowledge Proofs

Exercise 1. Answer: A.

Exercise 2. Answer: B. The probability of accepting the false statement after n rounds is $(1/8)^n = 2^{-3n}$. To guarantee 120 bits of security, we need $3n = 120$ or $n = 40$ rounds.

Exercise 3. Answer: C. To check a great explanation, see the answer from [this forum](#).

Exercise 4. Answer: C.

Exercise 5. Answer: B.

Exercise 6. Answer: D.

Sigma Protocols

Exercise 1. The protocol is sound and complete, but it is not zero-knowledge.

Exercise 2. The protocol is not complete. Yet, it is sound and zero-knowledge.

Exercise 3. While the protocol is sound and complete, it is not zero-knowledge: based on (a, r) , the verifier can find α as $a - r$.

Exercise 4. The protocol is complete and zero-knowledge, but it is not sound. Indeed, the prover does not prove that r is chosen randomly, so he can send $(g^z/h, z)$ to the verifier, which would make the verifier accept the statement, yet the prover might not know the discrete logarithm of h .

Exercise 5. This is a Schnorr's protocol. It is sound, complete, and zero-knowledge.

Exercise 6. Answer: B. Note that $w = g^{\alpha\beta} = (g^\alpha)^\beta = u^\beta$.

Exercise 7. Answer: C. $v = g^\beta$ and $w = u^\beta$.

Exercise 8. Answer: C. $v_r = g^{\beta r}$ and $w_r = u^{\beta r}$.

Exercise 9. Answer: A. We need to hash both the initial statement (u, v, w) and the commitment (v_r, w_r) , so the hash function should input five group elements.

Exercise 10. Answer: A. $c \leftarrow H((u, v, w), (v_r, w_r))$.

24 Acknowledgements

We are deeply thankful to Dmytro Zakharov, Anton Levochko, Oleg Fomenko, Roman Skovron, Nikita Masych, Denis Riabtsev, and Kyrylo Riabov for their direct participation in the creation of this book.

We additionally want to thank Oleksandr Kurbatov, Yaroslav Panasenko, Yevhen Hrubian, and Olena Voloshchuk for the valuable feedback and support.

Of course, nothing would be possible without Pavel Kravchenko, Vladymir Dubinin, and Lasha Antadze's desire to apply cryptography and zero knowledge in real applications.

References

- [1] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*. 2023.
Available at: <https://toc.cryptobook.us/#toc>.

contact@distributedlab.com